Modern Databases and Distributed Data Systems

Saman Siadati

September 2025

Modern Databases and Distributed Data Systems
© 2025 Saman Siadati
Edition 1.1
DOI: https://doi.org/10.5281/zenodo.23719816

Preface

The world today is not only driven by data—it is defined by how we store, process, and understand it at scale. From real-time applications and global platforms to artificial intelligence and large-scale analytics, modern systems rely on data infrastructures that go far beyond traditional databases. Distributed systems, NoSQL technologies, and specialized data models have become essential in handling the volume, velocity, and variety of today's data.

This book, Modern Databases and Distributed Data Systems, is written to provide a clear, structured, and practical guide to this evolving landscape.

My journey into this field began more than two decades ago with a background in Applied Mathematics. I started as a statistical data analyst, working on software systems where data played a secondary role. Over time, my focus shifted toward data mining, and eventually into data science and data engineering. As systems grew in complexity and scale, I came to understand that knowledge of traditional databases alone is not enough. To build modern systems, one must also understand distributed architectures, NoSQL paradigms, and new forms of data processing.

This realization motivated me to write this book as a continuation of that journey—moving beyond the foundations of SQL into the broader world of modern data systems.

This book is designed for readers who already have basic familiarity with databases and are ready to explore what comes next. Whether you are a developer, data engineer, researcher, or simply a curious learner, the goal is to help you build a deeper understanding of how modern data systems are designed and why they work the way they do.

Rather than presenting technologies in isolation, this book focuses on the principles behind them. You will learn not only what tools like document databases, graph systems, and streaming platforms do, but also when and why to use them. Concepts such as scalability, consistency, distributed trade-offs, and real-time processing are explained with a strong emphasis on intuition and practical application.

The aim is not to overwhelm you with complexity, but to guide you step by step toward a level where you can confidently think about, design, and work with modern data architectures.

You are free to use, share, and adapt any part of this book. If it contributes to your learning journey, a citation is appreciated but not required. This work is created to make modern data systems more accessible, practical, and engaging for everyone who wants to understand the technologies shaping today's data-driven world.

Saman Siadati
September 2025

Contents

Preface iii

I NoSQL Foundations 1

1 Beyond Relational Databases: Why NoSQL? 3
- 1.1 Introduction to Modern Data Challenges . . . 3
- 1.2 Limitations of Relational Databases . . . 4
  - 1.2.1 Rigid Schemas . . . 5
  - 1.2.2 The Object–Relational Impedance Mismatch . . . 5
  - 1.2.3 Join Cost at Scale . . . 5
  - 1.2.4 Difficult Horizontal Scaling . . . 5
  - 1.2.5 Write Throughput . . . 5
  - 1.2.6 Unstructured and Nested Data . . . 5
- 1.3 The Emergence of NoSQL . . . 6
  - 1.3.1 Historical Background . . . 6
  - 1.3.2 What Does "NoSQL" Mean? . . . 6
  - 1.3.3 Core Motivations . . . 6
  - 1.3.4 Polyglot Persistence . . . 7
- 1.4 Types of NoSQL Databases . . . 7
  - 1.4.1 Document Databases . . . 7
  - 1.4.2 Key-Value Stores . . . 8
  - 1.4.3 Wide-Column (Column-Family) Databases . . . 8
  - 1.4.4 Graph Databases . . . 8
  - 1.4.5 Comparison . . . 8
- 1.5 Trade-offs and Design Considerations . . . 9
  - 1.5.1 The CAP Theorem . . . 9
  - 1.5.2 Beyond CAP: PACELC . . . 10
  - 1.5.3 Consistency Models and Tunable Consistency . . . 10
  - 1.5.4 ACID versus BASE . . . 10
  - 1.5.5 Choosing a Database: A Practical Approach . . . 11

2 Document Databases and JSON Data Models 15
- 2.1 Introduction to Document Databases . . . 15
  - 2.1.1 Relational and Document Terminology . . . 16

2.1.2 Running Example and Scope . . . 16
2.2 JSON Data Model and Structure . . . 17
2.2.1 JSON Basics . . . 17
2.2.2 From JSON to BSON . . . 17
2.2.3 Schema Flexibility: Schema-on-Read and Schema-on-Write . . . 18
2.3 Data Modeling in Document Databases . . . 19
2.3.1 Embedding . . . 19
2.3.2 Referencing . . . 19
2.3.3 Choosing Between Embedding and Referencing . . . 20
2.3.4 Common Pitfalls . . . 21
2.3.5 A Workload-Driven Design Process . . . 21
2.4 Querying and Indexing Documents . . . 21
2.4.1 Basic Operations . . . 21
2.4.2 Indexing . . . 22
2.4.3 Aggregation Pipelines . . . 23
2.4.4 Full-Text Search . . . 23
2.4.5 Query Performance in Practice . . . 24
2.5 Scalability, Consistency, and Use Cases . . . 24
2.5.1 Replication . . . 24
2.5.2 Sharding . . . 24
2.5.3 Consistency and Transactions . . . 24
2.5.4 Use Cases . . . 25
2.5.5 Limitations . . . 25

3 Key-Value Stores and In-Memory Databases 29
3.1 Introduction to Key-Value Stores . . . 29
3.1.1 The Data Model . . . 29
3.1.2 Why Simplicity Matters . . . 30
3.1.3 Representative Systems . . . 30
3.2 Architecture of Key-Value Systems . . . 30
3.2.1 Storage Structures . . . 30
3.2.2 Partitioning (Sharding) . . . 31
3.2.3 Consistent Hashing . . . 32
3.2.4 Replication . . . 32
3.2.5 Consistency Models in Practice . . . 33
3.2.6 Request Routing . . . 33
3.3 In-Memory Databases . . . 33
3.3.1 Why Memory? . . . 33
3.3.2 Rich Data Structures . . . 33
3.3.3 Persistence and Durability . . . 34
3.3.4 Memory Management and Eviction . . . 35
3.4 Common Use Cases and Patterns . . . 36
3.4.1 Caching . . . 36
3.4.2 Session Management . . . 37
3.4.3 Counters, Rate Limiting, and Leaderboards . . . 37

3.4.4 Configuration and Coordination . . . 37
3.4.5 Real-Time Analytics and Messaging . . . 38
3.5 Designing Keys and Values . . . 38
3.6 Advantages and Limitations . . . 39
3.6.1 Advantages . . . 39
3.6.2 Limitations . . . 39
3.6.3 When to Choose a Key-Value Store . . . 39

4 Column-Family Databases and Wide-Column Models 43
4.1 Introduction to Column-Family Databases . . . 43
4.1.1 Origins . . . 43
4.1.2 Relational Versus Wide-Column Concepts . . . 44
4.2 Data Model and Structure . . . 44
4.2.1 A Sorted, Multidimensional Map . . . 44
4.2.2 Sparse Storage . . . 45
4.2.3 Cassandra's Model: Partition Key and Clustering Columns . . . 45
4.2.4 Timestamps and Versions . . . 46
4.3 Architecture and Distribution . . . 46
4.3.1 Partitioning and Replication . . . 46
4.3.2 Two Architectural Styles . . . 47
4.3.3 The Storage Engine: Why Writes Are Fast . . . 47
4.3.4 The Read Path and Bloom Filters . . . 48
4.3.5 Compaction Trade-offs . . . 48
4.3.6 Deletes and Tombstones . . . 48
4.3.7 Consistency Levels . . . 49
4.4 Data Modeling . . . 49
4.4.1 Query-First Design . . . 49
4.4.2 Denormalization: One Table per Query . . . 50
4.4.3 Designing Partitions . . . 50
4.4.4 Common Pitfalls . . . 51
4.5 Use Cases and Applications . . . 51
4.5.1 When Not to Use Them . . . 51
4.6 Challenges and Limitations . . . 52

5 Data Modeling in NoSQL Systems 55
5.1 Introduction to NoSQL Data Modeling . . . 55
5.1.1 Data-Driven Versus Workload-Driven Design . . . 55
5.1.2 Aggregates: The Unit of Design . . . 56
5.1.3 Flexibility With Responsibility . . . 56
5.1.4 A Modeling Process . . . 57
5.2 Denormalization and Data Duplication . . . 57
5.2.1 Why Denormalize . . . 57
5.2.2 When Duplication Pays Off . . . 58
5.2.3 Keeping Duplicated Data Consistent . . . 58
5.3 Modeling in Document Databases . . . 59

5.4 Modeling in Key-Value and Wide-Column Stores . . . 60
5.4.1 Key-Value Stores: Key Design Is the Model . . . 60
5.4.2 Wide-Column Stores: One Table per Query . . . 61
5.5 Graph Data Modeling . . . 61
5.5.1 The Property Graph Model . . . 61
5.5.2 Modeling Guidelines . . . 62
5.5.3 Example: Purchases and Recommendations . . . 62
5.6 One Domain, Several Models . . . 63
5.7 Anti-Patterns and a Design Checklist . . . 64
5.7.1 Common Anti-Patterns . . . 64
5.7.2 Design Checklist . . . 64

II Distributed Data Systems 69

6 Foundations of Distributed Systems 71
6.1 Introduction to Distributed Systems . . . 71
6.1.1 Why Distribute? . . . 71
6.1.2 What Makes Distribution Hard . . . 72
6.1.3 Transparency and Its Limits . . . 72
6.2 System Models and Failures . . . 73
6.2.1 Timing Assumptions . . . 73
6.2.2 Failure Models . . . 73
6.2.3 Failure Detection . . . 74
6.3 Communication in Distributed Systems . . . 74
6.3.1 Message Passing and Communication Styles . . . 74
6.3.2 Delivery Semantics . . . 74
6.4 Time, Clocks, and Ordering . . . 76
6.4.1 Physical Clocks . . . 76
6.4.2 Logical Clocks . . . 76
6.5 Consistency and Replication . . . 76
6.5.1 Why Replicate . . . 76
6.5.2 Consistency Models . . . 76
6.5.3 Replication Strategies . . . 77
6.5.4 Quorums and the CAP Theorem . . . 78
6.5.5 Conflict Resolution . . . 78
6.6 Consensus, Fault Tolerance, and Reliability . . . 79
6.6.1 Fault-Tolerance Techniques . . . 79
6.6.2 The Consensus Problem . . . 79
6.6.3 Paxos and Raft . . . 79
6.6.4 Distributed Transactions and Atomic Commit . . . 80
6.7 Scalability and Performance . . . 81
6.7.1 Measuring Performance . . . 81
6.7.2 Tail Latency . . . 81
6.7.3 Two Useful Laws . . . 81

6.7.4 Techniques for Scalability . . . 82

7 The CAP Theorem and System Trade-offs 87
7.1 Why Trade-offs Are Unavoidable . . . 87
7.2 The CAP Theorem . . . 88
7.2.1 Origins . . . 88
7.2.2 The Three Properties, Precisely . . . 88
7.2.3 The Theorem and a Proof Sketch . . . 88
7.2.4 Common Misconceptions . . . 89
7.3 Beyond CAP: Latency, PACELC, and Tunable Consistency . . . 90
7.3.1 PACELC . . . 90
7.3.2 Tunable Consistency . . . 91
7.3.3 Quantifying the Availability Cost . . . 91
7.3.4 Harvest and Yield . . . 92
7.4 ACID, BASE, and Isolation . . . 92
7.4.1 ACID . . . 92
7.4.2 BASE . . . 92
7.4.3 Isolation Levels . . . 93
7.5 Designing for Partitions . . . 94
7.5.1 Per-Operation Decisions . . . 94
7.5.2 Compensating Actions . . . 94
7.5.3 When Is Coordination Necessary? . . . 94
7.6 Other Trade-offs in System Design . . . 95
7.7 Evaluating Trade-offs in Practice . . . 95
7.8 A Decision Framework . . . 96

8 Consistency Models in Practice 101
8.1 From Theory to Practice . . . 101
8.2 Anomalies in Practice . . . 102
8.2.1 Read-Your-Writes Violations . . . 102
8.2.2 Monotonic-Read Violations . . . 103
8.2.3 Causality Violations . . . 103
8.2.4 Lost Updates . . . 103
8.2.5 Write Skew . . . 103
8.3 Session Guarantees . . . 104
8.3.1 How Session Guarantees Are Implemented . . . 104
8.4 Consistency in Real Systems . . . 105
8.4.1 Relational Replicas: PostgreSQL . . . 105
8.4.2 MongoDB: Write Concern, Read Concern, and Causal Sessions . . . 106
8.4.3 Cassandra: Consistency Levels and Lightweight Transactions . . . 106
8.4.4 DynamoDB: Strong Reads, Conditions, and Transactions . . . 107
8.4.5 A Spectrum as a Product Feature: Cosmos DB . . . 107
8.4.6 Redis and etcd . . . 107
8.5 How Eventual Consistency Converges . . . 108
8.5.1 How Eventual Is "Eventually"? . . . 109

8.6 Application-Level Techniques . . . 109
8.6.1 Prefer Atomic Operations to Read-Modify-Write . . . 109
8.6.2 Optimistic Concurrency Control . . . 109
8.6.3 Idempotence and Deduplication . . . 110
8.6.4 The Dual-Write Problem and the Outbox Pattern . . . 110
8.6.5 Hidden Sources of Staleness . . . 110
8.7 Testing and Monitoring Consistency . . . 111
8.8 Choosing Consistency Levels . . . 111

9 Replication and Data Distribution Strategies 117
9.1 Why Replicate and Distribute Data . . . 117
9.2 Replication Topologies . . . 118
9.2.1 Single-Leader Replication . . . 118
9.2.2 Multi-Leader Replication . . . 119
9.2.3 Leaderless Replication . . . 119
9.3 How Changes Are Replicated . . . 119
9.3.1 Synchronous, Asynchronous, and Semi-Synchronous . . . 119
9.3.2 Log Formats . . . 120
9.3.3 Replication Lag . . . 120
9.4 Failover . . . 120
9.5 Conflicts in Multi-Leader and Leaderless Systems . . . 121
9.6 Partitioning Strategies . . . 121
9.6.1 Range Partitioning . . . 122
9.6.2 Hash Partitioning . . . 122
9.6.3 Consistent Hashing and Virtual Nodes . . . 122
9.6.4 Fixed Partitions and Directories . . . 123
9.6.5 Hot Keys and Skew . . . 123
9.7 Secondary Indexes and Request Routing . . . 123
9.7.1 Secondary Indexes in a Partitioned Database . . . 123
9.7.2 Routing Requests to the Right Node . . . 123
9.8 Rebalancing . . . 124
9.9 Replication and Distribution in Real Systems . . . 124
9.9.1 PostgreSQL: Logical Replication . . . 124
9.9.2 MongoDB: Sharding and the Shard Key . . . 125
9.9.3 Cassandra: Replication Factor and Placement . . . 126
9.9.4 DynamoDB: Managed Partitioning . . . 126
9.10 Geo-Distribution and Data Placement . . . 126
9.11 Operating Replicated and Partitioned Systems . . . 127
9.12 Choosing a Strategy . . . 128

10 Distributed Transactions and Fault Tolerance 133
10.1 Why Distributed Transactions Are Hard . . . 133
10.2 Atomic Commit . . . 134
10.2.1 Two-Phase Commit . . . 134
10.2.2 The Blocking Problem . . . 134

10.2.3 Consensus-Based Commit . . . 135
10.2.4 Deterministic Alternatives . . . 135
10.3 Isolation Across Nodes . . . 136
10.3.1 Locking and Multi-Version Concurrency . . . 136
10.3.2 Snapshot Isolation and Serializability . . . 136
10.3.3 Timestamps and Clocks . . . 136
10.4 Transactions in Real Systems . . . 137
10.4.1 PostgreSQL: Prepared Transactions . . . 137
10.4.2 MongoDB and DynamoDB . . . 138
10.5 Sagas and Compensation . . . 139
10.5.1 Example: Order Fulfillment . . . 139
10.5.2 Orchestration and Choreography . . . 139
10.6 Idempotence and Delivery Guarantees . . . 140
10.7 The Fault Model . . . 141
10.7.1 Failure Detection . . . 141
10.8 Techniques for Fault Tolerance . . . 142
10.8.1 Retries Done Safely . . . 142
10.8.2 Containing Failures . . . 142
10.8.3 Graceful Degradation . . . 143
10.9 Recovery and Durability . . . 143
10.9.1 Write-Ahead Logging and Crash Recovery . . . 143
10.9.2 Backups and Disaster Recovery . . . 143
10.9.3 Resolving In-Doubt and Failed Transactions . . . 143
10.10 Testing Fault Tolerance . . . 144
10.11 Choosing an Approach . . . 145

III Specialized Databases 149

11 Graph Databases and Relationship Modeling 151
11.1 Why Graphs . . . 151
11.1.1 The Join Problem . . . 152
11.2 Graph Data Models . . . 152
11.2.1 The Property Graph . . . 152
11.2.2 RDF and Triple Stores . . . 153
11.2.3 Other Models . . . 154
11.3 Relationship Modeling . . . 154
11.3.1 A Modeling Process . . . 154
11.3.2 From Relational to Graph . . . 154
11.3.3 Modeling Patterns . . . 154
11.3.4 Common Mistakes . . . 155
11.4 Querying Graphs . . . 156
11.4.1 Cypher . . . 156
11.4.2 SPARQL . . . 156
11.4.3 Gremlin, GQL, and SQL/PGQ . . . 157

11.5 Graph Algorithms . . . 157
11.5.1 PageRank . . . 157
11.5.2 Where Algorithms Run . . . 158
11.6 Storage and Traversal Internals . . . 158
11.6.1 Index-Free Adjacency . . . 158
11.6.2 Native and Non-Native Storage . . . 158
11.6.3 Indexes Still Matter . . . 159
11.7 Scaling and Distributing Graphs . . . 159
11.7.1 The Partitioning Problem . . . 159
11.7.2 Common Scaling Strategies . . . 160
11.7.3 Transactions and Consistency . . . 160
11.8 Graph Databases in Practice . . . 160
11.8.1 Typical Applications . . . 160
11.9 Performance Pitfalls . . . 161
11.10Choosing a Graph Approach . . . 162

12 Time-Series Databases and Temporal Data **167**
12.1 Data That Arrives With a Timestamp . . . 167
12.2 Data Models for Time Series . . . 168
12.2.1 Series, Tags, and Fields . . . 168
12.2.2 Narrow and Wide Layouts . . . 169
12.2.3 The Cardinality Problem . . . 169
12.3 Ingestion and Storage . . . 170
12.3.1 The Write Path . . . 170
12.3.2 Compression . . . 170
12.3.3 Time-Based Partitioning . . . 171
12.4 Querying Time Series . . . 171
12.4.1 Typical Operations . . . 171
12.4.2 TimescaleDB: SQL for Time Series . . . 172
12.4.3 Prometheus: PromQL . . . 173
12.4.4 Windows in Stream Processing . . . 173
12.5 Retention, Downsampling, and Tiering . . . 173
12.6 Time-Series Systems in Practice . . . 174
12.6.1 Distribution and Replication . . . 174
12.6.2 Choosing Between a TSDB and a General Database . . . 174
12.7 Temporal Data: Modeling History . . . 174
12.7.1 Valid Time and Transaction Time . . . 175
12.7.2 System-Versioned Tables in SQL . . . 176
12.7.3 Validity Periods With Range Types . . . 176
12.7.4 Other Techniques for History . . . 177
12.8 Getting Time Right . . . 177
12.9 Operating Time-Series Systems . . . 178
12.10Choosing an Approach . . . 178

13 Search and Indexing Systems **183**

13.1 Why Databases Need Search Engines . . . 183
13.2 The Inverted Index . . . 184
13.2.1 Structure . . . 184
13.2.2 Compression . . . 185
13.2.3 Doc Values and Stored Fields . . . 185
13.3 Text Analysis . . . 185
13.4 Relevance Ranking . . . 186
13.4.1 TF-IDF . . . 186
13.4.2 BM25 . . . 186
13.4.3 Beyond Term Statistics . . . 187
13.4.4 Evaluating Search Quality . . . 187
13.5 Index Internals . . . 187
13.5.1 Immutable Segments . . . 187
13.5.2 Query Execution . . . 188
13.6 Distributed Search . . . 188
13.6.1 Shards and Replicas . . . 188
13.6.2 Scatter–Gather Execution . . . 188
13.6.3 Consequences for Ranking, Paging, and Sizing . . . 188
13.7 Querying and Faceted Search . . . 189
13.7.1 Search Engine Queries . . . 189
13.7.2 Full-Text Search in a Relational Database . . . 190
13.8 Vector Search and Hybrid Retrieval . . . 191
13.8.1 Embeddings and Similarity . . . 191
13.8.2 Approximate Nearest-Neighbor Indexes . . . 191
13.8.3 Hybrid Search . . . 191
13.8.4 Retrieval for Language-Model Applications . . . 192
13.9 Search Systems in Practice . . . 192
13.10 Keeping the Index in Sync . . . 192
13.11 Operating Search Systems . . . 192
13.12 Choosing an Approach . . . 193

14 Vector Databases and Embedding Search **199**
14.1 From Keywords to Meaning . . . 199
14.1.1 What Is New in This Data Model . . . 200
14.2 Embeddings and Distance Metrics . . . 200
14.2.1 Where Embeddings Come From . . . 200
14.2.2 Distance and Similarity . . . 200
14.2.3 Embedding Models Must Be Versioned . . . 201
14.3 Why Exact Search Does Not Scale . . . 201
14.4 Approximate Nearest-Neighbor Indexes . . . 201
14.4.1 Inverted File Index (IVF) . . . 202
14.4.2 Hierarchical Navigable Small World (HNSW) . . . 202
14.4.3 Quantization: Compressing Vectors . . . 203
14.5 Recall, Latency, and Memory . . . 203
14.5.1 Measuring and Tuning . . . 204

14.6 Filtering and Metadata . . . 204
14.7 Vector Database Architecture . . . 205
14.7.1 Segments and Updates . . . 205
14.7.2 Sharding and Replication . . . 205
14.7.3 Consistency and Freshness . . . 205
14.7.4 Storage Layout and Disaggregation . . . 206
14.7.5 Multi-Tenancy . . . 206
14.8 Systems in Practice . . . 206
14.8.1 PostgreSQL with pgvector . . . 207
14.8.2 Elasticsearch: Vectors Beside Keywords . . . 207
14.9 Retrieval Pipelines . . . 208
14.9.1 Chunking . . . 209
14.9.2 Hybrid Retrieval and Re-ranking . . . 209
14.9.3 Evaluating Retrieval . . . 209
14.9.4 Other Uses of Vector Search . . . 210
14.10 Operating Vector Systems . . . 210
14.10.1 Capacity and Cost . . . 210
14.10.2 Reindexing and Model Changes . . . 210
14.10.3 Keeping Vectors Synchronized . . . 210
14.10.4 Monitoring . . . 210
14.10.5 Security and Privacy . . . 211
14.11 Choosing an Approach . . . 211

IV Query Languages Beyond SQL 217

V Query Languages Beyond SQL 219

15 Querying Graph Data with Cypher 221
15.1 Why a Graph Query Language . . . 221
15.1.1 What Is Different from SQL . . . 222
15.2 A Running Example . . . 222
15.3 Patterns: The Heart of Cypher . . . 223
15.3.1 Naming Conventions . . . 223
15.4 Reading Data . . . 224
15.4.1 Projection, Ordering, and Paging . . . 224
15.4.2 Aggregation . . . 225
15.4.3 Chaining Query Parts with WITH . . . 225
15.4.4 Optional Matches, Existence Tests, and Unwinding . . . 225
15.5 Paths and Variable-Length Traversal . . . 226
15.5.1 Why Depth Must Be Bounded . . . 226
15.5.2 Supernodes . . . 226
15.6 Writing Data . . . 227
15.6.1 MERGE and Its Pitfall . . . 227

15.6.2 Updates and Deletes . . . 228
15.7 How a Cypher Query Is Executed . . . 228
15.7.1 Inspecting Plans . . . 228
15.8 Indexes and Constraints . . . 229
15.9 Cypher and SQL Side by Side . . . 230
15.10 Modeling for Queries . . . 231
15.11 Common Patterns . . . 232
15.11.1 Recommendation . . . 232
15.11.2 Fraud and Link Analysis . . . 232
15.11.3 Access Control and Hierarchies . . . 232
15.11.4 Combining Graph Traversal and Vector Search . . . 232
15.12 Standards and Related Languages . . . 233
15.13 Operating Graph Queries . . . 234
15.13.1 Parameters and Injection . . . 234
15.13.2 Bulk Loading and Batching . . . 234
15.13.3 Limits, Timeouts, and Access Control . . . 234
15.13.4 Monitoring . . . 235
15.14 Choosing an Approach . . . 235

16 Querying Distributed Data with CQL and Similar Languages 241
16.1 Why Distributed Databases Restrict Their Queries . . . 241
16.1.1 What Is Different from SQL and from Cypher . . . 242
16.2 CQL Fundamentals . . . 242
16.2.1 Keyspaces and Replication . . . 242
16.2.2 The Primary Key: Partition Key and Clustering Columns . . . 243
16.2.3 Data Types . . . 244
16.2.4 Writing: Upserts, Expiry, and Deletes . . . 244
16.2.5 Deletes Create Tombstones . . . 244
16.3 Reading: What a Query May Ask . . . 245
16.3.1 ALLOW FILTERING . . . 245
16.3.2 Aggregation and Other Limits . . . 246
16.4 Modeling for CQL . . . 246
16.4.1 Partition Design . . . 246
16.4.2 Keeping Denormalized Tables Consistent . . . 247
16.4.3 Counters, Static Columns, and Indexes . . . 247
16.5 Consistency, Quorums, and Lightweight Transactions . . . 248
16.5.1 Choosing a Level . . . 248
16.5.2 Lightweight Transactions . . . 249
16.6 How a Query Is Executed . . . 250
16.6.1 Paging . . . 250
16.6.2 Prepared Statements and Drivers . . . 250
16.6.3 Diagnosing Slow Queries . . . 251
16.7 Similar Languages in Other Distributed Stores . . . 251
16.7.1 PartiQL on DynamoDB . . . 252
16.7.2 The MongoDB Aggregation Pipeline . . . 252

16.7.3 Distributed SQL . . . 253
16.7.4 Search-Engine Languages . . . 253
16.8 Operating CQL Clusters . . . 253
16.8.1 Schema Evolution . . . 253
16.8.2 Repair, Compaction, and Capacity . . . 253
16.8.3 Security . . . 254
16.8.4 Monitoring . . . 254
16.9 Choosing an Approach . . . 254

17 JSON and Document Query Languages 259
17.1 Querying Nested Data . . . 259
17.1.1 Three Families of Languages . . . 260
17.2 A Running Example . . . 260
17.2.1 JSON and Its Binary Cousins . . . 261
17.3 Finding Documents . . . 261
17.3.1 Arrays: The Subtle Part . . . 261
17.3.2 Missing, Null, and Empty . . . 262
17.3.3 Paging . . . 262
17.4 Updating Documents . . . 263
17.4.1 Scope of Atomicity . . . 264
17.4.2 Unbounded Arrays and Document Size . . . 264
17.5 The Aggregation Pipeline . . . 264
17.5.1 Joining with $lookup . . . 265
17.5.2 Writing Efficient Pipelines . . . 266
17.6 Indexes and Query Plans . . . 266
17.6.1 Compound Indexes: Equality, Sort, Range . . . 266
17.6.2 Reading a Query Plan . . . 267
17.7 JSON in Relational Databases . . . 267
17.7.1 Document Store or JSON Column? . . . 269
17.8 Path and Transformation Languages . . . 269
17.8.1 JSONPath . . . 269
17.8.2 jq and JMESPath . . . 269
17.8.3 Safety of Path Expressions . . . 270
17.9 SQL++ and Other SQL-Like Document Languages . . . 270
17.10 Schemas and Validation . . . 271
17.11 Consistency and Transactions in Queries . . . 271
17.12 Operating Document Queries . . . 272
17.12.1 Query Injection . . . 272
17.12.2 Performance Diagnosis . . . 272
17.12.3 Monitoring . . . 272
17.13 Choosing an Approach . . . 272

18 Vector Similarity Queries and Retrieval Models 277
18.1 From Matching to Ranking by Similarity . . . 277
18.1.1 Kinds of Similarity Query . . . 278

18.2 Distances, Scores, and Thresholds . . . 278
18.2.1 Distance Versus Similarity . . . 279
18.2.2 Scores Are Not Probabilities . . . 279
18.2.3 Range Queries and Approximate Indexes . . . 279
18.3 Writing Similarity Queries . . . 279
18.3.1 PostgreSQL . . . 279
18.3.2 Elasticsearch . . . 280
18.3.3 MongoDB Atlas Vector Search . . . 281
18.4 Retrieval Models . . . 281
18.4.1 Lexical Retrieval: TF-IDF and BM25 . . . 282
18.4.2 Dense Retrieval: Bi-Encoders . . . 282
18.4.3 Sparse Learned Retrieval . . . 282
18.4.4 Late Interaction: Multi-Vector Models . . . 283
18.4.5 Cross-Encoders and Re-ranking . . . 283
18.5 Hybrid Retrieval and Rank Fusion . . . 283
18.5.1 Reciprocal Rank Fusion . . . 284
18.5.2 Weighted Score Fusion . . . 284
18.6 Improving the Query . . . 285
18.7 Shaping the Result List . . . 286
18.7.1 Grouping and Diversity . . . 286
18.7.2 Paging . . . 286
18.7.3 Re-ranking and Context Assembly . . . 287
18.8 Evaluating Retrieval Models . . . 287
18.8.1 Pitfalls . . . 288
18.9 Operating Similarity Queries . . . 288
18.9.1 Latency Budget . . . 288
18.9.2 Caching and Batching . . . 288
18.9.3 Monitoring . . . 289
18.9.4 Access Control, Privacy, and Safety . . . 289
18.10Choosing a Retrieval Approach . . . 289

VI Modern Data Architectures 295

19 Data Lakes and Lakehouse Architectures 297
19.1 From Warehouse to Lake to Lakehouse . . . 297
19.1.1 The Data Warehouse . . . 298
19.1.2 The Data Lake . . . 298
19.1.3 The Lakehouse . . . 298
19.2 Object Storage as the Foundation . . . 298
19.3 File Formats: Row and Columnar . . . 300
19.3.1 Why Columnar Formats . . . 300
19.3.2 Anatomy of a Parquet File . . . 300
19.3.3 Choosing Formats . . . 301
19.4 Open Table Formats . . . 302

19.4.1 The Common Idea: Snapshots and Atomic Commits . . . 302
19.4.2 Apache Iceberg . . . 302
19.4.3 Delta Lake . . . 303
19.4.4 Apache Hudi . . . 303
19.4.5 Tables in SQL . . . 303
19.5 Architecture of a Lakehouse . . . 305
19.5.1 The Medallion Pattern . . . 305
19.5.2 Ingestion . . . 306
19.5.3 Catalogs . . . 306
19.5.4 Compute Engines . . . 306
19.6 Partitioning, Clustering, and File Layout . . . 307
19.6.1 Partitioning . . . 307
19.6.2 Clustering and Sorting . . . 307
19.6.3 File Size and the Small-Files Problem . . . 307
19.7 Governance, Security, and Privacy . . . 308
19.7.1 Deletion in a System That Keeps History . . . 308
19.8 Beyond Tables: Unstructured Data and Machine Learning . . . 309
19.9 Operating a Lakehouse . . . 309
19.9.1 Backup and Disaster Recovery . . . 310
19.10 Choosing an Approach . . . 310

20 Streaming Systems and Event-Driven Architectures 315
20.1 From Batches to Streams . . . 315
20.1.1 Why Events . . . 316
20.2 Events, Commands, and Messages . . . 316
20.2.1 Designing an Event . . . 317
20.2.2 Event Time and Processing Time . . . 317
20.3 The Distributed Log . . . 318
20.3.1 Topics, Partitions, and Offsets . . . 318
20.3.2 Ordering . . . 318
20.3.3 Consumer Groups and Parallelism . . . 318
20.3.4 Replication and Durability . . . 319
20.3.5 Retention and Compaction . . . 319
20.3.6 Logs and Queues . . . 319
20.4 Delivery Guarantees and Exactly-Once Effects . . . 319
20.4.1 Building Exactly-Once Effects . . . 320
20.5 Stream Processing . . . 321
20.5.1 Windows . . . 321
20.5.2 Event Time, Watermarks, and Late Data . . . 322
20.5.3 Joins and Enrichment . . . 323
20.5.4 State and Backpressure . . . 323
20.6 Streams and Tables: Change Data Capture . . . 323
20.6.1 The Dual-Write Problem and the Outbox . . . 324
20.7 Event-Driven Architecture Patterns . . . 324
20.7.1 Event Sourcing . . . 325

20.7.2 CQRS . . . 325
20.7.3 Sagas . . . 326
20.7.4 Retries, Dead-Letter Queues, and Poison Messages . . . 326
20.8 Schemas and Their Evolution . . . 327
20.9 Architectures for Combining Batch and Stream . . . 327
20.10 Operating Streaming Systems . . . 328
20.10.1 Capacity Planning . . . 328
20.10.2 Monitoring . . . 328
20.10.3 Reprocessing and Replay . . . 328
20.10.4 Security and Privacy . . . 329
20.10.5 Multi-Region Operation . . . 329
20.11 Choosing an Approach . . . 329

21 Real-Time Data Pipelines and Processing Frameworks 335
21.1 From Batch to Streams . . . 335
21.1.1 Why Pipelines Rather Than Direct Integration . . . 336
21.2 The Log as Backbone . . . 336
21.2.1 The Distributed Log . . . 336
21.2.2 Producers and Consumers . . . 337
21.3 Getting Data Into the Pipeline . . . 338
21.3.1 Change Data Capture . . . 338
21.3.2 Schemas and Data Contracts . . . 339
21.4 Delivery Guarantees and Exactly-Once Effects . . . 340
21.4.1 Achieving Exactly-Once . . . 340
21.4.2 Failures in Individual Records . . . 340
21.5 Stream Processing Concepts . . . 341
21.5.1 Event Time and Watermarks . . . 341
21.5.2 Windows . . . 341
21.5.3 State . . . 341
21.5.4 Joins and Stream–Table Duality . . . 341
21.6 Processing Frameworks . . . 342
21.6.1 Architectural Differences . . . 342
21.7 State, Checkpoints, and Fault Tolerance . . . 343
21.8 Sinks and Serving . . . 344
21.9 Pipeline Architectures . . . 344
21.9.1 Lambda and Kappa . . . 345
21.9.2 Reprocessing and Backfills . . . 345
21.10 Data Quality, Governance, and Privacy . . . 346
21.11 Operating Real-Time Pipelines . . . 347
21.11.1 What to Monitor . . . 347
21.11.2 Capacity Planning . . . 347
21.11.3 Common Pitfalls . . . 347
21.11.4 Testing . . . 348
21.12 Choosing an Approach . . . 348

22 Polyglot Persistence and System Design **353**
22.1 One Database or Many? . . . 353
22.1.1 Two Opposite Mistakes . . . 354
22.2 From Workloads to Stores . . . 354
22.3 Systems of Record and Derived Data . . . 355
22.3.1 A Data Catalog . . . 356
22.4 Integration Patterns . . . 357
22.4.1 CQRS and Read Models . . . 357
22.4.2 Event Sourcing . . . 358
22.5 Consistency Across Stores . . . 358
22.5.1 Handling the Hardest Case: Inventory and Orders . . . 359
22.6 A Worked Design: An Online Shop . . . 359
22.6.1 Requirements and Constraints . . . 359
22.6.2 Store Selection . . . 360
22.6.3 Key Flows . . . 360
22.6.4 Failure Analysis . . . 361
22.6.5 Other Examples in Brief . . . 361
22.7 Estimation and Non-Functional Requirements . . . 362
22.7.1 Back-of-the-Envelope Estimation . . . 362
22.7.2 Availability Arithmetic . . . 363
22.7.3 Service-Level Objectives . . . 364
22.8 Evolving and Migrating Architectures . . . 364
22.8.1 Add Stores When Measurements Demand It . . . 364
22.8.2 Migrating Safely . . . 365
22.9 The Cost of Many Stores . . . 365
22.9.1 Restoring Many Stores . . . 365
22.10 Governance Across Stores . . . 366
22.11 Reviewing a Data Architecture . . . 367
22.12 Choosing an Approach . . . 367

23 Designing Scalable Data Systems **373**
23.1 What Scalability Means . . . 373
23.1.1 Describing Load . . . 374
23.1.2 Describing Performance . . . 374
23.1.3 Scaling Up and Scaling Out . . . 374
23.2 Quantitative Models . . . 375
23.2.1 Little's Law . . . 375
23.2.2 Queueing: Why Latency Explodes Near Saturation . . . 375
23.2.3 Amdahl's Law and the Universal Scalability Law . . . 376
23.3 Choosing Scaling Strategies . . . 377
23.3.1 The Scale Cube . . . 377
23.3.2 The Scaling Ladder . . . 377
23.4 Caching . . . 377
23.4.1 Why Caching Is So Effective . . . 377
23.4.2 Caching Patterns . . . 378

23.4.3 Invalidation and Staleness . . . 378
23.4.4 Cache Stampedes . . . 379
23.4.5 Other Hazards . . . 379
23.5 Scaling Reads . . . 379
23.5.1 The N+1 Query Problem . . . 380
23.6 Scaling Writes . . . 380
23.6.1 Hot Rows and Sharded Counters . . . 381
23.7 Partitioning Decisions . . . 382
23.8 Overload Control . . . 382
23.9 Data Modeling for Scale . . . 383
23.10 Scaling Across Regions . . . 384
23.11 Elasticity, Capacity, and Cost . . . 385
23.11.1 Capacity Planning . . . 385
23.11.2 Autoscaling . . . 385
23.11.3 Cost as a Design Dimension . . . 385
23.12 Load Testing and Performance Engineering . . . 385
23.13 Case Study: A URL Shortener . . . 386
23.13.1 Requirements and Estimates . . . 386
23.13.2 Design . . . 387
23.14 Case Study: A Social Timeline . . . 387
23.14.1 Fan-Out on Read and on Write . . . 387
23.14.2 The Hybrid Design . . . 388
23.15 Common Scalability Anti-Patterns . . . 389
23.16 Choosing an Approach . . . 389

Glossary of Key Terms **395**

Part I

NoSQL Foundations

Chapter 1

Beyond Relational Databases: Why NoSQL?

Learning Objectives

After reading this chapter, you will be able to:

- Describe the data trends (volume, velocity, variety) that challenge traditional data management.
- Explain the specific limitations of relational databases, and where they remain the right choice.
- Trace the historical emergence of NoSQL and the meaning of the term.
- Distinguish the four main NoSQL data models and match each to typical use cases.
- Reason about trade-offs using the CAP theorem, PACELC, and the ACID/BASE spectrum.
- Apply a structured approach to selecting a database for a given workload.

1.1 Introduction to Modern Data Challenges

The rapid growth of digital systems has fundamentally transformed how data is generated, stored, and processed. Modern applications collect data from mobile devices, sensors, social media platforms, and enterprise systems, often around the clock. These data sets are not only large, but also highly variable in structure and speed of arrival. Traditional approaches to data management were not designed for this environment, and they are increasingly strained by it.

The challenges are commonly summarized by the so-called *three Vs* of big data:

Volume. The sheer quantity of data, often measured in terabytes or petabytes, exceeds what a single machine can store or process economically.

Velocity. Data arrives continuously and at high rates, for example clickstreams, financial ticks, or telemetry from IoT devices.

Variety. Data takes many forms: structured tables, semi-structured JSON and XML, logs, images, and free text.

Four trends make these properties concrete.

Flexible data formats. Relational databases were designed in an era when most business data was structured and predictable. They excel at storing data with well-defined schemas and relationships. Modern applications, however, routinely handle semi-structured and unstructured formats such as JSON documents, event logs, and text streams, whose shape may change from one record to the next.

Scalability demands. Many modern systems must serve millions of concurrent users with low latency. Scaling a database *vertically* ("scaling up") by buying a larger server is expensive and eventually hits physical and financial limits. Scaling *horizontally* ("scaling out") by adding commodity machines is more economical, but it requires the database itself to distribute data and queries across nodes, something traditional systems were not originally built to do.

Real-time processing. Applications such as financial trading, recommendation engines, and IoT monitoring need insights from continuously generated data within milliseconds or seconds. This places simultaneous pressure on throughput and latency.

Always-on availability. Global services are expected to remain available despite hardware failures, network problems, and maintenance. Downtime translates directly into lost revenue and user trust.

These pressures, felt most acutely by large web companies in the early 2000s, led to a new family of database systems collectively known as *NoSQL*. They were designed to offer greater flexibility, horizontal scalability, and performance for specific workloads, often by making deliberate trade-offs against the guarantees of relational systems.

1.2 Limitations of Relational Databases

Before examining the limitations of relational databases, it is important to acknowledge their strengths. Since Codd's relational model (1970), relational database management systems (RDBMS) have provided a declarative query language (SQL), a rigorous theoretical foundation, mature tooling, and strong transactional guarantees (ACID: atomicity, consistency, isolation, durability). For many applications, especially those with structured data and complex transactional requirements, they remain an excellent choice. The limitations below describe pressure points, not fundamental failures. Readers who need a refresher on relational concepts and SQL will find a step-by-step introduction in Databases and SQL: From 0 to 100, Siadati (2021).

1.2.1 Rigid Schemas

A relational database requires the structure of data to be declared before it is stored. This guarantees consistency and integrity, but makes change costly. Altering a table in a large production system may require lengthy migrations, coordinated application updates, and downtime or careful online-migration tooling. Such rigidity sits uneasily with agile development, where data requirements evolve quickly.

1.2.2 The Object–Relational Impedance Mismatch

Applications typically work with nested, in-memory objects, whereas relational databases store flat rows spread across multiple tables. Translating between the two, often through object–relational mapping (ORM) frameworks, adds code complexity and can hide inefficient queries. This disparity is known as the *impedance mismatch* (Kleppmann, 2017).

1.2.3 Join Cost at Scale

Joins let relational databases combine related data across tables, but they can become expensive as data volumes grow. Queries spanning many tables may dominate execution time, and the problem is amplified when the joined data resides on different nodes, because it must then be transferred across the network.

1.2.4 Difficult Horizontal Scaling

Replication allows a relational system to scale reads, but scaling *writes* typically requires *sharding*: splitting data across independent database instances. Although many RDBMSs can be sharded, sharding is rarely transparent. The application or middleware must often decide where data lives, cross-shard joins and transactions become complex or impossible, and rebalancing data is operationally demanding.

1.2.5 Write Throughput

Maintaining ACID guarantees involves locking, logging, and coordination. For workloads dominated by very high write rates, such as logging systems or sensor networks, this overhead can become a bottleneck.

1.2.6 Unstructured and Nested Data

Storing deeply nested or highly variable data in a relational model often requires either many normalized tables or serializing the data into opaque text columns. The former complicates queries; the latter prevents efficient querying and indexing of the content.

The Gap Is Narrowing

The boundary between relational and NoSQL systems is increasingly blurred. Several relational databases now provide native JSON types and indexing (for example, PostgreSQL's `JSONB`), while *NewSQL* systems such as Google Spanner and CockroachDB combine SQL and ACID transactions with horizontal scalability. Conversely, many NoSQL systems have added SQL-like query languages and multi-document transactions. The choice is therefore about fit for a workload, not about which category is "better".

1.3 The Emergence of NoSQL

1.3.1 Historical Background

The roots of NoSQL lie in the engineering needs of large internet companies. Google's Bigtable (Chang et al., 2006) and Amazon's Dynamo (DeCandia et al., 2007) were built to store and serve data at a scale and availability that existing databases could not provide at acceptable cost. Their published designs inspired open-source systems such as Apache HBase, Apache Cassandra, and Riak, and the wider ecosystem that followed, including MongoDB, Redis, and Neo4j. The label "NoSQL" itself was popularized around 2009 as a hashtag and meetup theme for this emerging group of non-relational, distributed data stores.

1.3.2 What Does "NoSQL" Mean?

The term originally suggested databases that do not use SQL. It is now commonly read as "Not Only SQL", reflecting that these systems complement rather than replace relational databases, and that several of them support SQL-like query languages. Importantly, NoSQL is an umbrella term for a diverse set of systems rather than a single technology.

1.3.3 Core Motivations

Flexibility. Many NoSQL systems support schema-less or schema-flexible models, so developers can store records without declaring a structure up front and evolve it over time. This suits iterative development, although it shifts responsibility for data validation from the database to the application.

Horizontal scalability. NoSQL databases are typically designed to distribute data across many commodity machines through partitioning (sharding) and replication. Scaling out is built into the architecture rather than added afterwards.

High availability and fault tolerance. By replicating data across nodes, and often across data centers, these systems can continue to operate when individual components fail.

Performance for specific access patterns. By relaxing constraints such as strict consistency or general-purpose joins, and by optimizing the storage layout for particular query patterns, NoSQL systems can achieve very high throughput and low latency.

1.3.4 Polyglot Persistence

The emergence of NoSQL marks a shift from a "one-size-fits-all" approach to a more specialized ecosystem. Rather than forcing every workload into a single database, modern architectures often adopt *polyglot persistence*: using different data stores for different parts of an application. For instance, an e-commerce platform might keep orders in a relational database, product catalogs in a document store, shopping sessions in a key-value store, and recommendations in a graph database. The benefit is a better fit for each workload; the cost is higher operational complexity and the challenge of keeping data consistent across systems.

1.4 Types of NoSQL Databases

NoSQL databases are commonly grouped into four main families, based on their data model. Each addresses specific use cases and carries its own advantages and limitations.

1.4.1 Document Databases

Document databases store data as self-describing documents, typically in JSON or a binary variant such as BSON. Documents can contain nested structures and arrays, and documents in the same collection need not share an identical structure. Related data that is accessed together can be embedded in a single document, which avoids many joins. Typical uses include content management systems, product catalogs, and user profiles. Examples include MongoDB and Couchbase.

Listing 1.1: A user profile stored as a single JSON document.

```
{
  "_id": "user_1001",
  "name": "Ava Chen",
  "email": "ava@example.com",
  "addresses": [
    { "type": "home", "city": "Melbourne", "country": "AU" },
    { "type": "work", "city": "Sydney", "country": "AU" }
  ],
  "preferences": { "newsletter": true, "theme": "dark" }
}
```

In a relational design, the same information would typically be spread across at least three tables (users, addresses, preferences) and reassembled with joins, as sketched below.

Listing 1.2: Reassembling the same profile in a relational schema.

```
SELECT u.name, a.type, a.city, p.theme
FROM users u
```

```
JOIN addresses a ON a.user_id = u.id
JOIN preferences p ON p.user_id = u.id
WHERE u.id = 1001;
```

1.4.2 Key-Value Stores

Key-value stores are the simplest NoSQL model. Data is stored as a collection of key–value pairs, where the value is treated as an opaque blob and is retrieved by its key. Lookups are extremely fast and the model scales easily, but queries beyond key access are limited. Common uses include caching, session management, shopping carts, and real-time leaderboards. Examples include Redis, Amazon DynamoDB, and Riak.

1.4.3 Wide-Column (Column-Family) Databases

Wide-column stores organize data into tables whose rows are identified by a key, but whose columns are grouped into column families and may vary from row to row. Data is partitioned by key across many nodes, and the storage engines (often based on log-structured merge trees) are optimized for very high write throughput. They suit time-series data, event logging, and large-scale operational analytics. Examples include Apache Cassandra, Apache HBase, and Google Bigtable.

Do Not Confuse: Wide-Column vs. Columnar

Wide-column stores (such as Cassandra) are *not* the same as columnar analytical databases (such as ClickHouse or Amazon Redshift). The latter store each column contiguously on disk to accelerate analytical scans and aggregations, and usually remain relational in their data model.

1.4.4 Graph Databases

Graph databases treat relationships as first-class citizens. Data is modeled as nodes (entities) and edges (relationships), both of which can carry properties. Traversing connections is efficient because relationships are stored directly rather than computed by joins. This makes graph databases well suited to social networks, recommendation systems, fraud detection, and knowledge graphs. Examples include Neo4j and Amazon Neptune.

1.4.5 Comparison

Table 1.1 summarizes the trade-offs. In practice, many products are *multi-model* and support more than one of these data models, so the categories should be treated as a guide rather than a strict taxonomy.

Table 1.1: Comparison of the main NoSQL data models.

Model	Strengths	Limitations	Examples
Document	Flexible schema; natural fit for application objects; rich queries on fields	Cross-document relationships and joins are limited or costly	MongoDB, Couchbase
Key-value	Very fast, simple, highly scalable	Minimal query capability; no relationships	Redis, DynamoDB
Wide-column	High write throughput; scales across many nodes	Query patterns must be designed up front; limited ad hoc queries	Cassandra, HBase
Graph	Efficient relationship traversal; expressive queries over connections	Harder to partition across machines; narrower use cases	Neo4j, Neptune

1.5 Trade-offs and Design Considerations

Choosing a distributed database means understanding which properties you are trading for others. This section introduces the theoretical frameworks most commonly used to reason about those choices.

1.5.1 The CAP Theorem

The CAP theorem, conjectured by Brewer (2000) and later formally proved by Gilbert and Lynch (2002), concerns three properties of a distributed data store:

Consistency (C). Every read receives the most recent write (or an error). In CAP, this refers to *linearizability*, which differs from the "consistency" in ACID.

Availability (A). Every request to a non-failing node receives a (non-error) response.

Partition tolerance (P). The system continues to operate despite messages being lost or delayed between nodes.

A common but misleading summary is "pick any two of three". Network partitions are unavoidable in real distributed systems, so partition tolerance is not optional. The meaningful statement is therefore:

> *When a network partition occurs, a system must choose between consistency and availability.*

A **CP** system preserves consistency by refusing some requests during a partition, and an **AP** system remains available but may return stale or divergent data that is reconciled later. When there is no partition, a system can in principle provide both.

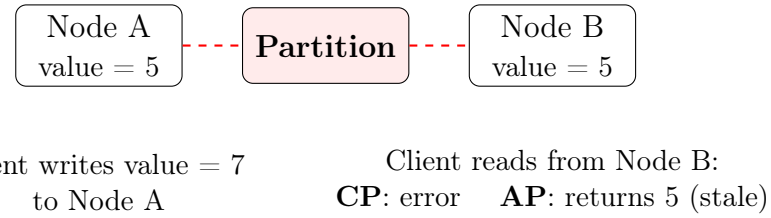


Figure 1.1: During a partition, Node B cannot learn about the write to Node A. A CP system rejects the read; an AP system answers with possibly stale data.

1.5.2 Beyond CAP: PACELC

CAP only describes behavior during partitions, yet partitions are rare. Abadi's (2012) PACELC formulation extends the model: if there is a **P**artition, choose between **A**vailability and **C**onsistency; **E**lse (during normal operation), choose between **L**atency and **C**onsistency. This captures a trade-off present at all times: stronger consistency requires more coordination between replicas, which increases latency.

1.5.3 Consistency Models and Tunable Consistency

Consistency is not binary. Systems offer a spectrum, from *strong consistency* (all clients observe writes in the same order immediately) to *eventual consistency* (replicas converge if no new updates arrive). Between these lie models such as read-your-writes and causal consistency.

Many NoSQL databases let developers tune consistency per operation. In quorum-based replication with N replicas, a write is acknowledged by W replicas and a read consults R replicas. If

$$R + W > N,$$

then every read quorum overlaps every write quorum, so a read observes the latest acknowledged write (assuming no concurrent writes or failures that affect quorums). Lowering R or W improves latency and availability at the cost of potentially stale reads.

1.5.4 ACID versus BASE

Relational systems emphasize ACID transactions. Many distributed NoSQL systems instead follow the looser **BASE** philosophy: *Basically Available*, *Soft state*, and *Eventually consistent*. BASE is best viewed as a design stance rather than a precise guarantee: it favors availability and scalability, and requires applications to tolerate temporary inconsistency. Note that the boundary is not rigid; several NoSQL systems now offer ACID transactions within limited scopes.

Table 1.2: Typical default orientation of selected systems (configurable in many cases).

System	Typical orientation	Notes
Relational (single node)	Strong consistency	ACID transactions; CAP is not a concern
MongoDB	CP-leaning	Single primary per shard; tunable read/write concerns
HBase	CP-leaning	Strong consistency per row
Cassandra	AP-leaning	Tunable consistency per query
DynamoDB	AP-leaning by default	Optional strongly consistent reads

1.5.5 Choosing a Database: A Practical Approach

No database is best in all circumstances. When evaluating options, consider the following questions:

1. **Data model.** Is the data tabular, hierarchical, relationship-heavy, or simple key-based?
2. **Access patterns.** Which queries must be fast? Are they known in advance, or ad hoc?
3. **Consistency requirements.** Can the application tolerate stale reads? What is the cost of an inconsistency (for example, in payments versus social feeds)?
4. **Scale and growth.** What are the expected data volume and read/write rates, and will they require horizontal scaling?
5. **Availability and latency targets.** What are the uptime, geographic distribution, and response-time goals?
6. **Operational factors.** What expertise, tooling, managed-service options, and cost constraints apply?

A useful rule of thumb is to start from the access patterns and consistency requirements rather than from a particular technology. Where requirements are modest and the data is structured, a well-tuned relational database is often sufficient; NoSQL becomes compelling when scale, flexibility, or availability needs genuinely outgrow it.

Summary

This chapter examined why NoSQL systems emerged. Growing data volume, velocity, and variety, together with demands for horizontal scalability, real-time processing, and high availability, exposed pressure points in traditional relational databases: rigid schemas, the impedance mismatch, join costs, difficult sharding, and write bottlenecks. Relational databases nonetheless remain powerful, and the line between the two worlds continues to blur.

We surveyed the four principal NoSQL data models (document, key-value, wide-column, and graph), noting their strengths, limitations, and typical use cases. We also introduced the key theoretical tools for reasoning about distributed trade-offs: the CAP theorem (correctly understood as a choice between consistency and availability during partitions), PACELC,

tunable quorum consistency, and the ACID/BASE spectrum. Finally, we outlined a practical approach to selecting a database. These foundations prepare the ground for the deeper treatment of individual systems and distributed-systems techniques in later chapters.

Review Questions

1. What are the main limitations of relational databases in modern applications, and in which situations do they remain the better choice?

2. Why is horizontal scaling important in modern data systems, and why is it harder for relational databases than for many NoSQL systems?

3. What are the key characteristics of NoSQL databases?

4. Compare the four main types of NoSQL databases and give one suitable use case for each.

5. Why is the statement "choose any two of C, A, and P" considered misleading? What choice does the CAP theorem actually describe?

6. What does PACELC add to the CAP theorem?

7. Explain the difference between ACID and BASE. Why might an online banking system and a social media feed make different choices?

8. What is polyglot persistence, and what are its benefits and costs?

Exercises

1. **Modeling.** Model a blog (authors, posts, comments, tags) first as a relational schema and then as JSON documents. Which queries become simpler or harder in each model?

2. **Quorums.** A system has $N = 5$ replicas. For each of $(R, W) = (1, 5)$, $(3, 3)$, and $(1, 1)$, state whether reads are guaranteed to see the latest write and discuss the latency and availability implications.

3. **Database selection.** For each scenario, propose a database type and justify your choice using the criteria in Section 1.5: (a) a web session store, (b) a payment ledger, (c) a fraud-detection network, (d) IoT sensor ingestion at 500,000 events per second.

4. **Research.** Choose one NewSQL system (for example, Spanner or CockroachDB) and explain how it attempts to combine SQL semantics with horizontal scalability. What trade-offs does it make?

References

- Abadi, D. (2012). Consistency Tradeoffs in Modern Distributed Database System Design: CAP is Only Part of the Story. *IEEE Computer*, 45(2), 37–42.
- Brewer, E. (2000). Towards Robust Distributed Systems. Keynote at the ACM Symposium on Principles of Distributed Computing (PODC).
- Chang, F., Dean, J., Ghemawat, S., et al. (2006). Bigtable: A Distributed Storage System for Structured Data. In *Proceedings of OSDI '06*.
- Codd, E. F. (1970). A Relational Model of Data for Large Shared Data Banks. *Communications of the ACM*, 13(6), 377–387.
- DeCandia, G., Hastorun, D., Jampani, M., et al. (2007). Dynamo: Amazon's Highly Available Key-value Store. In *Proceedings of SOSP '07*.
- Gilbert, S., & Lynch, N. (2002). Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services. *ACM SIGACT News*, 33(2), 51–59.
- Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.
- Sadalage, P. J., & Fowler, M. (2012). *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence*. Addison-Wesley.
- Siadati, S. (2021). *Databases and SQL: From 0 to 100*. Zenodo. https://doi.org/10.5281/zenodo.19880169
- Stonebraker, M. (2010). SQL Databases v. NoSQL Databases. *Communications of the ACM*, 53(4), 10–11.

Chapter 2

Document Databases and JSON Data Models

Learning Objectives

After reading this chapter, you will be able to:

- Explain the document model and how it relates to relational concepts.
- Describe JSON and BSON, and the role of schema flexibility and schema validation.
- Choose between embedding and referencing for one-to-one, one-to-many, and many-to-many relationships.
- Write basic queries and aggregation pipelines, and design indexes that support them.
- Explain how replication, sharding, and consistency settings work in document databases.
- Judge when a document database is, and is not, a good fit.

2.1 Introduction to Document Databases

Document databases are among the most widely adopted categories of NoSQL systems. As introduced in Chapter 1, they store data as self-describing *documents*, usually encoded as JSON or a binary variant of it. Each document holds the fields and values of an entity, including nested objects and arrays, so a complex entity such as a customer order can often be stored and retrieved as a single record.

Three ideas define the model:

- **Documents as the unit of storage.** A document groups together data that is typically read and written together.

- **Flexible structure.** Documents in the same collection may have different fields, and the structure can change without a database-wide migration.

- **Alignment with application objects.** In most programming languages, data is already represented as objects, dictionaries, or maps. Storing documents of the same shape reduces the object–relational impedance mismatch discussed in Chapter 1(kleppmann, 2017).

2.1.1 Relational and Document Terminology

Table 2.1 maps familiar relational concepts to their document-database counterparts. The correspondence is approximate: a document is not simply a row, because it may contain nested data that a relational design would spread across several tables.

Table 2.1: Approximate correspondence between relational and document concepts.

Relational	**Document (MongoDB)**	**Key difference**
Table	Collection	Collections do not enforce a fixed set of columns by default
Row	Document	Documents can hold nested objects and arrays
Column	Field	Fields can differ between documents
Primary key	`_id` field	Every document has a unique `_id`
Join	Embedding, or `$lookup`	Related data is often stored inside the document
Schema migration	Schema evolution in the application, optional validation	Structure changes are gradual rather than all-at-once

2.1.2 Running Example and Scope

Document databases include MongoDB, Couchbase, Apache CouchDB, Amazon DocumentDB, Azure Cosmos DB, and Google Cloud Firestore. Their query languages and guarantees differ, but the underlying concepts are shared. This chapter uses MongoDB(MongoDB Documentation) for concrete examples because it is the most widely used document database; the syntax shown follows the MongoDB shell (`mongosh`). When reading the examples, focus on the ideas, since the exact syntax varies between products and versions.

2.2 JSON Data Model and Structure

2.2.1 JSON Basics

JSON (JavaScript Object Notation) is a lightweight, text-based, human-readable data format standardized as RFC 8259 (Bray, 2017). It has only six kinds of values:

- **object**: an unordered set of name–value pairs, written `{ "name": value }`;
- **array**: an ordered list of values, written `[ v1, v2 ]`;
- **string**, **number**, **boolean** (`true`/`false`), and **null**.

Objects and arrays can be nested to any depth, which gives JSON its expressive power. Listing 2.1 shows a customer order with an embedded customer summary and an array of line items.

Listing 2.1: An order stored as a single JSON document.

```
{
  "_id": "order_5001",
  "status": "shipped",
  "orderDate": "2026-03-14T09:30:00Z",
  "customer": { "id": "cust_42", "name": "Ava Chen" },
  "items": [
    { "sku": "A1", "name": "Notebook", "qty": 2, "price": 4.50 },
    { "sku": "B7", "name": "Pen set", "qty": 1, "price": 12.00 }
  ],
  "tags": ["gift", "priority"]
}
```

Because JSON is supported natively or through mature libraries in nearly every programming language, documents can be passed directly between application code, web APIs, and the database with little translation.

2.2.2 From JSON to BSON

JSON has limitations as a storage format. It is text, so it is relatively verbose and slow to parse; it has a single `number` type with no distinction between integers and decimals; and it has no native type for dates or binary data. MongoDB therefore stores documents in *BSON* (Binary JSON; BSON Specification), a binary encoding that adds types and supports efficient traversal. Table 2.2 summarizes the differences.

For monetary values, prefer `Decimal128` (or integer minor units such as cents) over floating-point numbers to avoid rounding errors. In the shell, typed values are written explicitly:

Listing 2.2: Typed values in the MongoDB shell.

```
db.orders.insertOne({
  _id: "order_5002",
  orderDate: ISODate("2026-03-15T11:00:00Z"), // native date
```

Table 2.2: JSON compared with BSON as used by MongoDB.

Aspect	JSON	BSON
Encoding	Text	Binary
Number types	One generic number	32-bit and 64-bit integers, double, and `Decimal128`
Dates	Not native (stored as strings)	Native date type
Binary data	Not native (e.g., Base64 strings)	Native binary type
Extra types	—	`ObjectId`, timestamps, and others
Readability	Human-readable	Machine-oriented (shown as JSON by tools)

```
  total: NumberDecimal("16.50"), // exact decimal
  customerRef: ObjectId() // auto-generated id
})
```

2.2.3 Schema Flexibility: Schema-on-Read and Schema-on-Write

Document databases are often called "schemaless", but this is slightly misleading. The data still has a structure; the difference is *where* the structure is enforced. Relational databases use *schema-on-write*: the database checks every write against a declared schema. Document databases typically use *schema-on-read*: the database accepts documents of any shape, and the application interprets them when reading (kleppmann,2017). A more accurate description is therefore *schema-flexible*.

This flexibility has clear benefits. New fields can be added in the application without a migration, and heterogeneous records, such as product catalogs where each category has different attributes, are stored naturally. It also has a cost: without discipline, collections accumulate inconsistent documents (for example, `"phone"` in one document and `"phoneNumber"` in another), which complicates queries and maintenance.

Many document databases therefore offer optional *schema validation*, which lets teams enforce rules where they matter while keeping flexibility elsewhere. Listing 2.3 requires every user to have a string `name` and an `email` that looks like an address.

Listing 2.3: Optional schema validation with a JSON Schema validator.

```
db.createCollection("users", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["name", "email"],
      properties: {
        name: { bsonType: "string" },
```

```
      email: { bsonType: "string", pattern: "^.+@.+$" },
      age: { bsonType: "int", minimum: 0 }
    }
  }
 }
})
```

Best Practice

Treat flexibility as a tool, not an excuse for having no design. Agree on the expected fields, types, and naming conventions for each collection, validate the critical ones, and include a `schemaVersion` field in long-lived documents so that older and newer shapes can be told apart during gradual migrations.

2.3 Data Modeling in Document Databases

Relational design starts from the data: entities are normalized into tables, and queries are assembled later with joins. Document modeling starts from the *workload*: how will the application read and write this data? The goal is to shape documents so that the most common operations touch as few documents as possible. This usually means *denormalization*, storing related data together even if some of it is duplicated (Sadalage & Fowler, 2012).

2.3.1 Embedding

Embedding places related data inside the parent document, as the `customer` summary and `items` array do in Listing 2.1. Its main advantages are:

- reads are fast, because one query returns everything;
- writes to a single document are atomic, so related data is updated consistently;
- the document matches the structure that the application uses.

2.3.2 Referencing

Referencing stores the identifier of a related document, much like a foreign key, and the application (or a `$lookup`) retrieves the related document when needed. It avoids duplication and suits data that is large, shared, or updated independently, but it requires additional queries to assemble the full picture.

Listing 2.4: The same order using references instead of embedded items.

```
{
  "_id": "order_5001",
  "status": "shipped",
  "customerId": "cust_42",
  "itemIds": ["A1", "B7"]
}
```

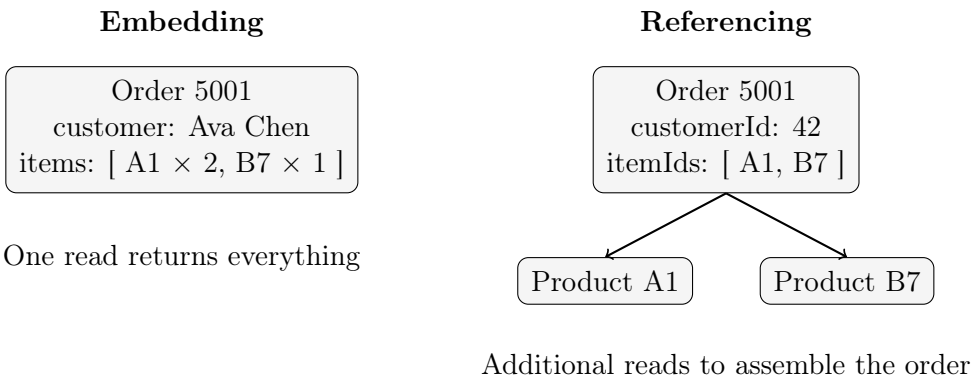


Figure 2.1: Embedding stores related data inside the parent document; referencing stores identifiers that must be resolved with further reads.

2.3.3 Choosing Between Embedding and Referencing

The decision depends on the *relationship*, the *access pattern*, and how the data *changes*. Table 2.3 gives widely used rules of thumb.

Table 2.3: Rules of thumb for embedding versus referencing.

Situation	**Prefer**	**Reason**
One-to-one (user and profile settings)	Embed	Always read together
One-to-few (order and its line items)	Embed	Bounded size; read together
One-to-many, large or growing (a user and years of log events)	Reference (child holds parent id)	Avoids unbounded documents
Many-to-many (students and courses)	Reference, possibly in both directions	Embedding duplicates data heavily
Child data is read independently of the parent	Reference	Avoids loading unneeded data
Child data changes frequently and is widely shared	Reference	Avoids updating many copies

A frequently useful compromise is the *extended reference* pattern: reference the related document, but also copy the few fields that are read most often. In Listing 2.1, the order stores the customer's `id` and `name`, so showing an order history does not need a second query. The price is that copied fields can become stale and must be updated, or tolerated as a historical snapshot, which is often exactly what is wanted for an order.

2.3.4 Common Pitfalls

- **Unbounded arrays.** An array that grows without limit (comments on a viral post, events from a sensor) slows updates and may eventually reach the maximum document size, which is 16 MB in MongoDB. Move such data to its own collection and reference the parent.
- **Over-embedding.** Embedding data that is also needed on its own, or that changes independently, causes duplication and costly multi-document updates.
- **Relational thinking.** Reproducing a fully normalized schema with references everywhere discards the main benefit of the model and pushes joins into application code.
- **Modeling without access patterns.** A model designed without knowing the queries is likely to perform poorly or need redesign.

2.3.5 A Workload-Driven Design Process

A practical sequence is:

1. List the entities and the relationships between them.
2. List the most important reads and writes, with expected frequency and latency goals.
3. Decide for each relationship whether to embed or reference, using Table 2.3.
4. Define indexes that support the main queries (Section 2.4).
5. Test with realistic data volumes and revise.

2.4 Querying and Indexing Documents

2.4.1 Basic Operations

Document databases support the four basic operations (create, read, update, delete) on JSON-like structures. Queries can filter on top-level fields, nested fields (using dot notation), and array contents. Listing 2.5 illustrates typical operations on the orders collection.

Listing 2.5: Basic create, read, update, and delete operations in the MongoDB shell.

```
// Create
db.orders.insertOne({ _id: "order_5003", status: "new",
                      customer: { id: "cust_42" }, items: [] })

// Read: shipped orders for one customer (dot notation for nested field)
db.orders.find({ status: "shipped", "customer.id": "cust_42" })

// Read: orders containing item A1 with quantity of at least 2
db.orders.find({ items: { $elemMatch: { sku: "A1", qty: { $gte: 2 } } } })

// Update: change a field and add an element to an array
```

```
db.orders.updateOne({ _id: "order_5003" },
  { $set: { status: "paid" },
    $push: { items: { sku: "A1", qty: 1, price: 4.50 } } })

// Delete
db.orders.deleteOne({ _id: "order_5003" })
```

Table 2.4 lists some common query operators.

Table 2.4: Selected MongoDB query and update operators.

Operator	Meaning	Kind
`$eq`, `$ne`	Equal, not equal	Comparison
`$gt`, `$gte`, `$lt`, `$lte`	Greater/less than (or equal)	Comparison
`$in`	Value is in a given list	Comparison
`$and`, `$or`	Combine conditions	Logical
`$elemMatch`	At least one array element satisfies all conditions	Array
`$set`, `$inc`	Set a field; increment a number	Update
`$push`	Append to an array	Update

2.4.2 Indexing

Without an index, the database must scan every document in a collection to answer a query (a *collection scan*), which becomes slow as data grows. An *index* is an auxiliary data structure, typically a B-tree, that lets the database locate matching documents directly. Document databases support several kinds of index, summarized in Table 2.5.

Table 2.5: Common index types in document databases.

Index type	Description	Typical use
Single-field	Index on one field	Lookups by a single attribute
Compound	Index on several fields in a defined order	Queries that filter and sort on multiple fields
Multikey	Indexes each element of an array	Querying array contents
Text	Supports word-based text search	Searching descriptions and articles
Geospatial	Supports location queries	"Find nearby" features
TTL	Automatically expires documents after a time	Sessions, temporary data

For compound indexes, field order matters. A widely used guideline is *Equality, Sort, Range* (ESR): place fields used in equality conditions first, then fields used for sorting, then fields used in range conditions. For the query "shipped orders, newest first", the following index is suitable, and `explain` shows whether the database actually used it.

Listing 2.6: A compound index and checking its use with `explain`.

```
db.orders.createIndex({ status: 1, orderDate: -1 })

db.orders.find({ status: "shipped" })
         .sort({ orderDate: -1 })
         .explain("executionStats")
// Look for an index scan (IXSCAN) rather than a collection scan (COLLSCAN)
```

Indexes are not free. Each index consumes storage and memory and must be updated on every write, so adding indexes speeds up reads but slows writes. Index the fields that the important queries use, and remove indexes that are never used.

2.4.3 Aggregation Pipelines

For analysis beyond simple retrieval, document databases offer an *aggregation pipeline*: a sequence of stages, each of which transforms the stream of documents produced by the previous one. Typical stages filter (`$match`), reshape (`$project`), flatten arrays (`$unwind`), group and summarize (`$group`), sort, limit, and join with another collection (`$lookup`). Listing 2.7 computes the five best-selling products by revenue.

Listing 2.7: An aggregation pipeline computing top products by revenue.

```
db.orders.aggregate([
  { $match: { status: "shipped" } },
  { $unwind: "$items" },
  { $group: { _id: "$items.sku",
              unitsSold: { $sum: "$items.qty" },
              revenue: { $sum: { $multiply: ["$items.qty", "$items.price"] } } } },
  { $sort: { revenue: -1 } },
  { $limit: 5 }
])
```

Because the first stage filters early and can use an index, placing `$match` (and `$sort` where possible) at the start of a pipeline keeps the amount of data flowing through later stages small. Compare this with the SQL equivalent, which would require a join between orders and line-item tables followed by `GROUP BY`; the document form works on data that is already stored together.

2.4.4 Full-Text Search

Many document databases support text search, either through text indexes or through dedicated search engines integrated with the database. These support tokenization, stemming, ranking, and in some products fuzzy matching. For demanding search requirements, such as

relevance tuning at scale, a specialized search engine is often used alongside the database, an example of the polyglot persistence introduced in Chapter 1.

2.4.5 Query Performance in Practice

Query performance is a continuing responsibility. Common causes of slow queries are missing or mismatched indexes, returning more fields than needed, unbounded result sets, and queries that scan large arrays. Use the database's profiling and `explain` tools, monitor slow queries, and revisit indexes as the workload changes.

2.5 Scalability, Consistency, and Use Cases

2.5.1 Replication

Document databases typically replicate data to provide availability and fault tolerance. In a MongoDB *replica set*, one node is the *primary* and accepts writes, while *secondaries* copy its changes. If the primary fails, the remaining members hold an election and one secondary is promoted, so the system recovers automatically without manual intervention. Secondaries can also serve reads, which can offload the primary, at the cost of possibly reading slightly stale data.

2.5.2 Sharding

To scale beyond a single machine, data is *sharded*: split across multiple replica sets according to a *shard key*. The choice of shard key is among the most important design decisions in a sharded cluster:

- A key with *high cardinality* spreads data across many shards.
- A key that changes *monotonically* (such as a timestamp or an auto-incrementing id) can send all new writes to one shard, creating a hotspot; hashed keys avoid this.
- The key should appear in the most common queries so that the database can route them to a single shard rather than querying all shards.

Range-based sharding keeps nearby keys together, which suits range queries; hash-based sharding distributes keys evenly, which suits uniform workloads.

2.5.3 Consistency and Transactions

The CAP and PACELC trade-offs from Section 1.5 appear here as configurable settings. In MongoDB, for example:

- **Write concern** controls how many replicas must acknowledge a write before it is reported successful (for example, `w: "majority"` for durability across a majority of nodes).

- **Read concern** and **read preference** control which data a read may return and which node serves it.

Stronger settings improve consistency and durability but increase latency and reduce availability during failures; weaker settings do the opposite. This is the quorum reasoning of Section 1.5 in practice.

Operations on a single document are atomic, which is one reason embedding is attractive: related data that lives in one document is updated together. Modern document databases such as MongoDB also support multi-document ACID transactions, but these carry a performance cost and are best used sparingly. If a workload requires them everywhere, the data model probably should be reconsidered, or a relational database may be a better fit.

2.5.4 Use Cases

Document databases fit well when:

- data is naturally hierarchical or varies between records, as in **content management systems** and **product catalogs**;
- the application needs a flexible **user profile** or **personalization** store that evolves quickly;
- **mobile and web backends** exchange JSON with the database;
- data structures evolve rapidly, as in **startups and prototypes**, where avoiding migrations speeds iteration;
- data is collected from varied sources, as in **event and catalog aggregation**.

2.5.5 Limitations

Document databases are not a universal solution. They are usually a weaker fit when:

- the data is highly relational, with many-to-many relationships that are queried in many different ways;
- the workload depends on complex multi-entity transactions with strict integrity guarantees, such as core financial ledgers;
- queries are unpredictable and join-heavy, as in ad hoc business analytics, where relational or columnar systems excel;
- duplicated data would be costly to keep consistent.

Matching the database to the workload, rather than following fashion, remains the central skill, a theme that continues in the chapters ahead.

Summary

Document databases store data as flexible, self-describing JSON or BSON documents that align closely with application objects. Their schema flexibility speeds development, but it moves responsibility for data quality to the team, which can use optional schema validation

where it matters. Good document modeling is workload-driven: embed data that is read and updated together and bounded in size, reference data that is large, shared, or independently changing, and consider the extended reference pattern as a compromise. Rich querying, well-designed indexes, and aggregation pipelines provide powerful retrieval and analysis, while replication and sharding provide availability and horizontal scale, with consistency controlled through configurable settings. Document databases excel for hierarchical, evolving data and are less suited to highly relational workloads with heavy multi-entity transactions.

Review Questions

1. What are the main differences between document databases and relational databases? Use the terminology in Table 2.1.

2. Why is "schema-flexible" a more accurate description than "schemaless"? What are schema-on-read and schema-on-write?

3. Name three ways in which BSON extends JSON, and explain why they matter.

4. What are the advantages and disadvantages of embedding versus referencing? Give an example where each is the better choice.

5. What is the extended reference pattern, and what trade-off does it make?

6. Why is an unbounded array in a document considered a design problem?

7. Why is indexing important in document databases, and what are the costs of adding indexes? What does the ESR guideline say?

8. Explain how write concern relates to the consistency and availability trade-offs from Chapter 1.

9. In what scenarios are document databases most effective, and when should you prefer another model?

Exercises

1. **Modeling.** Design a document model for a library system with books, authors, members, and loans. For each relationship, state whether you embed or reference, and justify your choice with the expected queries.

2. **Refactoring.** A blog stores each post with all of its comments embedded in an array. Explain why this may fail as the blog grows, and propose a better design.

3. **Indexing.** Given the query `db.orders.find({ status: "shipped", total: { $gt: 100 } }).sort({ orderDate: -1 })`, propose a compound index using the ESR guideline and explain your field order.

4. **Aggregation.** Write an aggregation pipeline that returns the total revenue per customer for shipped orders, sorted from highest to lowest.

5. **Shard key.** For an application that stores sensor readings with fields `sensorId`, `timestamp`, and `value`, evaluate `timestamp`, `sensorId`, and a hashed `sensorId` as shard keys.

References

- Bray, T. (Ed.). (2017). The JavaScript Object Notation (JSON) Data Interchange Format. RFC 8259, Internet Engineering Task Force.
- BSON Specification. https://bsonspec.org
- Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.
- MongoDB, Inc. MongoDB Documentation. https://www.mongodb.com/docs
- Sadalage, P. J., & Fowler, M. (2012). *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence*. Addison-Wesley.

Chapter 3

Key-Value Stores and In-Memory Databases

Learning Objectives

After reading this chapter, you will be able to:

- Describe the key-value data model, its basic operations, and why it is fast and scalable.
- Explain how key-value systems partition and replicate data, including consistent hashing.
- Compare in-memory and disk-based designs, and explain persistence, eviction, and durability trade-offs.
- Apply common patterns such as cache-aside, session storage, counters, and leaderboards.
- Design keys and values that avoid common pitfalls such as hot keys and oversized values.
- Decide when a key-value store is, and is not, the right choice.

3.1 Introduction to Key-Value Stores

Key-value stores are the simplest form of NoSQL database, yet they are among the most powerful when used for the right job. A key-value store holds a collection of *key–value pairs*: each *key* is a unique identifier, and the *value* is the data associated with it. The model is conceptually the same as a dictionary, map, or hash table in a programming language, except that the data is shared across applications and (usually) persists beyond a single process.

3.1.1 The Data Model

A key-value store exposes a very small interface. Table 3.1 shows the core operations.

Two properties define the model:

- **Access by key.** The primary way to find data is by its exact key. Most key-value

Table 3.1: Core operations of a key-value store.

Operation	Meaning
`PUT(key, value)`	Store the value under the key, replacing any existing value
`GET(key)`	Return the value stored under the key, or "not found"
`DELETE(key)`	Remove the key and its value

stores cannot efficiently answer questions such as "find all values where the customer is Ava" without additional structures that the application maintains.

- **Opaque or loosely structured values.** In the purest form, the database treats the value as an uninterpreted blob of bytes: a string, a number, a serialized JSON document, an image. Some systems go further and understand richer value types, as discussed in Section 3.3.

3.1.2 Why Simplicity Matters

The limited interface is a deliberate design choice rather than a weakness. Because a lookup needs only the key, the database can locate data in (amortized) constant time, spread keys evenly across machines, and avoid the planning, joins, and locking that complex queries require. This is why key-value stores are often used as a building block for scalable systems: the same simplicity that limits what they can express makes them easy to distribute and very fast.

3.1.3 Representative Systems

Key-value stores range from caches to durable, globally distributed databases. Table 3.2 lists some well-known examples.

Note that these systems occupy different points in the design space. Memcached is a cache that may lose data at any time; etcd prioritizes consistency over raw speed; DynamoDB is a durable system of record. The label "key-value" tells you the data model, not the guarantees, so always examine the durability and consistency properties of a specific product.

3.2 Architecture of Key-Value Systems

3.2.1 Storage Structures

Inside a single node, the data structure used to map keys to values determines performance characteristics:

- **Hash tables** map each key to a memory location through a hash function, giving constant-time lookups on average. They are the foundation of in-memory stores such as Redis and Memcached.

Table 3.2: Representative key-value systems.

System	Primary storage	Notable characteristics
Redis	Memory, with optional persistence	Rich value types (lists, sets, sorted sets, and others); replication and clustering
Memcached	Memory only	Simple, multithreaded cache; no built-in persistence or replication
Amazon DynamoDB	Managed, disk-based (SSD)	Partition key and optional sort key; automatic partitioning and replication; also supports document values
etcd	Disk, with strong consistency	Consensus-based (Raft); used for configuration and coordination
RocksDB	Embedded, disk-based	A library rather than a server; LSM-tree engine used inside many other databases

- **Log-structured merge trees (LSM trees)** (O'Neil et al., 1996) buffer writes in memory, then flush them as sorted, immutable files that are merged in the background. This turns random writes into sequential ones, giving very high write throughput on disk. LSM-based engines such as RocksDB, and designs inspired by Bigtable (Chang et al., 2006), are common.

- **B-trees** keep keys in sorted order on disk and are efficient for both point lookups and range scans, at the cost of more random I/O on writes.

Hash-based structures excel at exact lookups but cannot scan keys in order; sorted structures such as LSM trees and B-trees also support range queries over keys. Kleppmann (2017) gives a detailed comparison of the two sorted designs.

3.2.2 Partitioning (Sharding)

To store more data than one machine can hold, a distributed key-value store *partitions* the key space across nodes, so that each node is responsible for a subset of the keys. Two simple strategies exist:

- **Range partitioning** assigns contiguous key ranges to nodes. It supports range scans but can create hotspots if some ranges are accessed much more than others.

- **Hash partitioning** applies a hash function to the key and assigns the result to a node. It spreads keys evenly but gives up efficient range queries across partitions.

A naive hash scheme, such as node = hash(key) mod N for N nodes, has a serious flaw: when a node is added or removed, N changes and almost every key maps to a different node, forcing a massive data reshuffle.

3.2.3 Consistent Hashing

Consistent hashing (Karger et al., 1997), used in Dynamo-style systems (DeCandia et al., 2007) and many caches, avoids this problem. Both nodes and keys are hashed onto the same circular space, called a *hash ring*. Each key is stored on the first node encountered when moving clockwise around the ring from the key's position (Figure 3.1).

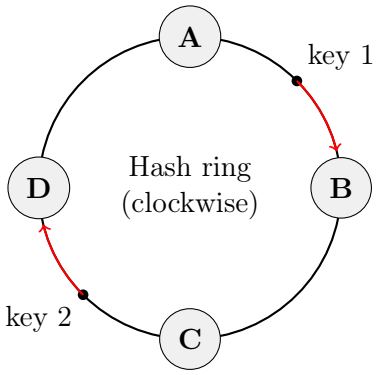


Figure 3.1: Consistent hashing. Key 1 is stored on node B and key 2 on node D: each key belongs to the first node found clockwise from its position.

The benefit appears when the cluster changes. If node B is removed, only the keys that belonged to B move, to the next node clockwise; all other keys stay where they are. Similarly, adding a node takes over a portion of the keys of only one neighbor. On average, only about K/N of the K keys move when a cluster of N nodes changes, compared with nearly all keys in the modulo scheme.

A plain ring can still distribute load unevenly, because the nodes' positions are random. Systems therefore assign each physical machine many positions on the ring, called *virtual nodes*, which smooths the distribution and lets more powerful machines take more virtual nodes. Redis Cluster uses a related fixed scheme, dividing the key space into 16,384 *hash slots* that are assigned to nodes (Redis Documentation).

3.2.4 Replication

To survive node failures, each piece of data is stored on several nodes. In a Dynamo-style design with replication factor N, a key is stored on the node that owns it plus the next nodes around the ring. If one fails, others can serve the data and the system remains available. Single-leader designs, such as Redis replication, instead copy the data from a primary to one or more replicas.

Replication introduces the consistency trade-offs of Chapter 1. With quorum-based replication, writes go to W replicas and reads consult R replicas; setting $R + W > N$ ensures that reads overlap the most recent write. For example, with $N = 3$, the common setting $R = 2$, $W = 2$ tolerates one failed replica while still returning up-to-date data (barring concurrent writes). Dynamo-style systems add further mechanisms to maintain convergence: *hinted handoff* (a healthy node temporarily holds writes for a failed one), *anti-entropy* using Merkle trees (replicas compare and repair differences in the background), and conflict resolution with

version information such as vector clocks or last-write-wins (DeCandia et al., 2007).

3.2.5 Consistency Models in Practice

Different products choose different points on the consistency spectrum:

- **Strong consistency**: systems based on a consensus protocol, such as etcd with Raft (Ongaro & Ousterhout, 2014), order all writes and reject operations they cannot safely complete.
- **Tunable or eventual consistency**: DynamoDB reads are eventually consistent by default, with an option for strongly consistent reads within a region at higher cost and latency (Amazon Web Services, DynamoDB Developer Guide).
- **Asynchronous replication**: Redis replicates from primary to replicas asynchronously by default. This is fast, but if the primary fails right after acknowledging a write, that write may be lost on failover.

3.2.6 Request Routing

Clients must find the node that owns a key. Three approaches are common: the *client* computes the owner itself using the ring or slot map (as with many Memcached clients); a *proxy* layer routes requests; or any node acts as a *coordinator* and forwards the request, as in Dynamo-style databases and Redis Cluster redirections. Each balances client complexity, extra network hops, and operational simplicity differently.

3.3 In-Memory Databases

3.3.1 Why Memory?

An *in-memory database* keeps its working data primarily in main memory (RAM) rather than on disk. The reason is the large gap in access times. As rough orders of magnitude, which vary with hardware, a main-memory access takes about a hundred nanoseconds, a random read from a solid-state drive takes tens to hundreds of microseconds, and a seek on a spinning disk takes several milliseconds. Memory is therefore faster by factors of hundreds to tens of thousands, which allows a single server to execute hundreds of thousands, or more, of simple operations per second.

In practice, the network round trip within a data center (typically a fraction of a millisecond) often dominates end-to-end latency, so a well-placed in-memory store usually responds in well under a few milliseconds. This is still far faster than a query that reads from disk, joins several tables, and applies business logic.

3.3.2 Rich Data Structures

Some in-memory stores go beyond opaque values and offer server-side data structures with operations that run atomically inside the database. Redis, often described as a *data structure server*, is the best-known example (Table 3.3).

Table 3.3: Selected Redis value types and typical uses.

Type	Description	Typical use
String	Text or binary value; supports atomic increments	Cached objects, counters, flags
Hash	Field–value map inside one key	Object with several fields (a user profile)
List	Ordered sequence	Simple queues, recent-activity feeds
Set	Unordered collection of unique members	Tags, unique visitors, manual indexes
Sorted set	Members ordered by a numeric score	Leaderboards, ranking, scheduling
Stream	Append-only log of entries	Event processing, lightweight messaging

Listing 3.1 shows typical commands. Each individual command executes atomically, which is why operations such as `INCR` are safe even with many concurrent clients.

Listing 3.1: Typical Redis commands for sessions, counters, locks, and leaderboards.

```
SET session:abc123 "user=1001;cart=3" EX 1800
GET session:abc123
TTL session:abc123

INCR page:home:views

SET lock:order:5001 worker-7 NX EX 30

ZADD leaderboard 4200 ava 3900 ben 5100 cara
ZREVRANGE leaderboard 0 2 WITHSCORES
```

The first command stores a session that expires after 1,800 seconds. `INCR` atomically increments a counter. The `NX` option sets the key only if it does not already exist, which can be used to implement a simple lock with an expiry; correct distributed locking has subtle failure cases, so consult the documentation before relying on it for critical correctness (Redis Documentation). The sorted set commands record scores and retrieve the top three entries.

3.3.3 Persistence and Durability

Memory is volatile: a restart or crash erases it. In-memory databases therefore offer a range of choices between speed and durability. Using Redis terminology, Table 3.4 summarizes the main options.

Production systems often combine these: replication for availability, plus snapshots or an append-only log for recovery after a full restart. The key design question is how much

Table 3.4: Persistence options for an in-memory store (Redis terminology).

Option	**How it works**	**Trade-off**
No persistence	Data exists only in memory	Fastest; everything is lost on restart (pure cache)
Snapshots (RDB)	Periodically write a point-in-time copy of the dataset to disk	Compact and fast to restore; writes since the last snapshot are lost on a crash
Append-only file (AOF)	Log every write operation and replay it on restart	Less data loss; larger files and some write overhead, depending on how often the log is flushed to disk
Replication	Copy data to other nodes	Survives a single-node failure; asynchronous copies can lose the most recent writes

data loss is acceptable. A cache can lose everything and be rebuilt from the system of record; a store that *is* the system of record cannot.

3.3.4 Memory Management and Eviction

Because RAM is more expensive and limited than disk, in-memory stores must manage capacity. Two mechanisms are common:

- **Expiration.** Keys can be given a *time to live* (TTL); the database deletes them automatically when it elapses. This is ideal for sessions and cached results.
- **Eviction.** When memory is full, the database removes keys according to an *eviction policy* (Table 3.5), or rejects new writes, depending on configuration.

Other techniques, such as compact encodings and careful choice of value types, also reduce memory use. Capacity planning should account for overhead beyond the raw data, including indexes, replication buffers, and memory fragmentation.

Speed Versus Durability

Each step toward durability, such as flushing every write to disk or waiting for replicas to acknowledge, reduces speed. There is no free setting: in-memory databases let you choose the balance for each use case. This is the latency-versus-consistency trade-off of PACELC (Chapter 1) applied to durability.

Table 3.5: Common eviction policies.

Policy	Removes	Suited to
LRU (least recently used)	Keys not accessed for the longest time	General-purpose caches
LFU (least frequently used)	Keys accessed least often	Workloads with stable popular items
TTL-based	Keys closest to expiry	Data with meaningful lifetimes
Random	Randomly chosen keys	Simple, low-overhead caching
No eviction	Nothing; new writes fail when full	Stores that must not drop data

3.4 Common Use Cases and Patterns

3.4.1 Caching

Caching is the most common use of key-value stores. Frequently read data is kept in memory so that most requests avoid the slower primary database. Table 3.6 lists the main patterns.

Table 3.6: Common caching patterns.

Pattern	How it works	Trade-off
Cache-aside	The application checks the cache first; on a miss it reads the database and fills the cache	Simple and widely used; the application manages the cache; the first request after a miss is slow
Read-through	The cache itself loads missing data from the database	Simpler application code; needs cache-side integration
Write-through	Writes go to the cache and the database together	Cache stays fresh; writes are slower
Write-behind	Writes go to the cache and are saved to the database later	Fast writes; risk of data loss if the cache fails before saving

Cache-aside is the pattern most applications start with (Listing 3.2).

Listing 3.2: Cache-aside pattern in pseudocode.

```
def get_user(user_id):
    key = "user:" + str(user_id)
    value = cache.get(key)
    if value is not None:
```

```
    return value # cache hit
value = db.query_user(user_id) # cache miss: read the database
cache.set(key, value, ttl=300) # store with a time to live
return value
```

The benefit depends on the *hit ratio* h, the fraction of requests served from the cache. If a cache lookup takes t_c and a database read takes t_d, and a miss pays for both, the average latency is

$$T = t_c + (1 - h)\, t_d.$$

For example, with $t_c = 1$ ms, $t_d = 20$ ms, and $h = 0.95$, the average is $1 + 0.05 \times 20 = 2$ ms, compared with 20 ms without the cache. Raising the hit ratio to 99% reduces it to 1.2 ms.

Caching also introduces problems that must be designed for:

- **Stale data.** The cache may hold values that the database has since changed. Mitigations are short TTLs and invalidating (deleting) or updating the cached entry when the underlying data changes.
- **Cache stampede.** When a popular key expires, many concurrent requests may miss at once and overload the database. Remedies include adding random jitter to TTLs, allowing only one request to rebuild a value, and refreshing popular entries before they expire.
- **Cold start.** An empty cache sends all traffic to the database. Warm the cache or ramp up traffic gradually after restarts.

At very large scale, caching layers become distributed systems in their own right, as described in the experience of operating Memcached at Facebook (Nishtala et al., 2013).

3.4.2 Session Management

Web applications keep per-user state, such as login status and shopping carts, in *sessions*. Storing sessions in a shared key-value store, keyed by a session identifier and given a TTL, lets any application server handle any request and makes the servers stateless and easy to scale. The expiry removes abandoned sessions automatically.

3.4.3 Counters, Rate Limiting, and Leaderboards

Atomic increments make counters trivial: page views, likes, or inventory reservations can be updated by many clients without lost updates. A simple fixed-window rate limiter increments a counter whose key includes the user and the current minute, sets an expiry on first use, and rejects requests once the count exceeds a limit. Sorted sets keep members ranked by score, so leaderboards can be updated and queried in logarithmic time.

3.4.4 Configuration and Coordination

Strongly consistent key-value stores such as etcd hold configuration, feature flags, and cluster membership information, and provide primitives for leader election and coordination. Here, correctness and consistency matter much more than raw throughput.

3.4.5 Real-Time Analytics and Messaging

Streaming data can be ingested and summarized with low latency, for example by maintaining counters and recent-event windows for dashboards and alerting. Lists, streams, and publish/subscribe features also support lightweight messaging between components. Note, however, that some of these mechanisms offer weaker delivery guarantees than dedicated messaging systems: in Redis, for example, publish/subscribe messages are not stored for subscribers that are offline. For reliable, replayable event pipelines, durable log-based systems are usually more appropriate (see later chapters).

3.5 Designing Keys and Values

Since the key is the only access path, key design is the central modeling activity.

- **Use structured, namespaced keys.** A convention such as `entity:id:attribute` (for example, `user:1001:session`) prevents collisions, makes keys readable, and allows tooling to group related keys.
- **Keep keys reasonably short.** Keys are stored with every value, so very long keys waste memory at scale.
- **Keep values appropriately sized.** Very large values slow replication and network transfer and block other requests. Split large objects or store them elsewhere (for example, in object storage) and keep only a reference.
- **Choose value formats deliberately.** Serialized JSON is flexible and human-readable; compact binary formats save space and parsing time.
- **Plan expiry.** Decide which keys need a TTL so that the store does not fill with obsolete data.
- **Maintain your own indexes if needed.** To find values by something other than the key, the application can maintain additional keys, such as a set `tag:gift` listing the order keys with that tag. This is manual work and can become inconsistent if updates to the two structures are not coordinated; if lookups by many attributes dominate, consider a document or relational database instead.

Hot keys. A *hot key* is a single key that receives a disproportionate share of traffic, such as a global counter in a busy system or a viral item. Because one key lives on one primary node, it can overload that node while the rest of the cluster sits idle. Common mitigations are caching the value inside the application, replicating hot data for reads, and splitting a counter into several sub-keys (for example, `views:home:0` to `views:home:9`) that are summed when read.

3.6 Advantages and Limitations

3.6.1 Advantages

- **Simplicity.** The model is easy to understand and to integrate into applications.
- **Speed.** Direct key access, and for in-memory systems the absence of disk I/O, yields very low latency and high throughput.
- **Horizontal scalability.** Hash partitioning spreads data and load across many nodes.
- **Availability.** Replication and automatic failover keep data accessible during failures.
- **Predictable performance.** Operations have simple cost profiles, which makes capacity planning easier.

3.6.2 Limitations

- **Limited querying.** There is no general query language: finding data by anything other than the key requires extra structures.
- **No built-in relationships.** Joins, foreign keys, and referential integrity are the application's responsibility.
- **Limited transactions.** Operations are atomic per key (and sometimes within a small scope), but general multi-key transactions are restricted or absent in many systems.
- **Cost and capacity of memory.** Holding large datasets in RAM is expensive, and capacity is bounded by memory.
- **Durability risk.** Systems optimized for speed may lose recent writes on failure unless configured otherwise.
- **Hotspots.** Skewed access patterns can overload individual nodes.

3.6.3 When to Choose a Key-Value Store

A key-value store is a strong choice when access is naturally by a known identifier, latency is critical, and the data has a simple structure (sessions, caches, counters, configuration, carts). It is a weaker choice as the primary store when you need ad hoc queries, relationships between entities, or complex transactions, in which case a relational, document, or graph database is a better system of record. Often the best architecture combines them: a key-value store in front of another database, an example of the polyglot persistence described in Chapter 1. Compare this with the flexible, queryable model of Chapter 2.

Summary

Key-value stores hold data as unique keys mapped to values and are accessed primarily by key. This simple model enables very high speed and straightforward horizontal scaling, using techniques such as hash partitioning, consistent hashing with virtual nodes, and replication with tunable consistency. In-memory databases such as Redis and Memcached go further by keeping data in RAM, trading volatility and capacity for extremely low latency; persistence options (snapshots, append-only logs, replication), expiration, and eviction policies let designers balance speed against durability and cost. Common applications include caching,

session management, counters, leaderboards, rate limiting, and configuration, supported by patterns such as cache-aside. Good key design, awareness of hot keys, and clear decisions about durability are essential. Because the model offers little querying or relationship support, key-value stores are most often used alongside, rather than instead of, richer databases.

Review Questions

1. What is the basic data model of a key-value store, and what are its core operations?
2. Why can key-value stores achieve such high performance and scalability?
3. What problem does consistent hashing solve compared with the "hash modulo N" scheme, and what are virtual nodes for?
4. Explain how quorum settings N, R, and W affect consistency and availability in a replicated key-value store.
5. What are the main differences between in-memory and disk-based databases? Why is the network round trip often more important than memory speed?
6. Compare snapshots and append-only files as persistence mechanisms. What data can be lost in each case?
7. Describe the cache-aside pattern. What are cache stampedes and stale data, and how can they be mitigated?
8. What is a hot key, and how can it be handled?
9. When is a key-value store a poor choice as a primary database?

Exercises

1. **Consistent hashing.** A cluster of 4 nodes stores 1,000,000 keys using hash(key) mod N. A fifth node is added. Estimate how many keys change nodes. Compare with consistent hashing, where roughly K/N keys move, and explain the difference.
2. **Cache sizing.** An application has $t_c = 0.5$ ms and $t_d = 15$ ms. Compute the average latency for hit ratios of 80%, 95%, and 99%. What does this suggest about the value of improving the hit ratio?
3. **Key design.** Design keys and value types (using the Redis types in Table 3.3) for an online game that must show each player's profile, the top 100 players by score, and the number of players currently online.
4. **Persistence.** For each system, choose a persistence and replication configuration and justify it: (a) a pure web-page cache, (b) a session store, (c) a store holding the only copy of shopping carts.

5. **Quorums.** For $N = 3$, evaluate $(R, W) = (1, 1)$, $(2, 2)$, and $(1, 3)$. For each, state whether reads are guaranteed to see the latest write and how many replica failures can be tolerated for reads and writes.

6. **Research.** Compare Memcached and Redis for a caching workload. In which situations is Memcached's simplicity an advantage, and when are Redis's richer features worth the added complexity?

References

- Amazon Web Services. *Amazon DynamoDB Developer Guide*. https://docs.aws.amazon.com/dynamodb

- Chang, F., Dean, J., Ghemawat, S., et al. (2006). Bigtable: A Distributed Storage System for Structured Data. In *Proceedings of OSDI '06*.

- DeCandia, G., Hastorun, D., Jampani, M., et al. (2007). Dynamo: Amazon's Highly Available Key-value Store. In *Proceedings of SOSP '07*.

- Karger, D., Lehman, E., Leighton, T., Panigrahy, R., Levine, M., & Lewin, D. (1997). Consistent Hashing and Random Trees: Distributed Caching Protocols for Relieving Hot Spots on the World Wide Web. In *Proceedings of STOC '97*, 654–663.

- Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.

- Nishtala, R., Fugal, H., Grimm, S., et al. (2013). Scaling Memcache at Facebook. In *Proceedings of NSDI '13*.

- O'Neil, P., Cheng, E., Gawlick, D., & O'Neil, E. (1996). The Log-Structured Merge-Tree (LSM-Tree). *Acta Informatica*, 33(4), 351–385.

- Ongaro, D., & Ousterhout, J. (2014). In Search of an Understandable Consensus Algorithm (Raft). In *Proceedings of USENIX ATC '14*.

- Redis Documentation. https://redis.io/docs

Chapter 4

Column-Family Databases and Wide-Column Models

Learning Objectives

After reading this chapter, you will be able to:

- Describe the wide-column data model, including row keys, column families, partitions, and timestamps.
- Explain how wide-column systems store data using commit logs, memtables, SSTables, and compaction.
- Compare the architectures of Apache Cassandra (masterless) and Apache HBase (master and region servers).
- Reason about replication and tunable consistency levels using quorum arithmetic.
- Model data starting from queries, using denormalization and well-sized partitions.
- Recognize suitable use cases and the main limitations and operational challenges.

4.1 Introduction to Column-Family Databases

Column-family databases, also called *wide-column stores*, are distributed databases designed to store very large volumes of data across many machines with high write throughput and high availability. Instead of organizing data as fixed tables of rows and columns, they organize it into *rows* identified by a key, whose columns are grouped into *column families*. Rows in the same table may have very different columns, so the model is naturally suited to sparse and evolving data.

4.1.1 Origins

The model was introduced by Google's Bigtable (Chang et al., 2006), which was built to store petabytes of data for services such as web indexing and Google Earth. Its design inspired open-source systems: Apache HBase follows the Bigtable architecture closely and runs on the Hadoop ecosystem (George, 2011), while Apache Cassandra, originally developed at Facebook, combines the Bigtable data model with the distribution techniques of Amazon's

Dynamo (Hewitt, 2010; Lakshman & Malik, 2010; DeCandia et al., 2007). Managed and compatible offerings include Google Cloud Bigtable, Amazon Keyspaces, and ScyllaDB.

Terminology: Wide-Column Versus Columnar

As noted in Chapter 1, wide-column stores should not be confused with *columnar* (column-oriented) analytical databases such as ClickHouse or Amazon Redshift. Columnar databases store each column's values contiguously to speed up analytical scans over a few columns of many rows. Wide-column stores are operational databases that group columns into families and are organized around row keys, partitions, and fast writes. The names are similar, but the designs and use cases differ.

4.1.2 Relational Versus Wide-Column Concepts

Table 4.1 relates familiar relational ideas to the wide-column model. As in earlier chapters, the mapping is approximate.

Table 4.1: Approximate correspondence between relational and wide-column concepts.

Relational	**Wide-column**	**Key difference**
Table	Table (Cassandra) or table with column families (HBase)	Data is organized around query patterns
Primary key	Row key, or partition key plus clustering columns	Determines where data lives and how it is sorted
Row	Row (possibly very wide)	Rows can have different columns; sparse columns cost nothing
Join	None; data is denormalized	Related data is duplicated into query-specific tables
Single version per value	Timestamped cells	Each cell carries a write timestamp

4.2 Data Model and Structure

4.2.1 A Sorted, Multidimensional Map

The Bigtable paper describes its data model as a sparse, distributed, persistent, multidimensional sorted map (Chang et al., 2006). Each value (a *cell*) is addressed by three coordinates:

$$(\text{row key},\ \text{column family}:\text{column qualifier},\ \text{timestamp}) \rightarrow \text{value}.$$

Rows are kept sorted by row key, which makes range scans over adjacent keys efficient. A *column family* is a named group of related columns defined when the table is created; the individual *column qualifiers* inside a family can be created dynamically at write time. Each column family is stored separately, so a query that needs only one family reads less data. Values are uninterpreted bytes, and each cell can hold multiple timestamped versions.

4.2.2 Sparse Storage

Because columns exist only where values were written, missing values consume no space. Table 4.2 shows a logical view of three rows in an HBase-style table with two column families. A dash means that no cell exists.

Table 4.2: Logical view of a sparse table with column families `info` and `activity`.

Row key	`info:name`	`info:email`	`activity:last_login`	`activity:plan`
user1001	Ava Chen	ava@example.com	2026-03-14	—
user1002	Ben Ortiz	—	—	premium
user1003	Cara Singh	cara@example.com	2026-03-12	—

Listing 4.1 shows how this is written and read in the HBase shell.

Listing 4.1: Creating and using an HBase table with two column families.

```
create 'users', 'info', 'activity'

put 'users', 'user1001', 'info:name', 'Ava Chen'
put 'users', 'user1001', 'info:email', 'ava@example.com'
put 'users', 'user1001', 'activity:last_login', '2026-03-14'

get 'users', 'user1001'
scan 'users', {STARTROW => 'user1000', LIMIT => 10}
```

4.2.3 Cassandra's Model: Partition Key and Clustering Columns

Cassandra uses a tabular vocabulary through its query language, CQL (Cassandra Query Language), but the underlying model is the same wide-column idea. The primary key has two parts:

- the *partition key*, which is hashed to decide which nodes store the data. All rows with the same partition key form a *partition* stored together on the same replicas;
- optional *clustering columns*, which determine the sort order of rows inside a partition.

This is what makes partitions "wide": one partition (for example, one sensor's readings for one day) may contain thousands of rows ordered by time, so a time-range query reads a contiguous slice from one node. Unlike HBase, Cassandra tables do have a declared schema (columns and types), although columns can be added online without a migration of existing data. Listing 16.5 defines a time-series table.

Listing 4.2: A Cassandra time-series table with a composite partition key (CQL).

```
CREATE KEYSPACE metrics
  WITH replication = {'class': 'NetworkTopologyStrategy', 'dc1': 3};

CREATE TABLE metrics.sensor_readings (
  sensor_id text,
  day date,
  ts timestamp,
  value double,
  PRIMARY KEY ((sensor_id, day), ts)
) WITH CLUSTERING ORDER BY (ts DESC);

INSERT INTO metrics.sensor_readings (sensor_id, day, ts, value)
VALUES ('s-17', '2026-03-14', '2026-03-14␣10:05:00+0000', 21.4)
USING TTL 2592000;

SELECT ts, value FROM metrics.sensor_readings
WHERE sensor_id = 's-17' AND day = '2026-03-14' LIMIT 10;
```

Here `(sensor_id, day)` is the partition key and `ts` is the clustering column, so each sensor-day is one partition with readings sorted newest first. The `TTL` clause makes each reading expire automatically after 30 days (2,592,000 seconds). The query supplies the full partition key, which is how Cassandra locates the right nodes without scanning the cluster.

4.2.4 Timestamps and Versions

Every cell carries a write timestamp. HBase can retain several versions of a cell, letting applications read the latest or a historical value. In Cassandra, timestamps resolve conflicts between replicas: when two replicas hold different values for the same cell, the one with the newer timestamp wins (*last-write-wins*). This makes reasonably synchronized clocks important, because skewed clocks can cause an older write to overwrite a newer one (Kleppmann, 2017).

4.3 Architecture and Distribution

4.3.1 Partitioning and Replication

Wide-column systems distribute rows across nodes by key. Cassandra hashes the partition key and places it on a consistent-hashing ring with virtual nodes (Chapter 3), giving an even spread without a central coordinator. HBase instead divides the sorted key space into contiguous *regions*, each served by one region server; a region splits automatically as it grows. Hash partitioning spreads load evenly but makes range scans across partitions hard, while range partitioning keeps adjacent keys together at the risk of hotspots (for example, with monotonically increasing keys).

Data is replicated so that nodes can fail without data loss. In Cassandra, the *replication factor* (RF) sets the number of copies, and the replicas are placed on consecutive nodes of

the ring, optionally spread across racks and data centers. HBase relies on the underlying distributed file system (HDFS) to keep multiple copies of its files (Apache HBase Reference Guide).

4.3.2 Two Architectural Styles

The two best-known systems differ in how they coordinate (Table 4.3).

Table 4.3: Comparison of Apache Cassandra and Apache HBase.

Aspect	**Apache Cassandra**	**Apache HBase**
Architecture	Masterless, peer-to-peer ring; all nodes have the same role; nodes learn about each other with a gossip protocol	A master coordinates region servers; relies on ZooKeeper for coordination and on HDFS for storage
Partitioning	Hash of the partition key on a ring	Contiguous key ranges (regions)
Consistency	Tunable per request (eventual to strong)	Strongly consistent reads and writes per row
Availability under failure	Continues serving with remaining replicas, subject to chosen consistency level	A region is briefly unavailable until the master reassigns it to another server
Typical strengths	Always-on, multi-data-center writes; time-series; linear scaling	Tight Hadoop integration; strong row-level consistency; large range scans

"Masterless" means that there is no single node whose failure stops the cluster, which is why Cassandra is described as having no single point of failure. In HBase, the master is not on the data path for reads and writes, and standby masters can take over, but the system depends on more components (ZooKeeper, HDFS), which adds operational complexity.

4.3.3 The Storage Engine: Why Writes Are Fast

Wide-column systems use a log-structured merge (LSM) design (O'Neil et al., 1996), already met in Chapter 3, which turns random writes into sequential ones. Figure 4.1 shows the write path.

The steps are:

1. The write is appended to a *commit log* (HBase calls it the write-ahead log, WAL). This sequential append makes the write durable, so it can be replayed after a crash.
2. The write is applied to the *memtable* (in HBase, the MemStore), an in-memory sorted structure. The write is now acknowledged; no random disk access was needed.
3. When the memtable is full, it is *flushed* to disk as an immutable, sorted file called an *SSTable* (in HBase, an HFile).

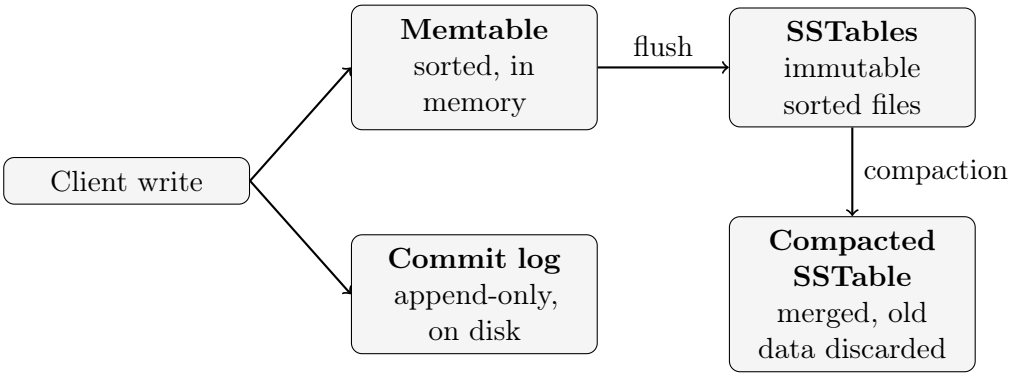


Figure 4.1: The LSM write path. A write is appended to the commit log (for durability) and added to the in-memory memtable. When full, the memtable is flushed to an immutable SSTable, and SSTables are periodically merged by compaction.

4. In the background, *compaction* merges SSTables, discarding overwritten and deleted data and keeping the number of files, and therefore read cost, under control.

4.3.4 The Read Path and Bloom Filters

Because a row may be spread over the memtable and several SSTables, a read may need to consult multiple places and merge the results by timestamp. To avoid reading files that cannot contain the key, each SSTable has a *Bloom filter* (Bloom, 1970): a compact probabilistic structure that answers "definitely not here" or "possibly here". It never produces a false negative, so a negative answer allows the file to be skipped entirely, at the price of occasional false positives that cost an unnecessary read. Together with indexes and caches, Bloom filters keep reads reasonably fast despite the multi-file layout.

4.3.5 Compaction Trade-offs

Compaction improves reads and reclaims space but consumes disk and CPU, so strategies trade off *write amplification* (data rewritten repeatedly), *read amplification* (files consulted per read), and *space amplification* (extra disk used). Cassandra offers strategies such as size-tiered compaction (good for write-heavy workloads), leveled compaction (better read performance, more rewriting), and time-window compaction (suited to time-series data with TTLs, because whole expired files can be dropped).

4.3.6 Deletes and Tombstones

Since SSTables are immutable, a delete cannot remove data in place. Instead, the system writes a marker called a *tombstone*, which hides older values and is removed later by compaction. Tombstones must be retained long enough to reach every replica; in Cassandra the default grace period is ten days, and replicas should be repaired within that window, otherwise deleted data can reappear. Workloads that delete or overwrite heavily can accumulate many tombstones, which slows reads, so patterns such as using a table as a queue should be avoided.

4.3.7 Consistency Levels

The quorum reasoning of Chapter 1 appears directly in Cassandra, where the client chooses a *consistency level* for each read and write (Table 4.4). With a replication factor N, a quorum is $\lfloor N/2 \rfloor + 1$ replicas; for $N = 3$ it is 2.

Table 4.4: Common Cassandra consistency levels (replication factor $N = 3$ in the examples).

Level	**Replicas that must respond**	**Trade-off**
`ONE`	One replica	Lowest latency, highest availability; may return stale data
`QUORUM`	A majority of all replicas (2 of 3)	Balanced; with quorum reads and writes, $R + W > N$
`LOCAL_QUORUM`	A majority in the local data center	Avoids cross-data-center latency
`ALL`	Every replica	Strongest; operation fails if any replica is down

For example, writing and reading at `QUORUM` with $N = 3$ gives $R + W = 2 + 2 = 4 > 3$, so every read overlaps the latest write. Writing and reading at `ONE` gives $1 + 1 = 2 \leq 3$, which is fast but can return stale data. Behind the scenes, Cassandra repairs divergence using hinted handoff, read repair, and periodic anti-entropy repair (DeCandia et al., 2007; Apache Cassandra Documentation). For the uncommon cases that need compare-and-set semantics, it also offers lightweight transactions, built on a consensus protocol, at significantly higher latency.

In terms of the CAP theorem (Section 1.5), Cassandra is typically classed as AP-leaning with tunable consistency, while HBase favors consistency.

4.4 Data Modeling

4.4.1 Query-First Design

Relational modeling starts from the entities and lets queries be written later. Wide-column modeling inverts this: because there are no joins and queries must supply the partition key, the *queries come first*. A common methodology (Chebotko et al., 2015) is:

1. Build a conceptual model of the entities and relationships.
2. List the application's queries (what is asked, filtered, and sorted).
3. Design one table per query (or per family of queries), choosing the partition key to match the filter and the clustering columns to match the sort order.
4. Choose physical details such as partition sizing, TTLs, and compaction strategy.

4.4.2 Denormalization: One Table per Query

Because the same data must be retrievable in several ways, the same facts are stored in several tables, each organized for one query. Listing 4.3 serves two queries: "orders of a customer, newest first" and "one order by its identifier".

Listing 4.3: Two query-specific tables holding the same order data (CQL).

```
CREATE TABLE orders_by_customer (
  customer_id text,
  order_ts timestamp,
  order_id text,
  total decimal,
  PRIMARY KEY (customer_id, order_ts)
) WITH CLUSTERING ORDER BY (order_ts DESC);

CREATE TABLE orders_by_id (
  order_id text PRIMARY KEY,
  customer_id text,
  order_ts timestamp,
  total decimal
);
```

When an order is created, the application writes to both tables (often in a logged batch to keep them in step). Storage is cheap and writes are fast, so duplicating data is an accepted cost; the price is extra application logic and the risk of inconsistency between copies. This echoes the denormalization discussed for document databases in Chapter 2, but here it is even more systematic.

4.4.3 Designing Partitions

The partition is the unit of distribution, so it must be designed with care:

- **Spread load evenly.** A partition key with high cardinality and balanced traffic distributes work across the cluster. A key with few values, or one value that is very popular, creates a *hot partition* on a few nodes.
- **Keep partitions bounded.** A partition that grows without limit (all readings of one sensor ever) becomes slow to read, compact, and repair. A common rule of thumb is to keep partitions to at most about a hundred megabytes. The usual remedy is *bucketing*: adding a time bucket (such as the day in Listing 16.5) to the partition key.
- **Match the query.** The most common queries should be answered from a single partition, using the partition key in the filter. Queries that touch many partitions are slow and are usually a sign that a table is missing.
- **Avoid monotonically increasing partition keys** in range-partitioned systems such as HBase (for example, a raw timestamp as row key), which send all new writes to one region. Prefixing the key with a salt or a high-cardinality identifier spreads the writes.

4.4.4 Common Pitfalls

- **Relational thinking.** Normalizing the schema and expecting joins or ad hoc filtering will lead to poor performance. Cassandra rejects queries that would need a full scan unless forced with `ALLOW FILTERING`, which is rarely appropriate in production.

- **Overuse of secondary indexes.** They are supported but less efficient than designing a table for the query, particularly on high-cardinality columns in large clusters.

- **Delete-heavy or queue-like patterns**, which create tombstones.

- **Unbounded partitions** and **unbounded collections** inside a row.

4.5 Use Cases and Applications

Wide-column databases fit workloads with very high write rates, large data volumes, predictable access patterns, and demanding availability requirements.

- **Time-series and IoT data.** Sensor readings, application metrics, and monitoring data are written continuously and read by time range. Partitioning by source and time bucket, with clustering by timestamp and TTL-based expiry, matches the model directly.

- **Logging and event data.** Large streams of events, audit trails, and transaction or call records are appended at high rates and retrieved by key and time.

- **User activity and messaging.** Activity feeds, message histories, and notifications are naturally "all items for a user, ordered by time".

- **Catalogs, profiles, and personalization.** Product catalogs and user profiles with many optional attributes use the sparse row model, and recommendation features need fast key-based lookups at scale.

- **Always-on, multi-region services.** Masterless replication across data centers lets a service survive the loss of a node or a whole data center.

4.5.1 When Not to Use Them

Wide-column stores are generally a poor fit when the application needs ad hoc queries or joins, complex multi-row transactions with strict integrity (a core financial ledger, for example), small data volumes that fit comfortably in a single relational server, or query patterns that are unknown or change constantly. For analytics over many rows and a few columns, a columnar analytical database is usually more appropriate.

4.6 Challenges and Limitations

- **Modeling complexity.** Data must be designed around known queries. A wrong partition key or a missing table is expensive to correct once large amounts of data exist, because it usually requires migrating the data into a new table.
- **Redundancy and consistency of copies.** Denormalized tables must be kept in step by the application, and duplicated data increases storage.
- **Limited querying.** There are no general joins, limited aggregation, and restricted filtering; secondary indexes are less efficient than purpose-built tables.
- **Consistency choices.** Eventual consistency and tunable levels require developers to understand the trade-offs. Wrong settings can produce stale reads, and last-write-wins resolution can silently discard concurrent updates.
- **Read performance can vary.** Reads may touch several files, and heavy deletion or poorly chosen compaction can degrade latency.
- **Operational complexity.** Running a cluster involves capacity planning, repair, compaction tuning, monitoring, upgrades, and (for HBase) managing HDFS and ZooKeeper as well. Managed services reduce this burden considerably.

These costs are real but manageable for the workloads the model is designed for. The central skill, as in earlier chapters, is matching the system to the workload.

Summary

Wide-column databases, introduced by Google's Bigtable and developed further in HBase and Cassandra, store data as sparse, sorted maps addressed by row key, column family, column, and timestamp. Cassandra adds partition keys and clustering columns, while HBase divides sorted key ranges into regions. Both use an LSM-tree storage engine: writes go to a commit log and an in-memory memtable and are later flushed to immutable SSTables that compaction merges, making writes fast and reads dependent on Bloom filters and compaction. Replication and tunable consistency, expressed through the quorum rule $R + W > N$, let designers trade consistency against latency and availability; Cassandra is masterless and AP-leaning, while HBase relies on a master, ZooKeeper, and HDFS and favors strong row-level consistency. Data modeling is query-first and denormalized, with carefully sized partitions. These systems excel at time-series, logging, activity, and always-on workloads, and are poorly suited to ad hoc queries, joins, and complex transactions.

Review Questions

1. What is a column-family (wide-column) database, and how does it differ from a relational database and from a columnar analytical database?

2. What are column families, column qualifiers, and cell timestamps? How is a cell addressed in the Bigtable model?

3. In Cassandra, what are the partition key and clustering columns, and what role does each play?

4. Describe the write path of an LSM-based system. Why are writes fast?

5. What is compaction, and what trade-offs do different compaction strategies make?

6. What is a tombstone, and why can deletes cause problems?

7. How do replication and tunable consistency work in Cassandra? Explain why $R+W > N$ provides read-your-latest-write behavior.

8. Compare the architectures of Cassandra and HBase.

9. Why is data modeling query-first, and what is denormalization?

10. What makes a good partition key? Describe hot partitions and bucketing.

Exercises

1. **Quorums.** For replication factor $N = 5$, compute the size of a quorum. For each of $(R, W) = (1, 1)$, $(3, 3)$, and $(1, 5)$, state whether reads are guaranteed to see the latest write and how many replica failures reads and writes can tolerate.

2. **Time-series design.** Design a Cassandra table for a fleet of vehicles that report GPS positions every second. Choose the partition key, clustering columns, and TTL, and explain how you keep partitions bounded. Estimate the partition size for your choice.

3. **Query-first modeling.** A social application needs to show (a) the latest 50 posts by a user and (b) all posts with a given hashtag from the last 24 hours. Design the tables for these queries and explain what data is duplicated.

4. **Row keys.** In HBase, explain why using the raw event timestamp as the row key causes a hotspot, and propose two row key designs that avoid it while still supporting time-range scans per device.

5. **Selection.** For each workload, decide whether a wide-column store is appropriate and justify your answer: (a) an e-commerce checkout with multi-table transactions, (b) clickstream ingestion at 200,000 events per second, (c) a monthly financial report with ad hoc joins.

References

- Apache Cassandra Documentation. https://cassandra.apache.org/doc/latest/
- Apache HBase Reference Guide. https://hbase.apache.org/book.html
- Bloom, B. H. (1970). Space/Time Trade-offs in Hash Coding with Allowable Errors. *Communications of the ACM*, 13(7), 422–426.
- Chang, F., Dean, J., Ghemawat, S., et al. (2006). Bigtable: A Distributed Storage System for Structured Data. In *Proceedings of OSDI '06*.
- Chebotko, A., Kashlev, A., & Lu, S. (2015). A Big Data Modeling Methodology for Apache Cassandra. In *Proceedings of the IEEE International Congress on Big Data*.
- DeCandia, G., Hastorun, D., Jampani, M., et al. (2007). Dynamo: Amazon's Highly Available Key-value Store. In *Proceedings of SOSP '07*.
- George, L. (2011). *HBase: The Definitive Guide*. O'Reilly Media.
- Hewitt, E. (2010). *Cassandra: The Definitive Guide*. O'Reilly Media.
- Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.
- Lakshman, A., & Malik, P. (2010). Cassandra: A Decentralized Structured Storage System. *ACM SIGOPS Operating Systems Review*, 44(2), 35–40.
- O'Neil, P., Cheng, E., Gawlick, D., & O'Neil, E. (1996). The Log-Structured Merge-Tree (LSM-Tree). *Acta Informatica*, 33(4), 351–385.

Chapter 5

Data Modeling in NoSQL Systems

Learning Objectives

After reading this chapter, you will be able to:

- Contrast workload-driven NoSQL modeling with data-driven relational modeling.
- Follow a systematic process from requirements and access patterns to physical design.
- Decide when to denormalize, and apply strategies to keep duplicated data consistent.
- Apply modeling patterns for document, key-value, wide-column, and graph databases.
- Compare how one domain is modeled in each family, and choose a model per requirement.
- Recognize common modeling anti-patterns and apply a design checklist.

5.1 Introduction to NoSQL Data Modeling

Data modeling in NoSQL systems is a shift in mindset from relational design. Earlier chapters introduced each data model separately: documents (Chapter 2), key-value stores (Chapter 3), and wide-column stores (Chapter 4). This chapter steps back and treats modeling as a discipline in its own right, showing what the approaches have in common, where they differ, and how to choose among them.

5.1.1 Data-Driven Versus Workload-Driven Design

Relational design is *data-driven*. The designer identifies entities and relationships, normalizes them to remove redundancy, and relies on the query language to answer any question later through joins. The schema is a general-purpose model of the domain that can serve many unforeseen queries.

NoSQL design is typically *workload-driven*. Because joins are limited or absent, and because data is distributed across machines, the structure of the data determines which

operations are fast. The designer therefore starts from the *access patterns*, meaning the specific reads and writes the application performs, and shapes the data so that each important operation touches as few records, and as few nodes, as possible (Sadalage & Fowler, 2012; Kleppmann, 2017). Table 5.1 summarizes the contrast.

Table 5.1: Relational versus NoSQL modeling.

Aspect	**Relational**	**NoSQL**
Starting point	Entities and relationships	Access patterns (queries and updates)
Redundancy	Minimized by normalization	Accepted, often deliberate (denormalization)
Relationships	Foreign keys and joins at query time	Embedding, references, duplicated data, or explicit edges (graph)
Flexibility	Many queries served by one schema	Fast for planned queries; new queries may need new structures
Schema	Declared up front, enforced on write	Flexible; enforced selectively, often in the application
Atomicity	Multi-row ACID transactions	Usually atomic within one record or partition

5.1.2 Aggregates: The Unit of Design

Sadalage and Fowler describe document, key-value, and wide-column stores as *aggregate-oriented*. An *aggregate* is a cluster of related data treated as a unit for storage, retrieval, and (usually) atomic update, such as an order together with its line items. Aggregates are also the natural unit of distribution: because an aggregate lives together on one node, operations on it avoid network coordination. The central modeling decision in these systems is therefore "where do the boundaries of my aggregates lie?" Graph databases are instead *relationship-oriented*: they make the connections between entities the primary object of modeling (Section 11.3).

5.1.3 Flexibility With Responsibility

NoSQL schemas are flexible, but this does not remove the need for design. Without a declared schema, the application carries the responsibility for structure: agreeing on field names and types, validating critical rules (many systems offer optional validation, as in Chapter 2), and versioning documents so that older and newer shapes can coexist during gradual migrations. Flexibility lowers the cost of change; discipline keeps that change under control.

5.1.4 A Modeling Process

Figure 5.1 shows a practical, iterative process that applies to all four families.

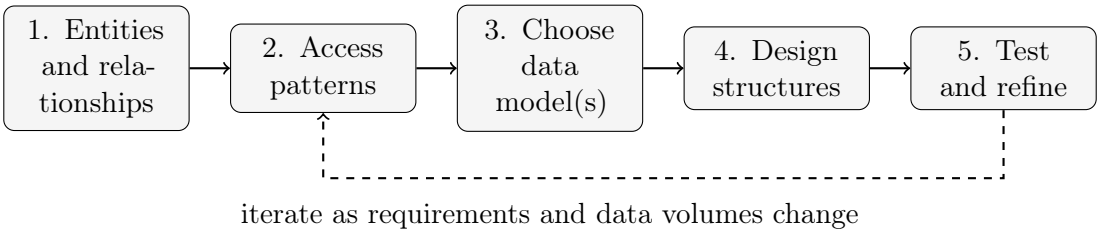


Figure 5.1: An iterative modeling process: understand the domain, list the access patterns, choose the model, design the structures, then test with realistic data and revise.

1. **Entities and relationships.** Build a conceptual model of the domain, which is useful for every database type.
2. **Access patterns.** For each read and write, record what is requested, how often, the latency target, and the consistency required. This list drives every later decision.
3. **Choose the data model(s).** Select the family (or families, in a polyglot architecture) that best matches the access patterns.
4. **Design structures.** Decide the aggregates, keys, partitions, and indexes.
5. **Test and refine.** Load realistic volumes, measure, and adjust.

Capture Access Patterns Explicitly

Write each access pattern as a short statement with numbers, for example: "Show a customer's 20 most recent orders; 500 requests per second; 50 ms at the 95th percentile; stale data of a few seconds is acceptable." Vague requirements lead to models that work on a laptop and fail in production.

5.2 Denormalization and Data Duplication

5.2.1 Why Denormalize

Denormalization stores redundant or pre-combined data so that common reads are answered from a single record or partition. It matters most in distributed systems, where a join may require data from several nodes: eliminating it removes network round trips and reduces tail latency. Embedding an order's line items in the order, or keeping the same order in two tables organized for two different queries (Chapter 4), are both denormalization.

The costs are equally clear:

- **Update complexity.** A change to duplicated data must be applied to every copy.

- **Consistency risk.** Copies can diverge, at least temporarily.
- **Storage.** Duplicated data consumes more space, although storage is usually cheap compared with the latency it saves, and compression reduces the overhead.

5.2.2 When Duplication Pays Off

The trade-off depends on how often the data is read, how often it changes, and how many copies exist. Suppose a seller's display name is copied into 50,000 product documents. If the name changes about once a month, updating all copies takes 50,000 writes per change, a small cost, while every product page view saves a separate lookup. If instead the duplicated value changed several times a day, the update cost and inconsistency window would dominate, and referencing would be better. As a rule of thumb, duplicate data that is read often, changes rarely, and is small; reference data that changes frequently, is large, or must always be current.

5.2.3 Keeping Duplicated Data Consistent

Table 5.2 lists strategies for managing copies. They are often combined.

Table 5.2: Strategies for managing duplicated data.

Strategy	How it works	Trade-off
Write all copies together	One operation writes every copy, using a transaction, an atomic multi-table batch, or an update inside one aggregate	Strong consistency where supported; slower writes; limited to what the database can make atomic
Asynchronous propagation	The primary write emits an event or change-stream record, and a background process updates the copies	Fast primary write; copies are eventually consistent; needs reliable event delivery
Snapshot semantics	Copy the value at a point in time and never update it, for example the price paid on an order	No consistency problem, because the copy is meant to be historical
Periodic reconciliation	A scheduled job compares copies against the source and repairs differences	Catches missed updates; temporary divergence remains
Reference instead	Store an identifier and read the current value when needed	Always current; costs an additional read

Snapshot semantics deserve emphasis, because many "duplicates" are not stale copies at all. An order should record the product name and unit price *at the time of purchase*, even if the catalog changes later, so that the order history stays correct. In such cases, duplication is the correct model, not a compromise.

Finally, decide which copy is the *source of truth*. Every duplicate is derived data that can be rebuilt from the source, which is what makes reconciliation and repair possible (Kleppmann, 2017).

5.3 Modeling in Document Databases

Chapter 2 covered embedding and referencing. In brief: embed data that is read together, bounded in size, and updated together; reference data that is large, shared, or independently changing. This section adds patterns that go beyond that choice. They are commonly used in practice and are described in the MongoDB documentation (MongoDB Documentation), although the ideas apply to other document databases.

Table 5.3: Common document modeling patterns.

Pattern	**Idea**	**Trade-off**
Extended reference	Reference the related document but copy the few fields read most often	Fewer reads; copies can become stale
Subset	Embed only the most relevant or recent items (for example, the latest reviews) and keep the rest in another collection	Keeps documents small; two places to query
Computed	Store pre-calculated values such as counts and averages	Fast reads; must be updated on writes
Bucket	Group many small records (such as readings) into one document per time window or entity	Fewer documents and indexes; bucket size must be managed
Polymorphic	Store records of different shapes in one collection, with a `type` field	Simple queries across variants; application handles variants
Schema versioning	Include a `schemaVersion` field so that old and new shapes coexist	Gradual migration; code must handle several versions

Listing 5.1 combines several of these patterns in a book document from an online bookstore. It embeds a small *subset* of recent reviews, stores *computed* statistics, and carries a schema version, while the full set of reviews lives in a separate collection because it is unbounded.

Listing 5.1: A book document using subset, computed, and versioning patterns, and its separate review documents.

```
// books collection
{
  "_id": "book_977",
```

```
  "schemaVersion": 2,
  "title": "Distributed Systems in Practice",
  "author": { "id": "auth_12", "name": "R. Alvarez" },
  "price": { "amount": 39.90, "currency": "AUD" },
  "stats": { "reviewCount": 1280, "avgRating": 4.6 },
  "latestReviews": [
    { "user": "Ava", "rating": 5, "date": "2026-03-10" },
    { "user": "Ben", "rating": 4, "date": "2026-03-08" }
  ]
}

// reviews collection (one document per review)
{ "_id": "rev_8841", "bookId": "book_977", "userId": "user_1001",
  "rating": 5, "body": "Clear and practical.", "createdAt": "2026-03-10T08:00:00Z"
     }
```

The book page can now be rendered from a single read, while adding a review involves two writes: the new review document and an update of the computed fields and the `latestReviews` array. This is exactly the duplication trade-off of Section 5.2.

5.4 Modeling in Key-Value and Wide-Column Stores

5.4.1 Key-Value Stores: Key Design Is the Model

In a key-value store the key is the only access path, so modeling consists of choosing *keys*, *value formats*, and *lifetimes*. Keys usually encode the entity type and identifier so that they are unique and readable. Table 5.4 shows typical key designs for the bookstore.

Table 5.4: Example key designs for an online bookstore (Redis-style).

Key pattern	Value type	Purpose
`cart:{sessionId}`	Hash (item to quantity)	Shopping cart with a time to live
`book:{id}:summary`	Serialized JSON	Cached book details for fast display
`bestsellers:{month}`	Sorted set	Books ranked by units sold
`idx:tag:{tag}`	Set of book ids	Manually maintained index by tag

Notice that the last row is a *manual index*: the application must update it whenever a book's tags change. If an application needs many such indexes, a store with richer querying is probably the better primary database, with the key-value store kept as a cache (Chapter 3).

5.4.2 Wide-Column Stores: One Table per Query

In wide-column databases, each query gets its own table, with the partition key matching the filter and the clustering columns matching the sort order (Chebotko et al., 2015; Apache Cassandra Documentation). Listing 5.2 serves two queries over the same reviews: "latest reviews of a book" and "all reviews by a user".

Listing 5.2: Two query-specific tables for the same reviews (CQL).

```
CREATE TABLE reviews_by_book (
  book_id text,
  created_at timestamp,
  review_id text,
  user_id text,
  rating int,
  body text,
  PRIMARY KEY ((book_id), created_at, review_id)
) WITH CLUSTERING ORDER BY (created_at DESC, review_id ASC);

CREATE TABLE reviews_by_user (
  user_id text,
  created_at timestamp,
  review_id text,
  book_id text,
  rating int,
  body text,
  PRIMARY KEY ((user_id), created_at, review_id)
) WITH CLUSTERING ORDER BY (created_at DESC, review_id ASC);
```

The same review is written to both tables; each read then comes from a single partition. For a very popular book, the partition `book_id` could grow without bound or become a *hot partition*, in which case a time bucket (for example, month) would be added to the partition key, as in Chapter 4.

5.5 Graph Data Modeling

5.5.1 The Property Graph Model

Graph databases such as Neo4j use the *property graph model*: *nodes* represent entities, *relationships* (edges) are directed, typed connections between nodes, and both can carry *properties* (key–value attributes). Nodes may also have *labels* that classify them, such as `Person` or `Product` (Robinson et al., 2015; Neo4j Documentation). Relationships are stored as first-class elements rather than reconstructed through joins, so, in native graph databases, traversing from a node to its neighbors costs time proportional to the part of the graph actually visited rather than to the total size of the data.

5.5.2 Modeling Guidelines

- **Nodes for things, relationships for connections.** Entities with identity become nodes. The verbs that connect them become relationships, named specifically (`PURCHASED`, `FOLLOWS`, `WROTE`).
- **Properties for attributes.** Facts about a node or relationship (a name, a purchase date, a quantity) are stored as properties.
- **Promote a relationship to a node when it has its own relationships.** If a purchase must itself link to a customer, several products, a store, and a payment, model it as a `Purchase` node connecting them (an intermediate node) rather than as a single edge.
- **Model for the questions asked.** As elsewhere, list the queries first: they decide which relationships to make explicit.
- **Beware of super nodes.** A *super node* is a node with an extremely large number of relationships (a celebrity account, a country). Traversals through it touch huge numbers of edges. Mitigations include more specific relationship types, intermediate nodes (for example, by date), and constraining traversals.
- **Index lookup properties.** Indexes on the properties used to find a starting node (such as a person's identifier) avoid scanning all nodes of a label.

5.5.3 Example: Purchases and Recommendations

Listing 5.3 creates a few nodes and relationships in Cypher, the declarative query language of Neo4j (Francis et al., 2018), and then asks a typical graph question: which books bought by customers who share a purchase with Ava has she not bought herself?

Listing 5.3: Creating and querying a purchase graph in Cypher.

```
CREATE (ava:Person {name: "Ava"}),
       (ben:Person {name: "Ben"}),
       (b1:Book {title: "Distributed Systems in Practice"}),
       (b2:Book {title: "Designing Data Models"}),
       (ava)-[:PURCHASED {date: date("2026-03-14")}]->(b1),
       (ben)-[:PURCHASED {date: date("2026-03-15")}]->(b1),
       (ben)-[:PURCHASED {date: date("2026-03-16")}]->(b2);

MATCH (me:Person {name: "Ava"})-[:PURCHASED]->(:Book)
      <-[:PURCHASED]-(other:Person)-[:PURCHASED]->(rec:Book)
WHERE NOT (me)-[:PURCHASED]->(rec)
RETURN rec.title AS recommendation, count(*) AS score
ORDER BY score DESC LIMIT 5;
```

In a relational design, the same question requires joining the purchases table with itself several times, and every additional hop ("customers of customers") adds another join:

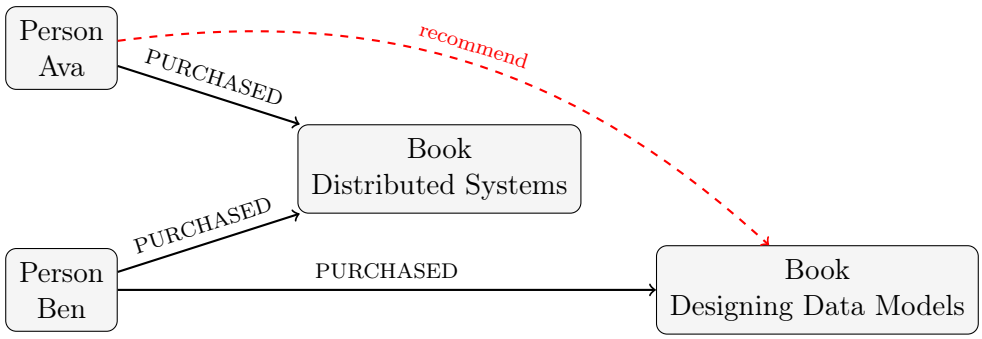


Figure 5.2: A small purchase graph. Ava and Ben share a purchase, and Ben has also bought a second book, which becomes a recommendation for Ava (dashed).

Listing 5.4: The same recommendation in SQL, using repeated joins.

```
SELECT p3.book_id, COUNT(*) AS score
FROM purchases p1
JOIN purchases p2 ON p2.book_id = p1.book_id
                 AND p2.customer_id <> p1.customer_id
JOIN purchases p3 ON p3.customer_id = p2.customer_id
WHERE p1.customer_id = 1
  AND p3.book_id NOT IN (SELECT book_id FROM purchases
                         WHERE customer_id = 1)
GROUP BY p3.book_id
ORDER BY score DESC
LIMIT 5;
```

The SQL version is workable for one or two hops and a moderate data size, but it becomes harder to write, read, and run efficiently as the traversal deepens. This is the kind of workload for which graph models are designed. Conversely, graph databases are less suited to bulk aggregation over all records or to simple key-based lookups, and partitioning a highly connected graph across machines is difficult.

5.6 One Domain, Several Models

To see the models side by side, consider the online bookstore with four requirements: (Q1) show a customer's order history; (Q2) display a book page with recent reviews; (Q3) track the best-selling books of the month; and (Q4) recommend books to customers. Table 5.5 shows how each family would serve them.

No single model is best for all four. This is the practical reason for *polyglot persistence* (Chapter 1): a bookstore might keep orders in a document or relational database, use a key-value store for carts and caching, and use a graph database for recommendations. The price is that the same facts now live in several stores. The remedy is to designate one store as the *source of truth* for each kind of data, and to feed the others as derived views through events or change streams, so that they can be rebuilt if they diverge (Kleppmann, 2017).

Table 5.5: Serving the bookstore requirements with different data models.

	Document	**Key-value**	**Wide-column**	**Graph**
Q1	Order documents with embedded items, indexed by customer	Possible, but needs a manual index per customer	Table per query: orders by customer, newest first	Possible but not the strength
Q2	Book document with subset of reviews and computed stats	Cached summary stored under a per-book key	Table per query: reviews by book	Not the strength
Q3	Aggregation pipeline over orders	Sorted set updated with each sale	Counter or precomputed table per month	Not the strength
Q4	Awkward and slow beyond one hop	Not suitable	Not suitable	Natural, as in Listing 5.3

5.7 Anti-Patterns and a Design Checklist

5.7.1 Common Anti-Patterns

- **Relational design in disguise.** Normalizing everything and expecting joins from a system that lacks them.
- **Modeling before knowing the queries.** Designing around the data alone, then discovering that key access patterns need full scans.
- **Unbounded growth.** Arrays, partitions, or collections that grow forever, such as all events of one user in a single document or partition.
- **Hot keys and hot partitions.** Concentrating traffic on a few keys, for example by partitioning on a low-cardinality value.
- **Uncontrolled flexibility.** No naming conventions, validation, or versioning, so that collections fill with inconsistent shapes.
- **Copies without ownership.** Duplicated data with no designated source of truth and no plan to keep copies in step.
- **Using one model for everything.** Forcing a workload into a database whose model does not match it.

5.7.2 Design Checklist

1. Have I listed every important access pattern with frequency, latency, and consistency needs?

2. Does each critical query read from a single record, partition, or short traversal?
3. Where have I duplicated data, and is each copy a snapshot, a derived view, or a risk?
4. Which copy is the source of truth, and how are the others updated or repaired?
5. Are all collections, partitions, and arrays bounded in size?
6. Is the load spread evenly across keys and partitions?
7. What are the atomicity needs, and do my aggregate boundaries support them?
8. How will the structure change over time (validation, versioning, migrations)?
9. Have I tested with realistic data volumes and traffic?

Summary

NoSQL data modeling is workload-driven: instead of normalizing data and relying on joins, designers start from access patterns and shape data so that important operations are fast and local. Document, key-value, and wide-column stores are aggregate-oriented, so the main decision is where aggregate boundaries lie, while graph databases are relationship-oriented and make connections explicit. Denormalization is the central technique, trading storage and update complexity for read performance; it is managed through strategies such as writing copies together, asynchronous propagation, snapshot semantics, reconciliation, and by designating a source of truth. Each family has its own patterns: extended references, subsets, computed values, and buckets for documents; careful key design for key-value stores; one table per query and well-sized partitions for wide-column stores; and typed relationships, intermediate nodes, and attention to super nodes for graphs. Because no model fits every requirement, real systems often combine several. A checklist of access patterns, duplication, growth, load distribution, atomicity, and evolution helps avoid the common anti-patterns.

Review Questions

1. What are the key differences between relational and NoSQL data modeling? Why is NoSQL modeling called workload-driven?
2. What is an aggregate, and why is it a natural unit of design and distribution?
3. Why is denormalization commonly used in NoSQL systems, and what are its costs?
4. Describe three strategies for keeping duplicated data consistent, and give one situation suited to each.
5. Explain snapshot semantics and why duplicated data is sometimes the correct model rather than a compromise.

6. Describe the subset, computed, and bucket patterns in document databases. What trade-off does each make?

7. How does key design act as the data model in a key-value store, and what is a manual index?

8. How does partition key design affect performance in wide-column databases?

9. What are the main elements of the property graph model? When should a relationship be promoted to a node?

10. What is a super node, and how can it be mitigated? Why might a graph database be unsuitable for some workloads?

Exercises

1. **Access patterns.** For a ride-sharing application, list at least six access patterns with frequency, latency target, and consistency need. Which data model would you choose for each?

2. **Denormalization decision.** A product's category name is copied into 2,000,000 order line records. Category names change roughly twice a year. Argue for or against keeping the copy, considering snapshot semantics.

3. **Document design.** Model a blog with posts, authors, comments, and tags, using the subset and computed patterns. State which data you embed, reference, and duplicate.

4. **Cross-model design.** Take the bookstore of Section 5.6 and design a wide-column table and a Redis key for the requirement "best-selling books in a given category this week". Compare the two designs.

5. **Graph modeling.** Model an organization with employees, teams, projects, and skills as a property graph. Write a Cypher query that finds employees who share at least two skills with a given employee.

6. **Critique.** A team stores all events from every IoT device inside one document per device. Identify the anti-patterns and propose a redesign.

References

- Apache Cassandra Documentation. https://cassandra.apache.org/doc/latest/

- Chebotko, A., Kashlev, A., & Lu, S. (2015). A Big Data Modeling Methodology for Apache Cassandra. In *Proceedings of the IEEE International Congress on Big Data*.

- Francis, N., Green, A., Guagliardo, P., et al. (2018). Cypher: An Evolving Query Language for Property Graphs. In *Proceedings of ACM SIGMOD '18*.

- Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.
- MongoDB Documentation: Data Modeling. https://www.mongodb.com/docs/manual/data-modeling/
- Neo4j Documentation. https://neo4j.com/docs/
- Robinson, I., Webber, J., & Eifrem, E. (2015). *Graph Databases* (2nd ed.). O'Reilly Media.
- Sadalage, P. J., & Fowler, M. (2012). *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence*. Addison-Wesley.

Part II

Distributed Data Systems

Chapter 6

Foundations of Distributed Systems

Learning Objectives

After reading this chapter, you will be able to:

- Define a distributed system and explain why distribution brings both benefits and new kinds of failure.
- Describe timing and failure models, and why failures cannot always be detected reliably.
- Explain message delivery semantics, retries, and idempotence in distributed communication.
- Reason about time and ordering using logical and vector clocks.
- Compare consistency models and replication strategies, and apply quorum arithmetic.
- Explain consensus, leader election, and atomic commit, and the roles of Paxos, Raft, and two-phase commit.
- Apply basic scalability and performance concepts, including tail latency, Little's law, and Amdahl's law.

6.1 Introduction to Distributed Systems

A *distributed system* is a collection of independent computers that cooperate by exchanging messages over a network and that, ideally, appear to their users as a single coherent system (Tanenbaum & Van Steen, 2017). Every NoSQL database discussed in this book is a distributed system: Cassandra's ring, a MongoDB replica set, and a sharded Redis cluster all consist of many machines that must agree on what the data is. This chapter provides the concepts needed to understand why those systems are designed the way they are.

6.1.1 Why Distribute?

Systems are distributed for several reasons:

- **Scalability.** One machine cannot hold all the data or serve all the requests; adding

machines (horizontal scaling) increases capacity.
- **Availability and fault tolerance.** If data and services are replicated, the failure of one machine need not stop the system.
- **Latency.** Placing data close to users in different regions reduces response times.
- **Sharing and organization.** Resources and data are inherently spread across organizations, sites, and devices.

6.1.2 What Makes Distribution Hard

Distribution does not simply multiply a single computer; it changes the problem. Table 6.1 compares the two settings.

Table 6.1: A single machine compared with a distributed system.

Aspect	Single machine	Distributed system
Failure	Usually all-or-nothing	*Partial failure*: some parts fail while others keep running
Communication	Fast, reliable function calls and shared memory	Messages over a network that can delay, lose, duplicate, or reorder them
Time	One clock	Each machine has its own clock, and clocks drift apart
State	One shared memory	No shared memory; knowledge of other nodes is always out of date
Knowledge of failure	A crash is obvious	A slow node cannot be told apart from a crashed one

The most important difference is *partial failure*: the system as a whole is often working while some component is failing, and the failures are nondeterministic. A program that assumes the network behaves like a local function call will eventually fail in surprising ways. These mistaken assumptions are summarized in the *eight fallacies of distributed computing*, attributed to Peter Deutsch and colleagues at Sun Microsystems: the network is reliable; latency is zero; bandwidth is infinite; the network is secure; the topology does not change; there is one administrator; transport cost is zero; and the network is homogeneous. Each assumption is false, and robust systems are designed without relying on any of them.

6.1.3 Transparency and Its Limits

A goal of distributed systems is *transparency*: hiding the distribution from users and programmers. Common forms include *access transparency* (local and remote resources are used the same way), *location transparency* (users need not know where a resource is), *replication transparency* (users do not see that data has several copies), and *failure transparency* (failures are masked). Full transparency is neither achievable nor always desirable. Waldo and

colleagues argued that remote and local operations differ fundamentally in latency, memory access, partial failure, and concurrency, so pretending otherwise leads to fragile systems (Waldo et al., 1994). Good designs hide the complexity they can and expose the rest in a controlled way, for example through timeouts, error codes, and consistency settings.

6.2 System Models and Failures

To reason about a distributed algorithm, we state assumptions about how the system behaves, called a *system model* (Coulouris et al., 2011). Two aspects matter: timing and failures.

6.2.1 Timing Assumptions

- In a *synchronous* model, message delays, processing times, and clock drift have known upper bounds. This simplifies algorithms but rarely holds on real networks.
- In an *asynchronous* model, there are no timing bounds at all, so a node cannot know whether a silent peer is slow or dead.
- The *partially synchronous* model, closest to reality, assumes that the system is usually well behaved but occasionally exceeds its bounds for unbounded periods. Most practical systems are designed for it.

6.2.2 Failure Models

Table 6.2 lists the common failure models, from the most benign to the most severe.

Table 6.2: Common failure models.

Model	**Behavior**	**Example**
Crash-stop	A node halts and never returns	Permanent hardware failure
Crash-recovery	A node halts, then restarts, possibly losing volatile memory	Process restart or reboot
Omission	Messages are lost, or a node fails to send or receive some	Overloaded network or full buffers
Network partition	Groups of nodes cannot communicate with one another	Failed switch or cut link
Byzantine	A node behaves arbitrarily or maliciously, including sending conflicting information	Software bug, corruption, or an attacker

Most database systems assume crash-recovery failures and network problems, and do not defend against Byzantine behavior, which is much more expensive to tolerate. Byzantine fault tolerance is typically reserved for settings where participants do not trust one another, such as some blockchain systems.

6.2.3 Failure Detection

Since a slow node cannot be distinguished from a crashed one, failure detection is based on *timeouts* and is inherently unreliable. A common mechanism is the *heartbeat*: each node periodically sends a signal, and a node that is silent for too long is suspected to have failed. Short timeouts detect failures quickly but cause false suspicions, which can trigger needless recovery actions; long timeouts avoid false alarms but delay recovery. Adaptive detectors, such as the phi accrual failure detector used by Cassandra, report a suspicion level based on observed heartbeat history instead of a fixed threshold (Hayashibara et al., 2004). The lesson is that a distributed system must always operate under uncertainty about which nodes are alive.

6.3 Communication in Distributed Systems

6.3.1 Message Passing and Communication Styles

Nodes coordinate by exchanging messages whose formats are defined by protocols. In *synchronous* communication, the sender blocks until it receives a response, as in a typical request–response call. In *asynchronous* communication, the sender continues without waiting, and the response, if any, arrives later or through another channel. Synchronous calls are simple to reason about but couple the caller to the availability and speed of the callee; asynchronous messaging decouples them and absorbs bursts, at the cost of more complex control flow.

Remote procedure calls (RPC) make a remote function look like a local one, hiding serialization and network details; examples include gRPC and Apache Thrift. RPC is convenient, but it can mislead: a remote call can be slow, can fail in ways a local call cannot, and its outcome can be unknown, as shown next. *Message-oriented middleware*, such as message queues and publish–subscribe systems, places a broker between components. Producers publish messages to a queue or topic, and consumers receive them independently, which improves decoupling and scalability.

6.3.2 Delivery Semantics

Because networks lose messages, senders usually retry. Retrying leads to three delivery guarantees (Table 6.3).

True exactly-once delivery over an unreliable network is impossible in general, because the sender cannot know whether a lost acknowledgment means the message or only the acknowledgment was lost. Systems that advertise “exactly-once” achieve it by combining at-least-once delivery with deduplication, so that processing the same message twice has the same effect as processing it once. Figure 6.1 shows the problem and its solution.

An operation is *idempotent* if applying it several times has the same effect as applying it once. Setting a value (`SET balance = 100`) is idempotent; incrementing it (`INCR balance`) is not. Non-idempotent operations can be made safe by attaching a unique *idempotency key* that the server uses to detect repeats, as in Listing 6.1.

Table 6.3: Message delivery semantics.

Semantics	**Behavior**	**Consequence**
At-most-once	Send once, never retry	No duplicates, but messages can be lost
At-least-once	Retry until acknowledged	No loss, but duplicates are possible
Exactly-once (effectively)	At-least-once delivery combined with duplicate suppression	Each message takes effect once; requires idempotence or deduplication

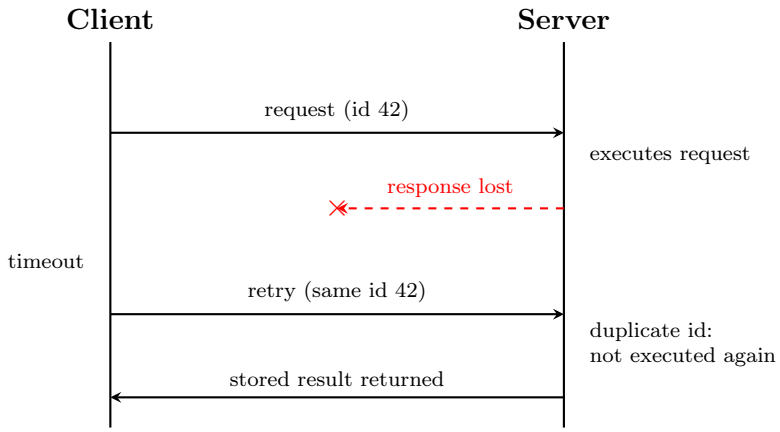


Figure 6.1: An unknown outcome. The client cannot tell whether the request or the response was lost. Retrying with the same request identifier is safe if the server remembers identifiers it has already processed.

Listing 6.1: Retrying with exponential backoff, jitter, and an idempotency key (pseudocode).

```
def charge_card(order_id, amount):
    key = "charge-" + order_id # same key on every retry
    delay = 0.1 # seconds
    for attempt in range(5):
        try:
            return payments.charge(amount, idempotency_key=key,
                                   timeout=2.0)
        except Timeout:
            sleep(delay + random.uniform(0, delay)) # jitter
            delay = delay * 2 # exponential backoff
    raise GiveUp("payment status unknown")
```

Backoff prevents retries from overwhelming a struggling service, and jitter (a random component) prevents many clients from retrying at the same moment. When all attempts fail, the outcome is still unknown, so the application must check the status rather than assume failure.

6.4 Time, Clocks, and Ordering

6.4.1 Physical Clocks

Each machine has a hardware clock that drifts, typically by many milliseconds per day unless corrected. Protocols such as NTP synchronize clocks to a reference, but synchronization is imperfect: accuracy over the internet is generally in the range of milliseconds to tens of milliseconds, and clocks can jump when corrected. Consequently, timestamps from different machines cannot be trusted to order events precisely. This is why last-write-wins conflict resolution (Chapter 4) can silently discard a newer write when clocks are skewed (Kleppmann, 2017). Google's Spanner takes a hardware-based approach, using GPS and atomic clocks to bound clock uncertainty and wait it out before committing (Corbett et al., 2012), but this is a specialized, costly design.

6.4.2 Logical Clocks

Often what matters is not the exact time, but the *order* of events. Lamport observed that events can be ordered by causality: event a *happens before* event b ($a \to b$) if a precedes b on the same process, or a is the sending of a message and b its receipt, or by transitivity (Lamport, 1978). Events with no such relationship are *concurrent*.

A *Lamport clock* gives each process a counter L updated by these rules:

Local event or send:	$L \leftarrow L + 1$ (attach L to any message sent),
Receive message with timestamp m :	$L \leftarrow \max(L,\, m) + 1$.

If $a \to b$, then $L(a) < L(b)$. The converse does not hold: a smaller timestamp does not prove that one event caused the other, so Lamport clocks cannot detect concurrency.

Vector clocks fix this by keeping one counter per process, so each event carries a vector $V = (V_1, \ldots, V_n)$. A vector V is *less than or equal to* W if every component of V is at most the corresponding component of W. Event a happened before b if $V(a) \le V(b)$ and $V(a) \neq V(b)$; if neither vector is less than or equal to the other, the events are concurrent. Dynamo-style stores use this idea to detect conflicting concurrent writes (Chapter 3). Table 6.4 compares the two.

6.5 Consistency and Replication

6.5.1 Why Replicate

Replication keeps copies of data on several nodes, to survive failures, to serve reads from nearby or numerous machines, and to scale read throughput. The difficulty is keeping the copies in agreement while updates arrive concurrently and nodes fail.

6.5.2 Consistency Models

A *consistency model* is a contract about which values a read may return. Stronger models are easier to program against but cost more latency and availability. Table 6.5 lists the main

Table 6.4: Lamport clocks compared with vector clocks.

Aspect	**Lamport clock**	**Vector clock**
Size per timestamp	One number	One number per process
Captures causality	If $a \to b$ then $L(a) < L(b)$	$a \to b$ if and only if $V(a) < V(b)$
Detects concurrency	No	Yes
Typical use	Total ordering of events	Conflict detection in replicated data

models from strongest to weakest.

Table 6.5: Selected consistency models, from strongest to weakest.

Model	**Guarantee**	**Typical cost or example**
Linearizability	Every operation appears to take effect instantly at one point between its start and finish, so the system behaves like a single copy	Coordination on every operation; etcd, single-leader systems with synchronous reads
Sequential consistency	All processes see operations in the same order, which respects each process's own order, but not necessarily real time	Slightly weaker than linearizability
Causal consistency	Operations that are causally related are seen in the same order by everyone; concurrent ones may differ	Tracking dependencies
Eventual consistency	If updates stop, all replicas eventually converge	Fast and available; stale reads possible

Note that the common phrase "all nodes see the same data at the same time" describes the strongest of these models only; in practice, replicas always differ briefly, and the design question is how visible the difference is. Weaker models also have practical session guarantees that make applications easier to build: *read-your-writes* (a client sees its own updates), *monotonic reads* (a client never sees data go backward in time), and *consistent prefix* (a client sees updates in the order they happened).

6.5.3 Replication Strategies

Three architectures are common (Table 6.6).

Table 6.6: Replication strategies.

Strategy	How it works	Trade-offs
Single-leader (primary-backup)	All writes go to one leader, which streams changes to followers	Simple and consistent; the leader is a bottleneck and needs failover (Redis, MongoDB replica sets, HBase regions)
Multi-leader	Several leaders accept writes, typically one per data center, and exchange changes	Low write latency in each region and tolerance of data-center outages; write conflicts must be resolved
Leaderless	Any replica accepts writes; reads and writes use quorums	High availability; relies on quorums, read repair, and conflict resolution (Cassandra, Dynamo)

Replication can be *synchronous* (the leader waits for followers to confirm, which is safer but slower and blocks if a follower is down) or *asynchronous* (the leader does not wait, which is faster but allows *replication lag* and loss of recent writes if the leader fails). Many systems use a mix, such as waiting for one follower and replicating asynchronously to the rest. A related, more general approach is *state machine replication*, in which replicas apply the same ordered log of commands, so they end in the same state. Consensus protocols (Section 6.6) are what agree on that order.

Leader-based systems face the risk of *split brain*: after a partition or a false failure suspicion, two nodes may both believe they are the leader and accept conflicting writes. Remedies include requiring a majority to elect a leader, and *fencing tokens*, increasing numbers that storage systems use to reject requests from a former leader (Kleppmann, 2017).

6.5.4 Quorums and the CAP Theorem

With N replicas, writing to W and reading from R replicas guarantees that a read overlaps the latest write if $R + W > N$, as introduced in Chapter 1 and applied in Chapter 4. The CAP theorem, proved by Gilbert and Lynch (2002), is often misquoted as "pick two of three". Because partitions cannot be ruled out, the real statement is that *during a network partition* a system must choose between consistency (linearizability) and availability, as discussed in Section 1.5. PACELC refines this by noting the latency–consistency trade-off even when there is no partition (Abadi, 2012).

6.5.5 Conflict Resolution

When replicas accept concurrent writes, conflicts must be resolved. Options include:

- **Last-write-wins (LWW):** keep the write with the highest timestamp. It is simple but can lose updates and depends on clocks.
- **Version vectors:** detect concurrent writes and either keep all versions (siblings) for the application to merge, or reject one.

- **Application-level merging:** domain logic combines the versions (for example, merging two shopping carts by taking the union of their items).
- **Conflict-free replicated data types (CRDTs):** data structures such as counters and sets designed so that concurrent updates merge automatically and deterministically, with replicas converging without coordination (Shapiro et al., 2011).

6.6 Consensus, Fault Tolerance, and Reliability

6.6.1 Fault-Tolerance Techniques

Fault tolerance combines several techniques: *redundancy* (replicating data and services), *failure detection* (Section 6.2), *failover* (promoting a replica to replace a failed primary), and *recovery*. A common recovery technique is *checkpointing*, which periodically saves the state of a computation so that, after a failure, it restarts from the last checkpoint instead of from the beginning. Databases combine a checkpoint with a write-ahead log, as seen with the commit logs of Chapter 4: the log records changes since the checkpoint and is replayed after a crash. The overall goal is a system that fails gracefully, degrades in a controlled way, and recovers automatically.

6.6.2 The Consensus Problem

Many coordination tasks reduce to *consensus*: getting a set of nodes to agree on a single value even if some nodes fail. A correct consensus algorithm provides *agreement* (no two nodes decide differently), *validity* (the decided value was proposed by some node), and *termination* (every non-failed node eventually decides). Consensus underlies leader election, agreement on the order of log entries, atomic commit, and configuration management.

An important theoretical result, the FLP impossibility theorem, shows that in a purely asynchronous system, no deterministic algorithm can guarantee consensus if even one node may crash (Fischer et al., 1985). Practical algorithms therefore rely on timeouts, which assume partial synchrony: they always remain *safe* (never decide inconsistently) but guarantee *liveness* (making progress) only when the network is sufficiently well behaved.

6.6.3 Paxos and Raft

Paxos, introduced by Lamport (1998), was the first widely studied fault-tolerant consensus algorithm and is the basis of many production systems, but it is notoriously difficult to understand and to implement completely. *Raft* was designed explicitly for understandability and is used in etcd, Consul, and many databases (Ongaro & Ousterhout, 2014). Raft decomposes consensus into three parts:

- **Leader election.** Time is divided into *terms*. A node starts as a *follower*. If it hears nothing from a leader before a randomized election timeout, it becomes a *candidate*, increments the term, and requests votes. A candidate that receives votes from a majority becomes the leader.

- **Log replication.** The leader appends client commands to its log and sends them to followers. An entry is *committed* once a majority of nodes have stored it, and then applied to the state machine.
- **Safety.** Rules guarantee that a new leader always holds all committed entries, so committed data is never lost.

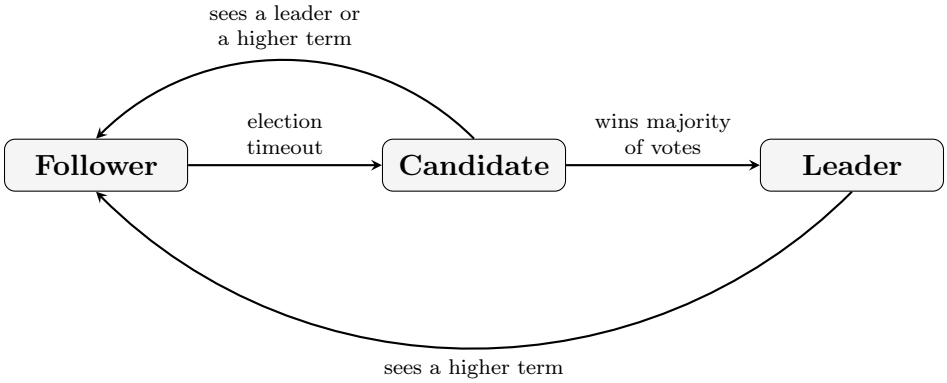


Figure 6.2: Node roles in Raft. A node starts as a follower, becomes a candidate if it hears no leader, and becomes leader by winning a majority of votes.

Because all decisions need a majority, consensus-based systems tolerate failures up to a limit. To tolerate f crash failures, a cluster needs $2f + 1$ nodes (Table 6.7). Byzantine failures are costlier: tolerating f Byzantine nodes requires $3f + 1$ nodes (Lamport et al., 1982).

Table 6.7: Majority quorums and tolerated crash failures.

Cluster size n	Majority needed	Crash failures tolerated
3	2	1
5	3	2
7	4	3

Even-sized clusters add cost without adding tolerance: four nodes still tolerate only one failure, since a majority is three. For this reason, consensus clusters are usually sized at three, five, or seven nodes, and the cost of each write grows with the need to reach a majority, which is why consensus is used for small, critical state (configuration, leadership, metadata) rather than for bulk data.

6.6.4 Distributed Transactions and Atomic Commit

When one transaction updates data on several nodes, all participants must either commit or abort. *Two-phase commit* (2PC) achieves this with a coordinator:

1. **Prepare.** The coordinator asks every participant whether it can commit. Each participant makes its changes durable and votes yes or no.

2. **Commit or abort.** If all vote yes, the coordinator records the decision and tells everyone to commit; otherwise it tells everyone to abort.

The weakness of 2PC is blocking: a participant that has voted yes must wait for the coordinator's decision, and if the coordinator fails at that moment, the participant holds its locks until the coordinator recovers. Atomic commit is related to, but different from, consensus; fault-tolerant systems run the coordinator on a consensus-replicated service to avoid the single point of failure. Because distributed transactions are slow and reduce availability, many NoSQL systems restrict transactions to a single record or partition (Chapter 5). Where a multi-step business process spans services, the *saga* pattern is often used instead: a sequence of local transactions, each with a *compensating action* that undoes it, so a failure midway is handled by running the compensations (Garcia-Molina & Salem, 1987). Sagas give up isolation (intermediate states are visible) in exchange for availability.

6.7 Scalability and Performance

6.7.1 Measuring Performance

Scalability is the ability to handle growing load by adding resources. To discuss it, we need measurements:

- **Throughput:** work completed per unit time, such as requests per second.
- **Latency:** the time to complete one request. Report *percentiles* rather than averages: the median (p50) shows the typical case, while p95, p99, and p99.9 show the slowest requests.
- **Saturation and errors:** how full the resources are, and how often requests fail. Together with latency and traffic, these form the "four golden signals" of monitoring (Beyer et al., 2016).

6.7.2 Tail Latency

Averages hide the slow requests that users notice. Tail latency matters even more in distributed systems because a single user request often fans out to many servers and must wait for the slowest. If each of 100 servers is slow for 1% of requests, the probability that at least one is slow is $1 - 0.99^{100} \approx 63\%$, so most user requests experience slowness (Dean & Barroso, 2013). Mitigations include hedged requests (sending a second request if the first is slow), timeouts with sensible retries, and reducing variability in the sources of delay.

6.7.3 Two Useful Laws

Little's law relates the number of requests in a system L, the arrival rate λ, and the average time each request spends in the system W:

$$L = \lambda W.$$

For example, a service receiving 2,000 requests per second with an average latency of 50 ms has $L = 2000 \times 0.05 = 100$ requests in flight on average, which helps size thread pools and connection limits.

Amdahl's law bounds the speedup from parallelism. If a fraction p of the work can be parallelized across N processors and the remainder is serial, the speedup is

$$S(N) = \frac{1}{(1-p) + p/N}.$$

With $p = 0.95$, ten processors give $S(10) = 1/(0.05 + 0.095) \approx 6.9$, and one hundred give about 16.8, while even infinitely many can never exceed $1/0.05 = 20$. The serial fraction, whether it is a lock, a single leader, or a coordination step, sets a ceiling on scalability.

6.7.4 Techniques for Scalability

Table 6.8 summarizes the main techniques and where they appear in this book.

Table 6.8: Techniques for scalability and performance.

Technique	Purpose	Notes
Horizontal scaling	Add nodes to increase capacity	Requires partitioning and coordination
Partitioning (sharding)	Split data across nodes so work runs in parallel	Choose keys that spread load (Chapters 3 and 4)
Replication	Serve reads from many copies; survive failures	Brings consistency trade-offs
Load balancing	Spread requests across nodes (round-robin, least connections, consistent hashing)	Hardware or software; health checks remove failed nodes
Caching	Avoid repeated computation or database reads	Needs invalidation and expiry (Chapter 3)
Asynchronous processing	Queue work and smooth bursts	Adds delay; needs idempotent consumers
Backpressure and rate limiting	Protect overloaded services by slowing or rejecting excess load	Prevents cascading failure

Scalability is not only about adding hardware. Systems must also avoid shared bottlenecks (a single leader, a hot key, a global lock), degrade gracefully under overload, and be monitored continuously, so that performance problems are found as load grows.

Summary

A distributed system is a set of independent machines cooperating over an unreliable network. It offers scalability, availability, and low latency, but introduces partial failures, unreliable

and delayed messages, unsynchronized clocks, and uncertainty about which nodes are alive. Designers therefore state explicit timing and failure models, use timeouts and heartbeats for failure detection, and handle communication failures with retries, backoff, idempotent operations, and deduplication. Because physical clocks are imprecise, logical and vector clocks capture causal order. Replication improves availability and performance but forces a choice among consistency models, replication strategies (single-leader, multi-leader, leaderless), and conflict-resolution techniques; quorums and the CAP theorem describe the trade-offs. Consensus algorithms such as Paxos and Raft let nodes agree despite crashes, needing $2f + 1$ nodes to tolerate f failures, while two-phase commit and sagas coordinate multi-node transactions. Finally, scalability depends on measuring throughput and tail latency, and on techniques such as partitioning, load balancing, caching, and backpressure, within limits set by laws such as Amdahl's. These foundations underlie the behavior of the database systems in the rest of the book.

Review Questions

1. What are the main characteristics and motivations of distributed systems? Why is partial failure the central difficulty?

2. Name four of the eight fallacies of distributed computing and explain how each can cause failures.

3. Why can a node not reliably tell whether a peer has crashed or is merely slow? How do heartbeats and timeouts address this, and what are the trade-offs?

4. Explain the difference between synchronous and asynchronous communication. What does an RPC hide, and what can go wrong?

5. Compare at-most-once, at-least-once, and effectively exactly-once delivery. What is an idempotent operation, and why does it matter for retries?

6. What does the happens-before relation capture, and how do vector clocks improve on Lamport clocks?

7. Compare linearizability, causal consistency, and eventual consistency. Which would you use for a bank balance, and which for a "likes" counter?

8. Compare single-leader, multi-leader, and leaderless replication. What is split brain?

9. What is the consensus problem? Why does Raft need a majority, and how many failures can a five-node cluster tolerate?

10. Why can two-phase commit block, and how do sagas avoid some of its costs?

11. What is tail latency, and why does it matter more in systems that fan out requests to many servers?

Exercises

1. **Delivery semantics.** A payment service receives a request, charges the card, and its response is lost. Describe what the client sees under at-most-once and at-least-once delivery, and design an idempotency scheme that prevents a double charge.

2. **Lamport clocks.** Processes P_1 and P_2 start with $L = 0$. P_1 performs a local event, then sends a message to P_2, then performs another local event. P_2 performs a local event before the message arrives, then receives it. Give the Lamport timestamp of every event.

3. **Vector clocks.** Three events have vector clocks $(2, 1, 0)$, $(1, 2, 0)$, and $(2, 2, 1)$. Which pairs are causally ordered and which are concurrent?

4. **Quorums.** For $N = 5$, evaluate $(R, W) = (2, 2)$, $(3, 3)$, and $(1, 5)$. Which guarantee that reads see the latest write? How many replica failures can each tolerate for reads and for writes?

5. **Consensus sizing.** Your team proposes a four-node consensus cluster to "be safer than three". Explain why this does not improve fault tolerance, and recommend a size to tolerate two simultaneous failures.

6. **Amdahl's law.** A job is 90% parallelizable. Compute the speedup with 8, 64, and 1,024 processors, and the theoretical maximum. What does this imply about adding more machines?

7. **Tail latency.** A page request calls 50 services in parallel, each exceeding 200 ms on 2% of calls. What fraction of page requests are expected to see at least one slow call? Suggest two mitigations.

References

- Abadi, D. (2012). Consistency Tradeoffs in Modern Distributed Database System Design: CAP is Only Part of the Story. *IEEE Computer*, 45(2), 37–42.

- Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (Eds.) (2016). *Site Reliability Engineering*. O'Reilly Media.

- Corbett, J. C., Dean, J., Epstein, M., et al. (2012). Spanner: Google's Globally-Distributed Database. In *Proceedings of OSDI '12*.

- Coulouris, G., Dollimore, J., Kindberg, T., & Blair, G. (2011). *Distributed Systems: Concepts and Design* (5th ed.). Addison-Wesley.

- Dean, J., & Barroso, L. A. (2013). The Tail at Scale. *Communications of the ACM*, 56(2), 74–80.

- Fischer, M. J., Lynch, N. A., & Paterson, M. S. (1985). Impossibility of Distributed Consensus with One Faulty Process. *Journal of the ACM*, 32(2), 374–382.
- Garcia-Molina, H., & Salem, K. (1987). Sagas. In *Proceedings of ACM SIGMOD '87*, 249–259.
- Gilbert, S., & Lynch, N. (2002). Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services. *ACM SIGACT News*, 33(2), 51–59.
- Hayashibara, N., Défago, X., Yared, R., & Katayama, T. (2004). The φ Accrual Failure Detector. In *Proceedings of the IEEE Symposium on Reliable Distributed Systems (SRDS)*.
- Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.
- Lamport, L. (1978). Time, Clocks, and the Ordering of Events in a Distributed System. *Communications of the ACM*, 21(7), 558–565.
- Lamport, L. (1998). The Part-Time Parliament. *ACM Transactions on Computer Systems*, 16(2), 133–169.
- Lamport, L., Shostak, R., & Pease, M. (1982). The Byzantine Generals Problem. *ACM Transactions on Programming Languages and Systems*, 4(3), 382–401.
- Ongaro, D., & Ousterhout, J. (2014). In Search of an Understandable Consensus Algorithm (Raft). In *Proceedings of USENIX ATC '14*.
- Shapiro, M., Preguiça, N., Baquero, C., & Zawirski, M. (2011). Conflict-Free Replicated Data Types. In *Proceedings of SSS 2011*, 386–400.
- Tanenbaum, A. S., & Van Steen, M. (2017). *Distributed Systems* (3rd ed.). distributed-systems.net.
- Waldo, J., Wyant, G., Wollrath, A., & Kendall, S. (1994). *A Note on Distributed Computing*. Sun Microsystems Laboratories, TR-94-29.

Chapter 7

The CAP Theorem and System Trade-offs

Learning Objectives

After reading this chapter, you will be able to:

- State the CAP theorem precisely, sketch why it holds, and avoid its common misinterpretations.
- Explain PACELC and the latency–consistency trade-off that exists even without partitions.
- Quantify how consistency settings affect availability and latency.
- Contrast ACID and BASE, and relate transaction isolation to the trade-offs.
- Design systems that detect partitions, limit risk during them, and recover afterwards.
- Identify which operations and invariants need coordination and which do not.
- Apply a structured framework to choose trade-offs for each component of an application.

7.1 Why Trade-offs Are Unavoidable

Every design in this book involves a trade-off. Replication improves availability but creates the problem of keeping copies consistent. Caching reduces latency but risks stale data. Denormalization speeds up reads but complicates writes. Chapters 1 and 6 introduced the CAP theorem and the main consistency models; this chapter examines them in depth and turns them into practical design guidance.

The central message is that there is no universally best configuration. A bank ledger, a shopping cart, and a social media feed require different guarantees, and a single application usually contains components with different needs. The skill lies in understanding precisely what each guarantee costs, and in choosing deliberately instead of accepting the defaults of a product.

7.2 The CAP Theorem

7.2.1 Origins

Eric Brewer presented CAP as a conjecture in a 2000 keynote on distributed systems (Brewer, 2000). Seth Gilbert and Nancy Lynch gave a formal proof two years later, for a precisely defined model (Gilbert & Lynch, 2002). The theorem concerns replicated data held by several nodes that communicate over an unreliable network.

7.2.2 The Three Properties, Precisely

The informal slogans for C, A, and P are easy to misread. Table 7.1 gives the informal meaning and the more careful meaning that the theorem actually uses.

Table 7.1: The properties in the CAP theorem.

Property	Informal reading	Precise meaning in the theorem
Consistency (C)	"Everyone sees the same data"	Linearizability: operations appear to execute one at a time in an order consistent with real time, so every read returns the latest completed write
Availability (A)	"The system is up"	Every request received by a non-failing node eventually receives a non-error response, with no guarantee about how fast
Partition tolerance (P)	"The system survives network problems"	The system continues to function when the network drops or delays an arbitrary number of messages between nodes

Two points deserve emphasis. First, "consistency" in CAP is not the "C" of ACID, which refers to preserving application-defined integrity rules. Second, "availability" in CAP is a binary property of the algorithm (every request to a live node gets an answer), not the percentage of uptime promised by a service-level agreement.

7.2.3 The Theorem and a Proof Sketch

> *In an asynchronous network, it is impossible for a replicated service to provide linearizable consistency and availability in the presence of network partitions.*

The argument is short. Suppose two nodes, G_1 and G_2, both store a value v that is initially v_0, and the network between them is partitioned so that no messages pass.

1. A client writes v_1 to G_1. Because the system is available, G_1 must accept the write and respond.
2. Afterward, another client reads v from G_2. Because the system is available, G_2 must respond, without waiting for the partition to heal.
3. G_2 has received no message about the write, so it cannot return v_1. If it returns v_0, the read is stale and linearizability is violated.
4. The only alternatives are to make G_2 refuse or wait indefinitely, which violates availability.

So, during a partition, a system must give up either consistency or availability. Figure 7.1 summarizes the choices facing a replica, including the extension discussed in Section 7.3.

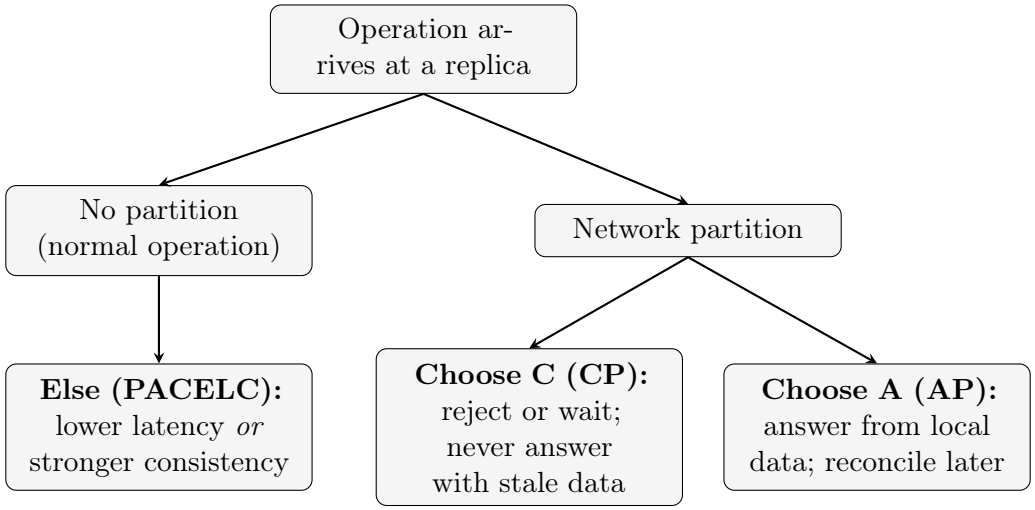


Figure 7.1: The choices behind CAP and PACELC. Only during a partition is the choice between consistency and availability forced; at other times the trade-off is between latency and consistency.

7.2.4 Common Misconceptions

The theorem is frequently misquoted. The most common errors are:

- **"Pick any two of three."** Partitions are a fact of life in any network, so partition tolerance is not optional. The real choice is between C and A, and only while a partition lasts (Brewer, 2012).
- **"CA systems exist."** A single-node database is trivially consistent and available, but it is not a distributed system, so CAP does not apply. A distributed system cannot choose to "forgo" partitions.
- **"A database is either CP or AP."** The choice is made per operation and per configuration. The same database may offer strongly consistent reads for one query and eventually consistent reads for another. Kleppmann argues that the labels CP and

AP are too coarse to be useful and that the theorem's definitions are narrower than they seem (Kleppmann, 2015).

- **"Partitions are rare, so CAP does not matter."** Partitions may be rare, but the design decisions that determine what happens during one also shape everyday latency and complexity (Section 7.3). Real networks do partition, and partial partitions, in which only some nodes cannot talk to each other, are especially troublesome.

- **"Giving up consistency means chaos."** Systems that choose availability typically provide weaker but well-defined guarantees, such as eventual or causal consistency, and handle conflicts in a controlled way.

CAP Is Narrower Than It Looks

CAP considers one consistency model (linearizability), one availability definition, and one failure type (partitions). It says nothing about latency, about other consistency models, about crash failures, or about throughput. It is a useful starting point but only a fraction of the trade-off space.

7.3 Beyond CAP: Latency, PACELC, and Tunable Consistency

7.3.1 PACELC

Abadi observed that CAP says nothing about the common case in which the network is healthy, yet systems still face a trade-off then: stronger consistency requires coordination among replicas, and coordination takes time (Abadi, 2012). His PACELC formulation reads: if there is a **P**artition, choose between **A**vailability and **C**onsistency; **E**lse, choose between **L**atency and **C**onsistency. A system is therefore described by two pairs of letters: its behavior during partitions (PA or PC) and its behavior otherwise (EL or EC).

The latency cost is concrete. A write that must be acknowledged by a replica on another continent pays at least one inter-continental round trip, which is on the order of a hundred milliseconds, no matter how fast the hardware is. A write acknowledged only locally takes a millisecond or less, but remote replicas learn of it later. No engineering can remove this trade-off, because it is a consequence of the finite speed of signals and the need to coordinate.

Table 7.2 places several systems in this classification. These are commonly cited default orientations; most are configurable, so the table is a starting point for reading the documentation, not a substitute for it.

Spanner deserves a comment, because Brewer himself has discussed it: it is formally a CP system, but with redundant private networks, careful operations, and a very low rate of partitions it delivers availability so high that, in practice, users rarely perceive the trade-off (Brewer, 2017). This illustrates that CAP governs what the system *does* in a partition, while engineering determines *how often* that choice must be made.

Table 7.2: Typical PACELC orientation of selected systems (defaults; configurable in many cases).

System	Partition	Else	Notes
Cassandra	PA	EL	Tunable per query; quorum levels move it toward EC
Amazon DynamoDB	PA	EL	Eventually consistent reads by default; strongly consistent reads optional
HBase	PC	EC	Strongly consistent per row; region briefly unavailable on failover
MongoDB	Depends	Depends	With majority write and read concerns it behaves as PC/EC; weaker settings favor availability and latency
Google Spanner	PC	EC	Synchronous replication with consensus; uses bounded clock uncertainty to give external consistency (Corbett et al., 2012)
etcd	PC	EC	Consensus-based (Raft); small, critical data

7.3.2 Tunable Consistency

Many databases expose the trade-off as configuration. In quorum-based replication with N replicas, writing to W and reading from R replicas gives overlap, and therefore up-to-date reads, when $R+W > N$ (Chapters 1 and 4). Lowering R or W gives faster and more available operations that may be stale. This is the practical form of the trade-off: the application selects, operation by operation, how much latency and availability to spend on consistency.

7.3.3 Quantifying the Availability Cost

The effect on availability can be calculated. Suppose each of $N = 3$ replicas is independently available with probability $p = 0.99$, and an operation needs a certain number of replicas to respond. Then:

- **One replica suffices** (level `ONE`): the operation fails only if all three are down, so availability is $1 - (1 - p)^3 = 1 - 10^{-6} = 99.9999\%$.
- **A majority is needed** (`QUORUM`, two of three): availability is the probability that at least two are up, $3p^2(1 - p) + p^3 \approx 99.9702\%$.
- **All replicas are needed** (`ALL`): availability is $p^3 \approx 97.03\%$.

Table 7.3 converts these to expected unavailability per year.

Table 7.3: Availability of an operation needing k of 3 replicas, each 99% available (independent failures).

Replicas needed	Availability	Unavailable per year	Consistency
1 of 3 (`ONE`)	99.9999%	about 32 seconds	Weakest
2 of 3 (`QUORUM`)	99.9702%	about 2.6 hours	Strong if $R + W > N$
3 of 3 (`ALL`)	97.03%	about 10.8 days	Strongest

The calculation assumes independent failures. In reality, failures are correlated (a shared power supply, a software bug that hits all nodes, a regional outage), so true availability is lower. Still, the pattern is instructive: requiring more replicas to respond improves consistency but multiplies unavailability, and the middle option (a majority quorum) often gives the best balance.

7.3.4 Harvest and Yield

CAP treats availability as all-or-nothing, but systems can degrade partially. Fox and Brewer proposed two measures: *yield*, the fraction of requests that receive an answer, and *harvest*, the fraction of the complete data that is reflected in an answer (Fox & Brewer, 1999). Under failure, a system can sacrifice harvest to preserve yield. For example, a search engine that has lost one shard of its index can still return results from the remaining shards, flagged as possibly incomplete, instead of failing the request. This reframes the trade-off: instead of choosing between a perfect answer and an error, a system may choose a slightly degraded answer.

7.4 ACID, BASE, and Isolation

7.4.1 ACID

Relational databases provide transactions with the ACID properties, a term associated with Härder and Reuter (1983): **A**tomicity (all or nothing), **C**onsistency (a transaction takes the database from one valid state to another, according to integrity constraints), **I**solation (concurrent transactions do not interfere), and **D**urability (committed data survives crashes). On a single node these properties are comparatively cheap, but providing them across nodes requires coordination such as two-phase commit (Chapter 6), which costs latency and availability.

7.4.2 BASE

Many large-scale systems instead follow the BASE philosophy (Pritchett, 2008): **B**asically **A**vailable, **S**oft state, and **E**ventually consistent. Under BASE, the system remains responsive despite partial failures, state may change over time without new input as replicas converge, and consistency is reached eventually (Vogels, 2009). BASE is a design stance rather than a

precise guarantee, and its main burden falls on application developers, who must tolerate and sometimes repair temporary inconsistency. Table 7.4 contrasts the approaches.

Table 7.4: ACID compared with BASE.

Aspect	**ACID**	**BASE**
Priority	Correctness of each transaction	Availability and scalability
Consistency	Strong, immediately after commit	Eventual, with a window of inconsistency
Failure behavior	May refuse or block when it cannot coordinate	Continues operating on available replicas
Complexity borne by	The database	The application
Typical use	Financial ledgers, order processing	Feeds, carts, catalogs, telemetry

The two are ends of a spectrum rather than opposites, and modern systems blend them. Many NoSQL databases now offer ACID transactions within a limited scope, such as a single document or partition, while some relational systems offer replicated, partition-tolerant modes.

7.4.3 Isolation Levels

Even within ACID, the "I" is a spectrum of trade-offs between safety and performance. Table 7.5 lists three widely used levels. Stronger isolation prevents more anomalies but reduces concurrency.

Table 7.5: Selected transaction isolation levels.

Level	**Guarantee**	**Anomalies that can remain**
Read committed	Reads see only committed data; writes overwrite only committed data	Non-repeatable reads; some write conflicts
Snapshot isolation	Each transaction sees a consistent snapshot of the database taken at its start	Write skew (two transactions each read overlapping data and write disjoint data, jointly violating a rule)
Serializability	The outcome equals some serial execution of the transactions	None, at the highest cost in concurrency

The lesson is that "strong consistency" is not a single setting. The consistency of replicas (CAP's C) and the isolation of concurrent transactions (ACID's I) are different dimensions, and a complete design considers both (Kleppmann, 2017).

7.5 Designing for Partitions

If partitions will happen, what should the system do? Brewer suggested treating a partition as a mode of operation to be managed, instead of a failure to be avoided (Brewer, 2012). Figure 7.2 shows the lifecycle.

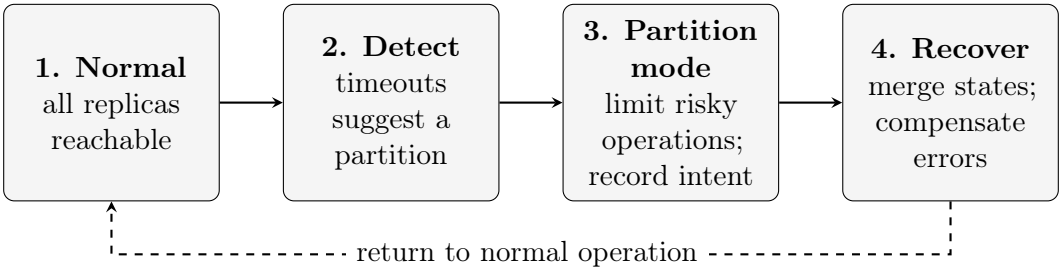


Figure 7.2: Managing a partition: detect it, continue in a restricted mode that records what happened, then merge and compensate when connectivity returns.

7.5.1 Per-Operation Decisions

The right behavior depends on the operation. Table 7.6 gives examples.

7.5.2 Compensating Actions

Availability-oriented systems often accept an operation that may later turn out to violate a business rule, and then fix the problem afterwards. Familiar examples are airlines overbooking flights and compensating passengers, banks charging fees for overdrafts instead of blocking every transaction at the point of sale, and retailers refunding an order that could not be fulfilled. This works when mistakes are rare, the cost of compensation is lower than the cost of refusing or delaying many operations, and compensation is possible. The same idea underlies sagas (Chapter 6).

7.5.3 When Is Coordination Necessary?

Coordination is the expensive ingredient, so the useful question is: which operations genuinely need it? Research on *invariant confluence* shows that whether an operation can safely run without coordination depends on the invariant (the rule) it must preserve (Bailis et al., 2014). Related work, the CALM principle, states that programs whose logic is *monotonic*, meaning that more input only adds to the output and never retracts it, can be made consistent without coordination (Hellerstein & Alvaro, 2020). Table 7.7 gives intuition.

Table 7.6: Choosing consistency or availability by operation.

Operation	Favor	Reasoning
Transfer money between accounts	Consistency	Wrong balances or double spending are costly; refusing temporarily is acceptable
Add an item to a shopping cart	Availability	A lost or delayed cart update is minor; the Dynamo design was motivated by cart availability (DeCandia et al., 2007)
Like or view counters	Availability	Approximate, slightly stale counts are fine
Reserve the last seat or unit	Consistency	Overselling is visible and costly, unless the business can compensate
Update a user profile	Availability	Rare conflicts can be resolved by last-write-wins or a merge
Elect a leader or hold a lock	Consistency	Two leaders (split brain) can corrupt data
Resolve a name (DNS)	Availability	Cached, slightly stale answers are acceptable; availability is vital

The practical conclusion is to apply strong coordination narrowly to the few invariants that need it, such as uniqueness and non-negative balances, while letting the rest of the system run with low-latency, highly available, weakly coordinated operations.

7.6 Other Trade-offs in System Design

CAP and PACELC concern distributed consistency, but designers face many other trade-offs. Table 7.8 lists common ones and where earlier chapters illustrate them.

These dimensions interact. Choosing strong consistency may increase latency and reduce availability; adding replicas to raise availability increases the work of keeping them consistent; denormalizing for read speed raises storage and write cost. No configuration is best on every axis, which is why requirements must be stated explicitly and ranked.

7.7 Evaluating Trade-offs in Practice

Product descriptions and labels such as "CP" or "strongly consistent" are not enough. To know what a system really does:

- **Read the defaults and the failure behavior in the documentation.** Check the default consistency level, how failover works, and what happens to acknowledged writes during a leader change.
- **Test under failure.** Independent analyses, such as the Jepsen project (`https://jepsen.io`), inject network partitions and crashes into real databases and check whether

Table 7.7: Which operations and invariants need coordination (intuition).

Operation or invariant	**Coordination?**	**Why**
Increment a counter; add to a set (a CRDT)	Usually not	Updates commute, so replicas converge in any order (Shapiro et al., 2011)
Insert records with unique identifiers generated per node	Not for the insert	Identifiers from separate ranges or random UUIDs avoid conflicts
Enforce that an identifier or username is unique across nodes	Yes	Two nodes could grant the same name concurrently
Keep an account balance non-negative under withdrawals	Yes	Concurrent withdrawals can each look valid but together overdraw
Delete a record that others reference	Often yes	A concurrent insert could create a dangling reference

the claimed guarantees hold. Such tests have found violations in widely used systems.

- **Inject faults in your own environment.** Practice partitions, node loss, and slow networks in staging, and observe latency percentiles, error rates, and data divergence, not just whether the service stays up.
- **Measure the consistency window.** For eventually consistent systems, measure how long replicas take to converge under normal and peak load, so the application's assumptions are based on data.
- **Define objectives.** Write service-level objectives (for example, 99.9% of reads below 50 ms; reads reflect writes within five seconds) so that trade-offs are judged against explicit targets.

7.8 A Decision Framework

A structured approach turns the concepts of this chapter into decisions. For each component of an application, ask:

1. **What is the cost of stale or conflicting data?** Financial loss, safety risk, or mild annoyance?
2. **What is the cost of unavailability or delay?** Lost sales, user frustration, or breach of an agreement?
3. **What invariants must always hold?** Uniqueness, non-negative balances, referential integrity? Only these need coordination.

Table 7.8: Common trade-off dimensions in database design.

Dimension	**The tension**	**Examples in this book**
Consistency vs. latency	Coordination between replicas adds delay	Quorum levels, synchronous replication
Durability vs. write speed	Flushing every write to disk is safer but slower	Append-only files, commit logs (Chapters 3, 4)
Read vs. write performance	Indexes and precomputed data speed reads but slow writes	Indexing, compaction strategies
Normalization vs. query speed	Duplicated data reads faster but updates are harder	Denormalization (Chapter 5)
Flexibility vs. integrity	Schema-free storage is agile but pushes validation to the application	Schema validation (Chapter 2)
Memory vs. disk cost	RAM is fast but costly and volatile	In-memory databases (Chapter 3)
Simplicity vs. capability	Richer features raise operational complexity	Wide-column and consensus-based systems

4. **What latency target applies, and where are the replicas?** Cross-region coordination has a hard latency floor.

5. **Can mistakes be detected and compensated afterwards?** If so, favor availability and reconcile.

6. **What can the team operate?** A system that is correct on paper but beyond the team's operational ability is a poor choice.

Table 7.9 applies the framework to the components of an online store, showing that one application can use several different positions.

This is polyglot persistence (Chapter 1) guided by trade-offs instead of by tools: each component gets the weakest guarantee that is still safe, and therefore the best latency and availability available to it.

Summary

The CAP theorem states that a replicated service cannot be both linearizable and available during a network partition. Because partitions cannot be excluded, the real choice is between consistency and availability, and only while a partition lasts; the theorem uses narrow definitions and says nothing about latency or other failures. PACELC adds that even without partitions there is a trade-off between latency and consistency, which follows from the cost of

Table 7.9: Trade-off decisions for the components of an online store.

Component	Likely choice	Rationale
Shopping cart	Highly available key-value or document store; merge conflicts	Never block shopping; occasional merge anomalies are cheap
Payment and order placement	Transactional store with strong consistency	Correctness outweighs a brief refusal
Inventory	Reserve stock with expiry; coordinate on the final decrement	Avoid overselling without blocking browsing
Product catalog and search	Replicated, eventually consistent, cached	Reads dominate; a few seconds of staleness is acceptable
Sessions	In-memory key-value store with a time to live	Fast; loss is a minor inconvenience
Recommendations	Eventually consistent derived data (for example, a graph)	Approximate and rebuildable from the source of truth

coordination. Quorum settings make these trade-offs configurable, and simple calculations show how requiring more replicas to respond improves consistency but reduces availability. ACID and BASE represent two design stances, and isolation levels show that even "strong" consistency has gradations. Rather than treating partitions only as failures, designers can detect them, operate in a restricted mode, and recover through merging and compensation, applying coordination only to the invariants that need it. Other trade-offs, such as durability against write speed or normalization against query speed, complete the picture. Choices should be made per component, with explicit requirements, tested under failure, and checked against documentation rather than labels.

Review Questions

1. State the CAP theorem precisely. What do consistency, availability, and partition tolerance mean in the theorem, and how do they differ from their everyday meanings?

2. Walk through the proof sketch with two nodes. Why can a system not remain both consistent and available during a partition?

3. Explain why "pick any two of three" and "a database is either CP or AP" are misleading.

4. Why is a single-node database not a counterexample to, or an instance of, CAP?

5. What does PACELC add to CAP, and why does the latency–consistency trade-off exist even without partitions?
6. For $N = 3$ replicas, each 99% available, compute the availability of an operation that needs one, two, or three replicas. What assumption does the calculation make, and when does it fail?
7. What are harvest and yield? Give an example of trading harvest for yield.
8. Compare ACID and BASE. Why is "consistency" in ACID different from "consistency" in CAP?
9. Describe the lifecycle for managing a partition. What is a compensating action, and when is it appropriate?
10. Which invariants require coordination, and which operations can run without it? Give two examples of each.
11. Why should you test a database under failures instead of relying on its documentation alone?

Exercises

1. **Classify.** For each system or configuration, state its behavior during a partition and in normal operation (PACELC) and justify it: (a) Cassandra with `ONE`/`ONE`, (b) Cassandra with `QUORUM`/`QUORUM`, (c) a single-leader database with synchronous replication to one follower, (d) a three-node etcd cluster.
2. **Availability.** Repeat the availability calculation of Table 7.3 for $N = 5$ replicas, each 99% available, for operations needing 1, 3, and 5 replicas. Compare with $N = 3$.
3. **Latency.** A database has replicas in Sydney, Frankfurt, and Virginia with round-trip times of about 280 ms (Sydney–Frankfurt), 200 ms (Sydney–Virginia), and 90 ms (Frankfurt–Virginia). Estimate the minimum latency of a quorum write issued from Sydney, and from Virginia. How would you reduce latency for users in Sydney?
4. **Partition mode.** Design the partition-mode behavior for a hotel booking system with two data centers. Which operations continue, which are restricted, and how do you reconcile double bookings afterwards?
5. **Invariants.** For each rule, decide whether it needs coordination and explain: (a) usernames are unique, (b) every order references an existing customer, (c) page-view counts only increase, (d) inventory never goes below zero.
6. **Critique.** A vendor claims: "Our database is CA, so it is both consistent and available." Evaluate the claim and list questions you would ask the vendor.
7. **Framework.** Apply the decision framework of Section 7.8 to a ride-hailing app with driver location tracking, ride matching, payments, and ratings.

References

- Abadi, D. (2012). Consistency Tradeoffs in Modern Distributed Database System Design: CAP is Only Part of the Story. *IEEE Computer*, 45(2), 37–42.
- Bailis, P., Fekete, A., Franklin, M. J., Ghodsi, A., Hellerstein, J. M., & Stoica, I. (2014). Coordination Avoidance in Database Systems. *Proceedings of the VLDB Endowment*, 8(3), 185–196.
- Brewer, E. (2000). Towards Robust Distributed Systems. Keynote at the ACM Symposium on Principles of Distributed Computing (PODC).
- Brewer, E. (2012). CAP Twelve Years Later: How the "Rules" Have Changed. *IEEE Computer*, 45(2), 23–29.
- Brewer, E. (2017). Spanner, TrueTime and the CAP Theorem. Google Research white paper.
- Corbett, J. C., Dean, J., Epstein, M., et al. (2012). Spanner: Google's Globally-Distributed Database. In *Proceedings of OSDI '12*.
- DeCandia, G., Hastorun, D., Jampani, M., et al. (2007). Dynamo: Amazon's Highly Available Key-value Store. In *Proceedings of SOSP '07*.
- Fox, A., & Brewer, E. A. (1999). Harvest, Yield, and Scalable Tolerant Systems. In *Proceedings of HotOS VII*.
- Gilbert, S., & Lynch, N. (2002). Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services. *ACM SIGACT News*, 33(2), 51–59.
- Härder, T., & Reuter, A. (1983). Principles of Transaction-Oriented Database Recovery. *ACM Computing Surveys*, 15(4), 287–317.
- Hellerstein, J. M., & Alvaro, P. (2020). Keeping CALM: When Distributed Consistency Is Easy. *Communications of the ACM*, 63(9), 72–81.
- Jepsen. Distributed systems safety research. https://jepsen.io
- Kleppmann, M. (2015). A Critique of the CAP Theorem. arXiv:1509.05393.
- Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.
- Pritchett, D. (2008). BASE: An ACID Alternative. *ACM Queue*, 6(3), 48–55.
- Shapiro, M., Preguiça, N., Baquero, C., & Zawirski, M. (2011). Conflict-Free Replicated Data Types. In *Proceedings of SSS 2011*, 386–400.
- Vogels, W. (2009). Eventually Consistent. *Communications of the ACM*, 52(1), 40–44.

Chapter 8

Consistency Models in Practice

Learning Objectives

After reading this chapter, you will be able to:

- Recognize the consistency anomalies that users and applications actually experience.
- Explain session guarantees and the techniques that implement them.
- Configure consistency in real systems such as MongoDB, Cassandra, DynamoDB, Cosmos DB, Redis, etcd, and PostgreSQL.
- Describe how eventually consistent systems converge and how quickly they do so.
- Write application code that tolerates and corrects weak consistency, using conditional writes, idempotence, and the outbox pattern.
- Identify hidden sources of staleness, and test and monitor consistency in production.
- Choose a consistency level for each use case.

8.1 From Theory to Practice

Chapters 6 and 7 defined consistency models, from linearizability to eventual consistency, and showed that they trade consistency against latency and availability. This chapter takes the perspective of the developer who must build applications on real databases. The questions are practical:

- What goes wrong, concretely, when consistency is weak?
- Which settings does my database offer, and what do they really guarantee?
- What can the application do to prevent or repair anomalies?
- How do I find out whether my system behaves the way I assume?

A recurring theme is that consistency is not a single property of a database, but a combination of the database's guarantees, the configuration of each operation, and the way the application uses the results. Many consistency bugs arise not from the database failing

to honor its contract, but from the application assuming a stronger contract than the one it was given (Kleppmann, 2017).

8.2 Anomalies in Practice

A *consistency anomaly* is behavior that would be impossible if the system were a single, instantaneous copy of the data. Weaker models permit more anomalies, so understanding the anomalies is the practical way to understand the models. Table 8.1 lists the ones that most often affect applications.

Table 8.1: Common consistency anomalies and how to prevent them.

Anomaly	**What the user or application sees**	**Typical prevention**
Stale read	A read returns an old value although a newer one exists	Read from the leader, or use a stronger read level
Read-your-writes violation	A user posts a comment, refreshes, and the comment is missing	Route the user's reads to an up-to-date replica; session tokens
Monotonic-read violation	Successive refreshes show newer, then older data, as if time moved backward	Pin a user to one replica; session tokens
Causality violation (consistent prefix)	A reply appears before the question it answers	Causal consistency, or reading all related data from one partition
Lost update	Two concurrent read-modify-write operations overwrite each other	Atomic operations, conditional writes, transactions
Write skew	Two transactions each check a rule, then write different data, jointly breaking the rule	Serializable isolation, or explicit locking of the checked data

8.2.1 Read-Your-Writes Violations

The best-known anomaly occurs with asynchronous replication. A user submits a change, which is written to the leader. The next page load is served by a follower that has not yet received the change, so the user's own update seems to have vanished. Figure 8.1 shows the timeline. The user may resubmit the change, producing duplicates, or conclude that the application is unreliable, even though no data was ever lost.

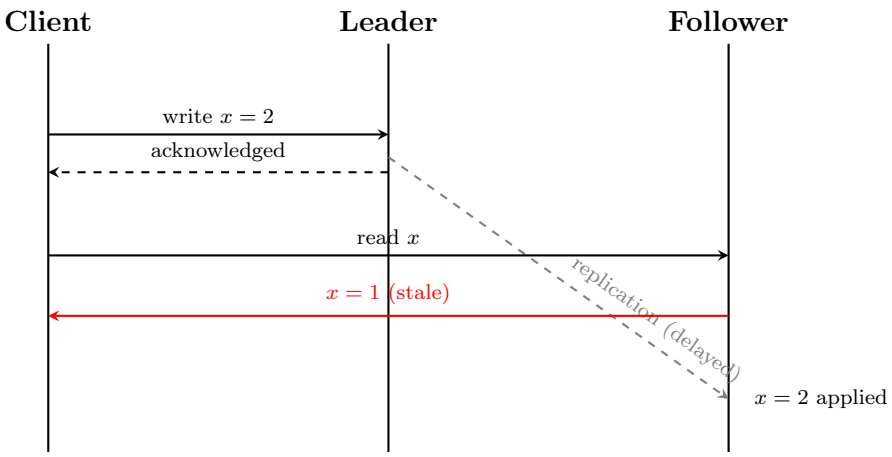


Figure 8.1: A read-your-writes violation. The client's write is acknowledged by the leader, but its next read is served by a follower that has not yet received the replicated update.

8.2.2 Monotonic-Read Violations

If successive reads of the same data are served by different replicas with different amounts of lag, a user may see a comment, refresh the page, and see it disappear, because the second request reached a more lagging replica. Reads appear to travel backward in time. This is usually more disturbing than plain staleness, because it contradicts the user's memory of what they have just seen.

8.2.3 Causality Violations

Related writes may be stored on different partitions that replicate at different speeds. An observer reading from replicas can then see an answer before the question that prompted it. This is a violation of *consistent prefix* reads: each replica should apply writes in an order consistent with the order in which they were issued. The problem arises mostly in sharded systems, where there is no single global order of writes, and it can be avoided by placing causally related data in one partition or by using a causally consistent system.

8.2.4 Lost Updates

A *read-modify-write* sequence is unsafe under concurrency. Suppose the stock of an item is 10, and two clients each read 10, subtract 1 in application code, and write 9. The correct final value is 8, but the second write overwrites the first, and one sale is lost. This anomaly does not need replication at all, because it occurs even on a single node unless the database makes the update atomic. Section 8.6 shows remedies.

8.2.5 Write Skew

Write skew is a subtler anomaly allowed by snapshot isolation (Berenson et al., 1995). Two on-call doctors each check that at least two doctors are on duty and each remove themselves

from the schedule. Each transaction saw a valid state and wrote a different row, so there is no write–write conflict, but together they leave nobody on call. Preventing write skew requires serializable isolation or explicitly locking the data that was read for the check.

8.3 Session Guarantees

Full strong consistency is expensive, while pure eventual consistency can surprise users. *Session guarantees* fill the gap: they promise sensible behavior to a single client or session without requiring global coordination (Terry et al., 1994). Table 8.2 lists the four classic ones.

Table 8.2: Session guarantees (Terry et al., 1994).

Guarantee	**Meaning**	**Prevents**
Read-your-writes	A session's reads reflect its own earlier writes	Updates "disappearing" for the author
Monotonic reads	Once a session has seen a value, it never later sees an older one	Time moving backward
Monotonic writes	A session's writes are applied in the order issued	Updates applied out of order
Writes-follow-reads	A write is ordered after the reads that influenced it	Answers preceding their questions

Together, these guarantees are close to *causal consistency*, in which operations that are causally related are seen in the same order by everyone. Session guarantees and causal consistency are attractive because they can be provided without sacrificing availability, so they remain achievable during a partition (Bailis et al., 2013).

8.3.1 How Session Guarantees Are Implemented

Several techniques are used, singly or combined:

- **Read from the leader.** Serve reads of data the user may have just modified (for example, their own profile) from the leader. Other users' data can come from followers. The cost is more load on the leader.
- **Time-based leader reads.** After a write, route the user's reads to the leader for a short window, such as a minute, longer than the typical replication lag.
- **Sticky sessions.** Route all requests of a user to the same replica, which gives monotonic reads (unless that replica fails). Hashing the user identifier to a replica is a simple implementation.
- **Session tokens.** The system returns a logical timestamp or version with each response. The client sends it with its next request, and a replica serves the read only once it has

caught up to that point, or forwards it elsewhere. Several databases implement this, as the next section shows.

- **Client-side overlay.** The client keeps its own recent writes and merges them into what it displays, or the write response returns the updated object so the interface does not need to re-read it.

8.4 Consistency in Real Systems

Table 8.3 summarizes how widely used systems expose consistency. The details change between versions, so treat it as a map to the documentation, not as a specification.

Table 8.3: Consistency controls in selected systems (check current documentation for defaults).

System	Controls	Typical default	Notes
PostgreSQL (replicas)	Synchronous or asynchronous replication; `synchronous_commit` levels	Asynchronous replicas	Reads on replicas can lag the primary
MongoDB	Write concern, read concern, read preference, causal sessions	Reads from the primary; recent versions default to majority write concern in most deployments	Majority concerns give durability across failover
Cassandra	Consistency level per request; lightweight transactions	Chosen by the client or driver	Quorum levels are strong for sequential operations, not always linearizable
DynamoDB	Strongly consistent reads; conditional writes; transactions	Eventually consistent reads	Strong reads not available on global secondary indexes
Redis	Asynchronous replication; `WAIT` command	Asynchronous	`WAIT` reduces but does not eliminate loss on failover
etcd	Linearizable or serializable reads	Linearizable reads	Serializable reads are faster but may be stale

8.4.1 Relational Replicas: PostgreSQL

In a typical single-primary relational deployment, one primary accepts writes and streams its log to replicas used for read scaling (PostgreSQL Documentation). Replication is usually asynchronous, so a replica can be seconds or more behind the primary, especially under heavy

load or after a network problem. Synchronous replication waits for acknowledgment from at least one replica before a commit returns, which protects against losing committed data on failover but adds latency and makes commits depend on the replica's availability. The application must therefore decide which queries can use replicas (reports, search pages) and which must use the primary (anything that follows a write by the same user, or any decision based on the latest state).

8.4.2 MongoDB: Write Concern, Read Concern, and Causal Sessions

MongoDB (MongoDB Documentation) lets each operation choose three settings:

- **Write concern** sets how many replica-set members must acknowledge a write; `w: "majority"` waits for a majority, and `j: true` waits for the write to reach the on-disk journal.
- **Read concern** sets which data a read may return; `"majority"` returns only data acknowledged by a majority, which will not be rolled back, whereas `"local"` may return data that is later rolled back after a failover. `"linearizable"` gives the strongest, and slowest, reads on the primary.
- **Read preference** chooses which member serves reads (primary, secondaries, or the nearest).

Listing 8.1 shows a debit written durably and read back with majority read concern, and a causally consistent session that provides read-your-writes even when reads are served by secondaries.

Listing 8.1: Write concern, read concern, and a causally consistent session in the MongoDB shell.

```
// Durable write, acknowledged by a majority
db.accounts.updateOne(
  { _id: "acc_1" },
  { $inc: { balance: -50 } },
  { writeConcern: { w: "majority", j: true } }
)

// Read only majority-acknowledged data
db.accounts.find({ _id: "acc_1" }).readConcern("majority")

// Causally consistent session (use majority concerns for the guarantees)
const session = db.getMongo().startSession({ causalConsistency: true })
const accounts = session.getDatabase("bank").accounts
accounts.updateOne({ _id: "acc_1" }, { $set: { nickname: "Savings" } })
accounts.find({ _id: "acc_1" }).readPref("secondaryPreferred")
```

8.4.3 Cassandra: Consistency Levels and Lightweight Transactions

In Cassandra (Apache Cassandra Documentation), the client picks a consistency level for each read and write, from `ONE` to `ALL`, including `QUORUM` and the data-center-aware `LOCAL_QUORUM`,

as described in Chapter 4. Using `QUORUM` for both reads and writes gives $R + W > N$ and so strong consistency for sequential operations. It does not give full linearizability in every case: concurrent writes resolved by timestamps, failed writes that are partially applied, and sloppy quorums can all produce anomalies (Kleppmann, 2017). For operations that need compare-and-set semantics, such as creating a unique username, Cassandra provides *lightweight transactions* (LWT), which run a consensus protocol between replicas and are therefore much slower than ordinary writes (Listing 16.8).

Listing 8.2: Consistency level and lightweight transactions in CQL.

```
CONSISTENCY QUORUM;

-- Succeeds only if the username is not taken
INSERT INTO users (username, email)
VALUES ('ava', 'ava@example.com') IF NOT EXISTS;

-- Compare-and-set on a balance
UPDATE accounts SET balance = 50 WHERE id = 'acc_1' IF balance = 100;
```

8.4.4 DynamoDB: Strong Reads, Conditions, and Transactions

DynamoDB (Amazon DynamoDB Developer Guide) reads are eventually consistent by default, with a per-request option for strongly consistent reads that costs more capacity and has higher latency; strongly consistent reads are not available on global secondary indexes, which are always eventually consistent. *Conditional writes* apply a change only if a condition on the current item holds, which is the basis of optimistic concurrency (Section 8.6). DynamoDB also supports transactions that group several reads or writes across items, subject to limits on their size. Global tables replicate across regions, and concurrent writes in different regions are reconciled automatically, typically by last-writer-wins, so applications that write the same item from several regions need care.

8.4.5 A Spectrum as a Product Feature: Cosmos DB

Azure Cosmos DB exposes consistency as a configurable choice among five named levels, a clear example of the spectrum from Chapter 7 (Azure Cosmos DB Documentation). Table 8.4 lists them.

The usefulness of this design is that it names guarantees the application can reason about, and, as Figure 8.2 shows, it lets the developer move along the spectrum per account or per request.

8.4.6 Redis and etcd

Redis replicates asynchronously from primary to replicas, so a write acknowledged to the client can be lost if the primary fails before replicating it. The `WAIT` command blocks until a given number of replicas have acknowledged the write, which narrows the window of loss, but it does not make Redis a strongly consistent store, because a failover may still promote a

Table 8.4: The five consistency levels of Azure Cosmos DB.

Level	Guarantee
Strong	Linearizability: reads return the latest committed write
Bounded staleness	Reads may lag writes by at most a configured number of versions or time interval, in order
Session (default)	Within a session: read-your-writes, monotonic reads, and monotonic writes
Consistent prefix	Reads never see writes out of order, but may lag
Eventual	No ordering guarantee; replicas converge eventually

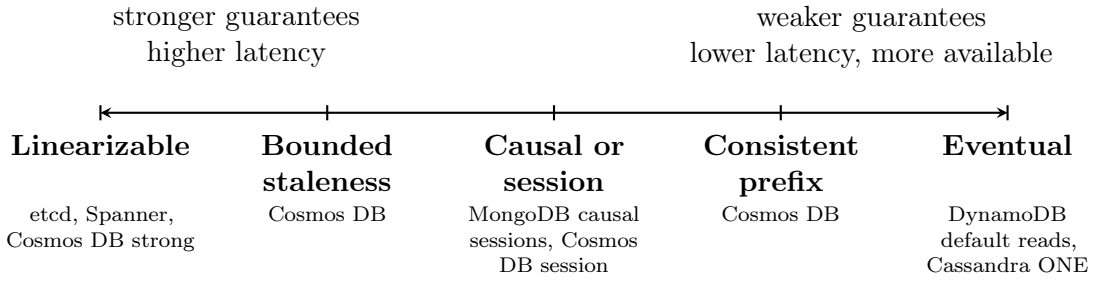


Figure 8.2: The consistency spectrum with example systems and settings. Moving left strengthens guarantees at the cost of latency and availability.

replica lacking the write (Redis Documentation). At the other end, etcd serves linearizable reads by default, using its Raft consensus; clients can choose faster serializable reads from a single member that may be stale (etcd Documentation). The contrast shows that even two key-value stores sit at very different points on the spectrum.

8.5 How Eventual Consistency Converges

"Eventually consistent" says that replicas converge if updates stop (Vogels, 2009), but not how. Leaderless and multi-leader systems use several mechanisms, often together:

- **Read repair.** When a read contacts several replicas and finds that one returned an older value, the system writes the newer value back to the stale replica. This repairs frequently read data quickly.
- **Hinted handoff.** When a replica is unreachable during a write, another node temporarily stores the write as a "hint" and delivers it when the replica returns.
- **Anti-entropy.** Background processes compare replicas, often using hash trees (Merkle trees) to find differences efficiently, and repair rarely read data (DeCandia et al., 2007).
- **Conflict resolution.** When replicas hold concurrent, conflicting versions, the system

applies last-write-wins, keeps both versions for the application to merge, or uses conflict-free replicated data types that merge deterministically (Chapter 6).

8.5.1 How Eventual Is "Eventually"?

In practice, convergence is usually fast. Measurements of Dynamo-style stores indicate that in a healthy cluster, replicas typically agree within milliseconds to tens of milliseconds, though with a long tail, and the probability of reading stale data can be estimated from the replication latency distribution and the quorum settings (Bailis et al., 2012). But convergence can take far longer, from seconds to hours, during overload, failures, or after a node has been down for a long time. Applications should therefore treat the window as unbounded in principle and measure it in practice (Section 8.7).

8.6 Application-Level Techniques

Because databases rarely give every guarantee for free, applications complete the picture with several techniques.

8.6.1 Prefer Atomic Operations to Read-Modify-Write

The lost-update example of Section 8.2 is fixed by letting the database perform the change atomically instead of reading, modifying, and writing in the application. For counters and stock, use increments or decrements with a guard condition.

8.6.2 Optimistic Concurrency Control

When a change depends on a value read earlier, *optimistic concurrency control* adds a version number to the record and makes the write conditional on the version being unchanged. If another writer got there first, the condition fails and the application re-reads and retries. Listing 8.3 shows the pattern in SQL, and the same idea underlies DynamoDB's condition expressions and Cassandra's lightweight transactions.

Listing 8.3: Atomic update and optimistic concurrency control in SQL.

```
-- Best: let the database do the arithmetic atomically, with a guard
UPDATE products SET stock = stock - 1
WHERE id = 7 AND stock > 0;

-- When a decision depends on earlier reads: version check
UPDATE products SET price = 19.90, version = version + 1
WHERE id = 7 AND version = 12;
-- 0 rows updated: someone else changed the row; re-read and retry
```

8.6.3 Idempotence and Deduplication

Retries are essential in distributed systems but can apply an operation twice (Chapter 6). Operations should be idempotent or carry a unique identifier that lets the database ignore repeats. Many consistency problems that look like replication anomalies are actually duplicate processing.

8.6.4 The Dual-Write Problem and the Outbox Pattern

Applications often need to update the database and notify another system, such as publishing an event to a message broker. Doing both separately is unsafe: if the process fails between them, one happens without the other, and no ordering guarantees can be given across two systems. The *transactional outbox* pattern solves this by writing the event to an "outbox" table in the same local transaction as the data change; a separate relay process reads the outbox and publishes the events, retrying until they are delivered (Richardson, 2018). Consumers must be idempotent, because delivery is at-least-once. Listing 8.4 shows the idea.

Listing 8.4: Transactional outbox: the data change and the event commit together.

```
BEGIN;
UPDATE orders SET status = 'paid' WHERE id = 5001;
INSERT INTO outbox (event_id, topic, payload)
VALUES (gen_random_uuid(), 'order.paid', '{"orderId": 5001}');
COMMIT;
-- A relay process publishes unsent outbox rows, then marks them sent
```

8.6.5 Hidden Sources of Staleness

Staleness does not arise only in the database. Caches, search indexes, and other derived data each add their own delay, and these delays add up. Table 8.5 lists common layers.

Table 8.5: Where staleness hides in an architecture.

Layer	Source of delay	Mitigation
Read replicas	Replication lag	Leader reads after writes; session tokens
Application cache	Time to live and invalidation delays (Chapter 3)	Invalidate on write; short TTLs for critical data
Content delivery network	Cached responses at the edge	Cache-control headers; versioned URLs
Search index or materialized view	Asynchronous indexing from the source of truth	Show "updating" states; read key data from the source
Event consumers	Queue backlog and processing delay	Idempotent consumers; monitor consumer lag

A cache in front of a replica that is itself behind the primary can serve data that is stale by the sum of both delays. When reasoning about what a user can see, examine the whole path, not only the database.

8.7 Testing and Monitoring Consistency

Documentation states intended behavior; production reveals actual behavior, especially under failure.

- **Monitor replication lag and consumer lag.** Alert when lag exceeds what the application assumes, and chart it against traffic.
- **Measure staleness.** Periodically write a timestamped marker to the primary and read it from other replicas or regions to measure how long it takes to appear, and track the percentiles (p50, p99), not only the average.
- **Test under fault injection.** Independent tools such as Jepsen test databases by injecting partitions, crashes, and clock skew while checking histories of operations against a consistency model (Jepsen). Run similar experiments in staging for your own configuration.
- **Check invariants.** Run periodic jobs that verify business rules, such as "no negative balances" or "every order has a customer", and alert on violations, so that anomalies are noticed even when no one complains.
- **Set explicit objectives.** For example, "99% of reads reflect a user's own writes" or "search results are updated within 10 seconds", and measure against them.

8.8 Choosing Consistency Levels

Choosing is easiest when done per use case, using the cost of an anomaly, as in Chapter 7. Table 8.6 gives guidance for common cases.

A sound process is: (1) list operations and the anomalies each could suffer; (2) rank the cost of each anomaly; (3) pick the weakest level that prevents the costly ones; (4) add application techniques for the rest; and (5) test and monitor. A useful default is to start from the strongest level the latency budget allows for critical paths, and relax it only where measurements justify the saving. For a survey of the formal landscape behind these levels, see Viotti and Vukolić (2016).

Summary

In practice, consistency appears as anomalies: stale reads, disappearing updates, time moving backward, effects before causes, lost updates, and write skew. Session guarantees such as read-your-writes and monotonic reads, and causal consistency, prevent the most disturbing ones

Table 8.6: Consistency guidance by use case.

Use case	Suggested level	Techniques
Account balance and payments	Strong, with transactions	Serializable or conditional updates; idempotency keys
Profile shown after the user edits it	Read-your-writes	Leader or session-token reads; return the updated object
Social feed and comments	Session or causal	Sticky sessions; keep a thread's data in one partition
Inventory at checkout	Strong on the final decrement	Atomic guarded update or reservation with expiry; display approximate stock elsewhere
Product catalog and search	Eventual	Caching; show freshness where it matters
Analytics dashboards	Eventual, bounded	Read from replicas or warehouses; display data age
Locks, leader election, configuration	Linearizable	Consensus-based stores (etcd, ZooKeeper); fencing tokens

without global coordination, and can be implemented through leader reads, sticky sessions, session tokens, and client-side overlays. Real databases expose consistency through different controls: replicas and synchronous commit in PostgreSQL, write and read concerns and causal sessions in MongoDB, per-request levels and lightweight transactions in Cassandra, strongly consistent reads, conditions, and transactions in DynamoDB, five named levels in Cosmos DB, and linearizable or serializable reads in etcd. Eventually consistent systems converge through read repair, hinted handoff, anti-entropy, and conflict resolution, typically quickly but without a hard bound. Applications add atomic operations, optimistic concurrency control, idempotence, and the outbox pattern, and must remember that caches, indexes, and consumers add their own staleness. Teams should monitor lag, measure staleness, test under faults, check invariants, and choose a consistency level for each use case based on the cost of an anomaly.

Review Questions

1. Describe a read-your-writes violation. What causes it, and name three ways to prevent it.

2. What is a monotonic-read violation, and why can it be more disturbing to users than simple staleness?

3. Explain how a lost update happens and why it can occur even without replication. How do atomic updates and conditional writes prevent it?

4. What is write skew? Give an example and say which isolation level prevents it.

5. List the four session guarantees and the implementation technique for each.

6. Compare MongoDB's write concern, read concern, and read preference. What does read concern `"majority"` protect against?

7. Why are Cassandra quorum reads and writes not always linearizable? When would you use a lightweight transaction?

8. Explain the five Cosmos DB consistency levels. Which would you choose for a collaborative document editor, and why?

9. Describe three mechanisms by which eventually consistent systems converge. Why is "eventually" not a bound?

10. What is the dual-write problem, and how does the transactional outbox pattern solve it?

11. Where in a typical architecture can staleness accumulate besides the database?

12. How would you test and monitor whether a system delivers the consistency that the application assumes?

Exercises

1. **Anomaly analysis.** An online forum stores threads in a sharded database with asynchronous replicas. List the anomalies that users could experience when posting and reading replies, and propose a design that prevents each.

2. **Concurrency.** Two clients run `stock = read(); write(stock - 1)` on an item with stock 10, interleaved so both read before either writes. What is the final value? Rewrite the operation using a guarded atomic update and using a version check.

3. **Session tokens.** Describe how a session token (a logical timestamp) can provide read-your-writes across replicas. What does a replica do if it has not yet reached the token?

4. **Configuration.** For a banking application on MongoDB, choose write concern, read concern, and read preference for (a) a balance transfer, (b) a statement query for last month, and (c) a "recent activity" view right after the user makes a payment. Justify each.

5. **Outbox.** An order service must save an order and publish an event. Show what can go wrong with two separate writes, then design the outbox table, the relay, and the consumer's deduplication.

6. **Staleness budget.** A page shows data from a database replica behind a cache with a 60-second TTL and a search index updated every 30 seconds. What is the worst-case staleness a user can see for (a) the database data, (b) the search results? How would you reduce it?

7. **Measurement.** Design an experiment to measure the replication lag of a read replica in production, including what you write, where you read, how often, and which statistics you report.

References

- Amazon Web Services. *Amazon DynamoDB Developer Guide*. https://docs.aws.amazon.com/dynamodb
- Apache Cassandra Documentation. https://cassandra.apache.org/doc/latest/
- Azure Cosmos DB Documentation: Consistency levels. https://learn.microsoft.com/azure/cosmos-db/consistency-levels
- Bailis, P., Venkataraman, S., Franklin, M. J., Hellerstein, J. M., & Stoica, I. (2012). Probabilistically Bounded Staleness for Practical Partial Quorums. *Proceedings of the VLDB Endowment*, 5(8), 776–787.
- Bailis, P., Davidson, A., Fekete, A., Ghodsi, A., Hellerstein, J. M., & Stoica, I. (2013). Highly Available Transactions: Virtues and Limitations. *Proceedings of the VLDB Endowment*, 7(3), 181–192.
- Berenson, H., Bernstein, P., Gray, J., Melton, J., O'Neil, E., & O'Neil, P. (1995). A Critique of ANSI SQL Isolation Levels. In *Proceedings of ACM SIGMOD '95*, 1–10.
- DeCandia, G., Hastorun, D., Jampani, M., et al. (2007). Dynamo: Amazon's Highly Available Key-value Store. In *Proceedings of SOSP '07*.
- etcd Documentation. https://etcd.io/docs/
- Jepsen. Distributed systems safety research. https://jepsen.io
- Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.
- MongoDB Documentation. https://www.mongodb.com/docs/manual/
- PostgreSQL Documentation. https://www.postgresql.org/docs/
- Redis Documentation. https://redis.io/docs
- Richardson, C. (2018). *Microservices Patterns*. Manning Publications.
- Terry, D. B., Demers, A. J., Petersen, K., Spreitzer, M. J., Theimer, M. M., & Welch, B. B. (1994). Session Guarantees for Weakly Consistent Replicated Data. In *Proceedings of PDIS '94*, 140–149.

- Viotti, P., & Vukolić, M. (2016). Consistency in Non-Transactional Distributed Storage Systems. *ACM Computing Surveys*, 49(1), Article 19.
- Vogels, W. (2009). Eventually Consistent. *Communications of the ACM*, 52(1), 40–44.

Chapter 9

Replication and Data Distribution Strategies

Learning Objectives

After reading this chapter, you will be able to:

- Explain why data is replicated and partitioned, and how the two techniques complement each other.
- Compare single-leader, multi-leader, and leaderless replication.
- Distinguish synchronous, asynchronous, and semi-synchronous replication, and the log formats used to ship changes.
- Describe failover, its failure modes, and how fencing prevents split brain.
- Compare range, hash, and consistent-hash partitioning, and explain how hot spots and rebalancing are handled.
- Explain local and global secondary indexes and how requests are routed in a partitioned system.
- Configure replication and distribution in real systems such as PostgreSQL, MongoDB, Cassandra, and DynamoDB.
- Design a multi-region data placement strategy and choose a strategy for each use case.

9.1 Why Replicate and Distribute Data

Chapter 8 described what applications experience when data lives on several machines. This chapter explains how the data comes to live on several machines in the first place. Two techniques are used, and almost every large system combines them:

- **Replication** keeps a copy of the same data on several nodes.
- **Partitioning** (also called *sharding*) splits a dataset into subsets, each stored on different nodes.

Replication is chosen for *availability* (the data survives the loss of a node), *latency* (a copy can be placed near the user), and *read throughput* (reads are spread over several copies).

Partitioning is chosen for *capacity* (the dataset exceeds one machine) and *write throughput* (writes are spread over several machines). Replication does not increase the amount of data that can be stored, and partitioning alone does not protect against failure, which is why the two are combined (Kleppmann, 2017). Figure 9.1 shows the combination.

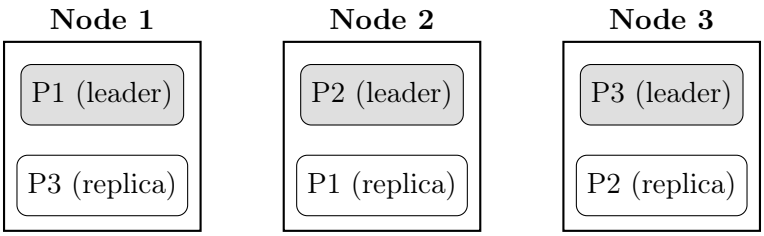


Figure 9.1: Replication combined with partitioning. The dataset is split into three partitions (P1–P3), each stored on two nodes. Every node is the leader for one partition and a replica for another, so load and risk are spread.

A recurring theme is that every choice here is a trade among consistency, latency, availability, and operational complexity, the same trade-offs introduced in Chapter 7.

9.2 Replication Topologies

A replication topology defines which nodes accept writes and how changes flow between nodes. Table 9.1 compares the three main designs.

Table 9.1: Replication topologies compared.

Topology	How writes work	Strengths	Weaknesses
Single-leader	All writes go to one leader, which streams changes to followers	Simple; no write conflicts; easy to reason about order	Leader is a bottleneck and a failover point; remote writers pay latency
Multi-leader	Several leaders accept writes and replicate to each other	Local writes in each region; tolerates region loss	Write conflicts must be detected and resolved
Leaderless	Clients write to several replicas directly; quorums decide success	No failover; highly available	Weaker ordering; relies on repair and conflict resolution

9.2.1 Single-Leader Replication

In single-leader (primary–replica) replication, clients send writes to the leader, which records them in a log and sends the log to followers. Followers apply the changes in the same order,

so they converge on the leader's state. Reads may go to the leader or to followers. This design is used by PostgreSQL, MySQL, MongoDB replica sets, and many others. It avoids write conflicts completely, because one node orders all writes, but it concentrates writes on one machine and makes the leader's failure a significant event (Section 9.4).

9.2.2 Multi-Leader Replication

Multi-leader replication lets more than one node accept writes. The typical motivation is geography: each region has a leader, so users write to a nearby node, and the leaders exchange changes asynchronously. Other uses include offline-capable clients, where each device acts as a small leader, and collaborative editing. The price is that two leaders can accept conflicting writes to the same item before they hear about each other. Conflict handling (Section 9.5) becomes part of the design, and features that depend on global order, such as unique constraints and auto-increment keys, become harder (Kleppmann, 2017).

9.2.3 Leaderless Replication

Leaderless designs, popularized by Dynamo (DeCandia et al., 2007) and used by Cassandra and Riak, let a client (or a coordinator node) send each write to several replicas and wait for a configured number of acknowledgments. A read similarly contacts several replicas and picks the most recent value. If N replicas hold each item, W must acknowledge a write, and R must answer a read, then $R + W > N$ ensures that every read quorum overlaps every write quorum (Chapter 8). Failed replicas are tolerated without any failover procedure, because the remaining ones still form a quorum. Convergence of lagging replicas relies on read repair, hinted handoff, and anti-entropy, described in Chapter 8.

9.3 How Changes Are Replicated

9.3.1 Synchronous, Asynchronous, and Semi-Synchronous

The key decision in leader-based replication is when the leader tells the client that a write succeeded.

- **Asynchronous.** The leader acknowledges after writing locally and ships the change in the background. Writes are fast and unaffected by slow followers, but a leader crash can lose acknowledged writes that had not yet been shipped, and followers lag.

- **Synchronous.** The leader waits until followers confirm the change. No acknowledged write is lost on failover, but each write is as slow as the slowest awaited follower, and an unavailable follower can block writes.

- **Semi-synchronous.** The leader waits for one (or a configured number of) followers, while others are asynchronous. This bounds data loss to a small window while keeping most of the speed, and it is the practical compromise in many deployments.

Synchronous replication to *all* followers is rarely used, because one failure would halt writes. Modern consensus-based systems (such as those built on Raft or Paxos) replicate synchronously to a *majority*, which tolerates a minority of failures without blocking.

9.3.2 Log Formats

The log the leader ships can take several forms, each with trade-offs:

- **Statement-based.** The leader sends the SQL statements it executed. This is compact, but nondeterministic functions (such as the current time or a random number) and statements with side effects can produce different results on followers.
- **Write-ahead log (WAL) shipping.** The leader sends the low-level storage log. This is exact, but it ties replicas to the same storage format and usually the same database version.
- **Logical (row-based) log.** The leader sends a description of which rows were inserted, updated, or deleted. It is independent of the storage engine, allows replication between different versions, and can feed other systems, as in change data capture.
- **Trigger- or application-level.** Custom code replicates selected changes. It is flexible but more error-prone.

Logical logs are increasingly used as a general mechanism, because the same stream that updates replicas can update search indexes, caches, and data warehouses (Chapter 8 showed the staleness such derived data may add).

9.3.3 Replication Lag

With asynchronous replication, followers trail the leader by some delay called *replication lag*. It is normally small but grows under heavy write load, slow networks, long-running queries on a replica, or a replica that was offline. Lag is the root of the read-your-writes, monotonic-read, and consistent-prefix anomalies of Chapter 8, so it must be monitored and its effect designed for.

9.4 Failover

When a leader fails, a follower must be promoted. This *failover* may be manual or automatic, and it consists of three steps: detecting the failure, choosing a new leader, and reconfiguring clients and followers to use it. Each step has pitfalls:

- **Failure detection** usually uses timeouts. A timeout that is too short triggers failover during a temporary slowdown, which can make a load problem worse; one that is too long prolongs the outage.
- **Choosing the new leader** should favor the most up-to-date follower, ideally by agreement among nodes (a consensus or election protocol).

- **Lost writes.** With asynchronous replication, the new leader may lack some writes the old leader had acknowledged. Those writes are discarded when the old leader returns, which can violate application expectations or conflict with data in other systems, such as reused identifiers.
- **Split brain.** Two nodes may both believe they are the leader and accept writes, corrupting data. The old leader may not know it has been replaced, for example after a network partition or a long pause.

The standard defense against split brain is *fencing*: each leadership term carries a monotonically increasing number (an *epoch* or *fencing token*), and storage or followers reject requests carrying an older number than the highest one seen. A node that was demoted while paused is then unable to do harm even if it wakes up and believes it is still the leader. Consensus-based stores such as etcd and ZooKeeper provide this kind of coordination (Chapter 8). Many teams choose a manual or semi-automatic failover for critical databases, since automatic promotion in an unusual failure can be worse than a short outage.

9.5 Conflicts in Multi-Leader and Leaderless Systems

When two replicas accept writes to the same item concurrently, a conflict arises. Strategies include:

- **Avoid conflicts.** Route all writes for a given item to the same leader, for example the leader in the user's home region. This is the simplest approach and works when each item has one natural owner.
- **Last-write-wins (LWW).** Each write carries a timestamp, and the highest one wins. It is simple and converges, but it silently discards one of the concurrent writes and depends on clocks that may be skewed.
- **Keep siblings.** The system stores all concurrent versions and returns them to the application, which merges them (as in the shopping-cart example of Dynamo).
- **Version vectors.** Metadata per replica lets the system tell whether one version supersedes another or whether they are truly concurrent.
- **CRDTs and operational transformation.** Data types and algorithms designed so that concurrent updates merge deterministically, used in collaborative editing and counters (Chapter 6).

The right strategy depends on the data: LWW is acceptable for a "last seen" timestamp, dangerous for an account balance, and a poor fit for a shared document.

9.6 Partitioning Strategies

Partitioning divides the data so that each record belongs to exactly one partition. A good scheme spreads data and load evenly and lets the system find a record's partition quickly. Table 9.2 compares the main approaches.

Table 9.2: Partitioning strategies compared.

Strategy	How a record is placed	Strengths	Weaknesses
Range	Contiguous key ranges map to partitions	Efficient range scans; ordered data	Hot spots for sequential keys (such as timestamps)
Hash	Hash of the key selects the partition	Even spread of keys	Range queries must contact all partitions
Consistent hashing	Keys and nodes are placed on a hash ring	Adding or removing a node moves little data	Needs virtual nodes for balance
Directory-based	A lookup service maps keys to partitions	Flexible placement	Directory is an extra component to keep available

9.6.1 Range Partitioning

Range partitioning assigns each partition a continuous interval of the key space, like the volumes of an encyclopedia. Scans over a key range touch few partitions, which suits ordered access such as "all events for a user between two dates". The danger is a *hot spot*: if keys are timestamps, all current writes land on the partition holding the latest range. A common remedy is to prefix the key with another attribute, such as the sensor identifier, so that writes spread out while scans within one sensor remain efficient.

9.6.2 Hash Partitioning

Hash partitioning applies a hash function to the key and assigns the result to a partition. Keys that are close together are scattered, which spreads load evenly but loses efficient range scans. The hash should be stable across processes and versions, which excludes functions seeded per run. A naive scheme of computing $\text{hash}(k) \bmod N$ is a trap, because changing the number of nodes N remaps most keys; the following techniques avoid it.

9.6.3 Consistent Hashing and Virtual Nodes

In *consistent hashing* (Karger et al., 1997), both nodes and keys are mapped onto a ring of hash values, and each key belongs to the first node clockwise from its position. When a node joins or leaves, only the keys in its neighboring segment move, roughly $1/N$ of the data, rather than most of it. A single position per node gives uneven segments, so each physical node is assigned many positions, called *virtual nodes*, which smooths the load and lets stronger machines take more positions. Dynamo and Cassandra use variants of this idea (DeCandia et al., 2007). Replicas are commonly placed on the next several distinct nodes along the ring.

9.6.4 Fixed Partitions and Directories

Many systems instead create many more partitions than nodes (for example, hundreds of partitions on ten nodes) and assign whole partitions to nodes. Adding a node means moving some entire partitions to it, and the key-to-partition mapping never changes. A *directory* (a metadata service or table) records which node holds which partition, which allows arbitrary placement and manual control at the price of one more component that must be highly available, often built on a consensus store.

9.6.5 Hot Keys and Skew

Even a good hash cannot help when one key receives most of the traffic, such as a celebrity account or a viral item. Remedies include splitting the hot key into several sub-keys by appending a random suffix to writes and reading all suffixes (which costs read effort), caching, and giving known hot items dedicated capacity. Whether this is needed should be decided from measurement, not guesswork.

9.7 Secondary Indexes and Request Routing

9.7.1 Secondary Indexes in a Partitioned Database

Partitioning by primary key makes lookups by that key easy, but queries on other attributes ("all orders with status shipped") need a secondary index. There are two designs:

- **Local (document-partitioned) indexes.** Each partition indexes only its own records. Writes are cheap, since only one partition is updated, but a query on the indexed attribute must be sent to *all* partitions and the results combined ("scatter–gather"), which makes tail latency worse.
- **Global (term-partitioned) indexes.** The index covers all partitions and is itself partitioned, by the indexed value. Reads are efficient because one index partition answers the query, but a single write may need to update several index partitions, often asynchronously, so the index can lag behind the data (Chapter 8).

DynamoDB's global secondary indexes, which are eventually consistent, are an example of the second design, while Cassandra's native secondary indexes and MongoDB's per-shard indexes follow the first.

9.7.2 Routing Requests to the Right Node

A client needs to know which node serves a given key. Three approaches are used:

1. The client contacts any node, which forwards the request if it is not the owner (gossip-based systems such as Cassandra).
2. A routing tier knows the partition map and forwards requests (MongoDB's `mongos` router).

3. The client library holds the partition map and connects directly (many drivers).

In all cases the partition map changes as nodes fail and as data is rebalanced, so it must be kept consistent enough that requests are not sent to nodes that no longer own the data. Coordination services or gossip protocols are used to spread this metadata.

9.8 Rebalancing

Rebalancing moves data between nodes when capacity is added, a node is removed, or load becomes uneven. A good procedure has these properties:

- After rebalancing, load is shared fairly among nodes.
- The database keeps accepting reads and writes during the move.
- Only the necessary data moves, to minimize network and disk load.

Moving a large amount of data stresses the network and the nodes involved and can make a struggling cluster worse, so rebalancing should be throttled. It also interacts with failure detection: a node that is slow because it is busy moving data may be wrongly declared dead, triggering still more data movement. For this reason many operators prefer rebalancing to be started or confirmed by a human rather than being fully automatic. Systems with dynamic partitioning (splitting a partition when it grows too large and merging small ones) adapt to data volume automatically, while fixed-partition systems require choosing the number of partitions wisely at the start.

9.9 Replication and Distribution in Real Systems

Table 9.3 summarizes how common systems combine the techniques of this chapter. As in Chapter 8, defaults and features change between versions, so use it as a map to the documentation.

9.9.1 PostgreSQL: Logical Replication

PostgreSQL's physical (streaming) replication copies the whole cluster byte for byte and is the usual choice for high-availability standbys and read replicas. *Logical replication* instead publishes row-level changes of selected tables, which subscribers can apply to a different version or to a database that also holds other data. Listing 9.1 shows the setup (PostgreSQL Documentation).

Listing 9.1: Logical replication of one table in PostgreSQL.

```
-- On the publisher (requires wal_level = logical)
CREATE PUBLICATION orders_pub FOR TABLE orders;

-- On the subscriber (the table must already exist)
CREATE SUBSCRIPTION orders_sub
  CONNECTION 'host=primary.example.com dbname=shop user=repl'
  PUBLICATION orders_pub;
```

Table 9.3: Replication and partitioning in selected systems (check current documentation).

System	Replication	Partitioning	Notes
PostgreSQL	Single-leader; physical streaming and logical replication	Declarative table partitioning inside a node; sharding needs extensions or tools	Logical replication allows selective copies and upgrades
MySQL	Single-leader; asynchronous or semi-synchronous; group replication	Table partitioning; sharding via external layers	Row-based binary log is common
MongoDB	Replica sets with automatic election	Sharded clusters with chosen shard key; range or hashed	Router and config servers hold the partition map
Cassandra	Leaderless; replication factor per keyspace; tunable consistency	Consistent hashing with virtual nodes	Replication strategy can be data-center aware
DynamoDB	Managed, multi-replica within a region; global tables across regions	Hash partitioning on the partition key; automatic splitting	Global tables reconcile concurrent writes, typically last-writer-wins
Spanner-like systems	Consensus (Paxos or Raft) per partition	Range partitions that split and merge automatically	Synchronous majority replication; stronger consistency at higher write latency

9.9.2 MongoDB: Sharding and the Shard Key

In MongoDB (MongoDB Documentation), each shard is itself a replica set, so partitioning and replication are layered. The most important decision is the *shard key*, which determines how documents are distributed and is difficult to change later. A monotonically increasing key (such as a timestamp or ObjectId) concentrates inserts on one shard when ranged, so a hashed key is often used when writes must be spread evenly, at the cost of efficient range queries on that key. Listing 9.2 shows both options.

Listing 9.2: Sharding a collection with a hashed key, and with a compound range key.

```
sh.enableSharding("shop")

// Even spread of writes, no efficient ranges on customerId
sh.shardCollection("shop.orders", { customerId: "hashed" })

// Alternative: range key that keeps one customer's orders together
```

```
// sh.shardCollection("shop.orders", { customerId: 1, orderDate: 1 })
```

9.9.3 Cassandra: Replication Factor and Placement

In Cassandra (Apache Cassandra Documentation), replication is configured per keyspace through a replication strategy. `NetworkTopologyStrategy` sets the number of replicas in each data center, and the partitioner places each row by the hash of its partition key. Listing 9.3 creates a keyspace with three replicas in each of two data centers; combined with `LOCAL_QUORUM` (Chapter 8), reads and writes stay within the local data center while still being replicated to the other.

Listing 9.3: A keyspace replicated across two data centers in CQL.

```
CREATE KEYSPACE shop WITH replication = {
  'class': 'NetworkTopologyStrategy',
  'dc_europe': 3,
  'dc_asia': 3
};

-- The partition key (customer_id) decides the replica set;
-- the clustering column (order_date) orders rows within it.
CREATE TABLE shop.orders (
  customer_id uuid,
  order_date timestamp,
  total decimal,
  PRIMARY KEY ((customer_id), order_date)
);
```

9.9.4 DynamoDB: Managed Partitioning

DynamoDB (Amazon DynamoDB Developer Guide) hides partitions from the developer, who chooses only a partition key (and optionally a sort key). The service hashes the key to a partition and splits partitions as data or traffic grows. The developer's main responsibility is therefore a key design that spreads load; a poorly chosen partition key, such as a date or a status with few values, creates hot partitions regardless of the service's automation. Global tables add multi-region replication on top.

9.10 Geo-Distribution and Data Placement

Placing data in several regions serves three goals: lower latency for users around the world, resilience against the loss of a whole region, and compliance with rules requiring certain data to stay within a country or jurisdiction. These goals sometimes conflict, so placement must be designed explicitly. Common patterns are:

- **Single write region, read replicas elsewhere.** Writes are slower for distant users, but there are no conflicts and the design is simple. Reads in other regions may be stale, so the techniques of Chapter 8 apply.
- **Home-region partitioning.** Each user or tenant is assigned a home region that owns their data, so writes are local for most users and conflicts are avoided because each item has one owner. It also fits data-residency rules.
- **Active–active multi-region.** Several regions accept writes to the same data and reconcile, with the conflict strategies of Section 9.5. It gives the best availability and write latency but the most difficult correctness reasoning.
- **Consensus across regions.** Majority replication across regions gives strong consistency and survives the loss of a region, but every write pays at least one inter-region round trip.

Distances impose physical limits: a round trip between continents costs on the order of a hundred milliseconds, which no software can remove. Designs that require synchronous cross-region coordination on every request therefore have a latency floor, and should be reserved for data that genuinely needs it.

9.11 Operating Replicated and Partitioned Systems

Replication is not a backup: a mistaken deletion or a corrupting bug is replicated as faithfully as any other change. Operations should therefore include:

- **Backups and point-in-time recovery**, kept separate from replicas and tested by actually restoring them.
- **Monitoring** of replication lag, replica health, partition size and traffic skew, and the progress of rebalancing.
- **Failover drills**, carried out regularly in staging, and sometimes in production, to confirm that promotion, client reconnection, and fencing work as expected.
- **Capacity planning** for the loss of a node or zone: the remaining nodes must be able to carry the load of the failed ones.
- **Safe schema and version changes**, using rolling upgrades and, where available, logical replication to move between versions with little downtime.
- **Shard-key and partition-key reviews**, since poor choices are the hardest mistakes to repair once data volume is large.

The testing techniques of Chapter 8, in particular measuring staleness and injecting faults, apply equally here.

9.12 Choosing a Strategy

As in Chapter 8, the choice is made per use case, by comparing requirements with costs. Table 9.4 gives guidance for common situations.

Table 9.4: Replication and distribution guidance by use case.

Use case	Suggested approach	Reasoning
Business application, moderate size	Single-leader with one or two replicas, semi-synchronous	Simplicity and no conflicts; one node suffices for the data
Read-heavy website	Single-leader with many read replicas and caching	Reads scale out; writes remain on one node
Very large dataset or write rate	Partitioning (hash or range) with replicas per partition	Capacity and write throughput scale with nodes
Time-series and event data	Range partitioning with a prefixed key, or hash with time-bucketed keys	Fast scans per source while avoiding a hot latest partition
Global users, local writes	Home-region partitioning, or multi-leader with conflict rules	Low write latency; conflicts avoided or handled
Always-on availability across data centers	Leaderless with data-center-aware replication	No failover; tolerates node and site loss
Critical data needing strong consistency worldwide	Consensus-based replication per partition	Strong guarantees, at the price of cross-region latency

A sound process is: (1) estimate data size, read rate, and write rate to decide whether one node suffices; (2) decide how many failures must be tolerated and whether that includes a whole region; (3) choose the topology that provides this with the least complexity; (4) choose partition keys from the actual query patterns and check them for skew; (5) decide how conflicts will be avoided or resolved; and (6) rehearse failure, rebalancing, and recovery. A useful default is to begin with the simplest design that meets the requirements, a single leader with replicas, and to add partitioning or multiple leaders only when measurements show that it is necessary. For a survey of replication in the formal literature, see Terry (2013) and Kleppmann (2017).

Summary

Replication keeps copies of data on several nodes for availability, latency, and read throughput, while partitioning splits data across nodes for capacity and write throughput; large

systems combine both. Single-leader replication is simple and conflict-free but depends on failover; multi-leader replication gives local writes at the price of conflict resolution; leaderless replication tolerates failures through quorums and repair. Asynchronous replication is fast but can lose acknowledged writes and creates lag, synchronous replication prevents loss at a cost in latency and availability, and semi-synchronous replication compromises. Failover requires careful detection, election, and fencing to avoid lost writes and split brain. Partitioning by range supports scans but risks hot spots, hashing spreads load but loses ranges, and consistent hashing with virtual nodes limits data movement when the cluster changes. Secondary indexes are either local, with scatter–gather reads, or global, with more expensive and often asynchronous writes. Real systems such as PostgreSQL, MongoDB, Cassandra, and DynamoDB expose these choices in different ways, and key design is the developer's main lever. Geo-distribution brings latency, resilience, and compliance benefits, constrained by the speed of light. Replication is not a backup, and operating these systems requires monitoring, drills, and capacity planning. The recommended approach is to start with the simplest design that satisfies the requirements and to add complexity only when measurements justify it.

Review Questions

1. Why are replication and partitioning usually combined? What does each provide that the other does not?
2. Compare single-leader, multi-leader, and leaderless replication. Name one advantage and one drawback of each.
3. Explain the difference between synchronous, asynchronous, and semi-synchronous replication. Which would you use for a payments database, and why?
4. Compare statement-based, WAL-shipping, and logical replication logs. Why is logical replication useful beyond keeping replicas in sync?
5. Describe the steps of a failover and three things that can go wrong. How does fencing prevent split brain?
6. List four strategies for resolving write conflicts. For which kind of data is last-write-wins acceptable, and for which is it dangerous?
7. What is a hot spot in range partitioning, and how can key design avoid it?
8. Why is $\text{hash}(k) \bmod N$ a poor partitioning scheme when nodes are added? How does consistent hashing improve on it, and what do virtual nodes add?
9. Compare local and global secondary indexes in terms of read cost, write cost, and consistency.
10. Why should rebalancing be throttled and, in many systems, human-approved?
11. Explain why replication is not a substitute for backup.
12. Describe four patterns of multi-region data placement and the trade-offs among them.

Exercises

1. **Quorums.** A leaderless system stores each item on $N = 5$ replicas. For (W, R) equal to $(3, 3)$, $(1, 5)$, $(5, 1)$, and $(2, 2)$, state whether $R + W > N$ holds, how many replica failures writes and reads each tolerate, and which setting you would pick for a read-heavy workload.

2. **Failover.** A leader with one asynchronous follower fails, and the follower is promoted. Describe what happens to writes the old leader had acknowledged but not shipped, and what could go wrong when the old leader rejoins. Propose a design using fencing tokens that prevents the old leader from corrupting data.

3. **Key design.** An IoT platform receives readings from 100 000 sensors, each sending one reading per second, and queries mostly "the last hour for one sensor." Compare partitioning by timestamp, by hash of the sensor identifier, and by a compound key, and justify your choice.

4. **Consistent hashing.** Nodes A, B, and C are placed on a hash ring. Explain which keys move when node D is added, and how adding virtual nodes changes the amount of data each existing node gives up. Roughly what fraction of data moves with consistent hashing, compared with $\bmod N$ hashing, when a fourth node joins three?

5. **Secondary indexes.** A sharded orders table is partitioned by customer identifier, and the application must support "find all orders with status = shipped." Design the solution with a local index and with a global index, and compare read latency, write cost, and freshness.

6. **Geo-distribution.** A service has users in Europe, North America, and Asia. Propose a data placement for (a) user profiles, (b) a global product catalog, and (c) account balances. State the topology, how conflicts are avoided or resolved, and the latency a user experiences for writes.

7. **Configuration.** Write the configuration for a Cassandra keyspace with three replicas in each of two data centers, and say which consistency level you would use for (a) a balance update and (b) a profile read, and why.

8. **Operations.** Draft a one-page runbook for failing over a PostgreSQL primary to a replica, including checks before promotion, steps to redirect clients, and verification afterward.

References

- Amazon Web Services. *Amazon DynamoDB Developer Guide*. https://docs.aws.amazon.com/dynamodb

- Apache Cassandra Documentation. https://cassandra.apache.org/doc/latest/

- Corbett, J. C., Dean, J., Epstein, M., et al. (2012). Spanner: Google's Globally-Distributed Database. In *Proceedings of OSDI '12*.
- DeCandia, G., Hastorun, D., Jampani, M., et al. (2007). Dynamo: Amazon's Highly Available Key-value Store. In *Proceedings of SOSP '07*.
- Karger, D., Lehman, E., Leighton, T., Panigrahy, R., Levine, M., & Lewin, D. (1997). Consistent Hashing and Random Trees: Distributed Caching Protocols for Relieving Hot Spots on the World Wide Web. In *Proceedings of ACM STOC '97*, 654–663.
- Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.
- MongoDB Documentation. https://www.mongodb.com/docs/manual/
- Ongaro, D., & Ousterhout, J. (2014). In Search of an Understandable Consensus Algorithm. In *Proceedings of USENIX ATC '14*.
- PostgreSQL Documentation. https://www.postgresql.org/docs/
- Terry, D. (2013). Replicated Data Consistency Explained Through Baseball. *Communications of the ACM*, 56(12), 82–89.

Chapter 10

Distributed Transactions and Fault Tolerance

Learning Objectives

After reading this chapter, you will be able to:

- Explain why transactions that span several nodes are difficult, and what atomicity and isolation require in a distributed setting.
- Describe two-phase commit, its blocking problem, and how consensus-based commit addresses it.
- Explain how distributed databases provide snapshot isolation and serializability using timestamps and clocks.
- Design a saga with compensating actions, and decide between orchestration and choreography.
- Use idempotency keys and explain why "exactly-once" delivery is really at-least-once delivery plus deduplication.
- Describe the fault model of distributed systems and how failures are detected.
- Apply timeouts, retries with backoff, circuit breakers, bulkheads, and redundancy to build fault-tolerant services.
- Plan recovery with write-ahead logs, backups, and recovery objectives, and test fault tolerance in practice.

10.1 Why Distributed Transactions Are Hard

On a single node, a transaction groups several operations into one unit with the ACID properties: atomicity (all or nothing), consistency (the application's invariants are preserved), isolation (concurrent transactions do not interfere), and durability (committed data survives crashes). Chapter 9 showed that data is routinely partitioned and replicated across nodes. A *distributed transaction* is one whose operations touch data on more than one node, for example transferring money between two accounts stored in different partitions, or updating a database and a message broker together.

The difficulty is that nodes fail independently and the network can delay, drop, or reorder messages. If a transaction updates two partitions, one may commit while the other crashes or becomes unreachable, leaving the database in a state that no serial execution could produce. Making all participants agree on one outcome, despite such failures, is the *atomic commitment* problem. Making concurrent distributed transactions appear to run one at a time is the *distributed isolation* problem. Both have well-understood solutions, but they are costly, and many large systems therefore avoid distributed transactions where they can (Helland, 2007).

This chapter first presents the protocols for atomic commit and isolation, then the alternatives that applications use when those protocols are too expensive, and finally the broader subject of fault tolerance: how failures are modeled, detected, contained, and recovered from.

10.2 Atomic Commit

10.2.1 Two-Phase Commit

Two-phase commit (2PC) is the classic protocol for atomic commitment (Gray, 1978). One node acts as the *coordinator* and the others are *participants*. The protocol has two phases:

1. **Prepare (voting) phase.** The coordinator sends a `prepare` request to every participant. Each participant checks whether it can commit, makes the transaction's changes durable in its log, and replies *yes* or *no*. A participant that votes yes has promised that it can commit and must not change its mind later; it holds its locks until it learns the outcome.

2. **Commit (decision) phase.** If every participant voted yes, the coordinator records the decision *commit* durably and sends `commit` to all; otherwise it records *abort* and sends `abort`. Participants apply the decision and release their locks.

Figure 10.1 shows a successful run. The coordinator's durable decision record is the point of no return: once it is written, the transaction's outcome is fixed even if the coordinator crashes immediately afterward.

10.2.2 The Blocking Problem

2PC is correct, but it can *block*. If a participant has voted yes and then cannot reach the coordinator, it is *in doubt*: it cannot commit, because the decision may have been abort, and it cannot abort, because the decision may have been commit. It must wait, holding its locks, until the coordinator recovers or can be reached. During that time other transactions that need the locked rows are blocked as well, so a failed coordinator can stall a significant part of the system. Practical consequences include:

- The coordinator's log is a critical resource and must be durable and recoverable.
- In-doubt transactions may need manual resolution by an operator when the coordinator is lost permanently.

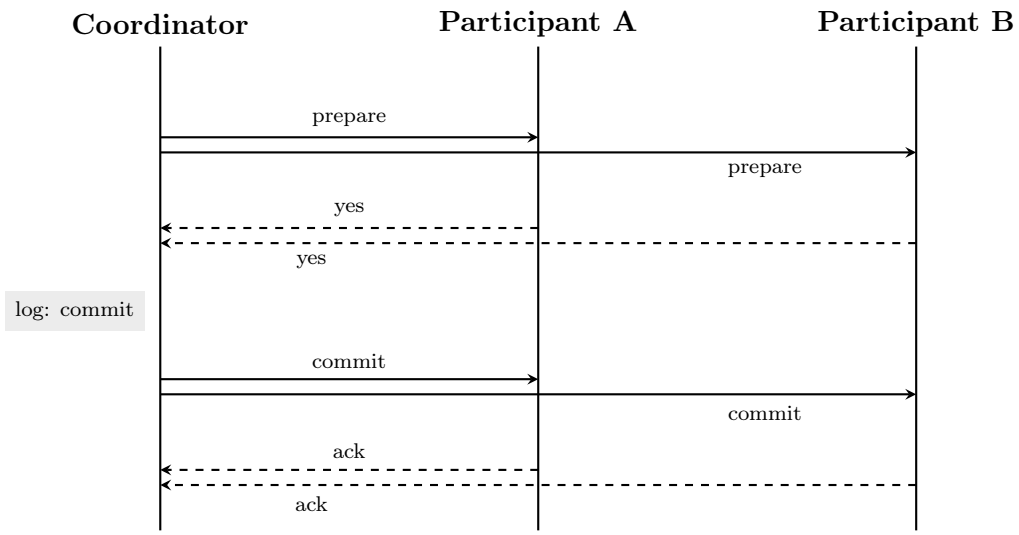


Figure 10.1: A successful two-phase commit. After both participants vote yes and the coordinator logs its decision, the outcome is fixed and is delivered to all participants.

- Latency is at least two round trips plus durable writes, and locks are held across them, which limits throughput under contention.
- Availability is reduced: the transaction can proceed only if *all* participants are reachable, unlike quorum-based writes that need only a majority.

Three-phase commit (3PC) adds a phase so that participants can decide safely when the coordinator fails, but it assumes bounded message delays and cannot tolerate network partitions, so it is rarely used in practice (Skeen, 1981). The practical remedy is to make the coordinator itself fault tolerant.

10.2.3 Consensus-Based Commit

A coordinator that is replicated by a consensus protocol (Paxos or Raft) no longer has the single-point-of-failure property: if the leader of the coordinator group fails, another member takes over with the logged state and finishes the protocol. Gray and Lamport (2006) showed that replacing the coordinator's decision record with a consensus instance gives a commit protocol that is non-blocking as long as a majority of the coordinator group is alive. Google Spanner applies this idea: each partition is a Paxos-replicated group, and a transaction that spans partitions uses 2PC *between* the groups, so each participant and the coordinator are themselves fault-tolerant (Corbett et al., 2012). CockroachDB and TiDB follow related designs. The cost is additional replication latency on every commit, which in a multi-region deployment includes cross-region round trips (Chapter 9).

10.2.4 Deterministic Alternatives

Some systems avoid 2PC by ordering transactions in advance. In a *deterministic* database such as Calvin, a sequencing layer agrees on a global order of transaction inputs, and every

replica then executes them in that order, so replicas reach the same result without a commit protocol (Thomson et al., 2012). This removes coordination at commit time, but it requires that a transaction's read and write sets be known up front or discovered in advance, which limits interactive transactions.

10.3 Isolation Across Nodes

Atomic commit gives all-or-nothing, but it does not decide how concurrent transactions see each other's effects. Chapter 8 introduced lost updates and write skew; this section summarizes how distributed databases prevent them.

10.3.1 Locking and Multi-Version Concurrency

Two-phase locking (2PL) makes transactions acquire locks as they go and hold them until commit, which yields serializability but causes blocking and deadlocks, and in a distributed database the locks are spread across nodes, so deadlock detection needs cooperation. *Multi-version concurrency control* (MVCC) keeps several versions of each item and lets readers see a consistent snapshot without blocking writers. Most distributed databases use MVCC, because long read-only queries (reports, backups) would otherwise stall writers.

10.3.2 Snapshot Isolation and Serializability

Under *snapshot isolation*, each transaction reads from a snapshot taken at its start, and a write conflicts only with a concurrent write to the *same* item. This prevents lost updates and most read anomalies but permits write skew (Berenson et al., 1995). *Serializable* isolation also prevents write skew and can be implemented by 2PL, by optimistic validation, or by serializable snapshot isolation, which tracks read–write dependencies and aborts transactions that would form a dangerous cycle. The stronger the isolation, the more aborts and waiting an application must expect, so it should be chosen for the transactions that need it, as in Chapter 8.

10.3.3 Timestamps and Clocks

Distributed MVCC needs a way to assign each transaction a position in time that all nodes agree on. Several approaches exist:

- **A central timestamp oracle** hands out increasing numbers. It is simple and correct, but it adds a round trip and is a bottleneck and a single point of failure unless replicated.
- **Hybrid logical clocks (HLC)** combine physical time with a logical counter, so timestamps stay close to real time while respecting causality, with no special hardware. Several open-source databases use them.
- **Bounded-uncertainty clocks.** Spanner's TrueTime API reports the current time as an interval whose width is the clock uncertainty. A transaction chooses its commit

timestamp and then waits until that timestamp is certainly in the past before releasing its locks ("commit wait"). This guarantees that timestamp order matches real-time order, which gives *external consistency* (linearizable transactions), at the price of a wait of a few milliseconds that depends on the clock error (Corbett et al., 2012).

The lesson is that clock quality directly affects transaction latency and correctness. Systems that rely on timestamps from unsynchronized clocks, such as last-write-wins, can lose updates when clocks drift (Chapter 9).

10.4 Transactions in Real Systems

Table 10.1 shows how widely used systems expose transactions. Limits and defaults change between versions, so use it as a map to the documentation.

Table 10.1: Transaction support in selected systems (check current documentation).

System	Transaction scope	Isolation	Notes
PostgreSQL	Full ACID on one node; `PREPARE TRANSACTION` for external coordinators	Read committed by default; repeatable read, serializable available	Prepared transactions hold locks until resolved
MongoDB	Multi-document transactions on replica sets and across shards	Snapshot isolation with majority concerns	Cross-shard transactions cost more; keep them short
Cassandra	Single-partition lightweight transactions; batches	Linearizable per partition for LWT	No general multi-partition ACID transactions
DynamoDB	Transactional reads and writes across items, within size limits	Serializable for the transaction's items	Conditions on items; higher capacity cost
Spanner-like systems	Distributed ACID across partitions and regions	Serializable, often externally consistent	Commit latency includes consensus and 2PC

10.4.1 PostgreSQL: Prepared Transactions

PostgreSQL exposes the participant side of 2PC to external coordinators, such as transaction managers using the XA standard, through `PREPARE TRANSACTION` (PostgreSQL Documentation). After the prepare, the transaction survives a crash and keeps its locks until it is committed or rolled back by name. Listing 10.1 shows the sequence. Prepared transactions that an application forgets to resolve hold locks and prevent cleanup, so monitoring for stale ones is essential.

Listing 10.1: A participant in two-phase commit using prepared transactions in PostgreSQL.

```
-- Requires max_prepared_transactions > 0
BEGIN;
UPDATE accounts SET balance = balance - 50 WHERE id = 'acc_1';
PREPARE TRANSACTION 'transfer_8841'; -- vote "yes"; survives a crash

-- Later, after the coordinator has decided:
COMMIT PREPARED 'transfer_8841'; -- or ROLLBACK PREPARED 'transfer_8841';

-- Find transactions waiting for a decision
SELECT gid, prepared FROM pg_prepared_xacts;
```

10.4.2 MongoDB and DynamoDB

MongoDB supports multi-document transactions on a replica set and across shards (MongoDB Documentation). Listing 10.2 shows a transfer written with a session and majority concerns. Transactions that span shards run a commit protocol among the shards and are noticeably slower than single-shard ones, so a good data model keeps data that is updated together in one document, which is atomic by itself, or at least in one shard.

Listing 10.2: A multi-document transaction in the MongoDB shell.

```
const session = db.getMongo().startSession()
const accounts = session.getDatabase("bank").accounts

session.startTransaction({
  readConcern: { level: "snapshot" },
  writeConcern: { w: "majority" }
})
try {
  accounts.updateOne({ _id: "acc_1" }, { $inc: { balance: -50 } })
  accounts.updateOne({ _id: "acc_2" }, { $inc: { balance: 50 } })
  session.commitTransaction()
} catch (e) {
  session.abortTransaction()
  throw e
}
```

DynamoDB offers transactional writes and reads that group several items, with per-item conditions, subject to limits on the number and size of items (Amazon DynamoDB Developer Guide). Cassandra deliberately omits general multi-partition transactions, offering lightweight transactions for single-partition compare-and-set instead (Chapter 8); applications that need more build it with the patterns in the next section.

10.5 Sagas and Compensation

When a business process spans several services, each with its own database, a single ACID transaction is usually unavailable, because the services cannot share a transaction manager and holding locks across them would reduce availability. A *saga* replaces one long transaction with a sequence of local transactions, each committed independently (Garcia-Molina and Salem, 1987). If a step fails, the saga runs *compensating transactions* that semantically undo the steps already completed: a cancelled hotel booking is compensated by a refund, not by erasing the booking.

10.5.1 Example: Order Fulfillment

Consider an order that must reserve stock, charge a payment, and schedule shipping. Table 10.2 lists the steps and their compensations.

Table 10.2: A saga for order fulfillment, with compensating actions.

Step	Action	Compensation	Service
1	Reserve stock	Release reservation	Inventory
2	Charge payment	Refund payment	Payments
3	Schedule shipment	Cancel shipment	Shipping

If shipping fails at step 3, the saga runs the refund and the release in reverse order. Two properties matter for correct design:

- **Compensations must be reliable and idempotent**, since they may themselves be retried, and they cannot be allowed to fail permanently without alerting an operator.
- **Sagas lack isolation.** Other transactions can see the intermediate states, for example stock that is reserved but whose order later fails. Applications handle this with semantic locks (a "pending" status), commutative updates, or by ordering the steps so that the hardest-to-undo ones (such as sending an email or a payment capture) come last.

10.5.2 Orchestration and Choreography

A saga can be coordinated in two ways. In *orchestration*, a central saga coordinator tells each service what to do next and records the saga's state, which makes the flow explicit and easy to monitor but concentrates logic in one component. In *choreography*, each service listens for events and reacts by performing its step and publishing the next event, which has no central controller but spreads the process logic across services and makes it harder to see the whole flow (Richardson, 2018). Orchestration is generally easier to reason about as the number of steps grows. Either way, the saga's state and its events must be persisted reliably, which is exactly the use of the transactional outbox pattern from Chapter 8. Listing 10.3 shows a simple saga state record.

Listing 10.3: Persisted state of an orchestrated order saga.

```
{
  "sagaId": "saga_5001",
  "type": "order-fulfillment",
  "status": "COMPENSATING",
  "steps": [
    { "name": "reserve-stock", "state": "DONE", "compensation": "release-stock" },
    { "name": "charge-payment", "state": "DONE", "compensation": "refund-payment"
        },
    { "name": "schedule-shipment", "state": "FAILED", "compensation": "cancel-
        shipment" }
  ],
  "nextAction": "refund-payment",
  "attempts": 1
}
```

10.6 Idempotence and Delivery Guarantees

Distributed protocols depend on retries: if a request or its reply is lost, the sender cannot tell which, so it sends again. A retried request may therefore be executed twice. Message systems offer three delivery semantics:

- **At-most-once:** messages may be lost but are never duplicated.
- **At-least-once:** messages are never lost but may be duplicated.
- **Exactly-once (effect):** each message has its effect once. This cannot be provided by the network alone; it is achieved by at-least-once delivery combined with deduplication or idempotent processing.

The practical technique is the *idempotency key*: the client generates a unique identifier for each logical operation and sends it with every attempt, and the server records the keys it has already processed, together with their results, and returns the stored result on a repeat. For this to be safe, recording the key and applying the effect must happen in the same local transaction. Listing 10.4 shows the pattern in SQL.

Listing 10.4: Idempotent payment processing with an idempotency key.

```
BEGIN;
-- Fails with a unique violation if this key was already processed
INSERT INTO processed_requests (idempotency_key, result)
VALUES ('pay_7f3c', 'pending');

UPDATE accounts SET balance = balance - 50 WHERE id = 'acc_1' AND balance >= 50;

UPDATE processed_requests SET result = 'ok' WHERE idempotency_key = 'pay_7f3c';
COMMIT;
-- A retry with the same key hits the unique constraint: return the stored result
```

Idempotency keys must be kept long enough to cover the longest plausible retry window, and the stored result should include enough information to answer the retry exactly as the original request would have been answered.

10.7 The Fault Model

Fault tolerance requires a clear statement of which faults are expected. Common models, from mildest to most severe, are:

- **Crash-stop and crash-recovery.** A node stops working, and in the second case later restarts, possibly with its durable state intact and its memory lost.
- **Omission faults.** Messages or operations are lost, though nodes are otherwise correct.
- **Timing faults.** Nodes or networks respond, but too slowly: pauses from garbage collection, overloaded disks, or congested links.
- **Network partitions.** Groups of nodes cannot communicate with each other, although each group is working (Chapter 7).
- **Byzantine faults.** Nodes behave arbitrarily, including maliciously (Lamport et al., 1982). These are addressed by specialized protocols used in blockchains and some safety-critical systems, and most databases, which run on trusted infrastructure, assume they do not occur.

A distinctive and difficult kind of failure is the *gray failure*: a component that is degraded but not dead, for example a disk that is ten times slower, a node that answers health checks but fails real requests, or a link that drops a small percentage of packets. Gray failures defeat simple up/down detection and are a frequent cause of large outages.

10.7.1 Failure Detection

In an asynchronous network, a slow node cannot be told apart from a failed one, and Fischer, Lynch, and Paterson (1985) proved that consensus cannot be guaranteed in such a model if even one node may crash. Practical systems sidestep this impossibility by using timeouts as *unreliable failure detectors* (Chandra and Toueg, 1996): they may wrongly suspect a live node, but they eventually become accurate enough to make progress, and safety is never made to depend on their accuracy. Typical mechanisms are:

- **Heartbeats** sent periodically, with a node suspected after a timeout.
- **Adaptive detectors**, such as the φ accrual detector, which outputs a continuous suspicion level based on the observed distribution of heartbeat arrival times instead of a fixed threshold (Hayashibara et al., 2004), and is used in Cassandra.
- **Gossip**, in which nodes exchange liveness information with random peers, so that no central monitor is needed.

The timeout is a trade-off: a short one detects failures quickly but causes false alarms and unnecessary failovers under load, while a long one delays recovery.

10.8 Techniques for Fault Tolerance

Fault-tolerant systems combine redundancy in the data layer, covered in Chapter 9, with defensive behavior in the services that call it. Table 10.3 summarizes the common patterns.

Table 10.3: Common fault-tolerance patterns.

Pattern	What it does	Caution
Timeouts	Bound the time spent waiting for a response	Too short causes false failures; too long ties up resources
Retries with backoff and jitter	Repeat failed requests after growing, randomized delays	Only safe for idempotent operations; can amplify overload
Circuit breaker	Stops calling a failing dependency for a period and fails fast	Needs sensible thresholds and a fallback
Bulkhead	Isolates resources (threads, connections) per dependency so one failure does not exhaust all	Reduces peak efficiency by reserving capacity
Hedged requests	Sends a duplicate request after a delay and uses the first reply, reducing tail latency	Increases load; needs idempotent reads
Load shedding and backpressure	Rejects or slows work when overloaded, to protect the core	Needs priorities so important work survives
Replication and quorums	Keep working while a minority of replicas fail	Needs correct failover and fencing (Chapter 9)

10.8.1 Retries Done Safely

Retries are the most common and the most dangerous technique. If every client retries immediately when a server is slow, the extra load can push the server over the edge, a pattern called a *retry storm*. Good practice is to retry only idempotent operations (Section 10.6), use *exponential backoff* (delays that double up to a limit), add random *jitter* so that clients do not retry in lockstep, cap the number of attempts, and, when many layers each retry, set a retry budget so that the multiplication of attempts across layers does not occur. A request that passes through three layers that each retry three times can generate up to $3^3 = 27$ attempts at the bottom.

10.8.2 Containing Failures

A circuit breaker watches the error rate of calls to a dependency. When it exceeds a threshold, the breaker "opens" and subsequent calls fail immediately (or use a fallback such as cached data) instead of waiting for timeouts; after a cooldown it lets a few trial calls through and

closes again if they succeed. Bulkheads partition resources so that a slow dependency can consume only its own allotment of threads or connections, keeping the rest of the service responsive. Together with timeouts, they turn a failure of one component into a degraded service instead of a total outage.

10.8.3 Graceful Degradation

The design question for each dependency is "what should the user see if this is unavailable?" An e-commerce page can still render without recommendations; a product page can show cached prices with a note; a checkout may be limited to payment methods that are working. Deciding these fallbacks in advance, per feature, is cheaper and safer than improvising them during an incident, and they connect directly to the use-case approach to consistency in Chapter 8.

10.9 Recovery and Durability

10.9.1 Write-Ahead Logging and Crash Recovery

A database survives a crash by writing changes to a sequential *write-ahead log* (WAL) and forcing it to stable storage *before* acknowledging a commit; modified data pages can be written later. After a crash, the recovery procedure replays the log to redo committed work that had not reached the data files and undoes the effects of uncommitted transactions, as in the ARIES algorithm (Mohan et al., 1992). Periodic *checkpoints* limit how much log must be replayed. In a distributed transaction, a participant's `prepare` record must be in the log before it votes yes, which is how a participant keeps its promise across a crash (Section 10.2).

10.9.2 Backups and Disaster Recovery

As Chapter 9 stressed, replication does not replace backup. Two measures set the targets for recovery planning:

- **Recovery point objective (RPO):** how much recent data the business can afford to lose, measured in time.
- **Recovery time objective (RTO):** how long the service may be unavailable before it is restored.

Table 10.4 relates common strategies to these objectives.

The values depend heavily on implementation and are given only to show the order of magnitude. An untested backup is only a hope; teams should restore regularly into a separate environment and verify the data.

10.9.3 Resolving In-Doubt and Failed Transactions

After a failure, transactions may be left in intermediate states: prepared but undecided 2PC participants, sagas halted between steps, or outbox rows not yet published. Robust systems

Table 10.4: Disaster-recovery strategies and typical objectives.

Strategy	Description	Typical RPO	Typical RTO
Periodic backups	Full and incremental copies stored elsewhere	Hours	Hours
Backups plus log archiving (point-in-time recovery)	Continuous WAL archive allows restoring to a chosen moment	Seconds to minutes	Tens of minutes to hours
Warm standby in another region	Asynchronous replica ready for promotion	Seconds	Minutes
Active–active multi-region	Several regions serve traffic, with synchronous or conflict-resolving replication	Near zero	Near zero

include recovery processes that periodically scan for such states and resolve them according to the logged decision, retry or compensate stuck sagas, and alert an operator when automatic resolution is impossible. Reports of "stuck" transactions that grow over time are an early warning that a coordinator or a downstream service is unhealthy.

10.10 Testing Fault Tolerance

Fault-tolerance mechanisms that are never exercised tend not to work when needed. Practices include:

- **Fault injection.** Tools such as Jepsen introduce partitions, crashes, and clock skew while checking recorded histories against the claimed consistency and isolation level (Jepsen), and have found serious defects in many well-known databases.
- **Chaos engineering.** Deliberately causing failures in a controlled way, such as terminating instances or adding network latency, to confirm that the system degrades as designed, with a clearly defined steady-state measure to compare against (Basiri et al., 2016).
- **Game days and failover drills.** Scheduled exercises in which teams respond to a simulated incident, which test not only software but also runbooks, alerts, and people.
- **Invariant checks and reconciliation.** Periodic jobs that verify business rules across services, for example that every captured payment has a matching order, catch the errors that sagas and retries may leave behind.
- **Observability.** Distributed tracing, metrics, and structured logs that make it possible to see where a transaction or saga is stuck and why.

10.11 Choosing an Approach

The cheapest distributed transaction is the one that is not needed. Table 10.5 gives guidance for common situations.

Table 10.5: Choosing a transaction and fault-tolerance approach by use case.

Use case	**Suggested approach**	**Reasoning**
Related data updated together	Model it in one document, row, or partition	Single-node atomicity is cheap and reliable
Transfer between accounts in one database	Distributed ACID transaction (serializable or snapshot with conditions)	Strong invariants justify the coordination cost
Business process across services	Saga with idempotent steps and compensations	Services stay independent; avoids cross-service locks
Database update plus message publication	Transactional outbox with idempotent consumers	Avoids the dual-write problem without 2PC across systems
Global strongly consistent data	Consensus-replicated, externally consistent store	Correctness across regions, at the price of latency
Calls to unreliable dependencies	Timeouts, backoff with jitter, circuit breakers, fallbacks	Contains failures and prevents cascades

A sound process is: (1) minimize the data that must change atomically by modeling it together; (2) for what remains, decide whether the invariant needs true isolation or can tolerate the temporary inconsistency of a saga; (3) make every operation idempotent; (4) state the fault model and recovery objectives; (5) add timeouts, retries, and containment around every remote call; and (6) test failure paths as carefully as the happy path. For a deeper treatment of these topics, see Kleppmann (2017) and the original papers cited below.

Summary

Distributed transactions must achieve atomicity and isolation across nodes that fail independently. Two-phase commit provides atomic commit but blocks when the coordinator fails after participants have voted, so practical systems replicate the coordinator with consensus, as in Spanner and its relatives, at the cost of extra latency. Distributed isolation is usually implemented with MVCC and timestamps, from timestamp oracles and hybrid logical clocks to bounded-uncertainty clocks with commit wait, and the choice between snapshot isolation and serializability determines which anomalies, such as write skew, remain. Where cross-service transactions are impractical, sagas replace them with local transactions and compensations, coordinated by orchestration or choreography, and at the price of weaker isolation. Because networks lose messages, retries are unavoidable, and correctness depends on idempotency

keys and deduplication, since exactly-once effects are built from at-least-once delivery. Fault tolerance begins with an explicit fault model, from crashes to partitions and gray failures, and with failure detectors that use timeouts and accept occasional mistakes. Timeouts, backoff with jitter, circuit breakers, bulkheads, load shedding, and graceful degradation contain failures, while write-ahead logging, backups with recovery objectives, and recovery processes restore state. Fault injection, chaos engineering, and drills confirm that these mechanisms work. The guiding principle is to avoid distributed transactions where the data model allows, use the weakest mechanism that preserves the invariants that matter, and test the failure paths.

Review Questions

1. What is the atomic commitment problem, and why can't a coordinator simply tell each node to commit?
2. Describe the two phases of two-phase commit. What must a participant guarantee after voting yes?
3. Explain the blocking problem of 2PC. Which failure causes it, and what resources stay locked?
4. How does replicating the coordinator with a consensus protocol remove the blocking problem? What does it cost?
5. Compare snapshot isolation and serializability. Which anomaly does the first allow?
6. How does Spanner's commit wait provide external consistency, and what determines how long it waits?
7. What is a saga? Explain compensating transactions and why sagas lack isolation.
8. Compare orchestration and choreography. When would you prefer each?
9. Why is "exactly-once delivery" not achievable by the network alone? How do idempotency keys provide exactly-once effects?
10. Why can't a timeout-based failure detector distinguish a slow node from a dead one, and how do practical systems cope with this?
11. What is a gray failure? Give two examples and explain why simple health checks miss them.
12. Describe how retries can cause an outage, and list four practices that make them safe.
13. Define RPO and RTO and relate them to three disaster-recovery strategies.
14. How would you test whether a distributed database really provides the isolation level it claims?

Exercises

1. **Two-phase commit.** A coordinator and two participants run 2PC. For each of the following crashes, state what the participants may do and whether they can block: (a) participant B crashes before voting; (b) the coordinator crashes after collecting both yes votes but before logging its decision; (c) the coordinator crashes after logging commit but before sending any message.

2. **Recovery protocol.** Write the recovery rules for a 2PC participant that restarts and finds in its log (a) no prepare record, (b) a prepare record and no decision, (c) a prepare record and a commit record.

3. **Write skew.** Two transactions run under snapshot isolation on a meeting-room booking table, each checking that the room is free and then inserting a booking for the same time under different identifiers. Explain how a double booking occurs and propose two ways to prevent it.

4. **Saga design.** Design a saga for booking a trip with a flight, a hotel, and a car. Give the compensating action for each step, say which step should come last and why, and show a state record similar to Listing 10.3 for a failure at the car rental.

5. **Idempotency.** A payment API receives the same request twice because a response was lost. Show how the idempotency-key table in Listing 10.4 handles (a) a sequential retry and (b) two concurrent retries arriving at the same time.

6. **Retry budgets.** A call passes through four services, each of which retries failed calls up to three times (four attempts in total). How many requests may the deepest service receive for one user request? Propose a retry policy that limits the amplification.

7. **Failure detection.** A heartbeat arrives every second, and a node is suspected after a fixed 3-second timeout. Discuss what happens during a 5-second garbage-collection pause, and how an adaptive detector would behave differently.

8. **Recovery objectives.** An online retailer states an RPO of 1 minute and an RTO of 15 minutes for its order database. Select a disaster-recovery strategy from Table 10.4, and describe how you would verify it meets both objectives.

9. **Chaos experiment.** Design a chaos experiment for a service that depends on a database and a payment provider. State the steady-state metric, the fault injected, the hypothesis, and the condition for aborting the experiment.

References

- Amazon Web Services. *Amazon DynamoDB Developer Guide*. https://docs.aws.amazon.com/dynamodb

- Basiri, A., Behnam, N., de Rooij, R., et al. (2016). Chaos Engineering. *IEEE Software*, 33(3), 35–41.

- Berenson, H., Bernstein, P., Gray, J., Melton, J., O'Neil, E., & O'Neil, P. (1995). A Critique of ANSI SQL Isolation Levels. In *Proceedings of ACM SIGMOD '95*, 1–10.
- Chandra, T. D., & Toueg, S. (1996). Unreliable Failure Detectors for Reliable Distributed Systems. *Journal of the ACM*, 43(2), 225–267.
- Corbett, J. C., Dean, J., Epstein, M., et al. (2012). Spanner: Google's Globally-Distributed Database. In *Proceedings of OSDI '12*.
- Fischer, M. J., Lynch, N. A., & Paterson, M. S. (1985). Impossibility of Distributed Consensus with One Faulty Process. *Journal of the ACM*, 32(2), 374–382.
- Garcia-Molina, H., & Salem, K. (1987). Sagas. In *Proceedings of ACM SIGMOD '87*, 249–259.
- Gray, J. (1978). Notes on Data Base Operating Systems. In *Operating Systems: An Advanced Course*, Springer, 393–481.
- Gray, J., & Lamport, L. (2006). Consensus on Transaction Commit. *ACM Transactions on Database Systems*, 31(1), 133–160.
- Hayashibara, N., Défago, X., Yared, R., & Katayama, T. (2004). The φ Accrual Failure Detector. In *Proceedings of SRDS '04*, 66–78.
- Helland, P. (2007). Life Beyond Distributed Transactions: An Apostate's Opinion. In *Proceedings of CIDR '07*.
- Jepsen. Distributed systems safety research. https://jepsen.io
- Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.
- Lamport, L., Shostak, R., & Pease, M. (1982). The Byzantine Generals Problem. *ACM Transactions on Programming Languages and Systems*, 4(3), 382–401.
- Mohan, C., Haderle, D., Lindsay, B., Pirahesh, H., & Schwarz, P. (1992). ARIES: A Transaction Recovery Method Supporting Fine-Granularity Locking and Partial Rollbacks Using Write-Ahead Logging. *ACM Transactions on Database Systems*, 17(1), 94–162.
- MongoDB Documentation. https://www.mongodb.com/docs/manual/
- PostgreSQL Documentation. https://www.postgresql.org/docs/
- Richardson, C. (2018). *Microservices Patterns*. Manning Publications.
- Skeen, D. (1981). Nonblocking Commit Protocols. In *Proceedings of ACM SIGMOD '81*, 133–142.
- Thomson, A., Diamond, T., Weng, S.-C., Ren, K., Shao, P., & Abadi, D. J. (2012). Calvin: Fast Distributed Transactions for Partitioned Database Systems. In *Proceedings of ACM SIGMOD '12*, 1–12.

Part III

Specialized Databases

Chapter 11

Graph Databases and Relationship Modeling

Learning Objectives

After reading this chapter, you will be able to:

- Explain why connected data is awkward in other data models and when a graph model is the better fit.
- Compare the property graph and RDF models.
- Model a domain as nodes, relationships, labels, and properties, and recognize common modeling patterns and anti-patterns.
- Write basic queries in Cypher, SPARQL, and SQL, and relate them to the emerging GQL and SQL/PGQ standards.
- Describe common graph algorithms and what questions they answer.
- Explain how graph databases store and traverse data, and why scaling them is difficult.
- Compare real graph systems and choose between them.
- Avoid performance pitfalls such as supernodes and unbounded traversals.

11.1 Why Graphs

The data models of the previous chapters organize data around *records*: rows, documents, key–value pairs, or wide columns (Chapters 3 and 4). Relationships between records are present in all of them, but they are second-class: a foreign key in a relational table, a reference inside a document, or an identifier stored in a value. Following these references means looking up the target record, and following a chain of them means a chain of lookups or joins.

For many questions this is perfectly adequate. For others, the relationships *are* the data and the question is about the structure they form:

- Who are the friends of my friends who are not yet my friends?
- Which accounts are connected, through shared devices, addresses, or payments, to an account flagged for fraud?

- If this server fails, which services and customers are affected?
- What is the shortest route between two stations, or the cheapest path of suppliers for a part?
- Which products do people with similar purchase histories also buy?

A *graph database* treats the connections as first-class citizens: they are stored explicitly, they carry their own data, and the query language is built to follow them. Graph models have long been studied in database research (Angles and Gutierrez, 2008), and they have become a mainstream category of systems because modern applications, from social networks to fraud detection and knowledge management, contain data that is naturally connected.

11.1.1 The Join Problem

In a relational database, a friendship network is typically stored in a table of pairs of person identifiers. Finding friends of friends requires joining this table to itself; finding friends at depth four requires four self-joins. The number of intermediate rows can grow roughly exponentially with the depth, and the query becomes hard to write and hard for the optimizer to execute well. Listing 11.1 shows friends-of-friends in SQL with a recursive query. It works, and relational systems with good indexes handle moderate depths efficiently, but the formulation is awkward compared with the graph equivalent in Listing 11.2. The advantage of a native graph engine grows with the depth of traversal and the connectedness of the data, not with the size of the dataset as such.

Listing 11.1: Friends of friends in SQL using a recursive query (depth at most 2).

```
WITH RECURSIVE reach(person_id, depth) AS (
  SELECT friend_id, 1 FROM friendships WHERE person_id = 1
  UNION
  SELECT f.friend_id, r.depth + 1
  FROM reach r
  JOIN friendships f ON f.person_id = r.person_id
  WHERE r.depth < 2
)
SELECT DISTINCT person_id FROM reach
WHERE depth = 2
  AND person_id <> 1
  AND person_id NOT IN (SELECT friend_id FROM friendships WHERE person_id = 1);
```

11.2 Graph Data Models

A graph consists of *vertices* (nodes) and *edges* (relationships) between them. Two data models dominate practice. Table 11.1 compares them.

11.2.1 The Property Graph

In the *property graph* model, a database contains nodes and relationships (Angles, 2018). Each node may have one or more *labels* (such as `Person` or `Product`) that classify it. Each

Table 11.1: The two dominant graph data models.

Aspect	**Property graph**	**RDF (triple store)**
Basic unit	Nodes and relationships, each with labels or types and key–value properties	Subject–predicate–object triples
Properties on relationships	Directly supported	Needs reification, named graphs, or RDF-star extensions
Identifiers	Internal or application-defined	Globally unique IRIs
Schema	Optional; labels and constraints	Ontologies and vocabularies (RDFS, OWL), with inference
Query languages	Cypher, Gremlin, GQL	SPARQL
Typical use	Applications: social, fraud, recommendation, networks	Data integration, linked data, knowledge graphs, semantics

relationship connects exactly two nodes, has a direction and a single *type* (such as `FOLLOWS` or `BOUGHT`). Both nodes and relationships may carry any number of *properties*, which are key–value pairs. Figure 11.1 shows a small example. The ability to attach properties to relationships, such as the date a person followed another or the quantity in a purchase, is one of the model's most useful features.

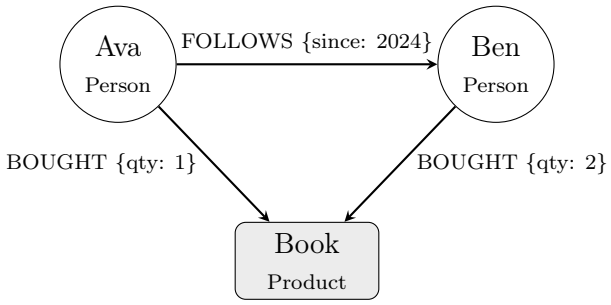


Figure 11.1: A small property graph. Nodes carry labels (Person, Product); relationships have a type and a direction, and may carry their own properties.

11.2.2 RDF and Triple Stores

The *Resource Description Framework* (RDF) represents all information as triples of the form (subject, predicate, object), such as (`ex:ava`, `ex:follows`, `ex:ben`). Subjects and predicates are identified by globally unique IRIs, and objects are either IRIs or literal values. A set of triples is a directed labeled graph. Because identifiers are global, data from different sources can be merged simply by taking the union of their triples, which is why RDF is the foundation of linked data and many knowledge graphs (Hogan et al., 2021). RDF is accompanied by vocabularies and ontology languages (RDFS and OWL) that describe classes

and relationships and allow *inference*: new triples can be derived from existing ones by rules, such as deducing that every `Manager` is also an `Employee`. The standard query language is SPARQL (Listing 11.4).

11.2.3 Other Models

Some systems use a *hypergraph*, in which an edge may connect more than two vertices, and multi-model databases such as ArangoDB combine documents, key–value pairs, and graphs behind a single engine. Relational databases can also treat tables as graphs, an idea now standardized in SQL (Section 11.4).

11.3 Relationship Modeling

Modeling for a graph database is closer to drawing a whiteboard diagram than to normalizing tables. A frequently quoted advantage is that the domain model and the physical model are almost the same, which reduces the translation effort between design and implementation (Robinson et al., 2015). Good models still require care.

11.3.1 A Modeling Process

1. **List the questions** the application must answer. A graph model is shaped by its traversals, much as column-family models are shaped by queries (Chapter 4).
2. **Identify the entities** in those questions; they become nodes with labels.
3. **Identify the connections** between entities; they become relationships with verbs as types (`PURCHASED`, `REPORTS_TO`, `LOCATED_IN`).
4. **Assign properties**: facts about an entity go on the node, and facts about the connection itself go on the relationship.
5. **Test the model** by writing the queries for each question and checking that they are natural and efficient.

11.3.2 From Relational to Graph

Table 11.2 shows how familiar relational constructs translate.

11.3.3 Modeling Patterns

Several patterns recur across domains:

- **Intermediate (hyper-edge) nodes.** A relationship connects exactly two nodes. When an event involves several parties (a meeting with attendees, a transaction with a buyer, a seller, and a payment method), model the event itself as a node and connect the participants to it. This also lets the event have its own relationships later.
- **Time-based structures.** To record history, add `valid_from` and `valid_to` properties to relationships, or build a linked list of state nodes (`NEXT` relationships) so that the latest state and the full history can both be traversed.

Table 11.2: Mapping relational constructs to a property graph.

Relational construct	**Graph equivalent**
Row of an entity table	Node, with the table name as its label
Column	Property on the node
Foreign key	Relationship from the referencing node to the referenced node
Join table with extra columns	Relationship with properties
Join table without extra columns	Plain relationship
Lookup or reference table (such as country)	Node, if it is traversed or shared; otherwise a property
Self-referencing foreign key (hierarchy)	Relationship between nodes of the same label

- **Hierarchies and categories.** Trees such as organization charts and product categories are natural: each node points to its parent, and a variable-depth traversal retrieves any subtree, which is awkward in plain SQL.

- **Promoting values to nodes.** If a value is shared and used for navigation, such as a city, a tag, or a device identifier, make it a node. Two accounts that share a device then become connected through that device, exactly the kind of link fraud analysis needs.

11.3.4 Common Mistakes

- **Relational thinking.** Creating nodes that only play the role of join tables, or storing identifiers of other nodes as properties instead of creating relationships, discards the advantage of the model.

- **Everything as one relationship type.** A generic `RELATED_TO` relationship with a `kind` property forces every traversal to filter, while specific types let the engine follow only the relevant edges.

- **Too many labels or types, or labels with no meaning.** Labels should express roles that are queried; hundreds of ad hoc ones make the model hard to maintain.

- **Large properties on nodes in traversal paths.** Big text or binary values slow traversals and are better stored in an object store or document database and referenced.

- **Ignoring direction and supernodes.** Section 11.9 treats this in detail.

11.4 Querying Graphs

Graph query languages are built around *pattern matching*: the query describes a shape, and the engine finds all subgraphs that match it.

11.4.1 Cypher

Cypher, created for Neo4j and later opened as the openCypher project, expresses patterns in an ASCII-art style: nodes in parentheses and relationships in square brackets with arrows (Francis et al., 2018). Listing 11.2 creates data and finds friends of friends, the same question as Listing 11.1.

Listing 11.2: Creating data and finding friends of friends in Cypher.

```
// Create two people and a relationship with a property
CREATE (a:Person {name: 'Ava', email: 'ava@example.com'})
       -[:FOLLOWS {since: 2024}]->
       (b:Person {name: 'Ben', email: 'ben@example.com'});

// Friends of friends that Ava does not already follow
MATCH (ava:Person {name: 'Ava'})-[:FOLLOWS]->(:Person)-[:FOLLOWS]->(fof:Person)
WHERE fof <> ava AND NOT (ava)-[:FOLLOWS]->(fof)
RETURN DISTINCT fof.name;
```

Variable-length patterns and path functions make questions about reachability short. Listing 11.3 finds the shortest connection between two people and produces product recommendations by following purchases of similar customers.

Listing 11.3: Shortest path and a collaborative-filtering recommendation in Cypher.

```
// Shortest chain of follows, ignoring direction, up to 6 hops
MATCH p = shortestPath(
  (a:Person {name: 'Ava'})-[:FOLLOWS*..6]-(z:Person {name: 'Zoe'}))
RETURN length(p);

// Products bought by people who bought what Ava bought
MATCH (u:Person {name: 'Ava'})-[:BOUGHT]->(:Product)
      <-[:BOUGHT]-(other:Person)-[:BOUGHT]->(rec:Product)
WHERE NOT (u)-[:BOUGHT]->(rec)
RETURN rec.name, count(*) AS score
ORDER BY score DESC
LIMIT 5;
```

11.4.2 SPARQL

SPARQL queries RDF data with triple patterns containing variables, which the engine matches against the stored triples (Listing 11.4). Because predicates and classes are IRIs, SPARQL queries can also federate over several endpoints, which supports integration of data published by different organizations.

Listing 11.4: Friends of friends in SPARQL.

```
PREFIX ex: <http://example.org/>

SELECT DISTINCT ?name
WHERE {
  ex:ava ex:follows ?friend .
  ?friend ex:follows ?fof .
  ?fof ex:name ?name .
  FILTER (?fof != ex:ava)
  FILTER NOT EXISTS { ex:ava ex:follows ?fof }
}
```

11.4.3 Gremlin, GQL, and SQL/PGQ

Gremlin, part of Apache TinkerPop, is a functional *traversal* language: a query is a chain of steps that move a "traverser" along the graph, which suits programmatic use and is supported by many systems (Rodriguez, 2015). The field has recently moved toward standardization. *GQL* (ISO/IEC 39075:2024) is an international standard for a declarative property-graph query language, strongly influenced by Cypher, and *SQL/PGQ*, part of the SQL:2023 standard, adds graph pattern matching over tables viewed as a graph, so that relational data can be queried with graph patterns without being copied (Deutsch et al., 2022). Support in products is still growing, so portability between graph systems today depends on the dialect each system supports.

11.5 Graph Algorithms

Beyond pattern matching, graphs support algorithms that reveal structure not visible in single records. Table 11.3 lists the most common families.

11.5.1 PageRank

PageRank, introduced to rank web pages, assigns each vertex a score reflecting the scores of the vertices that link to it (Page et al., 1999). For a graph with N vertices, damping factor d (commonly 0.85), and $L(u)$ the number of outgoing links of vertex u, the score of vertex v satisfies

$$PR(v) = \frac{1 - d}{N} + d \sum_{u \in B(v)} \frac{PR(u)}{L(u)}, \tag{11.1}$$

where $B(v)$ is the set of vertices that link to v. The equation is solved by iteration: start with equal scores and repeatedly recompute them until they stop changing. The same idea ranks influential people in a social network, important documents in a citation network, and critical components in a dependency graph.

Table 11.3: Common graph algorithms and the questions they answer.

Family	Example algorithms	Typical question
Traversal and paths	Breadth-first and depth-first search, shortest path, Dijkstra, A*	What is the quickest route? How far apart are two entities?
Centrality	Degree, betweenness, PageRank	Which nodes are most influential or most critical?
Community detection	Connected components, label propagation, Louvain	Which groups of nodes are tightly connected?
Similarity	Jaccard similarity, node embeddings	Which entities resemble each other?
Link prediction	Common neighbors, learned embeddings	Which connections are likely to appear?

11.5.2 Where Algorithms Run

Algorithms that touch the whole graph, such as PageRank and community detection, are *analytical* workloads and differ from the *transactional* workloads of pattern queries around a few start nodes. Many products distinguish the two: a transactional graph database serves application queries, while a graph analytics library or engine, often operating on an in-memory projection or an exported copy, runs global algorithms. Distributed analytics systems follow the *bulk synchronous parallel* model of Pregel, in which each vertex runs a function, sends messages to its neighbors, and waits for all vertices to finish a step (a "superstep") before the next begins (Malewicz et al., 2010).

11.6 Storage and Traversal Internals

11.6.1 Index-Free Adjacency

A native graph database stores each node together with direct references (pointers or record identifiers) to its relationships, and each relationship with references to its two end nodes and its neighbors in the chain. To follow a relationship, the engine follows a pointer rather than consulting an index or computing a join. This property is called *index-free adjacency*. The cost of a traversal step is then independent of the total size of the graph and depends only on the number of relationships actually visited, which is why traversal-heavy queries stay fast as the dataset grows, provided that the query starts from a small, indexed set of nodes.

11.6.2 Native and Non-Native Storage

Systems differ in how they store graphs:

- **Native graph storage and processing**, as in Neo4j, is designed around adjacency,

which gives fast traversals.

- **Graphs on top of other stores**, as in JanusGraph, which stores vertices and edges in a wide-column or key–value backend such as Cassandra or HBase, inherit the scalability and replication of the backend (Chapters 4 and 9) but pay extra lookups for each hop.

- **Multi-model systems** store several kinds of data in one engine and add graph traversal as one of the access paths.

- **Graph views over relational data**, using SQL/PGQ or extensions, avoid a separate system when graph queries are only an occasional need.

11.6.3 Indexes Still Matter

Index-free adjacency applies to moving from a node to its neighbors. Finding the *starting* nodes of a query, such as the person with a given e-mail address, still needs an ordinary index or constraint (Listing 11.5). A graph query that begins by scanning every node of a label negates the advantage of the structure.

Listing 11.5: Index and uniqueness constraint in Cypher (Neo4j 5 syntax).

```
CREATE INDEX person_name FOR (p:Person) ON (p.name);

CREATE CONSTRAINT person_email_unique
FOR (p:Person) REQUIRE p.email IS UNIQUE;
```

11.7 Scaling and Distributing Graphs

Chapter 9 showed that partitioning works best when data can be split into independent pieces. Graphs are the hard case, because their value comes from connections that cross any boundary.

11.7.1 The Partitioning Problem

Splitting a graph over several machines means that some relationships connect nodes on different machines, so a traversal must cross the network. A good partitioning minimizes these cut edges while keeping the machines balanced, which is an NP-hard problem in general, and real graphs such as social networks have no clean boundaries, so any partitioning leaves many edges cut. Hash partitioning of nodes balances the load but ignores structure, so most hops become remote calls; structure-aware partitioning reduces the cut but is expensive to compute and maintain as the graph changes. The practical result is that multi-hop traversals on very large, distributed graphs have much higher and more variable latency than on a single machine.

11.7.2 Common Scaling Strategies

- **Scale up first.** A single machine with large memory holds a surprisingly large graph, and many applications never need more. This is consistent with the advice in earlier chapters to begin with the simplest design.
- **Replicate for reads.** One writer with several read replicas scales query throughput, using the replication techniques of Chapter 9. Neo4j's cluster architecture, for example, uses Raft-based consensus among primary servers for writes and secondary servers for read scaling (Neo4j Documentation).
- **Shard by domain boundaries.** Split the graph along natural boundaries, such as one graph per tenant, or by region when relationships rarely cross; some systems support *composable* or federated queries across several graphs.
- **Use a distributed graph engine.** Systems designed for distribution partition vertices and edges across servers and process queries in parallel, accepting the cost of cross-partition hops.
- **Separate transactional and analytical workloads.** Run global algorithms on a replica, projection, or dedicated analytics cluster so that they do not disturb application queries.

11.7.3 Transactions and Consistency

Most native graph databases offer ACID transactions on a single instance or cluster, with writes ordered by a leader. A notable constraint arises from the structure itself: a relationship cannot exist without its two end nodes, so deleting a node requires first deleting its relationships (or using a "detach" delete), and concurrent transactions that touch the same node can contend for its locks. In a distributed graph, the issues of Chapters 8 and 10 apply as for any partitioned store: a write that creates a relationship across partitions needs a distributed commit or must accept eventual consistency, and replicas may serve stale traversals.

11.8 Graph Databases in Practice

Table 11.4 summarizes widely used systems. As elsewhere in this book, features and licensing change between versions, so treat it as a map to the documentation.

11.8.1 Typical Applications

Knowledge graphs have received renewed attention as a source of structured, verifiable context for language-model applications, where the explicit relationships help to ground answers and to trace where a fact came from (Hogan et al., 2021). In this role the graph is usually combined with a vector index or full-text search, with each answering the part of the question it handles best.

Table 11.4: Selected graph database systems (check current documentation).

System	Model and storage	Query languages	Notes
Neo4j	Property graph; native storage	Cypher	Large ecosystem; clustering with Raft; graph data science library
Amazon Neptune	Property graph and RDF; managed service	openCypher, Gremlin, SPARQL	Storage replicated across zones; read replicas
JanusGraph	Property graph; stores data in Cassandra, HBase, or others	Gremlin	Scalability inherited from backend; open source
TigerGraph	Property graph; distributed native engine	GSQL, with openCypher support	Designed for deep, parallel analytics
ArangoDB	Multi-model: documents, key–value, graph	AQL	One engine for several models
Memgraph	Property graph; in-memory	Cypher	Focus on streaming and low-latency queries
RDF stores (e.g. GraphDB, Virtuoso, Apache Jena)	RDF triples	SPARQL	Ontologies and reasoning; linked data

11.9 Performance Pitfalls

- **Supernodes (dense nodes).** A node with millions of relationships, such as a popular celebrity, a country, or a common tag, makes every traversal through it enormous. Remedies include modeling more specific relationship types, splitting the node (for example one node per country per year), restricting traversals by type or direction, and adding a filter early so that the engine does not expand the entire neighborhood.

- **Unbounded traversals.** A pattern such as `[:FOLLOWS*]` with no depth limit can explore the whole graph. Always bound the depth and start from a selective node.

- **Cartesian products.** Two unconnected patterns in one query multiply their results; connect the patterns or run them separately.

Table 11.5: Typical applications of graph databases.

Application	Graph structure	Typical queries
Fraud detection	Accounts, devices, addresses, cards, and transactions as nodes linked by shared attributes	Rings of accounts sharing devices; distance to a known fraudulent account
Recommendations	Customers, products, and purchases or views	Items bought by similar customers; shared interests
Knowledge graphs	Entities and facts from many sources, often RDF	Entity lookup, question answering, retrieval for AI systems
IT and network operations	Servers, services, dependencies, and users	Impact analysis: what depends on this component?
Identity and access	Users, groups, roles, resources, and permissions	Can this user reach this resource through any chain of memberships?
Supply chain	Suppliers, parts, products, and locations	Upstream and downstream effects of a disruption

- **Missing starting indexes.** Without an index or constraint on the starting property, the engine scans all nodes with a label.
- **Late filtering.** Filtering at the end of a long traversal does the work of exploring everything and then discards it. Put conditions into the pattern so that they prune early.
- **Using the graph for everything.** Aggregations over entire tables, bulk exports, and full-text search are often better served by other systems. A graph database complements, and does not always replace, relational or document stores.

Most graph systems provide an `EXPLAIN` or `PROFILE` facility that shows the query plan and the number of records touched; reading it is the first step in tuning, as with any database.

11.10 Choosing a Graph Approach

Table 11.6 gives guidance by situation.

A sound process is: (1) write down the questions and check whether they depend on variable-depth traversal or on relationship structure; (2) estimate the data size, the typical traversal depth, and the connectivity, including likely supernodes; (3) prototype the two or three most important queries on a realistic dataset in the candidate systems; (4) decide how the graph will be kept in sync with other systems of record, for instance through change data capture; and (5) plan for scaling by starting with a single, well-provisioned instance with read replicas and moving to distribution only when measurements require it.

Table 11.6: Choosing a data model for connected data.

Situation	**Suggested approach**	**Reasoning**
Mostly records, with occasional joins of depth one or two	Relational or document database	Joins at shallow depth are efficient and well understood
Variable-depth traversals, path finding, and relationship-centric questions	Native property graph	Traversal cost depends on the neighborhood, not the dataset size
Integrating many sources with shared vocabularies and inference	RDF triple store with SPARQL	Global identifiers and ontologies suit data integration
Occasional graph queries over existing relational data	SQL recursive queries or SQL/PGQ where available	No second system to operate or synchronize
Very large graph with global analytics	Distributed graph engine, or graph analytics on a data platform	Needs parallel processing of the whole graph
One database for mixed data and light graph needs	Multi-model database	Fewer systems, with moderate graph capabilities

Summary

Graph databases make relationships first-class by storing them explicitly, giving them types, directions, and properties, and by providing query languages built around pattern matching. The property graph model, with labeled nodes and typed relationships that carry properties, suits application development, while RDF, with its globally identified triples and ontologies, suits data integration and knowledge graphs. Modeling starts from the questions to be answered, turns entities into nodes and connections into specifically typed relationships, and uses patterns such as intermediate nodes, time-based structures, and promoted values, while avoiding relational habits and generic relationship types. Cypher, SPARQL, Gremlin, and the new GQL and SQL/PGQ standards express traversals far more compactly than recursive SQL. Graph algorithms such as shortest path, PageRank, and community detection reveal structure and are usually run as analytical workloads, often with Pregel-style distributed processing. Native storage with index-free adjacency makes traversal cost depend on the neighborhood visited and not on the size of the dataset, although starting nodes still need indexes. Distribution is the hard problem, because graphs resist partitioning and cut edges turn hops into network calls, so teams scale up, replicate for reads, shard along natural boundaries, and separate analytics from transactions. Supernodes, unbounded traversals, and late filtering are the main performance traps. Graphs complement other models, and the right choice follows from the questions asked of the data.

Review Questions

1. Why are relationships "second-class" in the relational and document models? Give an example of a question for which this becomes a problem.
2. Describe the property graph model. What are labels, relationship types, and properties?
3. Compare property graphs and RDF. When is each more appropriate?
4. Explain how a join table with extra columns is represented in a graph. Why is this useful?
5. What is an intermediate node, and when should an event be modeled as one?
6. Why is a single generic `RELATED_TO` relationship type a modeling mistake?
7. What is index-free adjacency, and why does it make traversal cost independent of total graph size? Why are indexes still needed?
8. Explain what PageRank computes and how it is calculated iteratively.
9. Distinguish transactional graph queries from analytical graph workloads. How do systems separate them?
10. Why is partitioning a graph harder than partitioning a key–value store? What are the consequences for latency?
11. What is a supernode? Give two ways to reduce its impact.
12. What are GQL and SQL/PGQ, and why do they matter for portability?
13. Describe three situations where a graph database is *not* the best choice.

Exercises

1. **Modeling.** A university stores students, courses, instructors, departments, and enrollments (with a grade and a semester). Draw a property graph model, list labels, relationship types, and properties, and decide whether the semester should be a property or a node.
2. **Relational to graph.** A relational schema has tables `customers`, `orders`, `order_items` (with quantity and price), and `products`. Convert it to a graph model and write a Cypher query for the products most often bought together with a given product.
3. **Queries.** Using the model of Figure 11.1, write Cypher queries for (a) all people who bought the same product as Ava, (b) the total quantity bought of each product, and (c) people who follow someone who bought a product that they have not bought themselves.

4. **SQL versus graph.** Rewrite Listing 11.3 (the recommendation query) in SQL. Compare readability and discuss how the cost of each version grows as the number of hops grows.

5. **PageRank.** For a graph with three pages where A links to B and C, B links to C, and C links to A, perform two iterations of PageRank with $d = 0.85$ starting from equal scores. Which page ranks highest, and why?

6. **Supernodes.** A social network has a node for each country, and 400 million users are connected to their country through `LIVES_IN`. Explain why queries through the country node are problematic and propose two remodelings.

7. **Fraud rings.** Design a graph model for detecting accounts that share devices, phone numbers, or addresses. Write a Cypher query that finds all accounts within three hops of a known fraudulent account, and explain how you would bound its cost.

8. **Distribution.** A graph with 10 billion relationships no longer fits on one machine. Compare hash partitioning with domain-based sharding for a multi-tenant application and for a single global social graph.

9. **Technology choice.** For each of (a) a company knowledge graph integrating data from 40 sources, (b) a real-time recommendation engine, and (c) an access-control checker for a large enterprise, choose a model and a product category from Table 11.4, and justify the choice.

References

- Amazon Web Services. *Amazon Neptune User Guide*. https://docs.aws.amazon.com/neptune

- Angles, R. (2018). The Property Graph Database Model. In *Proceedings of the 12th Alberto Mendelzon International Workshop on Foundations of Data Management (AMW)*.

- Angles, R., & Gutierrez, C. (2008). Survey of Graph Database Models. *ACM Computing Surveys*, 40(1), Article 1.

- Apache TinkerPop Documentation. https://tinkerpop.apache.org/docs/current/

- Deutsch, A., Francis, N., Green, A., et al. (2022). Graph Pattern Matching in GQL and SQL/PGQ. In *Proceedings of ACM SIGMOD '22*, 2246–2258.

- Francis, N., Green, A., Guagliardo, P., et al. (2018). Cypher: An Evolving Query Language for Property Graphs. In *Proceedings of ACM SIGMOD '18*, 1433–1445.

- Hogan, A., Blomqvist, E., Cochez, M., et al. (2021). Knowledge Graphs. *ACM Computing Surveys*, 54(4), Article 71.

- International Organization for Standardization. (2024). *ISO/IEC 39075:2024, Information technology — Database languages — GQL*.
- JanusGraph Documentation. https://docs.janusgraph.org
- Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.
- Malewicz, G., Austern, M. H., Bik, A. J. C., et al. (2010). Pregel: A System for Large-Scale Graph Processing. In *Proceedings of ACM SIGMOD '10*, 135–146.
- Neo4j Documentation. https://neo4j.com/docs/
- Page, L., Brin, S., Motwani, R., & Winograd, T. (1999). *The PageRank Citation Ranking: Bringing Order to the Web*. Technical Report, Stanford InfoLab.
- Robinson, I., Webber, J., & Eifrem, E. (2015). *Graph Databases* (2nd ed.). O'Reilly Media.
- Rodriguez, M. A. (2015). The Gremlin Graph Traversal Machine and Language. In *Proceedings of the 15th Symposium on Database Programming Languages (DBPL)*.
- World Wide Web Consortium. *RDF 1.1 Concepts and Abstract Syntax* and *SPARQL 1.1 Query Language*. https://www.w3.org/TR/

Chapter 12

Time-Series Databases and Temporal Data

Learning Objectives

After reading this chapter, you will be able to:

- Recognize the workload characteristics that distinguish time-series data from other data.
- Model time series using timestamps, tags (labels), and fields, and explain the cardinality problem.
- Describe how time-series databases ingest, compress, and partition data.
- Write time-oriented queries: bucketing, rates, windows, and downsampling.
- Design retention and rollup policies that balance cost and detail.
- Distinguish valid time from transaction time, and model history with temporal tables and range types.
- Handle time zones, clock skew, and late-arriving data correctly.
- Compare time-series systems and choose one for a given workload.

12.1 Data That Arrives With a Timestamp

Many of the most voluminous datasets in modern organizations are measurements of something over time: CPU load on thousands of servers, temperature readings from sensors, stock prices, clicks and requests, the position of vehicles, the consumption of electricity. Each observation consists of a timestamp, an identification of what was measured, and one or more values. The sequence of observations of one thing is a *time series*.

The previous chapters introduced data models organized around records, wide rows, and relationships (Chapters 3, 4, and 11). Time-series data can be stored in any of them, and for small volumes a plain relational table is perfectly adequate. At larger scale, however, its workload differs from the usual transactional one in ways that specialized systems exploit (Jensen et al., 2017; Dunning and Friedman, 2014):

- **Append-mostly writes.** New data arrives continuously in time order, and existing

points are rarely updated or deleted individually.

- **High ingest rates.** Millions of points per second are common, arriving in small, frequent writes from many sources.
- **Recent data is hot.** Most reads concern the last minutes or hours, for dashboards and alerts, while older data is read mainly in aggregate.
- **Queries are range scans and aggregations.** A typical question is "the average per five minutes of metric X for the last day," not a lookup of one record.
- **Value decays with age.** Detail is valuable now and much less so next year, so data is aggregated and eventually deleted according to policy.
- **Large, regular volume.** Data volume grows steadily and predictably, which makes capacity planning and compression important.

A *time-series database* (TSDB) is a system designed around this workload. This chapter describes its data model, its storage techniques, and its query features. The second half broadens the subject to *temporal data* in general: how to represent the history of ordinary business data, which has a time dimension too, and how to deal with the many ways time can go wrong.

12.2 Data Models for Time Series

12.2.1 Series, Tags, and Fields

Most TSDBs organize data into the following elements:

- A **measurement** or **metric name**, such as `cpu_usage` or `http_requests`.
- A set of **tags** (also called **labels**), which are key–value pairs that identify the source, such as `host=web-01` and `region=eu`. A unique combination of metric name and tag values identifies one *series*.
- One or more **fields** or **values**, the measured numbers (or sometimes strings), such as `usage=73.2`.
- A **timestamp**.

Tags are indexed and used for filtering and grouping; fields are not. The distinction affects both performance and correctness: values that identify a source belong in tags, and values that are measured belong in fields. Table 12.1 shows how three popular systems realize the model.

Table 12.1: How three systems represent a time-series point.

System	Model	Example
InfluxDB	Measurement, tags, fields, timestamp; written in line protocol	`cpu,host=web-01,region=eu usage=73.2 1735689600000000000`
Prometheus	Metric name with labels; one float per sample; samples are scraped (pulled) from targets	`http_requests_total{job="api",code="200"}`
TimescaleDB	A relational table with a time column; tags and fields are ordinary columns	Row: `(time, device_id, temperature)`

12.2.2 Narrow and Wide Layouts

A *narrow* layout stores one row per metric per timestamp, with columns such as (`time`, `metric`, `value`). It is flexible, since new metrics need no schema change, but repeats identifiers and makes queries across several metrics need joins or pivots. A *wide* layout stores several fields of the same source in one row, (`time`, `device_id`, `temperature`, `humidity`, `pressure`), which is compact and convenient when the metrics are always collected together. Relational TSDBs favor the wide layout, while Prometheus-style systems use the narrow one.

12.2.3 The Cardinality Problem

The number of distinct series is the *cardinality* of the data set. It equals the number of distinct combinations of tag values that actually occur, and in the worst case it is the product of the numbers of values of each tag:

$$\text{series} \le \prod_{i=1}^{k} |V_i|, \tag{12.1}$$

where V_i is the set of values of the i-th tag. For instance, 1 000 hosts, 50 endpoints, and 5 status codes can produce up to $1000 \times 50 \times 5 = 250\,000$ series. Each series carries index entries and in-memory structures, so cardinality, rather than raw point volume, is often what exhausts memory and slows queries. The most common mistake is to put an unbounded value such as a user identifier, a session identifier, a request identifier, or a full URL with parameters into a tag: every new value creates a new series, and the index grows without limit. Such values belong in logs, traces, or a separate analytic store, and tags should be limited to values drawn from a small, bounded set.

12.3 Ingestion and Storage

12.3.1 The Write Path

TSDBs usually use a log-structured design similar to the one used in the column-family stores of Chapter 4 (O'Neil et al., 1996). Incoming points are appended to a write-ahead log for durability (Chapter 10) and held in an in-memory structure. Periodically the memory contents are written to immutable, time-ordered files, which are later merged (compacted) into larger ones. Because writes are sequential appends, this design sustains very high ingest rates. Several practical issues arise:

- **Batching.** Clients should send points in batches rather than one at a time, since per-request overhead dominates otherwise.
- **Out-of-order and late data.** Points do not always arrive in time order: devices buffer data while offline, and networks reorder. Systems differ in how far in the past they accept writes, and some reject points older than a configured window.
- **Duplicates and idempotence.** A retried batch may deliver the same point twice. Many systems treat a point with the same series and timestamp as an overwrite, which makes writes idempotent (Chapter 10); others store both and rely on queries to deduplicate.
- **Push and pull.** In a push model, sources send data to the database; in a pull model, as in Prometheus, the server scrapes sources at intervals, which makes the server the authority on timing and gives a built-in health signal when a scrape fails.

12.3.2 Compression

Time-series data compresses extremely well, because consecutive timestamps are regular and consecutive values are similar. Typical techniques, used by Facebook's Gorilla system and many successors (Pelkonen et al., 2015), include:

- **Delta-of-delta encoding for timestamps.** If points arrive at regular intervals, the *difference between successive differences* is almost always zero and can be stored in a single bit. For timestamps 1000, 1060, 1120, 1181 the deltas are 60, 60, 61 and the delta-of-deltas are 0 and 1.
- **XOR encoding for floating-point values.** Successive values are combined by exclusive-or; when values change slowly, most bits of the result are zero, and only the changing bits are stored.
- **Dictionary and run-length encoding** for repeated strings and constant values.
- **Columnar storage and general-purpose compression** applied to blocks of one column, which works well because the values in a column are similar.

The Gorilla paper reported that production data could be stored in roughly one to two bytes per point, compared with 16 bytes for an uncompressed timestamp and double-precision value. Actual ratios depend strongly on the data: regular, slowly changing values compress much better than noisy ones.

12.3.3 Time-Based Partitioning

TSDBs split data into *chunks* by time interval, such as one chunk per day, as Figure 12.1 shows. The partitioning follows the access pattern: queries for a time range touch only the relevant chunks, recent chunks stay in memory or on fast storage, and old chunks can be compressed, moved to cheaper storage, or dropped as a whole, which is far cheaper than deleting individual rows.

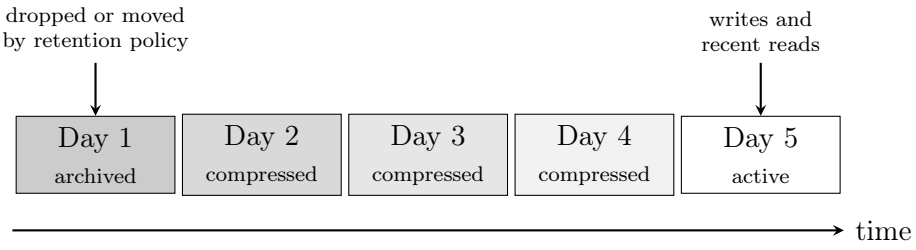


Figure 12.1: Time-based partitioning. New data goes to the active chunk; older chunks are compressed, archived, and eventually dropped as complete units.

Within a time range, data may also be partitioned by series or by a hash of a tag (such as the device identifier), which spreads load over several nodes and keeps one series' points together for efficient scans, the combined scheme discussed in Chapter 9.

12.4 Querying Time Series

12.4.1 Typical Operations

- **Time-range filters**, which the time partitioning makes cheap.
- **Bucketing** (time-window aggregation): grouping points into fixed intervals such as five minutes and computing the average, sum, minimum, maximum, count, or a percentile for each.
- **Rate of change.** Many metrics are *counters* that only increase (requests served, bytes sent). The meaningful quantity is the rate, computed from successive samples and corrected for counter resets when a process restarts.
- **Gap filling and interpolation**, which produce a regular series when samples are missing, by carrying forward the last value or interpolating.
- **Moving windows**, for smoothing and for comparisons with earlier periods, such as "this hour versus the same hour last week."

- **Downsampling**, which replaces a high-resolution series with a coarser summary, described in Section 12.5.

12.4.2 TimescaleDB: SQL for Time Series

TimescaleDB extends PostgreSQL with automatic time partitioning (*hypertables*), compression, and time-oriented functions, so applications keep SQL and the surrounding PostgreSQL ecosystem (TimescaleDB Documentation). Listing 12.1 creates a hypertable, runs a bucketed query, and sets up an incrementally maintained rollup, compression, and retention. Function and option names change between versions, so check the current documentation.

Listing 12.1: A hypertable, time bucketing, a continuous aggregate, compression, and retention in TimescaleDB.

```
CREATE TABLE readings (
  time timestamptz NOT NULL,
  device_id int NOT NULL,
  temperature double precision
);
SELECT create_hypertable('readings', 'time');

-- Average per 5 minutes for one device over the last day
SELECT time_bucket('5 minutes', time) AS bucket,
       avg(temperature) AS avg_temp
FROM readings
WHERE device_id = 42 AND time > now() - INTERVAL '1 day'
GROUP BY bucket
ORDER BY bucket;

-- Hourly rollup, maintained incrementally
CREATE MATERIALIZED VIEW readings_hourly
WITH (timescaledb.continuous) AS
SELECT time_bucket('1 hour', time) AS bucket, device_id,
       min(temperature) AS min_temp,
       max(temperature) AS max_temp,
       avg(temperature) AS avg_temp,
       count(*) AS samples
FROM readings
GROUP BY bucket, device_id;

-- Compress chunks older than 7 days, drop chunks older than 90 days
ALTER TABLE readings SET (timescaledb.compress,
                          timescaledb.compress_segmentby = 'device_id');
SELECT add_compression_policy('readings', INTERVAL '7 days');
SELECT add_retention_policy('readings', INTERVAL '90 days');
```

12.4.3 Prometheus: PromQL

Prometheus, widely used for infrastructure monitoring, has its own query language, PromQL, built around selecting series by label and applying functions over a time range (Prometheus Documentation). Listing 12.2 computes a per-second request rate from a counter and a 99th-percentile latency from a histogram.

Listing 12.2: Rate of a counter and a latency percentile in PromQL.

```
# Requests per second over the last 5 minutes, per job
sum by (job) (rate(http_requests_total{code="200"}[5m]))

# 99th percentile request duration from a histogram
histogram_quantile(0.99,
  sum by (le) (rate(http_request_duration_seconds_bucket[5m])))
```

12.4.4 Windows in Stream Processing

When time-series data is processed as a stream rather than queried from storage, windows define which events are grouped. Table 12.2 lists the standard types (Akidau et al., 2015).

Table 12.2: Window types for aggregating events over time.

Window	Definition	Example
Tumbling	Fixed-size, non-overlapping intervals	Requests per minute
Hopping (sliding)	Fixed-size intervals that overlap, advancing by a smaller step	Average over 10 minutes, updated every minute
Session	Groups of events separated by gaps of inactivity	A user's visit to a website

12.5 Retention, Downsampling, and Tiering

Because volume grows continuously, a time-series system needs a policy for the life of data. A common design keeps several resolutions, as in Table 12.3.

Downsampling loses information, so the choice of what to keep in a summary is important. An average alone cannot be recombined correctly: the average of hourly averages is wrong if the hours contain different numbers of samples, and it hides spikes. A good rollup stores the *count*, *sum*, *minimum*, and *maximum* (from which the correct average over any longer period can be computed), and, where percentiles matter, a mergeable sketch or histogram, because percentiles cannot be derived from averages (Listing 12.1 follows this practice). Policies should be agreed with the users of the data in advance, since deleted detail cannot be recovered.

Table 12.3: An example tiered retention policy.

Tier	Resolution	Retention	Storage
Raw	Every sample (for example every 10 seconds)	7 days	Fast local disk or memory
Rollup 1	1-minute summaries	30 days	Standard disk
Rollup 2	1-hour summaries	1 year or longer	Cheap or object storage

12.6 Time-Series Systems in Practice

Table 12.4 summarizes widely used options. As elsewhere in this book, features and availability change between versions and offerings, so treat it as a map to the documentation.

12.6.1 Distribution and Replication

Many TSDBs started as single-node systems, because a single node can ingest and store a large volume when the write path is efficient and data is compressed. Scaling beyond one node uses the strategies of Chapter 9: partition by time and series, replicate chunks for availability, and separate query nodes from storage nodes, with object storage holding older data. Prometheus is an instructive example: each server stores its data locally and independently, and projects such as Thanos and Mimir add global querying, replication, and long-term storage on top. Because telemetry is rarely overwritten and tolerates small delays, eventual consistency (Chapter 8) is generally acceptable, although dashboards and alerts must tolerate data that arrives late.

12.6.2 Choosing Between a TSDB and a General Database

A purpose-built TSDB is not always necessary. A relational or columnar database, with time-based partitioning and good indexes, handles substantial time-series workloads and avoids operating another system, particularly when the data must be joined with business tables. A specialized system pays off when the ingest rate is high, the retention policies are demanding, the series are numerous, or the ecosystem (collectors, alerting, dashboards) matters.

12.7 Temporal Data: Modeling History

Time-series databases record a stream of measurements. A different problem is representing the *history of ordinary data*: an employee's salary changes, a product's price is revised, an insurance policy is amended. The question may be "what is the value now?" or "what was it on 1 March?", or even "what did we believe on 1 April about the value on 1 March?"

Table 12.4: Selected systems for time-series data (check current documentation).

System	Approach	Query language	Notes
InfluxDB	Purpose-built TSDB; tags and fields	InfluxQL, SQL, Flux (depending on version)	Push model; strong fit for monitoring and IoT
TimescaleDB	PostgreSQL extension; hypertables	SQL	Full SQL and PostgreSQL ecosystem; joins with relational data
Prometheus	Pull-based monitoring; local storage	PromQL	Single-node storage; long-term storage via remote write
Thanos, Mimir, VictoriaMetrics	Scalable, long-term storage for Prometheus-style metrics	PromQL or compatible	Add replication, object storage, and horizontal scale
ClickHouse	Columnar analytical database used for time series	SQL	Very fast scans and aggregation; not time-series-specific
Apache Druid	Columnar, real-time analytics	SQL and native JSON queries	Time-partitioned segments; streaming ingestion
QuestDB	Columnar TSDB with SQL extensions	SQL	High ingest rate; time-ordered storage

12.7.1 Valid Time and Transaction Time

Temporal database theory distinguishes two time dimensions (Snodgrass and Ahn, 1985):

- **Valid time** (application time) is when a fact is true in the real world. A price may be valid from 1 March to 30 June.
- **Transaction time** (system time) is when the database knew the fact. A price change effective 1 March may have been entered on 5 March.

A table recording only valid time is *valid-time* temporal, one recording only transaction time is *system-versioned* (it can show earlier states of the database), and one recording both is *bitemporal*. Bitemporal tables answer audit questions and support corrections: if a price was entered wrongly, the correction creates a new transaction-time version while the earlier, wrong state remains visible for audit. Regulated domains such as finance and insurance often require this.

12.7.2 System-Versioned Tables in SQL

The SQL:2011 standard added temporal tables, with system versioning implemented by several products, including MariaDB, SQL Server, Db2, and Oracle (through related features) (Kulkarni and Michels, 2012). The database maintains the history automatically: each update or delete moves the old row version into history with its period of validity. Listing 12.3 shows the syntax used by MariaDB; other products differ in details.

Listing 12.3: A system-versioned table and an "as of" query (MariaDB syntax; others differ).

```
CREATE TABLE product_prices (
  product_id INT,
  price DECIMAL(10,2),
  PRIMARY KEY (product_id)
) WITH SYSTEM VERSIONING;

UPDATE product_prices SET price = 24.90 WHERE product_id = 7;

-- The price as the database held it at a given moment
SELECT product_id, price
FROM product_prices
FOR SYSTEM_TIME AS OF TIMESTAMP '2026-03-01 00:00:00'
WHERE product_id = 7;
```

12.7.3 Validity Periods With Range Types

Where the database lacks temporal tables, validity can be modeled explicitly with a pair of columns or, in PostgreSQL, with a *range type* that represents a period as a single value. Constraints can then forbid overlapping periods for the same entity, which is the correct integrity rule for both bookings and price histories (Listing 12.4) (PostgreSQL Documentation).

Listing 12.4: Non-overlapping validity periods with a PostgreSQL range type and exclusion constraint.

```
CREATE EXTENSION IF NOT EXISTS btree_gist;

CREATE TABLE price_history (
  product_id int,
  price numeric(10,2),
  valid tstzrange,
  EXCLUDE USING gist (product_id WITH =, valid WITH &&)
);

INSERT INTO price_history VALUES
  (7, 19.90, '[2026-01-01, 2026-03-01)'),
  (7, 24.90, '[2026-03-01, infinity)');

-- The price valid at a point in time
```

```
SELECT price FROM price_history
WHERE product_id = 7 AND valid @> timestamptz '2026-02-10';
```

12.7.4 Other Techniques for History

- **Slowly changing dimensions** in data warehouses keep history of descriptive attributes: type 1 overwrites, type 2 adds a new row with validity dates for each change, and type 3 adds a column for the previous value.
- **Audit tables and triggers** copy the old row into a history table on change, a manual form of system versioning.
- **Event sourcing** stores the sequence of events that changed the state, and derives the current state (and any past state) by replaying them. It gives full history by construction and fits naturally with the outbox pattern of Chapter 8, at the cost of more complex queries.
- **Change data capture** streams the changes of a database log to other systems, which can build their own history.

12.8 Getting Time Right

Time is deceptively difficult. Many data-quality problems in time-oriented systems come from a small number of recurring mistakes.

- **Store instants in UTC.** Keep timestamps as absolute points on the UTC timeline, and convert to a local time zone only for display. Types that carry no zone, such as a "timestamp without time zone," invite errors when applications in different zones write to the same column. Where the *local* wall-clock time is itself the fact, such as "the shop opens at 09:00 every day," store the local time together with the zone identifier.
- **Daylight saving time.** Local days can have 23 or 25 hours, and some local times occur twice or not at all. Bucketing by "day" in a local time zone must use zone-aware functions; fixed 24-hour arithmetic gives wrong results twice a year.
- **Event time versus processing time.** The time an event occurred may differ considerably from the time it was received. Analyses should use event time, and stream processors use *watermarks*, estimates of how far event time has progressed, to decide when a window is complete and how long to wait for late data (Akidau et al., 2015).
- **Clock skew and drift.** Clocks on different machines disagree and can jump when corrected. Ordering events from different machines by wall-clock timestamps can be wrong, which is the reason the distributed systems of Chapter 10 use logical or hybrid clocks and why last-write-wins can lose updates. For measuring durations on one machine, use a monotonic clock, not the wall clock.

- **Leap seconds and timestamp precision.** Different systems represent leap seconds differently and store timestamps with different precision (milliseconds, microseconds, nanoseconds). Mismatches cause silent truncation and ordering problems when data is exchanged.
- **Half-open intervals.** Represent periods as half-open intervals, including the start and excluding the end, so that adjacent periods neither overlap nor leave gaps.

12.9 Operating Time-Series Systems

- **Watch cardinality.** Track the number of active series and alert on sudden growth, which usually means that a label has been given an unbounded value.
- **Plan capacity from the ingest rate.** Estimate points per second, the bytes per point after compression, and the retention period, and add headroom for replication and compaction. For example, 100 000 points per second at 2 bytes per compressed point is about 17 GB per day before replication.
- **Test the retention and rollup policies.** Verify that rollups agree with the raw data over the overlapping period, and that dashboards and alerts still work after raw data expires.
- **Protect the write path.** Use buffering and backpressure so that a burst or a slow disk does not lose data or take down the collectors, and monitor the monitoring system itself, since an unnoticed outage of the TSDB hides all other outages.
- **Back up and test restores**, as for any database (Chapter 10), with particular attention to configuration, retention rules, and dashboards.

12.10 Choosing an Approach

A sound process is: (1) estimate the ingest rate, number of series, and retention needs; (2) decide which queries and alerts must be fast and how fresh they must be; (3) design tags to have bounded cardinality and put unbounded values elsewhere; (4) define the rollup and retention tiers with the users of the data; (5) settle the time conventions (UTC storage, event time, precision, handling of late data); and (6) prototype with realistic data, measuring ingest, query latency, and storage, before committing.

Summary

Time-series data is a sequence of timestamped measurements with a distinctive workload: continuous append-only writes at a high rate, mostly recent reads, range and aggregation queries, and value that decays with age. Time-series databases model each point with a metric name, tags, fields, and a timestamp, and the number of distinct tag combinations, the cardinality, is the main scalability risk, so unbounded values must not be tags. Storage

Table 12.5: Choosing an approach for time-oriented data.

Situation	**Suggested approach**	**Reasoning**
Infrastructure and application monitoring	Prometheus-style system, with long-term storage if needed	Pull model, labels, and alerting ecosystem
IoT with many devices and joins to business data	PostgreSQL with TimescaleDB, or a similar relational TSDB	SQL, joins, and compression in one system
Very high ingest with simple queries	Purpose-built TSDB, such as InfluxDB or QuestDB	Optimized write path and compression
Large-scale analytics over events with many dimensions	Columnar analytical database (ClickHouse, Druid)	Fast scans and aggregations across dimensions
Audit and regulatory history of business records	System-versioned or bitemporal tables; event sourcing	Preserves what was known and when
Moderate volume with existing relational database	Time-partitioned relational tables	No new system to operate

engines use log-structured writes, time-based chunking, and specialized compression such as delta-of-delta timestamps and XOR-encoded values, achieving a few bytes or less per point. Queries bucket, aggregate, and compute rates over time ranges, as in TimescaleDB's SQL and Prometheus's PromQL, and stream processors use tumbling, hopping, and session windows. Retention and downsampling policies keep detail for a short period and summaries for longer, with rollups storing count, sum, minimum, and maximum so that they can be combined correctly. Scaling uses time and series partitioning and replication, with eventual consistency usually sufficient. Beyond measurements, temporal data modeling distinguishes valid time from transaction time, and uses system-versioned tables, range types, slowly changing dimensions, and event sourcing to preserve history. Correctness depends on handling time zones, daylight saving time, event time, clock skew, precision, and half-open intervals deliberately. A specialized system is chosen when ingest, retention, or ecosystem needs justify it, and a time-partitioned general-purpose database serves many smaller workloads well.

Review Questions

1. List five characteristics of time-series workloads that differ from typical transactional workloads.

2. Explain the difference between tags and fields. Why is the choice important for performance?

3. What is cardinality in a TSDB? Why can a single unbounded tag cause an outage?

4. Describe delta-of-delta encoding and XOR encoding. Why do they work well on time-series data?

5. Why is data partitioned by time, and what operations does this make cheap?

6. Explain how a counter differs from a gauge, and why queries on counters use a rate.

7. Why can the average of hourly averages be wrong? What should a rollup store instead?

8. Compare pull-based and push-based collection. What does each imply for failure detection?

9. Distinguish valid time from transaction time. Give an example of a question that needs both.

10. How do system-versioned tables differ from a manually maintained audit table?

11. Why should periods be represented as half-open intervals?

12. List four sources of error when handling time, and how each is avoided.

13. When is a general-purpose relational database sufficient for time-series data, and when is a specialized TSDB justified?

Exercises

1. **Cardinality.** A monitoring system collects `http_requests` with tags for `host` (200 values), `endpoint` (80), `method` (4), and `status` (6). Compute the maximum number of series. A developer proposes adding a `user_id` tag, with 500 000 users. What happens, and what alternative would you propose?

2. **Compression.** Apply delta-of-delta encoding to the timestamps 5000, 5010, 5020, 5030, 5041, 5051. Show the deltas and delta-of-deltas, and explain why most would need very few bits.

3. **Capacity planning.** A fleet of 20 000 devices reports 5 values every 10 seconds. Compute the points per second and the storage per day at 2 bytes per compressed point. How much storage is needed for 30 days of raw data with one replica?

4. **Rollups.** Hour A has 10 samples with an average of 50, and hour B has 90 samples with an average of 70. What is the correct average over the two hours? What do you get by averaging the averages? Design a rollup table that avoids the error.

5. **Queries.** Using the `readings` table of Listing 12.1, write SQL for (a) the maximum temperature per device per day for the last week, (b) the hours in which the average temperature of a device exceeded 30 degrees, and (c) the devices that sent no data in the last 10 minutes.

6. **Retention design.** Design a retention and rollup policy for application latency metrics collected every 15 seconds from 500 services, to serve (a) on-call troubleshooting, (b) weekly capacity reviews, and (c) yearly trend reports. State resolutions, retention, and what each rollup stores.

7. **Bitemporal modeling.** A bank records interest rates. A rate of 3.0% effective 1 January is entered on 3 January; on 20 January the bank discovers that the rate should have been 3.5% and corrects it. Draw the rows of a bitemporal table after each step and write the query that returns "what the bank believed on 10 January about the rate valid on 1 January."

8. **Time zones.** A report counts orders "per day" for customers in Melbourne, using a column of UTC timestamps. Explain why grouping by the UTC date is incorrect, what changes at the start and end of daylight saving time, and how to write the grouping correctly in your favorite database.

9. **Late data.** A stream processor computes five-minute tumbling-window counts by event time. Describe the effect of a watermark that is too aggressive and of one that is too conservative, and propose a policy for events that arrive after a window has closed.

References

- Akidau, T., Bradshaw, R., Chambers, C., et al. (2015). The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing. *Proceedings of the VLDB Endowment*, 8(12), 1792–1803.

- Dunning, T., & Friedman, E. (2014). *Time Series Databases: New Ways to Store and Access Data*. O'Reilly Media.

- InfluxDB Documentation. https://docs.influxdata.com

- Jensen, S. K., Pedersen, T. B., & Thomsen, C. (2017). Time Series Management Systems: A Survey. *IEEE Transactions on Knowledge and Data Engineering*, 29(11), 2581–2600.

- Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.

- Kulkarni, K., & Michels, J.-E. (2012). Temporal Features in SQL:2011. *ACM SIGMOD Record*, 41(3), 34–43.

- MariaDB Documentation: System-Versioned Tables. https://mariadb.com/kb/en/system-versioned-tables/

- O'Neil, P., Cheng, E., Gawlick, D., & O'Neil, E. (1996). The Log-Structured Merge-Tree (LSM-Tree). *Acta Informatica*, 33(4), 351–385.

- Pelkonen, T., Franklin, S., Teller, J., et al. (2015). Gorilla: A Fast, Scalable, In-Memory Time Series Database. *Proceedings of the VLDB Endowment*, 8(12), 1816–1827.
- PostgreSQL Documentation. https://www.postgresql.org/docs/
- Prometheus Documentation. https://prometheus.io/docs/
- Snodgrass, R. T. (1999). *Developing Time-Oriented Database Applications in SQL*. Morgan Kaufmann.
- Snodgrass, R., & Ahn, I. (1985). A Taxonomy of Time in Databases. In *Proceedings of ACM SIGMOD '85*, 236–246.
- TimescaleDB Documentation. https://docs.timescale.com

Chapter 13

Search and Indexing Systems

Learning Objectives

After reading this chapter, you will be able to:

- Explain why full-text search needs its own data structures and how the inverted index works.
- Describe text analysis: tokenization, normalization, stemming, and synonyms.
- Explain relevance ranking with TF-IDF and BM25, and evaluate search quality with precision, recall, and nDCG.
- Describe how search engines store immutable segments, merge them, and provide near-real-time search.
- Explain how search is distributed with shards and replicas, and what this means for ranking, pagination, and consistency.
- Write search queries with filters, facets, and highlighting, in a search engine and in SQL.
- Explain vector search and hybrid retrieval, and when they complement keyword search.
- Keep a search index synchronized with a system of record, and choose a search technology.

13.1 Why Databases Need Search Engines

Earlier chapters concentrated on finding records by key, by key range, or by relationship. A different and very common need is to find records by *content*: the product whose description mentions "waterproof hiking boots," the support ticket that resembles a new complaint, the paragraph of a contract that discusses termination. A first attempt with a relational database uses a pattern match such as `LIKE '%boots%'`, which has three weaknesses:

- **Speed.** A pattern with a leading wildcard cannot use an ordinary B-tree index, so the database scans every row.
- **Language.** It matches characters, not words: it does not know that "boot" and "boots" are related, that "the" carries little information, or that a user's typo should still find

the right product.

- **Ranking.** It answers yes or no for each row, while users expect the best matches first.

Search systems solve these problems with specialized indexes and with scoring functions that rank documents by estimated relevance. The field of *information retrieval* that underlies them is older than relational databases (Manning et al., 2008). Search engines are now a standard part of data architectures, often alongside the system of record, and they are the backbone of site search, log analysis, e-commerce catalogs, enterprise search, and, increasingly, retrieval for applications built on language models.

13.2 The Inverted Index

13.2.1 Structure

A conventional index maps a record to its content. An *inverted index* reverses the direction: it maps each *term* to the list of documents that contain it, called its *postings list*. Looking up a word then gives the matching documents directly, without examining any others (Zobel and Moffat, 2006). Table 13.1 shows a small example.

Table 13.1: Three documents and their inverted index (terms lowercased).

Doc	Text	Term	Postings (document identifiers)
1	distributed databases scale	data	2, 3
2	databases replicate data	databases	1, 2, 3
3	search databases index data	distributed	1
		index	3
		replicate	2
		scale	1
		search	3

A query for `databases AND data` reads the two postings lists, 1, 2, 3 and 2, 3, and intersects them, giving documents 2 and 3. Because postings are kept sorted by document identifier, the intersection is a fast merge, and *skip pointers* let the algorithm jump over long runs of non-matching entries. A query with `OR` takes the union, and `NOT` removes documents.

The index usually stores more than document identifiers: the *term frequency* (how often the term occurs in the document), the *positions* of each occurrence, which allow *phrase queries* such as “distributed databases” (the two terms must be adjacent), and sometimes character offsets for highlighting matches in results. The set of all terms is the *term dictionary*, stored in a structure that supports fast lookup, prefix search, and compact storage, such as a sorted list with an index or a finite-state transducer.

13.2.2 Compression

Postings lists can be very long, so they are compressed. Because document identifiers are sorted, the index stores the *gaps* between successive identifiers, which are small numbers, and encodes them with variable-length or block-based schemes such as frame of reference and bit packing. This reduces storage and also speeds queries, since less data must be read from disk or memory (Zobel and Moffat, 2006). It parallels the delta encoding of timestamps in Chapter 12.

13.2.3 Doc Values and Stored Fields

The inverted index answers "which documents contain this term?" but is inefficient for the opposite question, "what is the value of this field in this document?", which is needed for sorting, aggregations, and facets. Search engines therefore also keep a column-oriented structure, often called *doc values*, which stores each field's values by document in a compact columnar layout (Chapter 4). A third structure, *stored fields*, holds the original document so that it can be returned in results. A single search engine index thus combines three representations of the same data, each optimized for a different access pattern.

13.3 Text Analysis

Before text enters the index, and before a query is looked up, it passes through *analysis*, a pipeline that turns raw text into normalized terms. The same analysis must be applied to documents and queries, so that the terms match. Table 13.2 lists the typical stages.

Table 13.2: Typical stages of text analysis.

Stage	Purpose	Example
Character filtering	Remove markup, map characters	Strip HTML tags
Tokenization	Split text into tokens	"Wi-Fi router" becomes `wi`, `fi`, `router`
Lowercasing and accent folding	Make matching case- and accent-insensitive	"Café" becomes `cafe`
Stop-word removal	Drop very common words with little meaning	Remove "the", "of"
Stemming or lemmatization	Reduce words to a common form	"running", "runs" become `run`
Synonyms	Treat different words as equal	"sofa" and "couch"
N-grams	Index fragments of words for partial matches	Edge n-grams of "boot": `b`, `bo`, `boo`, `boot`

Analysis choices change what users can find. Aggressive stemming increases *recall* (more relevant documents found) but may reduce *precision* (more irrelevant ones), as when “university” and “universe” are reduced to the same stem. Languages differ greatly: some, such as Chinese and Japanese, have no spaces between words and need dictionary- or model-based segmentation, and German compounds need decomposition. Fields that hold identifiers, such as an e-mail address, a SKU, or a country code, should *not* be analyzed in this way but indexed as a single exact term, which search engines call a *keyword* field. Choosing the right field type is one of the most important design decisions in a search system.

13.4 Relevance Ranking

13.4.1 TF-IDF

A document is likely to be about a term if the term occurs often in it, and a term is informative if it is rare in the collection. The classic weight multiplies the two ideas. With N documents in the collection and $\mathrm{df}(t)$ of them containing term t,

$$w(t, d) = \mathrm{tf}(t, d) \cdot \log \frac{N}{\mathrm{df}(t)}, \tag{13.1}$$

where $\mathrm{tf}(t, d)$ is the number of occurrences of t in document d. A word such as “the” appears in almost every document, so its logarithmic factor is close to zero, while a rare technical term receives a large weight (Manning et al., 2008).

13.4.2 BM25

TF-IDF has two weaknesses: the score grows without limit as a term is repeated, and long documents tend to mention more terms by chance. The *BM25* function, the default scoring in Lucene-based engines, corrects both with a saturating term-frequency factor and a document-length normalization (Robertson and Zaragoza, 2009). For a query Q containing terms q_i,

$$\mathrm{score}(D, Q) = \sum_{q_i \in Q} \mathrm{IDF}(q_i) \cdot \frac{f(q_i, D)\,(k_1 + 1)}{f(q_i, D) + k_1 \left(1 - b + b\,\frac{|D|}{\mathrm{avgdl}}\right)}, \tag{13.2}$$

where $f(q_i, D)$ is the frequency of q_i in D, $|D|$ is the length of the document, avgdl is the average document length in the collection, and

$$\mathrm{IDF}(q) = \ln\!\left(1 + \frac{N - n(q) + 0.5}{n(q) + 0.5}\right), \tag{13.3}$$

with $n(q)$ the number of documents containing q. The parameter k_1 (commonly about 1.2) controls how quickly the benefit of repeated occurrences saturates, and b (commonly 0.75) controls how strongly long documents are penalized. Implementations differ in details, so exact scores differ between systems, but the behavior is the same.

13.4.3 Beyond Term Statistics

Real relevance combines many signals:

- **Field boosting.** A match in the title counts more than a match in the body.
- **Phrase and proximity.** Terms that occur together or close to each other score higher.
- **Fuzzy matching.** Tolerating spelling errors by allowing a small *edit distance* (insertions, deletions, substitutions, or transpositions) between the query term and an indexed term.
- **Prefix and autocomplete.** Edge n-grams or prefix queries support "search as you type."
- **Business signals.** Popularity, recency, price, stock status, or personalization, combined with the text score by a function or learned ranking model.

Filters are different from scoring: a filter ("brand equals Acme," "price at most 200") includes or excludes documents without affecting the score, and engines can cache filter results, so conditions that do not need ranking should be expressed as filters.

13.4.4 Evaluating Search Quality

Ranking cannot be tuned without measurement. Given a set of queries with documents judged relevant by people, the basic measures are

$$\text{precision} = \frac{|\text{relevant} \cap \text{retrieved}|}{|\text{retrieved}|}, \qquad \text{recall} = \frac{|\text{relevant} \cap \text{retrieved}|}{|\text{relevant}|}. \tag{13.4}$$

Because users look mostly at the first results, rank-aware measures are common, among them *precision at k* and *normalized discounted cumulative gain* (nDCG), which gives more credit to relevant results near the top and allows graded judgments such as "perfect," "good," and "poor" (Järvelin and Kekäläinen, 2002). In production, click-through and conversion data supplement judged sets, with the caveat that clicks are biased toward the positions shown. Every change to analysis or ranking should be tested against a fixed set of queries, since an improvement for one query often harms another.

13.5 Index Internals

13.5.1 Immutable Segments

Lucene, the library underlying Elasticsearch, OpenSearch, and Solr, stores an index as a set of *segments*, each a complete, immutable mini-index (McCandless et al., 2010). New documents accumulate in memory and are periodically written as a new segment; a query searches all segments and merges their results. Immutability makes segments easy to cache, compress, and read concurrently without locks, and it matches the log-structured design of Chapters 4 and 12. The consequences are also similar:

- **Merging.** Many small segments slow searches, so a background process merges them into larger ones, rewriting the data and consuming disk and CPU.
- **Deletes and updates.** A deleted document is only *marked* as deleted and is physically removed when its segment is merged. An update is a delete of the old version and an insert of the new one, so documents that change frequently are expensive for a search index.
- **Near-real-time visibility.** A new document becomes searchable only when the segment that contains it is opened for search, an operation often called a *refresh*. In Elasticsearch the default refresh interval is about one second, which is why search is described as *near* real time. Durability is provided separately by a transaction log, as in Chapter 10.

13.5.2 Query Execution

For a ranked query, the engine need not score every matching document. Optimizations such as block-max pruning skip documents that cannot reach the current top results, so the cost depends on how many documents compete for the top places, not on how many match. This is why a query for a common word can still return in milliseconds.

13.6 Distributed Search

13.6.1 Shards and Replicas

A large index is split into *shards*, each a complete Lucene index holding a subset of the documents, and each shard has one or more *replicas* (Chapter 9). Documents are assigned to shards by hashing a routing value, normally the document identifier. Replicas serve two purposes: availability when a node fails, and extra capacity for queries. Replication of writes follows the primary–replica pattern: the primary shard applies the write and forwards it to its replicas.

13.6.2 Scatter–Gather Execution

A search request goes to a coordinating node, which executes it in two phases, as Figure 13.1 shows. In the *query phase* the request is sent to one copy of every shard; each shard returns the identifiers and scores of its own top results. The coordinator merges them into the global top results. In the *fetch phase* it asks the relevant shards for the stored content of only those winning documents. This is the scatter–gather pattern met in Chapter 9 for local secondary indexes, and it has the same property: latency is determined by the slowest shard, so tail latency grows with the number of shards.

13.6.3 Consequences for Ranking, Paging, and Sizing

- **Local statistics.** Each shard computes IDF from its own documents, so scores for the same document can differ slightly between shards, particularly when shards are small

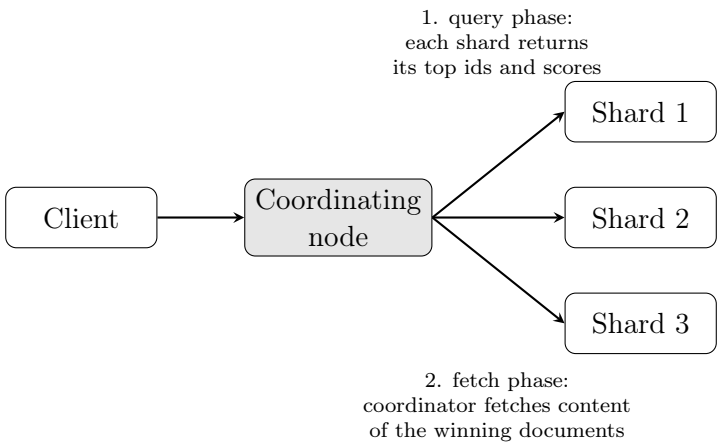


Figure 13.1: Distributed search by scatter–gather. The coordinating node queries every shard, merges the partial rankings, and then fetches the content of the winning documents.

or unevenly filled. Engines can compute global statistics at extra cost, and the effect fades as shards grow.

- **Deep pagination.** To return results 10 000 to 10 010, every shard must produce its top 10 010 results, and the coordinator must merge and discard nearly all of them, so the cost grows with the page depth. The remedy is a cursor-based approach, such as `search_after`, which continues from the sort values of the last result, or a snapshot-based scroll for bulk export.

- **Shard sizing.** Too few shards limit parallelism and make recovery slow, while too many shards waste memory and add coordination overhead to every query. A common rule is to size shards in the range of tens of gigabytes each, but the right size depends on the workload and must be measured. The number of primary shards is hard to change later, so it needs to be planned, as with the partition count in Chapter 9.

- **Consistency.** Search indexes are eventually consistent with the source of the data: a document is not searchable until it has been indexed and refreshed, and replicas may be briefly behind the primary. Applications that need read-your-writes behavior (Chapter 8) must either avoid using search for that read or request a refresh at a cost.

13.7 Querying and Faceted Search

13.7.1 Search Engine Queries

Search engines such as Elasticsearch and OpenSearch expose a JSON-based query language over HTTP (Elasticsearch Documentation). Listing 13.1 defines the fields of a product index, choosing which fields are analyzed text and which are exact keywords. Listing 13.2 then searches for text, filters by brand and price without affecting the score, and computes a facet count per brand.

Listing 13.1: An index mapping that distinguishes analyzed text from exact keywords.

```
PUT /products
{
  "mappings": {
    "properties": {
      "name": { "type": "text", "analyzer": "english" },
      "brand": { "type": "keyword" },
      "price": { "type": "float" }
    }
  }
}
```

Listing 13.2: A scored text query with filters, a facet, and highlighting.

```
GET /products/_search
{
  "query": {
    "bool": {
      "must": [ { "match": { "name": "wireless headphones" } } ],
      "filter": [ { "term": { "brand": "acme" } },
                  { "range": { "price": { "lte": 200 } } } ]
    }
  },
  "aggs": { "by_brand": { "terms": { "field": "brand" } } },
  "highlight": { "fields": { "name": {} } },
  "size": 10
}
```

Faceted search shows, next to the results, the counts of matching documents per category, such as brand, price range, or color, so that users can narrow the results interactively. The facet counts are computed with aggregations over doc values, which makes them efficient for fields with moderate numbers of distinct values.

13.7.2 Full-Text Search in a Relational Database

For moderate volumes and for applications that already use a relational database, built-in full-text search may be sufficient and avoids operating and synchronizing another system. PostgreSQL, for example, converts text into a `tsvector` of normalized terms, queries it with a `tsquery`, ranks results, and indexes the column with a GIN (generalized inverted) index, which is an inverted index (PostgreSQL Documentation). Listing 13.3 shows the steps.

Listing 13.3: Full-text search with a GIN index in PostgreSQL.

```
ALTER TABLE articles ADD COLUMN search tsvector
  GENERATED ALWAYS AS (
    to_tsvector('english', coalesce(title, '') || ' ' || coalesce(body, ''))
  ) STORED;
```

```
CREATE INDEX articles_search_idx ON articles USING GIN (search);

SELECT id, title, ts_rank(search, query) AS rank
FROM articles, websearch_to_tsquery('english', 'distributed consistency') AS query
WHERE search @@ query
ORDER BY rank DESC
LIMIT 10;
```

The advantages are transactional consistency with the data (the index is updated in the same transaction) and simplicity; the limitations are weaker relevance tuning, fewer language and analysis options, and less scalability for very large collections or demanding query loads than dedicated engines.

13.8 Vector Search and Hybrid Retrieval

13.8.1 Embeddings and Similarity

Keyword search matches words, so it misses documents that express the same idea in different words. *Vector search* addresses this by using *embeddings*: a machine-learning model maps a piece of text (or an image or other object) to a vector of a few hundred or thousand numbers so that items with similar meaning lie close together. Searching then means finding the stored vectors nearest to the query vector. A common measure of closeness is cosine similarity,

$$\cos(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \, \|\mathbf{b}\|}, \tag{13.5}$$

and Euclidean distance and the dot product are also used.

13.8.2 Approximate Nearest-Neighbor Indexes

Comparing the query with every vector is exact but too slow for large collections, so vector databases use *approximate nearest-neighbor* (ANN) indexes that trade a small loss of accuracy for large speedups. Two families are common: *graph-based* indexes such as HNSW (hierarchical navigable small world), which build a layered proximity graph and search it by greedy traversal, in the spirit of Chapter 11 (Malkov and Yashunin, 2020), and *clustering-based* indexes such as IVF, which partition the space into cells and search only the nearest ones, often combined with vector compression (Johnson et al., 2021). The trade-offs are the usual ones: memory use, build time, query latency, and *recall* of the true nearest neighbors, all of which can be tuned by index parameters. Updates and deletes are more expensive than in a keyword index, and combining a vector search with metadata filters ("only documents of this customer") needs care to keep results complete and fast.

13.8.3 Hybrid Search

Keyword and vector search have complementary strengths. Keyword search is precise for exact terms, names, codes, and rare words; vector search handles paraphrase and intent.

Hybrid search runs both and merges the two rankings. A simple, robust method is *reciprocal rank fusion* (RRF), which scores each document by its rank in each list rather than by incomparable raw scores (Cormack et al., 2009):

$$\mathrm{RRF}(d) = \sum_{r \in R} \frac{1}{k + \mathrm{rank}_r(d)}, \tag{13.6}$$

where R is the set of rankings and k is a constant, commonly 60. Many systems also add a second-stage *re-ranker*, a more expensive model that reorders the top candidates.

13.8.4 Retrieval for Language-Model Applications

Retrieval-augmented generation (RAG) supplies a language model with documents retrieved for each question, so that answers are based on current, private, or verifiable information (Lewis et al., 2020). The retrieval step is a search problem, and the quality of the answer depends heavily on it: documents are split into *chunks*, indexed by keywords and embeddings, retrieved with hybrid search, filtered by the user's access rights, and passed to the model. All the concerns of this chapter apply: analysis, relevance evaluation, freshness, and keeping the index synchronized with its sources. Where relationships matter, retrieval can be combined with a knowledge graph (Chapter 11).

13.9 Search Systems in Practice

13.10 Keeping the Index in Sync

A search index is almost always a *derived* copy: the system of record is a relational or document database, and the search engine holds a searchable projection. This brings back the staleness and dual-write issues of Chapter 8 (Table on hidden sources of staleness). Table 13.4 compares the usual strategies.

Whichever approach is used, three practices make it robust. First, indexing must be *idempotent*, for example by using the record's identifier and version as the document identifier and version, so that retries and duplicates are harmless (Chapter 10). Second, a *full reindex* must always be possible, because mappings and analysis change, and bugs happen. The standard technique is zero-downtime reindexing with *aliases*: build a new index in the background, then atomically switch the alias that applications use from the old index to the new one. Third, the index must be monitored for *drift*, with periodic comparison of counts and samples against the source.

13.11 Operating Search Systems

- **Mapping explosion.** Allowing the engine to create a field for every key it sees in dynamic documents can produce thousands of fields and exhaust memory. Define mappings explicitly and limit dynamic fields.

- **Too many small shards.** Every shard costs memory and file handles; thousands of tiny shards degrade clusters. Use time-based or size-based rollover for log-like data and plan capacity.
- **Expensive queries.** Leading wildcards, regular expressions, very deep pagination, and aggregations on high-cardinality fields can overload a cluster. Set timeouts and limits, and examine slow queries.
- **Memory and caches.** Search engines depend on the operating system's file cache and on memory for structures such as doc values and vector indexes. Capacity planning must include them.
- **Security.** Access control must be applied to search results as well as to the source: filter by the user's permissions inside the query, and do not index fields that should never be searchable. User input should be treated as untrusted, and passed to the engine through safe query constructs and not concatenated into query strings.
- **Not a system of record.** Search indexes are rebuilt from the source and are not designed for strict durability or multi-document transactions. Keep the source of truth elsewhere and test restoring by reindexing.
- **Relevance regressions.** Track search quality with a fixed query set and with online metrics such as zero-result rate and click-through, and review them whenever analysis, ranking, or data change.

13.12 Choosing an Approach

A sound process is: (1) collect real queries and define what a good result is, with a judged query set; (2) decide which fields are analyzed text, exact keywords, filters, and facets; (3) start with the simplest technology that meets the relevance and scale needs, often the database already in use; (4) design synchronization with the system of record, including reindexing; (5) evaluate relevance offline and online, and tune analysis and ranking with measurement rather than intuition; and (6) plan shards, capacity, security, and monitoring before launch.

Summary

Search systems find documents by content, with speed, linguistic awareness, and ranking that pattern matching in a database cannot provide. Their core structure is the inverted index, a mapping from terms to compressed, sorted postings lists that may also hold frequencies and positions, complemented by columnar doc values and stored fields. Text analysis, which tokenizes, normalizes, stems, and expands text, defines what can be matched and must be applied identically to documents and queries. Relevance is estimated from term statistics, notably by TF-IDF and the BM25 function, combined with field boosts, proximity, fuzziness, and business signals, and it is evaluated with precision, recall, and nDCG on judged queries. Lucene-based engines store immutable segments that are merged in the background, deletes

are marks, and new documents become visible after a refresh, which makes search near real time. Distributed search splits an index into replicated shards and executes queries by scatter–gather, with consequences for tail latency, per-shard statistics, deep pagination, shard sizing, and eventual consistency. Queries combine scored text conditions with filters and facets, and relational databases offer simpler built-in full-text search. Vector search with approximate nearest-neighbor indexes captures meaning, and hybrid search with rank fusion and re-ranking combines it with keyword search, which underlies retrieval for language-model applications. Because the index is a derived copy, it must be synchronized through an outbox, change data capture, or batch jobs, with idempotent indexing, aliases for reindexing, and drift monitoring. Mappings, shards, expensive queries, security, and relevance regressions are the main operational concerns, and the choice of technology follows from the relevance needs, scale, and the database already in use.

Review Questions

1. Give three reasons why `LIKE '%term%'` is a poor basis for search.
2. Describe the structure of an inverted index. How is a query for two terms joined by `AND` executed?
3. Why does the index store term positions? Which kind of query needs them?
4. What do doc values provide that the inverted index does not?
5. Name five stages of text analysis. Why must documents and queries use the same analysis?
6. Explain how stemming can both help and hurt search quality.
7. Why does BM25 improve on TF-IDF? What do the parameters k_1 and b control?
8. Define precision, recall, and nDCG. Why is recall alone insufficient to judge a search system?
9. Why are segments immutable? What consequences does this have for updates and deletes?
10. Explain the query and fetch phases of distributed search. Why does latency depend on the slowest shard?
11. Why is deep pagination expensive, and how does `search_after` avoid the problem?
12. What is the difference between a filter and a query clause? When should each be used?
13. What is hybrid search, and why combine keyword and vector retrieval? How does reciprocal rank fusion work?
14. Compare dual writes, the transactional outbox, and change data capture for synchronizing a search index.

15. Why should a search index not be the system of record, and what is the role of aliases in reindexing?

Exercises

1. **Inverted index.** Build the inverted index, with term frequencies and positions, for the documents (1) "the quick brown fox", (2) "the lazy brown dog", and (3) "quick quick dog". Show the result of the queries `brown AND dog`, `quick OR lazy`, and the phrase query "brown dog".

2. **TF-IDF.** In a collection of 1 000 documents, the term *database* occurs in 100 and *sharding* in 5. A document has tf = 3 for *database* and tf = 2 for *sharding*. Compute the TF-IDF weights (use base-10 logarithms) and say which term contributes more.

3. **BM25.** For $k_1 = 1.2$, $b = 0.75$, and a term with IDF = 2, compute the BM25 term score of a document with $f = 1$ and one with $f = 10$, both of average length. Explain what the comparison shows about saturation. Repeat with a document twice the average length.

4. **Analysis design.** Design the analysis for (a) the title of a product, (b) its SKU, (c) a customer e-mail address, and (d) a free-text review. State the tokenizer, filters, and whether the field is text or keyword, and justify each choice.

5. **Evaluation.** A query returns ten results, of which four are relevant; the collection holds eight relevant documents for this query. Compute precision, recall, and precision at 5 if two of the first five results are relevant. Propose how you would build a judged query set for a product-search system.

6. **Sharding.** An index of 600 GB is to be served with a target shard size of about 30 GB and one replica. How many primary shards and total shard copies result? Discuss the effects on query latency and recovery time if you choose 100 shards instead, and what happens when the data grows to 6 TB.

7. **Rank fusion.** A keyword search ranks documents A, B, C, D and a vector search ranks C, A, E, B. Compute the reciprocal-rank-fusion scores with $k = 60$ and give the merged order.

8. **Synchronization.** A web shop stores orders in PostgreSQL and searches them in Elasticsearch. Design the synchronization using the outbox pattern, specifying the outbox table, the relay, the idempotent indexing key, and the procedure for a zero-downtime reindex with an alias.

9. **Technology choice.** For each of (a) a 20 000-article help center in a Django application, (b) a retailer with 50 million products and heavy faceting, and (c) a legal-research tool for question answering over 2 million documents, select a technology from Table 13.3 and justify the decision, including how the data will be kept fresh.

References

- Cormack, G. V., Clarke, C. L. A., & Buettcher, S. (2009). Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. In *Proceedings of ACM SIGIR '09*, 758–759.
- Elasticsearch Documentation. https://www.elastic.co/guide/
- Järvelin, K., & Kekäläinen, J. (2002). Cumulated Gain-Based Evaluation of IR Techniques. *ACM Transactions on Information Systems*, 20(4), 422–446.
- Johnson, J., Douze, M., & Jégou, H. (2021). Billion-Scale Similarity Search with GPUs. *IEEE Transactions on Big Data*, 7(3), 535–547.
- Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.
- Lewis, P., Perez, E., Piktus, A., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*.
- Malkov, Y. A., & Yashunin, D. A. (2020). Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4), 824–836.
- Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press.
- McCandless, M., Hatcher, E., & Gospodnetić, O. (2010). *Lucene in Action* (2nd ed.). Manning Publications.
- OpenSearch Documentation. https://opensearch.org/docs/
- PostgreSQL Documentation: Full Text Search. https://www.postgresql.org/docs/
- Robertson, S., & Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval*, 3(4), 333–389.
- Apache Solr Reference Guide. https://solr.apache.org/guide/
- Zobel, J., & Moffat, A. (2006). Inverted Files for Text Search Engines. *ACM Computing Surveys*, 38(2), Article 6.

Table 13.3: Selected search technologies (check current documentation, licensing, and features).

System	**Type**	**Strengths**	**Notes**
Apache Lucene	Java search library	Foundation of several engines	Embedded, not a server
Elasticsearch	Distributed search and analytics engine on Lucene	Rich query language, aggregations, vector search, ecosystem for logs and observability	License terms have changed over time
OpenSearch	Community fork of Elasticsearch	Similar features, open governance	APIs diverge gradually from Elasticsearch
Apache Solr	Distributed search server on Lucene	Mature, strong faceting and text features	Configuration-oriented
PostgreSQL full-text search	Built into the relational database	Transactional, no extra system; `pgvector` extension adds vector search	Moderate scale and tuning
Typesense, Meilisearch	Lightweight search servers	Simple setup, typo tolerance, fast autocomplete	Smaller scale and feature set
Hosted services (for example Algolia, cloud search offerings)	Managed search	Little operational effort	Cost and lock-in
Vector databases (for example Milvus, Qdrant)	Systems built for ANN search	Large vector collections; filtering	Often paired with a keyword engine

Table 13.4: Strategies for synchronizing a search index with a system of record.

Strategy	How it works	Trade-offs
Dual write from the application	The application writes to the database and to the index	Simple, but unsafe: a failure between the two leaves them inconsistent
Transactional outbox	Index events are written in the same transaction as the data and relayed to the indexer	Reliable and ordered; adds a relay (Chapter 8)
Change data capture	The indexer reads the database's change log and applies changes	No application changes; depends on log access and connectors
Periodic batch reindex	A job rebuilds or refreshes the index on a schedule	Simple and self-healing; results are stale between runs

Table 13.5: Choosing a search approach.

Situation	Suggested approach	Reasoning
Modest data, search is one feature of a relational application	Built-in full-text search (for example PostgreSQL with GIN)	Transactional consistency and no extra system
Rich site or catalog search with facets and relevance tuning	Dedicated engine (Elasticsearch, OpenSearch, Solr)	Analysis, scoring, faceting, and scale
Log and event analytics, with search over large volumes	Search and analytics engine with time-based indices	Aggregations, retention, and ingestion pipelines
Semantic or question-answering retrieval	Vector index with hybrid keyword search and reranking	Combines meaning and exact matching
Small team, simple search, instant autocomplete	Lightweight or hosted search service	Low operational cost
Lookup by identifiers and exact attributes only	Ordinary database indexes	A search engine adds cost without benefit

Chapter 14

Vector Databases and Embedding Search

Learning Objectives

After reading this chapter, you will be able to:

- Explain what embeddings are, how distance metrics compare them, and why embedding models must be versioned.
- Explain why exact nearest-neighbor search does not scale and how approximate indexes (IVF, HNSW, graph-on-disk) and quantization address this.
- Quantify the trade-off among recall, latency, and memory, and measure recall against ground truth.
- Compare pre-filtering, post-filtering, and in-index filtering, and design metadata for filtered search.
- Describe the architecture of a vector database: segments, updates, sharding, replication, and multi-tenancy.
- Build a retrieval pipeline with chunking, hybrid search, and re-ranking, and evaluate it.
- Use vector search in PostgreSQL and in a search engine, and compare dedicated vector systems.
- Address cost, security, and privacy concerns specific to embeddings.

14.1 From Keywords to Meaning

Chapter 13 introduced vector search briefly as a complement to keyword search. This chapter treats it as a subject in its own right, because it brings new data structures, new trade-offs, and new operational problems.

The core idea is simple. A machine-learning model, the *embedding model*, converts an item (a passage of text, an image, an audio clip, a product, a user) into a *vector*, a list of numbers typically of a few hundred to a few thousand dimensions. The model is trained so that items that are similar in meaning produce vectors that are close to each other. Searching by meaning then reduces to a geometric problem: given a query vector, find the stored vectors nearest to it. This is the *nearest-neighbor* (NN) problem, and a system specialized in storing

vectors and answering such queries is a *vector database*.

Applications include semantic search, question answering and retrieval-augmented generation (RAG) (Lewis et al., 2020), recommendation, duplicate detection, image and audio search, and anomaly detection. Their common requirement is to find, among millions or billions of vectors, the few that are most similar to a given one, within milliseconds.

14.1.1 What Is New in This Data Model

Compared with the models of earlier chapters, vector data has unusual properties:

- **Similarity, not equality.** Queries ask for the *closest* items, not matching ones. There is no exact answer that an index can pinpoint in a single lookup.
- **Approximation is accepted.** For efficiency, systems usually return results that are *probably* the nearest, which is a design decision visible to users, unlike in exact indexes.
- **High-dimensional data.** Vectors with hundreds of dimensions defeat classical spatial indexes (Section 14.3).
- **Dependence on a model.** The vectors are meaningful only relative to the embedding model that produced them; the data and the model are coupled.

14.2 Embeddings and Distance Metrics

14.2.1 Where Embeddings Come From

Embeddings are produced by neural networks. Text embedding models (descendants of Sentence-BERT and dense passage retrieval) map sentences or passages to vectors (Reimers and Gurevych, 2019; Karpukhin et al., 2020); image models map pictures to vectors; *multimodal* models map text and images into the same space, so that a text query can retrieve images. Models differ in dimensionality, in the maximum length of input they accept, in the languages they cover, and in the domains on which they were trained. Some models produce vectors that can be truncated to shorter lengths with limited loss of quality, a property called *Matryoshka* representation (Kusupati et al., 2022), which allows a choice between cost and accuracy.

14.2.2 Distance and Similarity

Table 14.1 lists the measures in common use. For vectors $\mathbf{a}, \mathbf{b} \in \mathbb{R}^d$:

The measures are related. If the vectors are *normalized* to unit length, then

$$\|\mathbf{a} - \mathbf{b}\|_2^2 = 2 - 2\,\mathbf{a} \cdot \mathbf{b}, \tag{14.1}$$

so cosine similarity, the dot product, and Euclidean distance all produce the same ranking. Normalizing the vectors once at ingestion and using the dot product is therefore a common way to obtain fast cosine search. The important rule is to use the metric for which the embedding model was trained, as stated in the model's documentation, and to declare the same metric in the index, because an index built for one metric returns wrong neighbors for another.

Table 14.1: Common distance and similarity measures for embeddings.

Measure	Definition	Remarks
Euclidean (L2) distance	$\|\mathbf{a} - \mathbf{b}\|_2$	Smaller is closer; sensitive to vector length
Cosine similarity	$\frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\|\|\mathbf{b}\|}$	Compares direction only; larger is closer; most common for text
Dot (inner) product	$\mathbf{a} \cdot \mathbf{b}$	Larger is closer; fast; used when vector length carries meaning

14.2.3 Embedding Models Must Be Versioned

Vectors from different models, or different versions of the same model, live in unrelated spaces and cannot be compared. Replacing a model therefore requires *re-embedding* the entire collection and rebuilding the index, which for large collections is a major, expensive operation. Practical consequences are to record the model name, version, and dimension with every vector, to plan model changes like schema migrations (build a new index beside the old one, then switch), and to keep the source content so that it can be re-embedded (Section 14.10).

14.3 Why Exact Search Does Not Scale

The exact solution is *brute force*: compute the distance from the query to every stored vector and keep the best k. Its cost is proportional to $N \cdot d$ for N vectors of dimension d. For a collection of 100 million vectors with 768 dimensions stored as 4-byte floats, the data alone occupies

$$10^8 \times 768 \times 4 \text{ bytes} \approx 307 \text{ GB}, \tag{14.2}$$

and every query would scan all of it. Hardware with vectorized instructions or GPUs can make brute force fast for collections of up to some millions of vectors, and it remains a good choice for small collections because it is exact and needs no index. Beyond that, it is too slow and too costly.

Classical spatial indexes, such as k-d trees and R-trees, do not help much in high dimensions. This is one aspect of the *curse of dimensionality*: as the number of dimensions grows, distances between random points become nearly equal, so that the distinction between near and far, on which tree pruning relies, fades (Beyer et al., 1999). Search structures for vectors thus give up exactness and aim for *approximate nearest-neighbor* (ANN) search: return neighbors that are very likely, but not guaranteed, to be the true nearest ones (Indyk and Motwani, 1998).

14.4 Approximate Nearest-Neighbor Indexes

Table 14.2 compares the main families, which are described below.

Table 14.2: Approximate nearest-neighbor index families compared.

Index	Idea	Strengths	Weaknesses
Flat (brute force)	Compare with every vector	Exact; no build; simple	Cost grows linearly with N
IVF	Cluster vectors; search only the nearest clusters	Small memory overhead; fast build; works with compression	Recall depends on clustering; updates degrade cluster quality
HNSW	Layered proximity graph searched greedily	High recall at low latency; incremental inserts	High memory use; slow build; deletes are awkward
Disk-based graph (for example DiskANN)	Graph with compressed vectors in memory and full vectors on SSD	Billions of vectors on one machine at modest memory	More complex; SSD latency
LSH	Hash similar vectors to the same bucket	Simple theory; streaming data	Usually lower recall than graphs at the same cost

14.4.1 Inverted File Index (IVF)

An IVF index first clusters the vectors, usually with the k-means algorithm, into n_{list} cells, each represented by its centroid. Each vector is stored in the list of its nearest centroid. At query time the engine finds the n_{probe} centroids nearest to the query and scans only the vectors in those cells. The name reflects the analogy with the inverted index of Chapter 13: a centroid plays the role of a term, and its list is the postings list. Increasing n_{probe} raises recall and cost together. A vector whose true neighbors lie just across a cell boundary may be missed if the neighboring cell is not probed, which is the main source of error.

14.4.2 Hierarchical Navigable Small World (HNSW)

HNSW builds a multi-layer graph in which each vector is a node linked to a limited number of near neighbors (Malkov and Yashunin, 2020). The upper layers contain few nodes with long-range links, like an express network, and the lowest layer contains all nodes with short links. A search starts at an entry point on the top layer, moves greedily to the neighbor closest to the query, descends a layer when no neighbor is closer, and on the bottom layer explores a candidate list of a given size. Three parameters control its behavior:

- M: the maximum number of links per node, which raises recall and memory use.
- $ef_{\text{construction}}$: the size of the candidate list when building, which improves graph quality at the cost of build time.
- ef_{search}: the size of the candidate list at query time, the main knob for trading latency against recall.

HNSW gives excellent recall and speed and supports incremental insertion, which explains its popularity. Its costs are memory, because the graph and, usually, the full vectors are kept in RAM, and the awkwardness of deletion, since removing a node can disconnect the graph; many systems mark nodes as deleted and rebuild periodically. The graph connection to Chapter 11 is only structural: these are proximity graphs built by the index, not relationship graphs stored by users.

14.4.3 Quantization: Compressing Vectors

Memory is the main cost of vector search, so vectors are often *quantized*, stored in a lossy compressed form, and distances are computed on the compressed form, sometimes followed by re-scoring the best candidates with the full vectors.

- **Scalar quantization** replaces each 32-bit float with an 8-bit integer (a 4× reduction) or fewer bits.
- **Product quantization** (PQ) splits each vector into m sub-vectors and replaces each sub-vector by the identifier of its nearest centroid in a small codebook (typically 256 entries, one byte) (Jégou et al., 2011). A 768-dimensional float vector of 3 072 bytes encoded with $m = 96$ sub-vectors takes 96 bytes, a 32× reduction, and distances can be approximated from precomputed tables.
- **Binary quantization** keeps one bit per dimension, a 32× reduction with fast Hamming-distance comparison, usually followed by re-scoring; it works best with high-dimensional embeddings that are trained to tolerate it.

Quantization reduces recall, which re-scoring partly recovers. Combinations such as IVF with PQ are used to hold billions of vectors in a manageable amount of memory (Johnson et al., 2021), and disk-based graph indexes keep compressed vectors in memory while placing full vectors on fast SSDs (Subramanya et al., 2019).

14.5 Recall, Latency, and Memory

Every ANN index is a compromise among three quantities: *recall* (accuracy), *latency or throughput*, and *memory* (cost). The standard accuracy measure is *recall at* k:

$$\text{recall@}k = \frac{|\text{ANN results} \cap \text{exact top-}k|}{k}. \tag{14.3}$$

A recall of 0.95 means that, on average, 95 of the 100 true nearest neighbors appear in the returned 100. This differs from the recall of Chapter 13, which is measured against human relevance judgments; here the reference is the exact answer of the same embedding space, and the measure says nothing about whether the neighbors are *useful*, which is a property of the embedding model.

14.5.1 Measuring and Tuning

1. Take a representative sample of real queries and compute the exact answers by brute force; this is the *ground truth*.
2. Run the same queries against the ANN index at several parameter settings (such as ef_{search} or n_{probe}), recording recall at k and latency percentiles (p50, p99).
3. Plot the recall–latency curve and select the cheapest setting that meets the application's recall target.
4. Repeat after significant data growth or distribution change, since the best setting drifts.

Public benchmarks, such as ANN-Benchmarks, compare algorithms on standard datasets (Aumüller et al., 2020), but results depend on dimensionality, data distribution, hardware, and filtering, so a system must be tested on its own data.

14.6 Filtering and Metadata

Real queries are rarely "the nearest vectors overall." They are "the nearest vectors *among documents of this customer, in this language, published this year, which the user may read*." Combining a similarity search with conditions on metadata is the hardest practical problem in vector search. Table 14.3 lists the strategies.

Table 14.3: Strategies for filtered vector search.

Strategy	**How it works**	**Trade-offs**
Post-filtering	Retrieve the nearest k vectors, then discard those that fail the filter	Simple, but a selective filter can leave fewer than k (or zero) results
Pre-filtering	Find the matching rows with metadata indexes, then search only among them (exactly or with an ANN index on the subset)	Complete results; can be slow when many rows match, or when the ANN structure cannot be restricted
In-index filtering	Check the filter while traversing the index, continuing until k matching results are found	Good results at most selectivities; needs support from the engine
Partitioning	Keep separate indexes per tenant or category and route the query to the right one	Fast and secure for large, well-defined partitions; many small indexes are costly

The behavior depends on the *selectivity* of the filter, the fraction of vectors that pass. A filter that keeps half of the data is easy for any strategy; one that keeps one in a hundred thousand is better handled by pre-filtering and exact search over the few matching vectors, which is both fast and exact. Good systems choose a strategy per query from the estimated

selectivity, much as a relational optimizer chooses a join method (Chapter 9 discussed the analogous choice of partition keys). Metadata should therefore be designed deliberately: store the fields that queries filter on (tenant, access group, language, document type, date) as indexed attributes next to the vector, and keep large content elsewhere.

For multi-tenant applications, *access control* belongs in the filter: the query must include the user's permissions, so that results from other tenants or restricted documents never appear, even as candidates. Filtering after generation of an answer is too late, since retrieved text may already have influenced it.

14.7 Vector Database Architecture

A vector database must combine an ANN index with the facilities expected of any database: durable storage, updates and deletes, metadata queries, scaling, and replication (Wang et al., 2021).

14.7.1 Segments and Updates

Most systems follow the pattern of search engines and LSM-based stores (Chapters 4, 12, and 13): new vectors are buffered in memory and in a log, then sealed into immutable *segments*, each with its own index. Small segments are searched by brute force and large ones through their ANN index; a query searches all segments and merges the results. A delete marks a vector as removed, and background compaction merges segments and rebuilds indexes. An update is a delete plus an insert. Heavy update workloads cause frequent index rebuilds, which consume CPU and memory, so write patterns matter.

14.7.2 Sharding and Replication

A large collection is split into shards, each holding a subset of vectors and its own index, and each shard is replicated (Chapter 9). A query is sent to every shard (scatter–gather, as in Chapter 13); each returns its local top k, and a coordinator merges them. Latency is therefore dominated by the slowest shard. Because each shard returns approximate results, the merged result is approximate too, and shard counts influence recall and cost. Replicas add availability and query throughput.

14.7.3 Consistency and Freshness

Writes to a vector index are normally visible after a short delay: the vector must be added to a segment or to the index before it can be found. Systems offer settings from *eventual* visibility to *strong* or *bounded* guarantees at a cost in latency (Chapter 8). For applications such as RAG over documents that change, the practical issue is the time between the change of a source document and the update of its vectors, and the possibility that the old version is still retrieved.

14.7.4 Storage Layout and Disaggregation

Many modern systems separate storage from compute: segments and indexes are kept in object storage, and stateless query nodes cache the hot ones in memory or on local SSD. This makes it possible to scale and pay for storage and compute independently, and to serve large collections cheaply when queries are infrequent, at the price of higher latency for cold data. The same idea appears in other systems of this book, and the trade-offs are the same: cost and elasticity against tail latency.

14.7.5 Multi-Tenancy

When one deployment serves many customers, systems provide *namespaces*, *collections*, or *partitions* per tenant. Separate indexes give strong isolation and simple deletion of a tenant's data, but thousands of tiny indexes cost memory and management; a shared index with a tenant filter is cheaper, but depends on correct filtering for isolation. The choice depends on the number and size of tenants and on the isolation requirements.

14.8 Systems in Practice

Table 14.4: Selected systems for vector search (check current documentation).

Category	**Examples**	**Strengths**	**Limits**
Libraries	FAISS, hnswlib, ScaNN, Annoy	Fast, flexible index building blocks	No server, storage, or replication
Dedicated vector databases	Milvus, Qdrant, Weaviate, Pinecone, Chroma	Built for vectors: filtering, scale, tuning	Another system to operate or buy
Extensions to relational databases	PostgreSQL with `pgvector`	Transactions, SQL joins, one system	Scale and index options more limited
Search engines	Elasticsearch, OpenSearch, Solr	Hybrid keyword and vector search; filtering; mature operations	Memory-hungry at large scale
General-purpose stores with vector indexes	MongoDB, Redis, and various cloud databases	Vectors next to existing data	Features vary

14.8.1 PostgreSQL with pgvector

The `pgvector` extension adds a `vector` type, distance operators, and IVF and HNSW indexes to PostgreSQL (pgvector Documentation). Because the vectors live in an ordinary table, they can be updated transactionally and combined with SQL filters and joins, which makes it a strong starting point for applications that already use PostgreSQL. Listing 14.1 shows the steps. The operator `<=>` denotes cosine distance (one minus cosine similarity), `<->` Euclidean distance, and `<#>` the negative inner product.

Listing 14.1: Vector storage, an HNSW index, and a filtered similarity query in PostgreSQL with pgvector.

```
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE documents (
  id bigserial PRIMARY KEY,
  tenant_id int NOT NULL,
  content text NOT NULL,
  embedding vector(768) NOT NULL,
  model text NOT NULL -- embedding model and version
);

-- Approximate index for cosine distance
CREATE INDEX documents_embedding_idx
  ON documents USING hnsw (embedding vector_cosine_ops)
  WITH (m = 16, ef_construction = 64);

-- Query-time recall/latency knob
SET hnsw.ef_search = 100;

-- Five nearest documents of one tenant ($1 is the query vector)
SELECT id, content, embedding <=> $1 AS distance
FROM documents
WHERE tenant_id = 42
ORDER BY embedding <=> $1
LIMIT 5;
```

Note that with an approximate index and a selective `WHERE` clause, the planner may apply the filter after the index scan, returning fewer rows than requested; raising the search parameter, using partial or partitioned indexes per tenant, or relying on newer filtering features in the extension are the usual remedies, and they illustrate the filtering problem of Section 14.6.

14.8.2 Elasticsearch: Vectors Beside Keywords

Search engines add a vector field type and a k-nearest-neighbor query, which makes hybrid search (Chapter 13) a single request (Elasticsearch Documentation). Listing 14.2 defines a vector field and runs a filtered query.

Listing 14.2: A vector field and a filtered k-nearest-neighbor query in Elasticsearch.

```
PUT /docs
{
  "mappings": {
    "properties": {
      "text": { "type": "text" },
      "tenant": { "type": "keyword" },
      "embedding": { "type": "dense_vector", "dims": 768,
                     "index": true, "similarity": "cosine" }
    }
  }
}

GET /docs/_search
{
  "knn": {
    "field": "embedding",
    "query_vector": [0.12, -0.03, 0.57], // abbreviated; 768 values in practice
    "k": 5,
    "num_candidates": 50,
    "filter": { "term": { "tenant": "acme" } }
  }
}
```

Here `num_candidates` plays the role of ef_{search}: examining more candidates per shard raises recall and cost.

14.9 Retrieval Pipelines

A vector index is only one component of a retrieval system. Figure 14.1 shows the stages of a typical pipeline for retrieval-augmented generation.

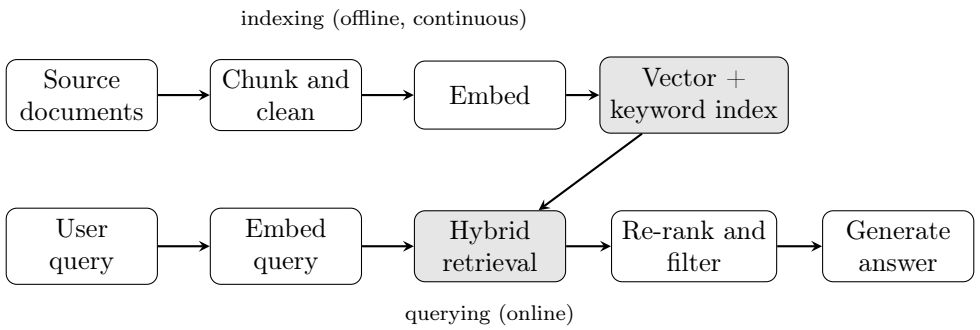


Figure 14.1: A retrieval pipeline. Documents are chunked, embedded, and indexed; at query time the question is embedded, candidates are retrieved with hybrid search, re-ranked and filtered, and passed to a generator.

14.9.1 Chunking

Documents are usually longer than an embedding model can accept, and a vector for a whole long document blurs its many topics, so documents are split into *chunks*. Chunking has a large effect on quality:

- **Fixed-size chunks** (for example a few hundred tokens) are simple but may cut through sentences and ideas.
- **Structure-aware chunks** split at headings, paragraphs, or sentences, preserving units of meaning.
- **Overlap** between neighboring chunks keeps context that would otherwise be cut at a boundary.
- **Hierarchical chunking** indexes small chunks for precise matching but returns the larger parent section for context.
- **Metadata and context.** Attaching the document title, section heading, source, and date to each chunk, and sometimes prepending them to the text before embedding, helps both retrieval and citation.

Chunk size is a compromise: small chunks are precise but lose context, and large chunks carry context but dilute the match. The best choice is found by measurement on real questions.

14.9.2 Hybrid Retrieval and Re-ranking

As described in Chapter 13, combining keyword and vector retrieval, for example with reciprocal rank fusion, handles both exact identifiers and paraphrases. A *re-ranker*, typically a cross-encoder model that reads the query and a candidate passage together, then reorders the top candidates (tens, not millions) with much better accuracy than vector similarity, at higher cost per item. The vector search thus provides cheap recall over the whole collection, and the re-ranker provides precision in the shortlist. Models that keep several vectors per item and compare them in a fine-grained way (*late interaction*, as in ColBERT) and *sparse* learned representations offer further points on the accuracy–cost curve (Khattab and Zaharia, 2020).

14.9.3 Evaluating Retrieval

Evaluate retrieval separately from the generation that follows it. Build a set of realistic questions with the passages that answer them, and measure whether the retriever returns those passages: recall at k against the labeled answers, *mean reciprocal rank* (MRR), which averages 1/rank of the first correct result, and nDCG (Chapter 13). Then measure the final answers for correctness and for faithfulness to the retrieved sources. Failures located by this separation are often retrieval failures (the right passage was never retrieved), which no change to the generating model can fix.

14.9.4 Other Uses of Vector Search

Beyond retrieval for language models, nearest-neighbor search drives *recommendation* (items near a user's history), *deduplication* and near-duplicate detection, *visual and audio search*, *anomaly detection* (points far from all clusters), and *classification* by the labels of nearest neighbors. In each case the same trade-offs apply: embedding quality, index recall, filtering, and freshness.

14.10 Operating Vector Systems

14.10.1 Capacity and Cost

Memory dominates cost, so estimate it early. A rough estimate for the vectors alone is $N \times d \times b$ bytes, where b is the bytes per dimension (4 for float32, 1 for int8, and 1/8 for binary), plus index overhead: for HNSW, the graph links add a few hundred bytes per vector depending on M. For 100 million 768-dimensional vectors, this means about 307 GB with float32, 77 GB with 8-bit scalar quantization, and about 10 GB with product quantization at 96 bytes per vector, before the graph, replicas, and headroom. These figures show why quantization and disk-based indexes matter at large scale, and why dimensionality (and truncation of Matryoshka-style vectors) is a cost decision.

14.10.2 Reindexing and Model Changes

Changing the embedding model, the chunking strategy, or the index type requires building a complete new index. The robust procedure is the same as for search indexes (Chapter 13): create the new index beside the old one, populate it from the source content (which must be retained), compare quality on the evaluation set, and then switch an alias or configuration atomically, keeping the old index available for rollback. Recording the model version with each vector makes it possible to run both versions during the transition and to detect accidental mixing.

14.10.3 Keeping Vectors Synchronized

The vector index is a derived copy of the source content, so the synchronization strategies of Chapter 13 apply: outbox or change data capture, idempotent writes keyed by chunk identifier and content hash, and periodic reconciliation. Two cases deserve special care. When a document changes, *all* of its old chunks must be replaced, or stale text will continue to be retrieved. When a document is deleted, for example to comply with a request under privacy law, its vectors must be deleted as well, and the deletion should be verified, since the embeddings are derived personal data.

14.10.4 Monitoring

Monitor the usual signals (query latency percentiles, throughput, memory, ingestion lag) and the vector-specific ones: recall on a canary query set compared against periodic brute-force

results, the distribution of similarity scores of returned results (a sudden shift suggests a data or model change), the proportion of queries returning fewer than k results because of filters, and the fraction of queries with no sufficiently close results, which reveals gaps in the content.

14.10.5 Security and Privacy

- **Embeddings are not anonymous.** Research has shown that text can often be partly reconstructed from its embedding (Morris et al., 2023), so vectors should be protected and governed like the source content: encrypted, access-controlled, and subject to the same retention and deletion rules.
- **Access control at retrieval.** Apply permission filters inside the query, as discussed in Section 14.6, so that restricted content is never retrieved.
- **Retrieved text is untrusted input.** A retrieved document may contain instructions aimed at the language model that consumes it (*prompt injection*), whether planted deliberately or not. The generating application must treat retrieved content as data, limit what actions the model can take on its basis, and not rely on the retrieval step to sanitize it.
- **Data poisoning.** An attacker who can add documents to the collection can influence what is retrieved. Control who may write to the index and track the provenance of every chunk.
- **Data location.** Sending content to an external embedding service transfers it to a third party. Check residency and contractual requirements, as with any managed service.

14.11 Choosing an Approach

A sound process is: (1) define the use case and collect real queries with known good answers; (2) choose and evaluate embedding models on that data, since model quality limits everything downstream; (3) design chunking and metadata, including the filters and permissions that every query needs; (4) start with the simplest storage that fits (exact search or the existing database) and measure recall and latency; (5) move to a dedicated index or system only when scale, filtering, or latency requires it, and tune recall against ground truth; (6) plan synchronization, reindexing for model changes, deletion, and monitoring from the start; and (7) treat the whole retrieval pipeline, not only the index, as the thing to evaluate.

Summary

Vector databases store embeddings, numerical representations produced by machine-learning models in which similar items lie close together, and answer nearest-neighbor queries. Distance measures such as cosine similarity, Euclidean distance, and the dot product are equivalent for normalized vectors, and the metric must match the model. Because vectors are tied to

Table 14.5: Choosing a vector search approach.

Situation	**Suggested approach**	**Reasoning**
Up to a few hundred thousand vectors; PostgreSQL already used	`pgvector`, possibly with exact search	Transactional, joins and filters in SQL; no new system
Existing search engine with keyword search	Add vector fields and hybrid queries to the engine	One system, one query, unified filtering
Tens of millions to billions of vectors with heavy filtering	Dedicated vector database, with quantization or disk-based index	Scale, filtered search, and tuning options
Prototype or embedded use	Library (FAISS, hnswlib) or lightweight database	Minimal setup
Many small tenants	Namespaces or partitions per tenant, or filtered shared index	Isolation and cost depend on tenant count and size
Exact identifiers, codes, names	Keyword index, not vectors (or hybrid)	Vectors blur exact matches

the model that produced them, model changes require re-embedding and reindexing. Exact search scales linearly with the collection and fails in high dimensions to benefit from classical indexes, so systems use approximate indexes: IVF partitions the space into cells and probes the nearest ones, HNSW navigates a layered proximity graph, disk-based graphs serve very large collections, and quantization (scalar, product, binary) compresses vectors by large factors at some loss of recall. Every index trades recall, latency, and memory, and recall must be measured against brute-force ground truth on the application's own data. Filtering by metadata and permissions is the hardest practical issue, handled by post-filtering, pre-filtering, in-index filtering, or partitioning, depending on selectivity. Architecturally, vector databases resemble search engines and LSM stores, with immutable segments, background compaction, sharding with scatter–gather, replication, eventual freshness, and increasingly disaggregated storage. Systems range from libraries to dedicated databases, relational extensions such as `pgvector`, and search engines with vector fields. A retrieval pipeline adds chunking, hybrid retrieval, re-ranking, and separate evaluation of retrieval and generation. Operations center on memory cost, safe reindexing when models change, synchronization and deletion, monitoring of recall and score distributions, and security: embeddings can leak information, retrieved text is untrusted, and permissions must be enforced at query time. The recommended approach is to begin simply, measure, and add scale and complexity only when the measurements demand it.

Review Questions

1. What is an embedding, and why does searching by embeddings find results that keyword search misses?
2. Explain the relationship among cosine similarity, dot product, and Euclidean distance for normalized vectors. Why must the index metric match the model's?
3. Why can vectors from two different embedding models not be compared? What follows for model upgrades?
4. Why does exact nearest-neighbor search not scale, and why do classical tree indexes fail in high dimensions?
5. Describe how an IVF index works and what n_{probe} controls. Where do its errors come from?
6. Describe HNSW search. What do M, $ef_{\text{construction}}$, and ef_{search} control?
7. Compare scalar, product, and binary quantization in terms of compression and loss.
8. Define recall at k for ANN search. How does it differ from recall in classical information retrieval?
9. Describe a procedure for tuning an ANN index to a target recall.
10. Compare post-filtering, pre-filtering, and in-index filtering. How does filter selectivity affect the choice?
11. Why are deletes and updates harder in vector indexes than in B-tree indexes?
12. Why do vector databases use sharding with scatter–gather, and what does it imply for latency and recall?
13. What is the role of a re-ranker in a retrieval pipeline? Why is it not used on the whole collection?
14. Why should retrieval be evaluated separately from answer generation?
15. Name four security or privacy concerns specific to embeddings and retrieval for language models.

Exercises

1. **Metrics.** Given $\mathbf{a} = (3, 4)$ and $\mathbf{b} = (6, 8)$, compute the Euclidean distance, the dot product, and the cosine similarity. Normalize both vectors and verify that $\|\mathbf{a} - \mathbf{b}\|^2 = 2 - 2\,\mathbf{a} \cdot \mathbf{b}$ holds for the normalized vectors. Which pair of vectors would be ranked differently by Euclidean distance and by cosine similarity?

2. **Capacity.** A collection has 250 million vectors of 1 024 dimensions. Estimate the memory needed with float32, 8-bit scalar quantization, and product quantization with 128 one-byte codes, ignoring index overhead. Which of these fit on a machine with 256 GB of RAM?

3. **Brute force.** Estimate the time to scan the collection of Exercise 2 for one query on a machine that can read memory at 100 GB/s. How many queries per second can one machine serve at best, and what does this imply for the choice of index?

4. **IVF.** A collection is divided into 4 096 cells; each query probes 16 cells. What fraction of the collection is scanned (assuming equal-sized cells)? Explain why a query whose true neighbors are spread over many cells has lower recall, and how increasing the probe count changes cost and recall.

5. **Recall measurement.** Design an experiment to tune an HNSW index for a recall of 0.95 at $k = 10$ and a p99 latency under 50 ms. Specify the query sample, the ground truth, the parameters varied, the measurements recorded, and how you would handle a data set that grows by 10% per month.

6. **Filtering.** A document store has 50 million chunks over 10 000 tenants. A query must search within one tenant. Compare a shared index with a tenant filter, one index per tenant, and partitioning of tenants into groups, discussing recall, latency, memory, isolation, and deletion of a tenant's data.

7. **Chunking.** Propose and compare two chunking strategies for (a) a product manual with headings and tables, and (b) a transcript of customer-support calls. State the chunk size, overlap, and metadata, and the experiment you would run to choose between them.

8. **Pipeline design.** Design a retrieval pipeline for a company knowledge base with per-department access rights. Specify the embedding model selection process, index and filters, hybrid retrieval, re-ranking, synchronization with source systems, deletion handling, and monitoring.

9. **Model migration.** The team wants to replace its embedding model with one of different dimensionality. Write a migration plan with the steps, the validation criteria, the rollback path, and the estimated cost of re-embedding 80 million chunks given an embedding throughput of 2 000 chunks per second per worker.

10. **Security review.** List the attack surfaces of a RAG system built on a vector database, including prompt injection through retrieved documents and poisoning of the index, and propose a mitigation for each.

References

- Aumüller, M., Bernhardsson, E., & Faithfull, A. (2020). ANN-Benchmarks: A Benchmarking Tool for Approximate Nearest Neighbor Algorithms. *Information Systems*, 87, 101374.

- Beyer, K., Goldstein, J., Ramakrishnan, R., & Shaft, U. (1999). When Is "Nearest Neighbor" Meaningful? In *Proceedings of ICDT '99*, 217–235.
- Elasticsearch Documentation: k-nearest neighbor (kNN) search. https://www.elastic.co/guide/
- Indyk, P., & Motwani, R. (1998). Approximate Nearest Neighbors: Towards Removing the Curse of Dimensionality. In *Proceedings of ACM STOC '98*, 604–613.
- Jégou, H., Douze, M., & Schmid, C. (2011). Product Quantization for Nearest Neighbor Search. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 33(1), 117–128.
- Johnson, J., Douze, M., & Jégou, H. (2021). Billion-Scale Similarity Search with GPUs. *IEEE Transactions on Big Data*, 7(3), 535–547.
- Karpukhin, V., Oğuz, B., Min, S., et al. (2020). Dense Passage Retrieval for Open-Domain Question Answering. In *Proceedings of EMNLP 2020*, 6769–6781.
- Khattab, O., & Zaharia, M. (2020). ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT. In *Proceedings of ACM SIGIR '20*, 39–48.
- Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.
- Kusupati, A., Bhatt, G., Rege, A., et al. (2022). Matryoshka Representation Learning. In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*.
- Lewis, P., Perez, E., Piktus, A., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*.
- Malkov, Y. A., & Yashunin, D. A. (2020). Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4), 824–836.
- Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press.
- Morris, J. X., Kuleshov, V., Shmatikov, V., & Rush, A. M. (2023). Text Embeddings Reveal (Almost) as Much as Text. In *Proceedings of EMNLP 2023*.
- pgvector Documentation. https://github.com/pgvector/pgvector
- Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks. In *Proceedings of EMNLP-IJCNLP 2019*, 3982–3992.
- Subramanya, S. J., Devvrit, Kadekodi, R., Krishnaswamy, R., & Simhadri, H. V. (2019). DiskANN: Fast Accurate Billion-Point Nearest Neighbor Search on a Single Node. In *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*.

- Wang, J., Yi, X., Guo, R., et al. (2021). Milvus: A Purpose-Built Vector Data Management System. In *Proceedings of ACM SIGMOD '21*, 2614–2627.

Part IV

Query Languages Beyond SQL

Part V

Query Languages Beyond SQL

Chapter 15

Querying Graph Data with Cypher

Learning Objectives

After reading this chapter, you will be able to:

- Explain why graph queries are expressed as patterns, and relate Cypher to openCypher, GQL, and SQL/PGQ.
- Read and write Cypher patterns for nodes, relationships, properties, and variable-length paths.
- Use `MATCH`, `WHERE`, `RETURN`, `WITH`, `OPTIONAL MATCH`, `UNWIND`, and aggregation to answer questions over a property graph.
- Create and modify graph data safely with `CREATE`, `MERGE`, `SET`, and `DELETE`, and avoid the common pitfalls of `MERGE`.
- Describe how a Cypher query is executed, and use indexes, constraints, and `PROFILE` to diagnose slow queries.
- Compare a Cypher query with its SQL formulation, and decide when a graph query language is the better tool.
- Model data for traversal queries, and combine graph traversal with vector search.
- Address bulk loading, parameterization, and security concerns specific to graph queries.

15.1 Why a Graph Query Language

Chapter 11 described the property graph model and the systems that store it. This chapter turns to the question of how to ask questions of a graph. The central observation is that questions about connected data are naturally questions about *shapes*: "people who work at the same company as a friend of Ada," "accounts linked by a chain of shared devices," "the shortest route between two stations." A relational query describes such a shape indirectly, as a sequence of joins between tables whose keys the author must remember. A graph query language lets the author *draw* the shape and leaves the search for matching subgraphs to the engine.

Cypher is the best-known language of this kind. It was created at Neo4j and released

in 2015 as the open specification *openCypher*, which was adopted by several other systems. Its design principle is a visual one: a pattern in a query looks like a diagram drawn with characters, with parentheses for nodes and arrows for relationships. Cypher is *declarative*: the query states which pattern to find and which data to return, and the planner decides the order of traversal, the indexes to use, and the join strategy (Francis et al., 2018).

Cypher is also part of a standardization story. In 2024 the ISO standard *GQL* (ISO/IEC 39075) was published as the first new database query language standard since SQL, and it draws heavily on Cypher's pattern syntax. In parallel, SQL:2023 added *SQL/PGQ* (Part 16 of ISO/IEC 9075), which lets SQL queries match graph patterns over tables (Deutsch et al., 2022). A reader who learns Cypher therefore learns the core of all three notations. Section 15.12 compares them.

15.1.1 What Is Different from SQL

- **Patterns, not joins.** Relationships are first-class and are traversed directly, not reconstructed by matching foreign keys.
- **Variable-length structure.** A query can ask for paths of unknown length ("reachable in one to five steps") without recursion syntax.
- **Cost follows the neighborhood.** In graph stores with direct adjacency, the cost of a traversal grows with the part of the graph actually visited, not with the total size of the data. This is the main performance argument for graph databases, and it holds only when a query starts from a small, well-indexed set of nodes.
- **Flexible schema.** Nodes with the same label may have different properties, and new relationship types can be added without migrations. Constraints are optional and must be declared (Section 15.8).

15.2 A Running Example

The examples in this chapter use a small graph about people, employers, and skills, shown in Figure 15.1. Listing 15.1 creates it. Dan has no employer, which will be useful to illustrate optional matches.

Listing 15.1: Creating the example graph.

```
CREATE
  (ada:Person {id: 1, name: 'Ada', born: 1990}),
  (ben:Person {id: 2, name: 'Ben', born: 1985}),
  (cara:Person {id: 3, name: 'Cara', born: 1992}),
  (dan:Person {id: 4, name: 'Dan', born: 1988}),
  (acme:Company {name: 'Acme', city: 'Berlin'}),
  (zeta:Company {name: 'Zeta'}),
  (gm:Skill {name: 'Graph␣modeling'}),
  (sql:Skill {name: 'SQL'}),
  (ada)-[:WORKS_AT {since: 2019}]->(acme),
  (ben)-[:WORKS_AT {since: 2015}]->(acme),
  (cara)-[:WORKS_AT {since: 2021}]->(zeta),
```

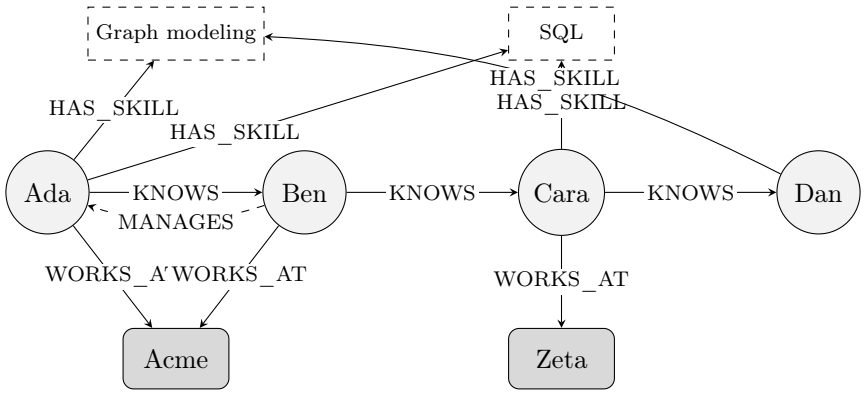


Figure 15.1: The example graph: people (circles), companies (rounded rectangles), and skills (dashed rectangles). Property values are omitted from the figure and given in Listing 15.1.

```
(ada)-[:KNOWS {since: 2018}]->(ben),
(ben)-[:KNOWS {since: 2020}]->(cara),
(cara)-[:KNOWS {since: 2022}]->(dan),
(ben)-[:MANAGES]->(ada),
(ada)-[:HAS_SKILL {level: 4}]->(gm),
(ada)-[:HAS_SKILL {level: 5}]->(sql),
(cara)-[:HAS_SKILL {level: 3}]->(sql),
(dan)-[:HAS_SKILL {level: 2}]->(gm);
```

15.3 Patterns: The Heart of Cypher

A *pattern* describes a subgraph. Table 15.1 lists its elements.

Three rules govern how patterns are matched. First, a relationship has exactly one *type* and one *direction* in storage, but a query may ignore the direction by omitting the arrowhead. Second, within a single `MATCH` clause, a relationship is never traversed twice in the same match (*relationship uniqueness*); nodes may repeat, but edges may not, which keeps variable-length matches finite on graphs with cycles. Third, labels and property maps in a pattern are filters: they restrict which nodes can bind to a variable, and the planner uses them to choose a starting point, ideally an indexed one.

15.3.1 Naming Conventions

By convention, labels are written in `CamelCase` (`Person`), relationship types in `UPPER_SNAKE_CASE` (`WORKS_AT`), and properties and variables in `camelCase`. Keywords are case-insensitive but are usually written in capitals. Labels, types, and property names are case-sensitive.

Table 15.1: Elements of a Cypher pattern.

Syntax	Meaning
`()`	Any node (anonymous)
`(p)`	Any node, bound to the variable `p`
`(p:Person)`	A node with label `Person`
`(p:Person {name: 'Ada'})`	A `Person` node whose property `name` equals `'Ada'`
`(a)-[:KNOWS]->(b)`	A `KNOWS` relationship from `a` to `b`
`(a)-[:KNOWS]-(b)`	A `KNOWS` relationship in either direction
`(a)-[r:WORKS_AT]->(c)`	As above, with the relationship bound to `r`, so its properties can be read
`(a)-[:KNOWS|MANAGES]->(b)`	A relationship of either type
`(a)-[:KNOWS*1..3]->(b)`	A chain of one to three `KNOWS` relationships
`p = (a)-[:KNOWS]->(b)`	The whole matched path bound to the variable `p`

15.4 Reading Data

A Cypher query is a sequence of clauses. Each clause receives a table of rows (bindings of variables) from the previous clause and produces a new table. The first query below finds the colleagues of Ada.

Listing 15.2: A basic pattern match with filtering and projection.

```
MATCH (a:Person {name: 'Ada'})-[:WORKS_AT]->(c:Company)<-[:WORKS_AT]-(colleague:
    Person)
WHERE colleague <> a
RETURN colleague.name AS colleague, c.name AS company
ORDER BY colleague;
```

The pattern joins two `WORKS_AT` relationships at the shared company node. On the example data the result is the single row (Ben, Acme). The `WHERE` clause is a predicate over bound variables; it supports comparison, boolean operators, string tests (`STARTS WITH`, `ENDS WITH`, `CONTAINS`), list membership (`IN`), and tests on null (`IS NULL`, `IS NOT NULL`). Like SQL, Cypher uses three-valued logic: a comparison with a missing property yields null, and a row is kept only if the predicate is true.

15.4.1 Projection, Ordering, and Paging

`RETURN` chooses the output columns and may rename them with `AS`. `DISTINCT` removes duplicate rows, `ORDER BY` sorts, and `SKIP` and `LIMIT` page through the results. As in SQL, paging without a deterministic ordering returns unstable pages.

15.4.2 Aggregation

Cypher has no `GROUP BY`. Instead, grouping is implicit: in a `RETURN` (or `WITH`) that mixes aggregate and non-aggregate expressions, the non-aggregate expressions form the grouping key. Common aggregates are `count`, `sum`, `avg`, `min`, `max`, and `collect`, which gathers values into a list.

Listing 15.3: Implicit grouping and collect.

```
// Number of employees and the list of their names, per company
MATCH (p:Person)-[:WORKS_AT]->(c:Company)
RETURN c.name AS company,
       count(p) AS employees,
       collect(p.name) AS names
ORDER BY employees DESC;
```

15.4.3 Chaining Query Parts with WITH

`WITH` ends one part of a query and starts the next. It passes selected variables forward, can aggregate, sort, and limit in the middle of a query, and discards everything not listed. Because `WHERE` may follow `WITH`, it also plays the role of SQL's `HAVING`.

Listing 15.4: WITH as an intermediate step and as HAVING.

```
// Skills held by at least two people, with their average level
MATCH (p:Person)-[h:HAS_SKILL]->(s:Skill)
WITH s, count(p) AS holders, avg(h.level) AS avgLevel
WHERE holders >= 2
RETURN s.name AS skill, holders, avgLevel
ORDER BY holders DESC;
```

15.4.4 Optional Matches, Existence Tests, and Unwinding

`OPTIONAL MATCH` is the counterpart of a left outer join: if the pattern is not found, the row is kept and the new variables are null. Existence and counting subqueries test a pattern without multiplying rows. `UNWIND` turns a list into rows, which is the basis of batch processing (Section 15.13).

Listing 15.5: OPTIONAL MATCH, an existence subquery, and UNWIND.

```
// Everyone, with their employer if any (Dan gets null)
MATCH (p:Person)
OPTIONAL MATCH (p)-[:WORKS_AT]->(c:Company)
RETURN p.name AS person, c.name AS employer;

// People without an employer
MATCH (p:Person)
WHERE NOT EXISTS { (p)-[:WORKS_AT]->() }
RETURN p.name;
```

```
// Rows from a list
UNWIND ['Ada', 'Dan'] AS name
MATCH (p:Person {name: name})
RETURN p.name, p.born;
```

15.5 Paths and Variable-Length Traversal

The ability to follow relationships to an unknown depth is the main reason to use a graph language. The notation `*1..3` after a relationship type bounds the number of hops. Binding the pattern to a path variable gives access to its nodes, relationships, and length.

Listing 15.6: Variable-length paths, a shortest path, and quantified relationships.

```
// Everyone Ada can reach through one to three KNOWS hops
MATCH p = (a:Person {name: 'Ada'})-[:KNOWS*1..3]->(b:Person)
RETURN b.name AS person, min(length(p)) AS hops
ORDER BY hops;

// Shortest connection between two people, ignoring direction
MATCH p = shortestPath(
  (a:Person {name: 'Ada'})-[:KNOWS*..6]-(d:Person {name: 'Dan'}))
RETURN [n IN nodes(p) | n.name] AS route, length(p) AS hops;

// Newer quantified syntax (Cypher 5.9 and later; check your version)
MATCH (a:Person {name: 'Ada'})-[:KNOWS]->{1,3}(b:Person)
RETURN DISTINCT b.name;
```

On the example data Ada reaches Ben in one hop, Cara in two, and Dan in three, and the shortest route from Ada to Dan has three hops. `allShortestPaths` returns every path of minimal length instead of one.

15.5.1 Why Depth Must Be Bounded

The number of paths grows exponentially with depth. If each node has on average 150 neighbors, there are about $150^2 = 22\,500$ paths of length two, $150^3 \approx 3.4$ million of length three, and $150^4 \approx 506$ million of length four. Since a variable-length match enumerates paths (not just reachable nodes), an unbounded `*` on a real social graph can exhaust memory and time. Practical rules are to bound the depth, to start from a small set of indexed nodes, to filter relationship types and node labels inside the pattern, and to use `shortestPath` when only the shortest connection matters, since it uses a bidirectional breadth-first search instead of enumerating all paths.

15.5.2 Supernodes

A *supernode* is a node with an extremely high degree, such as a country, a popular tag, or a celebrity account. Any traversal that passes through it fans out to millions of neighbors.

Mitigations include excluding such nodes in the pattern, splitting them by time or category in the model (Section 15.10), and making sure traversals through them are filtered by relationship properties or direction.

15.6 Writing Data

Cypher is also a data manipulation language. Table 15.2 lists the writing clauses.

Table 15.2: Writing clauses of Cypher.

Clause	Effect
`CREATE`	Always creates the nodes and relationships of the pattern, even if identical ones exist
`MERGE`	Matches the pattern if it exists; otherwise creates it (a conditional create, comparable to an upsert)
`SET`	Sets or updates properties, or adds labels; `SET n += {map}` merges a map of properties
`REMOVE`	Removes a property or a label
`DELETE`	Deletes a node or relationship; a node with relationships cannot be deleted unless they are deleted first
`DETACH DELETE`	Deletes a node together with all its relationships

15.6.1 MERGE and Its Pitfall

`MERGE` is the tool for idempotent loading, but it matches the *entire* pattern. If any part of the pattern is missing, `MERGE` creates the whole pattern, including duplicate copies of nodes that already exist. The safe habit is to merge nodes by a unique key first, and then merge the relationship between the bound nodes.

Listing 15.7: Idempotent loading: merge nodes by key, then the relationship.

```
// Parameters: $personId, $name, $company, $since
MERGE (p:Person {id: $personId})
  ON CREATE SET p.name = $name, p.createdAt = timestamp()
  ON MATCH SET p.lastSeen = timestamp()
MERGE (c:Company {name: $company})
MERGE (p)-[r:WORKS_AT]->(c)
  ON CREATE SET r.since = $since;
```

The pattern in the last `MERGE` binds `p` and `c` first, so the clause can create at most the relationship. Concurrent `MERGE` operations on the same key can still create duplicates unless a uniqueness constraint exists, which is the reason to declare one (Section 15.8).

15.6.2 Updates and Deletes

Listing 15.8: Updating properties and deleting data.

```
// Change a property and add a label
MATCH (p:Person {name: 'Dan'})
SET p.born = 1989, p:Candidate;

// Remove a relationship, keep the nodes
MATCH (:Person {name: 'Cara'})-[r:KNOWS]->(:Person {name: 'Dan'})
DELETE r;

// Remove a person and everything attached to them
MATCH (p:Person {name: 'Dan'})
DETACH DELETE p;
```

Writes run inside transactions with ACID guarantees in the main graph databases, as described in Chapter 11. Deleting large parts of a graph in a single transaction can exhaust memory, so large deletes are done in batches (Section 15.13).

15.7 How a Cypher Query Is Executed

Understanding execution explains most performance problems. The planner converts a query into a tree of *operators*, each of which consumes and produces rows. Typical operators are:

- **Scans and seeks**: `NodeByLabelScan` reads every node with a label; `NodeIndexSeek` and `NodeUniqueIndexSeek` look nodes up through an index. A scan over a large label is the usual cause of slow queries.
- **Expand**: follows relationships from bound nodes to their neighbors. The cost depends on the degree of the nodes visited.
- **Filter, projection, aggregation, sort**: row-by-row work, as in relational plans.
- **Joins**: when two sub-patterns must be combined, the planner may use a hash join; a missing connection between two patterns produces a `CartesianProduct`, which multiplies the row counts and is almost always an error.

The planner uses statistics about label counts and degrees to choose the starting points and the order of expansion. A good plan starts from the most selective, indexed pattern and expands outward; a bad one starts from a large label and filters late.

15.7.1 Inspecting Plans

`EXPLAIN` shows the plan without running the query, and `PROFILE` runs it and annotates each operator with the number of rows and *database hits*, the storage accesses it performed. The procedure parallels the one for relational systems (Chapter 13 discussed measuring queries against realistic data): run the query on realistic data, find the operator with the largest number of rows or hits, and ask why the planner started where it did.

Listing 15.9: Profiling a query.

```
PROFILE
MATCH (a:Person {name: 'Ada'})-[:KNOWS]->(f)-[:WORKS_AT]->(c:Company)
RETURN c.name, count(f) AS friends;
```

If `PROFILE` shows a `NodeByLabelScan` on `Person` followed by a filter on `name`, the query reads every person to find Ada; an index on `Person(name)` replaces the scan by a seek. Other common remedies are to remove accidental Cartesian products, to bound variable-length patterns, to filter early (before a `WITH` that aggregates), and to avoid returning entire nodes and paths when a few properties suffice.

15.8 Indexes and Constraints

Graph traversal is fast only after the starting nodes have been found, so indexes matter at the *entry points* of a query, not along the traversal itself. Table 15.3 summarizes the main kinds in Neo4j-style systems; the exact names and options vary between systems and versions.

Table 15.3: Index and constraint types for property graphs.

Kind	**Supports**	**Remarks**
Range index	Equality, ranges, ordering on a property	The default index for lookups by key or by value ranges
Text index	String matching such as `CONTAINS` and `ENDS WITH`	Compact; for string properties only
Full-text index	Tokenized, scored keyword search over several properties	The graph analogue of the search indexes of Chapter 13
Vector index	Approximate nearest-neighbor search on an embedding property	HNSW-based; see Chapter 14
Uniqueness constraint	Guarantees one node per key value; creates an index	Required for safe concurrent `MERGE`
Existence and type constraints	Mandatory properties and property types (in some editions)	Brings schema discipline to a schema-optional model

Listing 15.10: Constraints and indexes (Neo4j 5 syntax).

```
CREATE CONSTRAINT person_id IF NOT EXISTS
  FOR (p:Person) REQUIRE p.id IS UNIQUE;

CREATE INDEX person_name IF NOT EXISTS
```

```
  FOR (p:Person) ON (p.name);

CREATE VECTOR INDEX chunk_embedding IF NOT EXISTS
  FOR (c:Chunk) ON (c.embedding)
  OPTIONS { indexConfig: {
    `vector.dimensions`: 768,
    `vector.similarity_function`: 'cosine' } };
```

15.9 Cypher and SQL Side by Side

Consider the question: *whom can Ada reach through up to three steps of acquaintance, and at what minimum distance?* In SQL the answer needs a recursive common table expression over a table `knows(person_a, person_b)`.

Listing 15.11: The reachability query in SQL with a recursive CTE.

```
WITH RECURSIVE reach(id, depth) AS (
  SELECT k.person_b, 1
  FROM knows k
  WHERE k.person_a = 1 -- Ada's id
  UNION
  SELECT k.person_b, r.depth + 1
  FROM reach r
  JOIN knows k ON k.person_a = r.id
  WHERE r.depth < 3
)
SELECT p.name, MIN(r.depth) AS hops
FROM reach r
JOIN person p ON p.id = r.id
GROUP BY p.name
ORDER BY hops;
```

The Cypher version is the first query of Listing 15.6: one pattern, with the bound written as `*1..3`. SQL:2023 adds a third notation. With SQL/PGQ, a property graph is declared as a view over tables, and patterns are matched with the `GRAPH_TABLE` operator:

Listing 15.12: The same question in SQL/PGQ (illustrative; system support varies).

```
SELECT friend, MIN(hops) AS hops
FROM GRAPH_TABLE ( social_graph
  MATCH (a IS person WHERE a.name = 'Ada')
        -[IS knows]->{1,3} (b IS person)
  COLUMNS (b.name AS friend, 1 AS hops) ) -- path length reporting varies
GROUP BY friend;
```

The comparison in Table 15.4 highlights where each approach is strongest.

For queries that touch most of the data (totals per region, monthly reports) a relational or columnar engine remains the better choice. Graph databases earn their place when

Table 15.4: Cypher compared with SQL for connected data.

Aspect	**Cypher (graph store)**	**SQL (relational store)**
Multi-hop queries	Concise; cost follows the visited neighborhood	Self-joins or recursive CTEs; cost depends on join plans and indexes
Unknown depth	Built-in variable-length patterns and shortest paths	Recursion with explicit termination logic
Large aggregations	Workable, but not the main strength	Mature optimizers, columnar engines, parallel scans
Schema and integrity	Optional; constraints must be declared	Strong, declarative, enforced by default
Ecosystem	Smaller; fewer reporting and BI tools	Very large; universal tooling

the questions are about *connections* between a small number of starting points and their neighborhoods.

15.10 Modeling for Queries

In a graph database the model is shaped by the queries, much as in the document and column-family stores of earlier chapters. A few guidelines apply.

- **Entities become nodes, connections become relationships.** If a noun appears in questions as something to be found or counted (a person, an account, a device), it is a node. A verb that links two entities (works at, owns, transferred to) is a relationship.
- **Make a relationship a node when it needs to connect to more than two things.** A relationship connects exactly two nodes. An employment that involves a person, a company, a role, and a period is better modeled as an `Employment` node with a relationship to each, so that further links can be added later.
- **Use specific relationship types.** `(:Person)-[:WORKS_AT]->(:Company)` lets the engine follow only the relevant edges. A generic type with a `kind` property forces the traversal to read and filter every relationship.
- **Prefer properties for attributes and relationships for things you traverse.** A city stored as a string cannot be traversed; if questions ask "who lives in the same city," make `City` a node.
- **Keep traversal paths short and selective.** Avoid shortcuts through supernodes, and consider splitting a high-degree node into time-based or category-based buckets.
- **Store large content outside the graph.** Long texts, files, and binary data are better kept in an object store or document store, with an identifier or a link in the node.

15.11 Common Patterns

15.11.1 Recommendation

Recommendations follow the pattern of "people like you also did this." The query below suggests skills held by people who share a skill with Ada, which she does not yet have, ranked by how many such peers hold them.

Listing 15.13: A collaborative recommendation as a graph pattern.

```
MATCH (a:Person {name: 'Ada'})-[:HAS_SKILL]->(:Skill)<-[:HAS_SKILL]-(peer:Person),
      (peer)-[:HAS_SKILL]->(s:Skill)
WHERE peer <> a AND NOT (a)-[:HAS_SKILL]->(s)
RETURN s.name AS suggestion, count(DISTINCT peer) AS peers
ORDER BY peers DESC
LIMIT 5;
```

15.11.2 Fraud and Link Analysis

Fraud rings are groups of accounts linked through shared attributes such as devices, addresses, or phone numbers. In a graph, those attributes are nodes, so a ring is a connected subgraph: accounts that share a device, and devices used by the same accounts, within a few hops. Queries of this kind are hard to write as joins, because the number of hops is not known in advance, and they are a classic use of variable-length patterns.

15.11.3 Access Control and Hierarchies

Permissions that flow through groups and folders form a graph. "May this user read this document?" becomes a path question: is there a path from the user through memberships to a group that holds a read permission on the document or on one of its ancestor folders?

Listing 15.14: A permission check as a path existence test.

```
MATCH (u:User {id: $userId}), (d:Document {id: $docId})
RETURN EXISTS {
  (u)-[:MEMBER_OF*0..5]->(:Group)-[:CAN_READ]->(:Folder)<-[:INSIDE*0..8]-(d)
} AS allowed;
```

15.11.4 Combining Graph Traversal and Vector Search

Chapter 14 showed how vector search finds the passages that are *similar* to a question. A graph adds what similarity cannot express: which document a passage belongs to, which entities it mentions, and what else is connected to them. Systems that retrieve a few passages by vector similarity and then expand to their graph neighborhood are often called GraphRAG (Edge et al., 2024). Listing 15.15 shows the pattern.

Listing 15.15: Vector search followed by graph expansion and a permission filter.

```
// Fetch more candidates than needed, because the filter comes afterwards
CALL db.index.vector.queryNodes('chunk_embedding', 50, $queryVector)
YIELD node AS chunk, score
MATCH (chunk)-[:PART_OF]->(doc:Document)
WHERE EXISTS {
  (:User {id: $userId})-[:MEMBER_OF]->(:Group)-[:CAN_READ]->(doc)
}
OPTIONAL MATCH (chunk)-[:MENTIONS]->(e:Entity)<-[:MENTIONS]-(related:Chunk)
WHERE related <> chunk
RETURN doc.title AS title, chunk.text AS text, score,
       collect(DISTINCT related.text)[..3] AS related
ORDER BY score DESC
LIMIT 5;
```

Two points deserve attention. First, the vector search returns its nearest candidates *before* the permission filter is applied, which is the post-filtering problem of Section 14.6: a selective filter can leave too few results, so the query requests 50 candidates and keeps five. Second, the permission check is inside the query, so that restricted content is never retrieved, not removed afterwards.

15.12 Standards and Related Languages

Table 15.5: Graph query languages and standards.

Language	Model	Style	Remarks
Cypher / openCypher	Property graph	Declarative, ASCII-art patterns	Most widely taught; used by Neo4j and several others
GQL (ISO/IEC 39075:2024)	Property graph	Declarative; Cypher-like patterns	International standard for graph databases; adoption is still growing
SQL/PGQ (ISO/IEC 9075-16:2023)	Property graph over tables	Pattern matching inside SQL	Graph queries on relational data
Gremlin	Property graph	Imperative traversal steps	Part of Apache TinkerPop; flexible but lower-level
SPARQL	RDF triples	Declarative triple patterns	Standard of the semantic web; different data model

The practical consequence is that the pattern-matching skills of this chapter transfer: a team that learns Cypher can read GQL almost immediately, and the same pattern ideas

appear in SQL/PGQ. Vendor extensions, such as procedures for graph algorithms or for vector indexes, remain system-specific, and the syntax of recent features changes between versions, so consult the current documentation of the system in use.

15.13 Operating Graph Queries

15.13.1 Parameters and Injection

Queries should receive values through *parameters* (`$name`), not through string concatenation. Parameters prevent query injection, in which crafted input changes the meaning of a query, and they let the server cache and reuse plans. Labels, relationship types, and property names cannot be parameters in standard Cypher; when they come from user input, validate them against an allow-list before building the query text.

15.13.2 Bulk Loading and Batching

Loading data row by row is slow, and one huge transaction can exhaust memory. The standard approach is to send batches, using `UNWIND` over a list parameter or a subquery that commits in batches.

Listing 15.16: Batched import from CSV with periodic commits, and a batched delete.

```
LOAD CSV WITH HEADERS FROM 'file:///people.csv' AS row
CALL {
  WITH row
  MERGE (p:Person {id: toInteger(row.id)})
    ON CREATE SET p.name = row.name
} IN TRANSACTIONS OF 5000 ROWS;

// Delete old data in batches of 10000 nodes
MATCH (n:Event) WHERE n.ts < $cutoff
CALL { WITH n DETACH DELETE n } IN TRANSACTIONS OF 10000 ROWS;
```

Create the uniqueness constraints and indexes *before* loading, so that each `MERGE` is an index seek and not a label scan. For very large initial loads, most systems provide an offline import tool that is much faster than queries.

15.13.3 Limits, Timeouts, and Access Control

A single careless query can scan the whole graph or enumerate billions of paths. Production systems set transaction timeouts and memory limits, restrict variable-length depth in application code, and use role-based access control to limit who can run write queries or administrative procedures. Access at the level of individual nodes and relationships is usually enforced in the application or in the query (as in Listing 15.14), not by the query language itself.

15.13.4 Monitoring

Monitor query latency percentiles, the slowest queries with their plans, page cache hit ratio, transaction counts and conflicts, memory use, and replication lag in clustered deployments (Chapter 9). Keep `PROFILE` output for critical queries as a baseline, and repeat it after data growth, since a plan that was good for a million nodes may not be for a hundred million.

15.14 Choosing an Approach

Table 15.6: Choosing an approach for connected data.

Situation	Suggested approach	Reasoning
Questions of two or three joins over well-known keys	Relational database with SQL	Mature, simple, no new system
Occasional graph questions over existing relational data	SQL recursive CTEs or SQL/PGQ where available	Reuses the existing system and its integrity guarantees
Frequent multi-hop traversals, paths of unknown length, relationship-centric analysis	Graph database queried with Cypher or GQL	Concise queries; cost follows the neighborhood
Retrieval that needs both similarity and connections	Vector index combined with graph expansion	Similarity finds the entry points; the graph supplies context
Whole-dataset analytics and reporting	Columnar or analytical engine	Scans and aggregates over everything
Billions of relationships with simple lookups	Wide-column or key-value store, or a distributed graph system	Scale and partitioning requirements dominate

A sound process is: (1) collect the real questions and mark those that are about connections; (2) draw the model on a whiteboard as nodes and relationships, and check that each question is a short, selective traversal from an identifiable starting point; (3) write the queries first, with the constraints and indexes that give them indexed entry points; (4) load realistic data and use `PROFILE` to confirm that no query starts with a large scan or a Cartesian product; (5) bound variable-length patterns and plan for supernodes; (6) parameterize queries and set timeouts; and (7) revisit the plans as the data grows.

Summary

Cypher is a declarative query language for property graphs in which patterns are drawn with parentheses for nodes and arrows for relationships, and the engine finds all matching subgraphs.

A query is a pipeline of clauses that transform tables of variable bindings: `MATCH` finds patterns, `WHERE` filters, `WITH` passes and aggregates between parts, `OPTIONAL MATCH` behaves like an outer join, and `RETURN` projects the result, with grouping implied by the non-aggregated expressions. Variable-length patterns and shortest paths answer questions about connectivity that need recursion in SQL, but the number of paths grows exponentially with depth, so depth must be bounded and supernodes handled. Data is written with `CREATE`, `MERGE`, `SET`, and `DELETE`; `MERGE` matches the whole pattern, so nodes should be merged by a unique key first, and uniqueness constraints are needed for safety under concurrency. Execution follows an operator tree whose cost is governed by the starting points, which indexes must make selective, and `EXPLAIN` and `PROFILE` reveal scans, expansions, and accidental Cartesian products. Compared with SQL, Cypher excels at multi-hop, relationship-centric questions and is weaker for whole-dataset analytics. Modeling decisions (nodes versus relationships, specific relationship types, intermediate nodes) determine what can be queried efficiently. Cypher's pattern syntax underlies the ISO standard GQL and SQL/PGQ, and combined with vector indexes it supports retrieval that joins similarity with structure, provided that permission filters are applied inside the query. Operationally, queries should be parameterized, bulk loads batched, timeouts set, and plans monitored as data grows.

Review Questions

1. Why are questions about connected data naturally expressed as patterns? What does the planner decide that the author of a Cypher query does not?
2. Explain the elements of the pattern `(a:Person {name: 'Ada'})-[r:KNOWS]->(b)`.
3. What is relationship uniqueness, and why does it matter for variable-length patterns on graphs with cycles?
4. How does grouping work in Cypher without a `GROUP BY` clause?
5. What is the role of `WITH`? How can it replace SQL's `HAVING`?
6. Compare `MATCH` and `OPTIONAL MATCH`. When would a query return null values for the variables of an optional pattern?
7. Why can an unbounded variable-length pattern be dangerous? Give three ways to limit its cost.
8. What is a supernode, and how does it affect traversal queries? How can the model reduce the problem?
9. Explain the difference between `CREATE` and `MERGE`. Why can `MERGE` on a full pattern create duplicates?
10. Why is a uniqueness constraint needed even if every load uses `MERGE`?
11. What do `NodeByLabelScan`, `NodeIndexSeek`, `Expand`, and `CartesianProduct` indicate in a query plan?

12. Why do indexes matter at the start of a graph query but not along the traversal itself?
13. For which kinds of queries is Cypher preferable to SQL, and for which is SQL or a columnar engine preferable?
14. When should a relationship in the model be replaced by an intermediate node?
15. How do Cypher, GQL, and SQL/PGQ relate to each other?
16. Why should permission filters be part of a GraphRAG query and not applied to its output?

Exercises

1. **Reading patterns.** Using the graph of Listing 15.1, write Cypher queries that (a) list the names of all people who have the skill `SQL` at level 4 or higher, (b) list each company with the number of employees, including companies with none, and (c) list the people who know someone who works at a different company than they do (or who have no employer). State the expected results.
2. **Aggregation and WITH.** Write a query that returns, for each skill, the name of the person with the highest level. Then write a variant that returns only skills held by more than one person. Explain how implicit grouping decides the result in each query.
3. **Optional matches.** Write a query that lists every person with the name of their manager, or null if they have none, and the number of people they manage. Explain why `OPTIONAL MATCH` is needed, and what changes if it is replaced by `MATCH`.
4. **Paths.** Write queries that (a) find the shortest `KNOWS` route from Ada to Dan ignoring direction, (b) find all people reachable from Ada through `KNOWS` relationships created in 2020 or later, and (c) find pairs of people connected by a path of `KNOWS` relationships whose `since` values are strictly increasing along the path. Which of these needs a predicate over the relationships of a path?
5. **Path explosion.** In a graph where each node has on average 80 outgoing `FOLLOWS` relationships, estimate the number of paths of length one to four starting from a single node. Propose three changes to a query or model that would reduce the work, and explain the effect of each.
6. **MERGE.** Explain what happens when `MERGE (:Person {id: 1})-[:WORKS_AT]->(:Company {name: 'Acme'})` is executed on the graph of Listing 15.1. Rewrite it so that it creates nothing that already exists, and add the constraints that make the rewrite safe under concurrent execution.
7. **Plan analysis.** A query `MATCH (p:Person) WHERE p.email = $e RETURN p` runs in 800 ms on a graph with 20 million people. Describe the plan you expect from `PROFILE`, the change you would make, and the plan you expect afterwards. How would the answer change for the predicate `p.email CONTAINS $fragment`?

8. **SQL translation.** Given tables `person(id, name)`, `company(id, name)`, and `works_at(person_id, company_id)`, write the SQL equivalent of Listing 15.2. Compare the two queries in length and readability, and discuss at what number of hops you expect the SQL version to become impractical.

9. **Modeling.** A company wants to model projects: people take part in projects with a role and a period, projects belong to departments, and questions ask who has worked with whom, in which role. Draw a graph model and justify each choice of node, relationship, and property. Decide whether participation should be a relationship or a node, and write the query that finds all colleagues of Ada from projects in which she was a team lead.

10. **GraphRAG design.** Design a retrieval query for a knowledge base stored as a graph of documents, chunks, entities, and users with group-based permissions. Specify the vector index and its candidate count, the permission check, the graph expansion, the ranking, and how you would evaluate whether the expansion improves answers. Discuss the risk of retrieved text containing instructions to the language model (Chapter 14).

References

- Angles, R. (2018). The Property Graph Database Model. In *Proceedings of the 12th Alberto Mendelzon International Workshop on Foundations of Data Management (AMW 2018)*.

- Angles, R., Arenas, M., Barceló, P., Hogan, A., Reutter, J., & Vrgoč, D. (2017). Foundations of Modern Query Languages for Graph Databases. *ACM Computing Surveys*, 50(5), Article 68.

- Deutsch, A., Francis, N., Green, A., et al. (2022). Graph Pattern Matching in GQL and SQL/PGQ. In *Proceedings of ACM SIGMOD '22*, 2246–2258.

- Edge, D., Trinh, H., Cheng, N., et al. (2024). From Local to Global: A Graph RAG Approach to Query-Focused Summarization. arXiv:2404.16130.

- Francis, N., Green, A., Guagliardo, P., et al. (2018). Cypher: An Evolving Query Language for Property Graphs. In *Proceedings of ACM SIGMOD '18*, 1433–1445.

- Hogan, A., Blomqvist, E., Cochez, M., et al. (2021). Knowledge Graphs. *ACM Computing Surveys*, 54(4), Article 71.

- ISO/IEC 39075:2024. *Information technology – Database languages – GQL*.

- ISO/IEC 9075-16:2023. *Information technology – Database languages SQL – Part 16: Property Graph Queries (SQL/PGQ)*.

- Neo4j Cypher Manual. https://neo4j.com/docs/cypher-manual/

- openCypher Project. https://opencypher.org/

- Robinson, I., Webber, J., & Eifrem, E. (2015). *Graph Databases* (2nd ed.). O'Reilly Media.

Chapter 16

Querying Distributed Data with CQL and Similar Languages

Learning Objectives

After reading this chapter, you will be able to:

- Explain why query languages of distributed databases resemble SQL in syntax but restrict what can be asked.
- Define tables, partition keys, and clustering columns in CQL, and write the statements that read and write them.
- Predict which queries a CQL table can answer efficiently and which are rejected or dangerous, and design one table per query.
- Use collections, user-defined types, counters, time-to-live, and lightweight transactions, and state their limits.
- Choose consistency levels per statement and reason about quorums, batches, and tombstones.
- Trace how a query travels from the client to the replicas, and diagnose slow queries.
- Compare CQL with PartiQL, the MongoDB aggregation pipeline, search-engine query languages, and distributed SQL.
- Address schema evolution, security, and operational concerns specific to distributed query languages.

16.1 Why Distributed Databases Restrict Their Queries

Chapter 15 showed a language whose strength is flexibility: a pattern can start anywhere and wander through the data. The databases of this chapter make the opposite choice. In a system that spreads data over tens or hundreds of nodes, a query is cheap only if it can be sent to the few nodes that hold the answer. A query that has to ask every node, collect partial answers, and merge them is slow, consumes capacity on the whole cluster, and gets slower as the cluster grows. The query languages of wide-column, key-value, and similar

stores are therefore designed to *make the efficient query easy and the inefficient query difficult or impossible*.

The best-known example is the *Cassandra Query Language* (CQL), used by Apache Cassandra and compatible systems such as ScyllaDB (Lakshman and Malik, 2010). CQL borrows the vocabulary of SQL (`CREATE TABLE`, `SELECT`, `WHERE`), which makes it familiar and also misleading, because the semantics differ (Siadati, 2021). There are no joins and no subqueries, filtering is limited to columns that the table's key structure makes searchable, and writes are *upserts*. The reason is architectural, not a lack of maturity: the language exposes the data model of Chapter 4, in which a table is a collection of partitions placed on nodes by a hash of the partition key, and each partition is stored sorted.

16.1.1 What Is Different from SQL and from Cypher

- **The key structure decides the queries.** The primary key is not only a constraint but the access path: it determines where data lives and in what order.
- **Queries are designed first, tables second.** A table is built to serve a query; a second query pattern usually needs a second table with the same data (Section 16.4).
- **Writes are upserts.** `INSERT` and `UPDATE` both create the row if it is missing and overwrite it if it exists, without reading first.
- **Consistency is a per-request choice.** The client states how many replicas must answer, trading latency and availability against freshness (Chapter 8).
- **Cost is predictable.** A query restricted to one partition costs the same on a cluster of ten nodes and of a thousand, which is the property the restrictions protect.

16.2 CQL Fundamentals

16.2.1 Keyspaces and Replication

A *keyspace* is the top-level namespace and the unit at which replication is configured, comparable to a database or schema in SQL. Listing 16.1 creates one replicated over two data centers.

Listing 16.1: A keyspace with replication per data center.

```
CREATE KEYSPACE shop
  WITH replication = {
    'class': 'NetworkTopologyStrategy',
    'dc1': 3,
    'dc2': 3
  };

USE shop;
```

The strategy `NetworkTopologyStrategy` places the stated number of replicas in each data center; `SimpleStrategy` ignores data center boundaries and is suitable only for single-site test setups.

16.2.2 The Primary Key: Partition Key and Clustering Columns

The primary key has two parts, and understanding them is the key to the language (Table 16.1).

Table 16.1: The parts of a CQL primary key.

Part	Role	Consequence for queries
Partition key (first element, one or more columns)	Hashed to a token; the token decides which replicas store the partition	Every query must name the full partition key, normally by equality, to reach the right nodes
Clustering columns (remaining elements)	Define the sort order of rows inside a partition	Rows can be selected by equality or range, in the declared order, and are returned already sorted

In the declaration `PRIMARY KEY ((customer_id, order_month), order_ts, order_id)`, the inner parentheses enclose a *composite partition key* of two columns, and `order_ts` and `order_id` are clustering columns. Listing 16.2 defines the running example of this chapter: orders of a customer, grouped by month and sorted by time with the newest first, plus a second table that finds an order by its identifier.

Listing 16.2: Tables for the running example: one table per query.

```
CREATE TYPE item (
  sku text,
  qty int,
  price decimal
);

-- Query 1: the recent orders of a customer
CREATE TABLE orders_by_customer (
  customer_id uuid,
  order_month text, -- for example '2026-10'
  order_ts timestamp,
  order_id uuid,
  status text,
  total decimal,
  PRIMARY KEY ((customer_id, order_month), order_ts, order_id)
) WITH CLUSTERING ORDER BY (order_ts DESC, order_id ASC);

-- Query 2: one order with its items
CREATE TABLE orders_by_id (
  order_id uuid PRIMARY KEY,
  customer_id uuid,
  order_ts timestamp,
  status text,
  items list<frozen<item>>
);
```

16.2.3 Data Types

CQL provides the usual scalar types (`int`, `bigint`, `double`, `decimal`, `text`, `boolean`), `uuid` and `timeuuid` for identifiers that can be generated on any node without coordination, `timestamp`, `date`, and `time`. Three kinds of composite values are available. *Collections* (`list`, `set`, `map`) hold a modest number of elements inside a row. *User-defined types* group named fields, as `item` does above. A type marked `frozen` is stored and replaced as one value, which is required for types nested inside collections. Collections are meant for small, bounded sets of values, such as tags or phone numbers; a collection that grows without limit becomes expensive to read and should be a table of its own.

16.2.4 Writing: Upserts, Expiry, and Deletes

Listing 16.3: Writing, expiring, and deleting data.

```
-- Insert (an upsert: overwrites an existing row with the same key)
INSERT INTO orders_by_customer
  (customer_id, order_month, order_ts, order_id, status, total)
VALUES (?, '2026-10', '2026-10-27␣09:15:00', ?, 'paid', 129.90);

-- Update (also an upsert; the row need not exist)
UPDATE orders_by_id SET status = 'shipped' WHERE order_id = ?;

-- Expire a row automatically after one day
INSERT INTO sessions (session_id, user_id) VALUES (?, ?)
  USING TTL 86400;

-- Delete one column value, then a whole row
DELETE status FROM orders_by_id WHERE order_id = ?;
DELETE FROM orders_by_id WHERE order_id = ?;
```

Because `INSERT` and `UPDATE` do not read the existing row, they are fast and safe to retry, but they also never fail with "already exists" unless a condition is added (Section 16.5). Every write carries a *timestamp*, normally assigned by the coordinator, and when replicas disagree the value with the highest timestamp wins (*last write wins*, Chapter 9). The clause `USING TTL` gives a value a lifetime in seconds, after which it disappears.

16.2.5 Deletes Create Tombstones

A delete cannot simply remove the data, because replicas that were unreachable must learn about it later; otherwise a stale replica would bring the deleted value back. A delete therefore writes a marker called a *tombstone*, which is kept for a grace period (by default ten days, the table option `gc_grace_seconds`) and removed only by compaction afterward. Expired TTL values leave tombstones in the same way. Reads must scan past tombstones, so tables with many deletes, or a pattern that uses a table as a queue, can become slow even when little live data remains. This is one of the most common operational problems of the model (Section **??**).

16.3 Reading: What a Query May Ask

The rules for `SELECT` follow directly from the physical layout. A query must restrict the whole partition key by equality (or by `IN`); it may then restrict clustering columns left to right, with equality on each column except the last one restricted, which may use a range. Results come back in clustering order, and `ORDER BY` may only reverse that order.

Listing 16.4: Queries accepted and rejected by the `orders_by_customer` table.

```
-- Accepted: full partition key; newest orders first (the table's order)
SELECT order_id, status, total
FROM orders_by_customer
WHERE customer_id = ? AND order_month = '2026-10'
LIMIT 20;

-- Accepted: range on the first clustering column
SELECT order_id, total
FROM orders_by_customer
WHERE customer_id = ? AND order_month = '2026-10'
  AND order_ts >= '2026-10-01' AND order_ts < '2026-10-15';

-- Rejected: part of the partition key is missing
SELECT * FROM orders_by_customer WHERE customer_id = ?;

-- Rejected: status is not part of the key
SELECT * FROM orders_by_customer
WHERE customer_id = ? AND order_month = '2026-10' AND status = 'paid';

-- Rejected: ORDER BY a column that is not a clustering column
SELECT * FROM orders_by_customer
WHERE customer_id = ? AND order_month = '2026-10'
ORDER BY total;
```

The rejected queries are not oversights of the language. Each would require the database to read a large or unknown number of partitions on an unknown number of nodes, and CQL refuses to hide that cost.

16.3.1 ALLOW FILTERING

The clause `ALLOW FILTERING` tells the database to run a query that it would otherwise reject, by reading data and discarding the rows that do not match. The query then works, but its cost grows with the data scanned, and on a large table it can read the whole cluster. It is acceptable on small tables, inside a single partition, or for one-off investigations, and a warning sign in application code. The cure is almost always to create a table or index that serves the query directly.

16.3.2 Aggregation and Other Limits

CQL has `COUNT`, `SUM`, `AVG`, `MIN`, and `MAX`, and `GROUP BY` on primary key columns, but no joins, subqueries, or arbitrary grouping. Aggregates over a single partition are efficient; aggregates over many partitions are executed by the coordinator over data from many nodes and do not scale. Analytics over a wide-column store is usually done elsewhere: in a processing framework that reads the data in token ranges, or in an analytical store fed from the database (Chapter 4).

16.4 Modeling for CQL

The relational method is to model the data first and write queries later. In CQL the order is reversed: list the queries, then design one table for each. The same data may appear in several tables, written together by the application or the database. This is *denormalization by design*, and it trades storage and write work for fast, predictable reads (Chapter 5).

16.4.1 Partition Design

A partition is the unit of placement, replication, and reading, so its size matters. Guidelines that are commonly cited, and that should be checked against current documentation and measured on real data, are to keep a partition well below about one hundred megabytes and a few hundred thousand rows or cells, and to make sure no key receives a disproportionate share of the traffic (a *hot partition*). Two techniques control size and balance:

- **Bucketing.** Add a time or hash component to the partition key, as `order_month` does above, so that no partition grows without bound.
- **Choosing a high-cardinality key.** A key such as a country code puts millions of rows in a few partitions; a key such as a user identifier spreads them.

Listing 16.5 shows a time-series table of sensor readings with a daily bucket, newest rows first, automatic expiry, and a compaction strategy suited to data written in time order.

Listing 16.5: A time-series table with bucketing, expiry, and time-window compaction.

```
CREATE TABLE readings_by_sensor_day (
  sensor_id text,
  day date,
  ts timestamp,
  value double,
  PRIMARY KEY ((sensor_id, day), ts)
) WITH CLUSTERING ORDER BY (ts DESC)
  AND default_time_to_live = 7776000 -- 90 days
  AND compaction = {
    'class': 'TimeWindowCompactionStrategy',
    'compaction_window_unit': 'DAYS',
    'compaction_window_size': 1 };
```

A query for the last hour of a sensor reads one partition (today's), and a query for the last week reads seven partitions through a client loop or an `IN` on a small, bounded set of days. Chapter 12 discusses such designs further.

16.4.2 Keeping Denormalized Tables Consistent

When the same order is stored in `orders_by_customer` and in `orders_by_id`, both copies must be written. A *logged batch* makes the database guarantee that all statements of the batch are eventually applied, even if the coordinator fails in the middle:

Listing 16.6: A logged batch writing two denormalized tables.

```
BEGIN BATCH
  INSERT INTO orders_by_id
    (order_id, customer_id, order_ts, status)
    VALUES (?, ?, '2026-10-27␣09:15:00', 'paid');
  INSERT INTO orders_by_customer
    (customer_id, order_month, order_ts, order_id, status, total)
    VALUES (?, '2026-10', '2026-10-27␣09:15:00', ?, 'paid', 129.90);
APPLY BATCH;
```

The meaning of a batch is often misunderstood. A logged batch gives *atomicity of eventual application*, not isolation: other readers may briefly see one table updated and not the other. It also costs more than the same statements issued separately, because the coordinator writes a batch log to other nodes first. A batch is therefore a tool for keeping denormalized copies in step, and not a way to speed up bulk loading; large batches spanning many partitions overload the coordinator, and the database rejects or warns about batches above a configured size.

16.4.3 Counters, Static Columns, and Indexes

- **Counters.** A table with a `counter` column supports increments (`SET views = views + 1`) without reading. Counter tables may contain only counters besides the key, and an increment is *not idempotent*: a retry after a timeout can count twice or not at all, so counters suit approximate statistics but not money.

- **Static columns.** A column declared `STATIC` has one value per partition, shared by all its rows, which is useful for partition-level attributes such as a customer's name stored beside the orders.

- **Secondary indexes.** An index on a non-key column lets a query filter on it. Classic secondary indexes are local to each node, so a query without the partition key must ask many nodes, and they perform poorly on columns of very high or very low cardinality. Newer storage-attached indexes (SAI, in recent Cassandra versions) improve this considerably and support ranges, but a query that cannot name its partition remains a scatter query.

- **Materialized views.** A view maintained by the database as another table under a different key is convenient, but in Apache Cassandra it has long carried an experimental status and known consistency problems; many teams maintain denormalized tables in application code or with batches instead. Check the status in the version in use.

Listing 16.7: A counter table, a static column, and a storage-attached index (Cassandra 5 syntax).

```
CREATE TABLE page_views (
  page text PRIMARY KEY,
  views counter
);
UPDATE page_views SET views = views + 1 WHERE page = ?;

CREATE TABLE notes_by_customer (
  customer_id uuid,
  note_ts timestamp,
  customer text STATIC, -- one value per customer partition
  note text,
  PRIMARY KEY (customer_id, note_ts)
);

CREATE INDEX orders_status_idx ON orders_by_id (status) USING 'sai';
```

16.5 Consistency, Quorums, and Lightweight Transactions

Every read and write carries a *consistency level*, the number of replicas that must acknowledge it before the coordinator answers. Table 16.2 lists the common levels. With replication factor RF, a *quorum* is $\lfloor RF/2 \rfloor + 1$ replicas; for $RF = 3$ it is two, for $RF = 5$ three.

The rule from Chapter 8 applies: a read sees the latest acknowledged write if the replicas read (R) and written (W) overlap, that is, if $R + W > RF$. With $RF = 3$, writing and reading at `QUORUM` gives $2 + 2 > 3$ and a consistent read, while writing and reading at `ONE` gives $1 + 1 < 3$ and allows stale reads. Replicas that miss a write are repaired later by read repair, hinted handoff, and scheduled anti-entropy repair; repair must be run regularly, in particular within the tombstone grace period, or deleted data can come back.

16.5.1 Choosing a Level

The level is set per statement or per session, in the driver or in the shell. A common design uses `LOCAL_QUORUM` for both reads and writes in multi-site clusters, `ONE` for data that tolerates staleness (for example caches and telemetry), and the strongest level only for the few operations that need it. Lower levels keep the service available when replicas fail; higher levels do not.

Table 16.2: Common consistency levels in CQL (replication factor RF).

Level	Replicas that must respond	Typical use
`ONE`	Any single replica	Lowest latency; stale reads possible
`QUORUM`	A majority of all replicas in all data centers	Strong consistency within a cluster, at the price of cross-site latency
`LOCAL_QUORUM`	A majority of replicas in the coordinator's data center	The usual choice for multi-site clusters
`EACH_QUORUM`	A majority in every data center (writes only)	Rarely used; high latency
`ALL`	Every replica	Highest consistency; unavailable if any replica is down
`SERIAL`, `LOCAL_SERIAL`	Consensus among replicas	Reads that see the outcome of lightweight transactions

16.5.2 Lightweight Transactions

Plain writes are unconditional, which makes "insert only if the username is free" impossible with ordinary statements. CQL provides *lightweight transactions* (LWT) through the `IF` clause. They use a consensus protocol among the replicas of one partition (Paxos in classic Cassandra versions, with a newer protocol in recent ones) and return whether the condition held.

Listing 16.8: Conditional writes with lightweight transactions.

```
-- Succeeds only if no such user exists; result row has [applied] = true/false
INSERT INTO users_by_name (username, user_id) VALUES (?, ?)
  IF NOT EXISTS;

-- Compare-and-set on a column
UPDATE orders_by_id SET status = 'shipped'
WHERE order_id = ? IF status = 'paid';
```

An LWT costs several network round trips instead of one and serializes competing operations on the same partition, so it should guard the few steps where uniqueness or a state transition matters, not every write. A condition spans one partition only; CQL has no multi-partition transactions in the classic model, which is the main limitation of this family compared with distributed SQL (Section 16.7).

16.6 How a Query Is Executed

Figure 16.1 shows the path of a single-partition query. The client connects to any node, which becomes the *coordinator* for that request; a token-aware driver picks a node that already holds a replica of the partition, saving one hop. The coordinator hashes the partition key to a token, finds the replicas on the ring, sends the request to as many of them as the consistency level requires (and often a digest request to the others), compares the answers, and returns the newest data, repairing replicas that were behind.

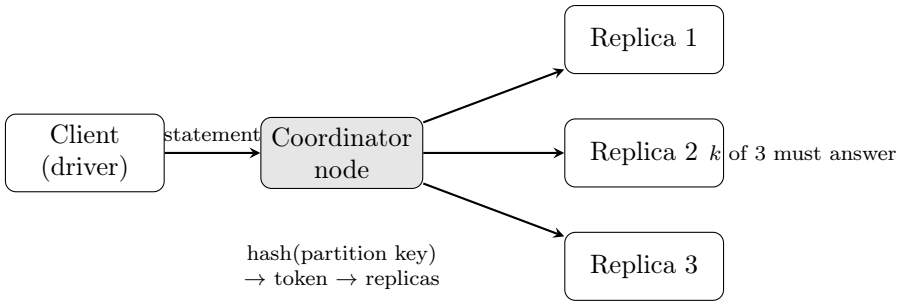


Figure 16.1: Execution of a single-partition query. The consistency level k determines how many replicas must respond before the coordinator answers.

On each replica the read combines the in-memory write buffer (memtable) with the immutable files on disk (SSTables), using Bloom filters and indexes to skip files that cannot contain the key, as in the LSM designs of Chapter 4. A query that restricts only part of the partition key, or none, cannot be routed this way: the coordinator must contact many nodes, and the cost grows with the cluster.

16.6.1 Paging

Large results are returned in pages. The driver fetches a fixed number of rows (the *fetch size*) and a *paging state* token that tells the server where to continue, so that no node holds the whole result. The paging state is opaque and should not be built or modified by applications. Deep paging through a very large partition is still expensive, and the cleaner solution is to design partitions so that the interesting rows come first.

16.6.2 Prepared Statements and Drivers

Applications should send *prepared statements*: the query text with placeholders (`?`) is parsed once and executed many times with bound values, which saves parsing, enables token-aware routing, and prevents injection. Listing 16.9 shows the pattern with the Python driver.

Listing 16.9: A prepared statement with a per-statement consistency level (Python driver).

```
from cassandra.cluster import Cluster
from cassandra import ConsistencyLevel

session = Cluster(["10.0.0.1", "10.0.0.2"]).connect("shop")
```

```
stmt = session.prepare(
    "SELECT order_id, status, total FROM orders_by_customer "
    "WHERE customer_id = ? AND order_month = ? LIMIT 20")
stmt.consistency_level = ConsistencyLevel.LOCAL_QUORUM

rows = session.execute(stmt, (customer_id, "2026-10"))
for row in rows:
    print(row.order_id, row.status, row.total)
```

16.6.3 Diagnosing Slow Queries

The shell command `TRACING ON` makes the coordinator report the steps of each query: which replicas were contacted, how many SSTables and tombstones were read, and the time of each step. The cluster's metrics add per-table latency percentiles, partition-size histograms, and tombstone counts. Table 16.3 lists the patterns that most often cause trouble.

Table 16.3: Frequent CQL anti-patterns and their remedies.

Anti-pattern	Symptom	Remedy
`ALLOW FILTERING` on large tables	Latency grows with table size; timeouts	A table or index that serves the query
Unbounded partitions	Slow reads, memory pressure, uneven nodes	Add a time or hash bucket to the partition key
Large `IN` lists over many partitions	Coordinator overload; high tail latency	Parallel single-partition queries from the client
Queue-like tables with frequent deletes	Reads scan thousands of tombstones	Time-bucketed tables with TTL; use a queueing system instead
Large multi-partition batches	Coordinator timeouts; batch size warnings	Individual statements; batches only for denormalized copies
Hot partition (one key with most traffic)	One replica set saturated while others idle	Spread the key (salting) or cache the hot item
`SELECT *` on wide rows	Large responses and wasted network	Name only the columns needed

16.7 Similar Languages in Other Distributed Stores

CQL is one answer to the question of how to query partitioned data; other systems give different answers (Table 16.4). The common thread is that each language mirrors the storage model of its system.

Table 16.4: Query languages of distributed data stores compared.

Language	Systems	Character	Main limit
CQL	Cassandra, ScyllaDB	SQL-like; key-driven; tunable consistency	No joins; one table per query
PartiQL	Amazon DynamoDB (and others)	SQL-compatible syntax over key-value and document items	Queries without a key condition become scans
MongoDB query language and aggregation pipeline	MongoDB	JSON queries; multi-stage pipelines with grouping, joins (`$lookup`)	Cross-shard queries and joins cost more than single-shard ones
Search-engine query languages	Elasticsearch, OpenSearch (Query DSL, ES|QL, SQL)	Relevance ranking, filtering, aggregations	Not a system of record; eventual freshness
Distributed SQL	Spanner, CockroachDB, YugabyteDB	Full SQL with joins and distributed transactions	Cross-region latency; higher cost per write

16.7.1 PartiQL on DynamoDB

PartiQL gives DynamoDB a SQL-compatible syntax. The statements are translated into the key-value operations of the service, so the same rule applies: a statement that names the partition key reads one partition, and a statement that does not is a *scan* of the whole table, with the cost that implies.

Listing 16.10: PartiQL statements for DynamoDB.

```
-- Key condition on the partition key: efficient
SELECT order_id, total FROM "Orders"
WHERE customer_id = 'c42' AND order_ts > '2026-10-01';

-- No key condition: a full table scan
SELECT * FROM "Orders" WHERE status = 'paid';

INSERT INTO "Orders" VALUE {'customer_id': 'c42', 'order_ts': '2026-10-27T09:15:00Z
    ', 'total': 129.9};
```

16.7.2 The MongoDB Aggregation Pipeline

MongoDB expresses complex queries as a pipeline of stages, each transforming the stream of documents. Filtering by the shard key early lets the router send the query to one shard; stages that regroup data across shards need a merge step.

Listing 16.11: An aggregation pipeline: monthly revenue of paid orders for one customer.

```
db.orders.aggregate([
  { "$match": { "customerId": 42, "status": "paid" } },
  { "$group": { "_id": "$month", "revenue": { "$sum": "$total" } } },
  { "$sort": { "_id": 1 } }
])
```

16.7.3 Distributed SQL

Systems such as Spanner and CockroachDB keep the full SQL language, including joins and multi-row transactions, and pay for it with consensus on each write and with transaction latency that grows with the distance between replicas (Corbett et al., 2013; Taft et al., 2020). They are the natural choice when the application needs relational features at a scale beyond a single server. The designs of this chapter, in contrast, accept a restricted language to obtain the lowest and most predictable latency at the largest scale.

16.7.4 Search-Engine Languages

The query languages of search engines (Chapter 13) are declarative and rich in ranking and aggregation, and are often combined with a wide-column store or other system of record that holds the authoritative data. Scatter–gather over all shards is the normal execution model there, which is acceptable because the data is organized for it and the results are ranked and limited.

16.8 Operating CQL Clusters

16.8.1 Schema Evolution

Adding a column is cheap (`ALTER TABLE ... ADD`), because existing rows simply have no value for it. The primary key, however, cannot be changed: a new access pattern means a new table, and migration means copying the data, usually with a dual-write period and a verification step, as for other derived stores (Chapter 13). Renaming or retyping columns is restricted. Because the key structure is the hardest part to change, design it with care and test it with realistic data volumes before production.

16.8.2 Repair, Compaction, and Capacity

Regular *repair* keeps replicas consistent and must complete within the tombstone grace period. Compaction merges SSTables in the background, and its strategy (size-tiered, leveled, or time-window) should match the workload: time-window for expiring time series, leveled for read-heavy tables with updates. Capacity planning looks at disk per node (with headroom for compaction, commonly cited as keeping utilization well below half to two thirds depending on the strategy), partition-size distribution, and the number of replicas.

16.8.3 Security

- **Authentication and roles.** Enable authentication and use roles with the least permissions each application needs (`GRANT SELECT ON KEYSPACE shop TO app_reader`).
- **Encryption.** Encrypt client-to-node and node-to-node traffic, and encrypt data at rest at the storage or volume level.
- **Injection.** CQL built by string concatenation is as vulnerable as SQL; use prepared statements with bound parameters.
- **Multi-tenancy.** Isolation is by keyspace and role, or by a tenant column in the partition key enforced by the application; the database does not filter rows per user.

16.8.4 Monitoring

Monitor read and write latency percentiles per table, timeouts and unavailable exceptions (which signal that consistency levels cannot be met), pending compactions, partition-size and tombstone histograms, hinted-handoff backlog, repair status, and disk usage. Alert on tombstone warnings in the logs: they usually point to a modeling problem, not a capacity problem.

16.9 Choosing an Approach

Table 16.5: Choosing a query model for distributed data.

Situation	Suggested approach	Reasoning
Known, stable access patterns; very high write rates; multi-site availability	Wide-column store with CQL	Predictable latency; tunable consistency; scales out
Single-key lookups at large scale on a managed service	Key-value or document store with its key-based API or PartiQL	Minimal operations; pay per request
Flexible documents; varied queries on one dataset	Document database with its query language	Rich filtering and aggregation; secondary indexes
Relational features (joins, transactions) at large scale	Distributed SQL	Familiar model; strong consistency across rows
Full-text and relevance-ranked retrieval	Search engine, fed from the system of record	Ranking and faceting that other stores lack
Ad hoc analytics over operational data	Export to an analytical engine	Keeps scans away from the serving cluster

A sound process is: (1) list the queries, with their frequency, latency target, and consistency need; (2) design one table per query, choosing partition keys that spread load

and bound size; (3) estimate partition sizes and write rates with real numbers; (4) choose consistency levels per operation, and use lightweight transactions only where a condition is needed; (5) load realistic data and use tracing and metrics to check that every query touches one partition; (6) plan repair, compaction, TTL, and tombstone behavior from the start; and (7) revisit the design when a new query arrives, as a new table, not as a filter.

Summary

The query languages of distributed databases expose the physical layout of the data and restrict queries to those that can be answered efficiently. In CQL a table's primary key consists of a partition key, which is hashed to place the partition on replica nodes, and clustering columns, which sort the rows within it. A query must name the full partition key and may restrict clustering columns in order; filtering on other columns, joins, and subqueries are not supported, and `ALLOW FILTERING` overrides the protection at the price of scanning. Tables are therefore designed from the queries, one per query, with the same data denormalized across them, and partitions are bounded by bucketing. Writes are upserts with timestamps and last-write-wins resolution; deletes and expired values leave tombstones that slow reads if they accumulate. Each statement carries a consistency level, and quorum reads and writes overlap when $R + W > RF$. Lightweight transactions provide conditional updates within one partition at higher cost, and logged batches keep denormalized tables in step without isolation. Execution goes through a coordinator that routes the statement to replicas by token, and prepared statements, paging, and tracing are the practical tools for correctness and diagnosis. Other systems offer similar languages shaped by their storage models: PartiQL over DynamoDB, the MongoDB pipeline, search-engine query languages, and distributed SQL, which restores joins and transactions at a cost in latency. Operations center on repair, compaction strategy, schema evolution through new tables, security, and monitoring of partition sizes and tombstones.

Review Questions

1. Why do the query languages of distributed databases restrict the queries they accept? How does CQL differ from SQL despite the similar syntax?

2. Explain the roles of the partition key and the clustering columns in a CQL primary key.

3. What does `PRIMARY KEY ((a, b), c, d)` mean? Which `WHERE` clauses are allowed on such a table?

4. Why are `INSERT` and `UPDATE` both upserts, and what are the consequences for retries and for uniqueness?

5. What is a tombstone? Why are tombstones needed, and why can they harm read performance?

6. What does `ALLOW FILTERING` do, and why is it dangerous on large tables?

7. Explain why a CQL schema is designed starting from the queries. What does this imply for denormalization?

8. Name two techniques to keep partitions bounded, and explain when each is appropriate.

9. What guarantees does a logged batch give, and what guarantees does it not give?

10. For $RF = 5$, what is a quorum? Under which combinations of read and write levels does a read see the latest acknowledged write?

11. Why is `LOCAL_QUORUM` the usual choice in multi-data-center clusters?

12. What does a lightweight transaction guarantee, what does it cost, and what is its scope?

13. Describe the path of a single-partition query from the client to the replicas. What does a token-aware driver improve?

14. Why are counters not suitable for balances, and what makes an operation idempotent?

15. Compare CQL with PartiQL and with distributed SQL in terms of query power, cost, and consistency.

16. Why can the primary key of a table not be changed, and how is a new access pattern introduced?

Exercises

1. **Reading keys.** For a table with `PRIMARY KEY ((region, day), sensor, ts)`, state which of the following queries are accepted and which are rejected, with reasons: (a) `region = 'eu' AND day = '2026-10-27'`, (b) `region = 'eu' AND sensor = 's1'`, (c) `region = 'eu' AND day = '2026-10-27' AND sensor = 's1' AND ts > '2026-10-27 08:00'`, (d) `region = 'eu' AND day = '2026-10-27' AND ts > '2026-10-27 08:00'`.

2. **Table design.** Design CQL tables for a messaging application with these queries: (a) the latest 50 messages of a conversation, (b) all conversations of a user ordered by the time of the last message, (c) one message by its identifier. Give the primary key and clustering order of each table, and explain how a new message is written to all of them.

3. **Partition size.** A sensor writes one reading per second; a row occupies about 40 bytes. Estimate the size of a partition if the key is the sensor alone, after one day, and after one year. Propose a bucketing scheme that keeps a partition below 10 MB, and state how a query for the last 48 hours would be executed.

4. **Quorums.** With $RF = 3$ in each of two data centers, state for each of the levels `ONE`, `LOCAL_QUORUM`, `QUORUM`, and `EACH_QUORUM` how many replicas must respond to a write, and which node failures each level tolerates. Which pairs of write and read levels guarantee that a read sees the latest write?

5. **Tombstones.** A table is used as a work queue: tasks are inserted and deleted after processing, and a query reads the oldest unprocessed tasks. Explain why reads slow down over time, and redesign the data model, or choose another system, to avoid the problem.

6. **Batches.** Rewrite the two inserts of Listing 16.6 so that the application reads the order back after a failure and repairs a missing copy, without a batch. Compare the two approaches for correctness, latency, and load on the coordinator.

7. **Lightweight transactions.** Design the statements for a seat reservation system in which a seat may be held by one customer at a time and a hold expires after five minutes. Use conditions and TTL, state the consistency levels, and explain what happens when two customers try to hold the same seat at the same moment.

8. **Cassandra and PartiQL.** Translate the query of Listing 16.4 (first query) to PartiQL for DynamoDB, choosing the key schema. Explain which queries of the running example would become scans, and how a secondary index changes the cost.

9. **Choosing a system.** A retailer needs (i) a product catalog with flexible attributes and faceted search, (ii) an order ledger with transactions across several rows, and (iii) a clickstream of one billion events per day, queried by user. Select a query model for each, and justify the choices using the table in this chapter.

10. **Migration.** The team must change the partition key of `orders_by_customer` from `(customer_id, order_month)` to `(customer_id, order_year)`. Write a migration plan with the steps, the consistency of data during the transition, the validation, and the rollback path, and estimate the time to copy 2 billion rows at 50 000 rows per second.

References

- Amazon DynamoDB Developer Guide: PartiQL. https://docs.aws.amazon.com/amazondynamodb/
- Apache Cassandra Documentation: Cassandra Query Language (CQL). https://cassandra.apache.org/doc/latest/
- Carpenter, J., & Hewitt, E. (2020). *Cassandra: The Definitive Guide* (3rd ed.). O'Reilly Media.
- Chang, F., Dean, J., Ghemawat, S., et al. (2006). Bigtable: A Distributed Storage System for Structured Data. In *Proceedings of OSDI '06*, 205–218.
- Corbett, J. C., Dean, J., Epstein, M., et al. (2013). Spanner: Google's Globally Distributed Database. *ACM Transactions on Computer Systems*, 31(3), Article 8.
- DeCandia, G., Hastorun, D., Jampani, M., et al. (2007). Dynamo: Amazon's Highly Available Key-value Store. In *Proceedings of ACM SOSP '07*, 205–220.

- Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.
- Lakshman, A., & Malik, P. (2010). Cassandra: A Decentralized Structured Storage System. *ACM SIGOPS Operating Systems Review*, 44(2), 35–40.
- MongoDB Documentation: Aggregation Pipeline. https://www.mongodb.com/docs/manual/
- Siadati, S. (2021). CQL similar to SQL, not the same. *Level Up Coding (Medium)*. https://levelup.gitconnected.com/cql-is-similar-to-sql-not-the-same-8202ea582d26
- Taft, R., Sharif, I., Matei, A., et al. (2020). CockroachDB: The Resilient Geo-Distributed SQL Database. In *Proceedings of ACM SIGMOD '20*, 1493–1509.

Chapter 17

JSON and Document Query Languages

Learning Objectives

After reading this chapter, you will be able to:

- Explain how querying nested, variable documents differs from querying flat tables, and name the main families of JSON query languages.
- Write MongoDB-style queries with dot notation, array conditions, `$elemMatch`, projection, sorting, and cursor-based paging.
- Update documents atomically with update operators, upserts, and array filters, and state the scope of atomicity.
- Build aggregation pipelines with `$match`, `$unwind`, `$group`, and `$lookup`, and order the stages for efficiency.
- Design compound, multikey, partial, and TTL indexes, apply the equality–sort–range guideline, and read `explain` output.
- Query JSON in a relational database with SQL/JSON operators, JSONPath, and `JSON_TABLE`, and decide between a document store and JSON columns.
- Compare JSONPath, jq, JMESPath, and SQL++, and handle missing, null, and typed values correctly.
- Address schema validation, query injection, and operational risks specific to document queries.

17.1 Querying Nested Data

Chapter 2 introduced document databases, in which a record is a self-contained, nested structure, usually written in JSON (Bray, 2017). Chapter 16 showed that a query language can be shaped by a storage layout; in document systems the shaping force is the *structure of the data itself*. A document may contain objects inside objects, arrays of objects, optional fields, and values whose type differs from one document to the next. A query language for such data must therefore answer questions that have no counterpart in a flat table:

- How does a query *navigate* into a nested object (`shipTo.city`) or into the elements of an array?

- What does "*the array contains a matching element*" mean when two conditions concern the same element, and when they concern different ones?
- What is the difference between a field that is *absent*, one that is *null*, and one that is *empty*?
- How can nested data be *unnested* to rows for grouping and joining, and *rebuilt* into documents for the result?

17.1.1 Three Families of Languages

Languages for JSON data fall into three families, summarized in Table 17.1.

Table 17.1: Families of languages for querying JSON data.

Family	Examples	Idea	Typical home
Document-native query languages	MongoDB query language and aggregation pipeline	Queries and pipelines are themselves JSON documents	Document databases
SQL extended for documents	SQL/JSON in PostgreSQL, MySQL, SQL Server, Oracle; SQL++ in Couchbase and AsterixDB; Cosmos DB SQL	Familiar `SELECT` with path access and unnesting	Relational and multi-model databases
Path and transformation languages	JSONPath, jq, JMESPath	Select or transform parts of one document	Libraries, command line, APIs, embedded in the other two

This chapter studies the three in turn, using MongoDB as the main example of the first family, because its language is the most widely used and its ideas are repeated in many other systems.

17.2 A Running Example

The examples use a collection `orders` of order documents. Each order embeds its shipping address and its line items, as recommended in Chapter 5 for data that is read together. Listing 17.1 inserts one.

Listing 17.1: An order document with nested object and arrays (mongosh).

```
db.orders.insertOne({
  _id: "o-1001",
  customerId: 42,
  status: "paid",
  orderDate: ISODate("2026-10-27T09:15:00Z"),
  shipTo: { city: "Berlin", country: "DE" },
```

```
  items: [
    { sku: "A-17", qty: 2, price: 19.95 },
    { sku: "B-02", qty: 1, price: 89.99 }
  ],
  tags: ["gift", "express"],
  total: 129.89
})
```

17.2.1 JSON and Its Binary Cousins

JSON has only a few types: string, number, boolean, null, array, and object. Databases store a richer binary form (BSON in MongoDB, `jsonb` in PostgreSQL) that adds typed dates, exact integers and decimals, binary data, and identifiers, and that can be traversed without parsing text. The difference is visible in queries: comparing a date with a string, or a number stored as text with a number, silently fails to match. Queries should use the same types as the stored data, and the types should be enforced when it matters (Section 17.10).

17.3 Finding Documents

A MongoDB query is a *filter document* that describes the shape of the documents to be returned. Fields listed together are combined with AND; operators, written with a leading dollar sign, express other conditions.

Listing 17.2: Filters, projection, sorting, and limits.

```
// Equality on two fields, only some fields returned, newest first
db.orders.find(
  { customerId: 42, status: "paid" },
  { status: 1, total: 1, orderDate: 1, _id: 0 }
).sort({ orderDate: -1 }).limit(10)

// Dot notation reaches into the nested object; comparison operators
db.orders.find({ "shipTo.country": "DE", total: { $gte: 100 } })

// Set membership, disjunction, negation
db.orders.find({ status: { $in: ["paid", "shipped"] } })
db.orders.find({ $or: [ { total: { $gt: 500 } }, { tags: "express" } ] })
db.orders.find({ status: { $ne: "cancelled" } })
```

The second argument of `find` is a *projection* that selects the fields to return; asking only for needed fields reduces network traffic and can make a query *covered* by an index (Section 17.6). Table 17.2 lists the most common query operators.

17.3.1 Arrays: The Subtle Part

A condition on a field that holds an array matches if *any* element satisfies it, so `{tags: "gift"}` finds orders whose tag list contains `"gift"`. The subtlety appears when two

Table 17.2: Common query operators of the MongoDB query language.

Operator	Meaning
`$eq`, `$ne`, `$gt`, `$gte`, `$lt`, `$lte`	Comparison; a plain `field: value` means `$eq`
`$in`, `$nin`	The value is (is not) in a list
`$and`, `$or`, `$nor`, `$not`	Boolean combination of conditions
`$exists`, `$type`	The field is present; the field has a given type
`$regex`	Pattern match on strings; an unanchored pattern cannot use an index efficiently
`$all`, `$size`, `$elemMatch`	Array conditions: contains all of, has a given length, has one element satisfying all conditions
`$expr`	Use aggregation expressions in a filter, for example to compare two fields of the same document

conditions concern an array of objects. Written separately, they may be satisfied by *different* elements; `$elemMatch` requires one element to satisfy all of them.

Listing 17.3: Conditions on array elements: separate versus the same element.

```
// Matches if SOME item has sku A-17 and SOME (possibly other) item has qty >= 2
db.orders.find({ "items.sku": "A-17", "items.qty": { $gte: 2 } })

// Matches only if ONE item has sku A-17 AND qty >= 2
db.orders.find({ items: { $elemMatch: { sku: "A-17", qty: { $gte: 2 } } } })
```

On the order of Listing 17.1, the first query matches because item `A-17` exists and the item `A-17` also has quantity two; if the quantities were reversed (`A-17` with one unit and `B-02` with two), the first query would still match, while the second would not. Confusing the two forms is one of the most common sources of wrong results in document queries.

17.3.2 Missing, Null, and Empty

The condition `{discount: null}` matches documents where `discount` is null *or absent*. To separate the cases, use `{discount: {$exists: false}}` for absent fields and `{discount: {$type: "null"}}` for explicit nulls. An empty array is neither absent nor null and matches `{items: {$size: 0}}`. Applications should choose one convention for "no value" and write it consistently, because queries and indexes must handle every variant that exists in the data.

17.3.3 Paging

Pages are produced with `skip` and `limit`, but `skip(n)` must still walk past the first n results, so deep pages become slow. The scalable alternative is *range-based paging*: sort by an indexed, unique key and ask for the documents after the last one seen.

Listing 17.4: Range-based paging with a tie-breaking key.

```
// First page
db.orders.find({ customerId: 42 }).sort({ orderDate: -1, _id: 1 }).limit(20)

// Next page: continue after the last (orderDate, _id) pair returned
db.orders.find({
  customerId: 42,
  $or: [
    { orderDate: { $lt: lastDate } },
    { orderDate: lastDate, _id: { $gt: lastId } }
  ]
}).sort({ orderDate: -1, _id: 1 }).limit(20)
```

17.4 Updating Documents

Updates describe a *change* and not a replacement of the whole document. Because the database applies the change itself, it avoids the read–modify–write cycle on the client and the lost updates that this cycle allows (Chapter 8). Table 17.3 lists the main update operators.

Table 17.3: Common update operators.

Operator	Effect
`$set`, `$unset`	Set a field (creating it if needed), or remove it
`$inc`, `$min`, `$max`	Increment; keep the smaller or larger of the old and new values
`$push`, `$addToSet`	Append to an array; append only if absent
`$pull`, `$pop`	Remove matching elements; remove the first or last element
`$rename`, `$currentDate`	Rename a field; set a field to the current time

Listing 17.5: Atomic updates, a conditional update, array filters, and an upsert.

```
// Change status and append to a history array in one atomic operation
db.orders.updateOne(
  { _id: "o-1001" },
  { $set: { status: "shipped" },
    $push: { history: { status: "shipped", at: new Date() } } })

// Compare-and-set: only moves from "paid" to "shipped"; check modifiedCount
db.orders.updateOne({ _id: "o-1001", status: "paid" },
                    { $set: { status: "shipped" } })

// Update only the array elements that match a condition
```

```
db.orders.updateOne(
  { _id: "o-1001" },
  { $set: { "items.$[i].qty": 3 } },
  { arrayFilters: [ { "i.sku": "A-17" } ] })

// Upsert: create the document if no match exists
db.counters.updateOne({ _id: "orders-2026-10" },
                      { $inc: { n: 1 } }, { upsert: true })
```

17.4.1 Scope of Atomicity

An update to a *single document* is atomic, however many fields and nested elements it changes, which is the reason to place together the data that must change together. Changes that span several documents are not atomic by default. Modern document databases offer multi-document transactions, but they cost more than single-document operations and are best reserved for the cases that need them; with a good model, most updates touch one document (Chapter 5).

17.4.2 Unbounded Arrays and Document Size

Appending to an array without limit makes the document grow until it reaches the maximum document size (16 MiB in MongoDB) and makes every read and write of it slower long before that. A history that grows forever, such as the events of an account, belongs in a separate collection with a reference back, or in a bucketed structure with a bounded number of entries per document. The cap is a protection against modeling mistakes and not a limit that should be approached.

17.5 The Aggregation Pipeline

When a query must group, reshape, or join documents, MongoDB provides the *aggregation pipeline*: an array of stages, each of which receives the documents produced by the previous stage and emits new ones (Figure 17.1). The model is the same as the operator trees of relational and graph engines, written out explicitly.

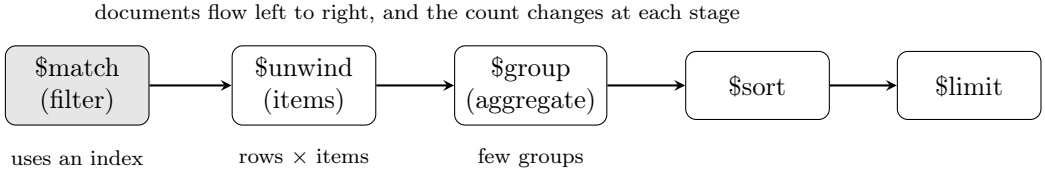


Figure 17.1: An aggregation pipeline. Early stages should reduce the data as much as possible; only the first stages can use indexes.

Listing 17.6: Units and revenue per month and product for paid orders shipped to Germany.

Table 17.4: Frequently used pipeline stages.

Stage	Purpose
`$match`	Filter documents, with the same syntax as `find`
`$project`, `$set` (`$addFields`)	Select, rename, and compute fields
`$unwind`	Produce one document per array element
`$group`	Group by an expression and compute accumulators (`$sum`, `$avg`, `$min`, `$max`, `$push`, `$first`)
`$sort`, `$limit`, `$skip`	Order and cut the stream
`$lookup`	Left outer join with another collection
`$facet`, `$bucket`	Several sub-pipelines on the same input; histogram buckets
`$merge`, `$out`	Write the result to a collection, for example a precomputed summary

```
db.orders.aggregate([
  { $match: { status: "paid", "shipTo.country": "DE",
              orderDate: { $gte: ISODate("2026-01-01") } } },
  { $unwind: "$items" },
  { $group: {
      _id: { month: { $dateToString: { format: "%Y-%m", date: "$orderDate" } },
             sku: "$items.sku" },
      units: { $sum: "$items.qty" },
      revenue: { $sum: { $multiply: ["$items.qty", "$items.price"] } } } },
  { $sort: { "_id.month": 1, revenue: -1 } }
])
```

17.5.1 Joining with $lookup

`$lookup` fetches matching documents from another collection and attaches them as an array. It is a left outer join, executed per input document, and it is most efficient when the joined field is indexed in the other collection.

Listing 17.7: A join with $lookup, followed by unwinding and projection.

```
db.orders.aggregate([
  { $match: { status: "paid" } },
  { $lookup: { from: "customers", localField: "customerId",
               foreignField: "_id", as: "customer" } },
  { $unwind: "$customer" },
  { $project: { total: 1, "customer.name": 1, "customer.tier": 1 } }
])
```

A document database is not a relational system, and a model that needs a `$lookup` in every query has probably embedded too little. Joins are acceptable for occasional reporting; if they appear on the hot path, reconsider the model (Chapter 5). In a sharded cluster, a join between collections with different shard keys requires data movement between nodes and is correspondingly expensive.

17.5.2 Writing Efficient Pipelines

- **Filter first.** A leading `$match` (and `$sort`) can use an index and shrinks the data for every later stage; the optimizer moves some filters forward, but the author should not depend on it.
- **Unwind late and narrowly.** `$unwind` multiplies the number of documents by the array length. Filter the arrays or the parent documents before unwinding.
- **Project early.** Dropping unused fields reduces memory and network use.
- **Respect memory limits.** Blocking stages such as `$group` and `$sort` have memory limits per stage and can spill to disk, which is slower; the limits and defaults depend on the version.
- **Pre-compute what is asked often.** Writing summaries to a collection with `$merge` on a schedule turns an expensive report into a cheap lookup.

17.6 Indexes and Query Plans

Without an index, a query reads every document of the collection (a *collection scan*). Document indexes are B-tree structures, as in relational systems, with several features specific to nested data (Table 17.5).

17.6.1 Compound Indexes: Equality, Sort, Range

In a compound index the order of fields matters. A well-known guideline is to put fields in the order *equality, sort, range* (ESR): fields tested for equality first, then the fields on which the query sorts, and finally fields tested with a range. The index then returns rows already in the requested order and narrows the scan with the equality fields. For the query of Listing 17.2 (equality on `customerId` and `status`, sort on `orderDate`), an effective index is the following:

Listing 17.8: A compound index following the ESR guideline, and a partial and a TTL index.

```
db.orders.createIndex({ customerId: 1, status: 1, orderDate: -1 })

// Only open orders are indexed: smaller and faster to maintain
db.orders.createIndex({ orderDate: 1 },
                      { partialFilterExpression: { status: "open" } })

// Remove sessions one hour after their expiry date
db.sessions.createIndex({ expiresAt: 1 }, { expireAfterSeconds: 3600 })
```

Table 17.5: Index types for document collections.

Index	Behavior	Use
Single-field and compound	Sorted by one or several fields, including nested paths such as `shipTo.country`	Equality, range, and sort queries
Multikey	Created automatically when an indexed field holds an array; one entry per element	Queries on array contents; a compound index may contain at most one array field
Partial	Indexes only documents that satisfy a filter	Smaller indexes, for example only open orders
TTL	Deletes documents a fixed time after a date field	Sessions, logs, temporary data
Text and wildcard	Tokenized text search; indexes all fields or paths under a pattern	Simple search; irregular documents
Geospatial	Two-dimensional queries on coordinates	"Near this point" searches

17.6.2 Reading a Query Plan

The method `explain("executionStats")` returns the plan and its measured work. The key figures are the plan stages (`COLLSCAN` reads the whole collection, `IXSCAN` uses an index, `FETCH` reads documents), the number of index keys examined, the number of documents examined, and the number of documents returned. A healthy query examines about as many keys and documents as it returns. A query that examines a million documents to return ten needs a better index. A query is *covered* when the index contains every field it filters on and returns, in which case no document is fetched at all.

Listing 17.9: Inspecting a query plan.

```
db.orders.find({ customerId: 42, status: "paid" })
        .sort({ orderDate: -1 })
        .explain("executionStats")
```

Every index speeds some reads and slows every write, and occupies memory and disk; indexes that no query uses should be dropped, and the working set (the indexes and documents in active use) should fit in memory.

17.7 JSON in Relational Databases

Relational systems have absorbed JSON. SQL:2016 added a family of JSON functions to the standard, SQL:2023 added a native JSON data type and simpler access syntax, and most systems implement their own variations (Liu et al., 2014; ISO/IEC TR 19075-6). PostgreSQL's

`jsonb` type is the most widely used example: it stores documents in a decomposed binary form, supports indexing, and allows documents and ordinary columns in the same table and the same transaction.

Listing 17.10: Orders as jsonb in PostgreSQL: navigation, containment, JSONPath, and unnesting.

```
CREATE TABLE orders (
  id text PRIMARY KEY,
  doc jsonb NOT NULL
);

-- Index supporting containment (@>) and path-existence queries
CREATE INDEX orders_doc_gin ON orders USING gin (doc jsonb_path_ops);

-- Index on one frequently used expression
CREATE INDEX orders_status_idx ON orders ((doc->>'status'));

-- Navigation: -> returns jsonb, ->> returns text
SELECT id, doc->'shipTo'->>'city' AS city
FROM orders
WHERE (doc->>'total')::numeric >= 100;

-- Containment: documents that contain this fragment
SELECT id FROM orders WHERE doc @> '{"status":␣"paid",␣"tags":␣["gift"]}';

-- JSONPath predicate: an item with sku A-17 and qty >= 2
SELECT id FROM orders
WHERE doc @? '$.items[*]␣?␣(@.sku␣==␣"A-17"␣&&␣@.qty␣>=␣2)';

-- Unnest the items array to rows
SELECT o.id, i->>'sku' AS sku, (i->>'qty')::int AS qty
FROM orders o, jsonb_array_elements(o.doc->'items') AS i;
```

Recent versions of PostgreSQL (version 17 and later) implement the standard SQL/JSON query functions as well, in particular `JSON_TABLE`, which maps parts of a document to relational columns declaratively:

Listing 17.11: JSON_TABLE: documents as relational rows (SQL/JSON, PostgreSQL 17 and later).

```
SELECT o.id, jt.sku, jt.qty, jt.price
FROM orders o,
     JSON_TABLE(o.doc, '$.items[*]'
       COLUMNS (sku text PATH '$.sku',
                qty int PATH '$.qty',
                price numeric PATH '$.price')) AS jt
WHERE jt.qty >= 2;
```

17.7.1 Document Store or JSON Column?

A JSON column is attractive when most of the data is relational and a part of it is variable (product attributes, user preferences, payloads of events), and when joins, constraints, and transactions across tables matter. A document store is attractive when the document is the unit of the application, access is mostly by document, and horizontal scale-out is expected. Two cautions apply to JSON columns: the database cannot enforce structure unless constraints or validation are added, and statistics about fields inside the JSON are limited, so the optimizer estimates selectivity poorly. A frequently queried field should be promoted to a real column, which can be generated from the JSON automatically.

17.8 Path and Transformation Languages

17.8.1 JSONPath

JSONPath selects values from a JSON document with a path expression. After years of divergent implementations it was standardized as RFC 9535 (Gößner et al., 2024). Table 17.6 shows the main syntax applied to the order of Listing 17.1.

Table 17.6: JSONPath expressions (RFC 9535) applied to the example order.

Expression	Selects	Result on the example
`$.shipTo.city`	A member of a nested object	`"Berlin"`
`$.items[*].sku`	The `sku` of every array element	`"A-17"`, `"B-02"`
`$.items[-1]`	The last element	The `B-02` item
`$.items[?@.qty >= 2].sku`	Elements that pass a filter	`"A-17"`
`$..price`	All `price` members at any depth (descendants)	19.95, 89.99

Dialects differ: PostgreSQL's `jsonpath` has its own syntax for filters (as in Listing 17.10) and *lax* and *strict* modes that decide whether structural errors raise errors or are ignored. A path expression that works in one system may need adjustment in another.

17.8.2 jq and JMESPath

jq is a command-line language that both selects and *transforms* JSON, with pipes, filters, and constructors, and is the standard tool for inspecting API responses and logs. *JMESPath* is a similar but smaller language with a formal specification, used in cloud command-line tools and configuration. Both treat a query as a pipeline of transformations on a document.

Listing 17.12: jq: select and reshape the paid orders of a file.

```
jq '[ .[] | select(.status == "paid")
      | { id: ._id, city: .shipTo.city,
          units: ([.items[].qty] | add) } ]' orders.json
```

These languages work on one document or a stream of documents in memory. They are not a replacement for a database query language, but they are the right tool for ad hoc inspection, for ETL scripts, and for extracting fields in an application.

17.8.3 Safety of Path Expressions

Path expressions that contain user-supplied input must be treated like any query: concatenating text into a filter can change its meaning. Some implementations evaluate filter expressions with the host language's evaluator, which has led to code-execution vulnerabilities; prefer implementations that parse filters themselves, and pass user values as variables or parameters where the language supports them.

17.9 SQL++ and Other SQL-Like Document Languages

Another approach is to extend SQL itself to nested, schema-flexible data. *SQL++* (Ong et al., 2014; Chamberlin, 2018), used by Couchbase and Apache AsterixDB, keeps the structure of `SELECT`, `FROM`, `WHERE`, and `GROUP BY`, lets paths such as `o.shipTo.country` appear anywhere, treats arrays as collections that `FROM` can iterate over (`UNNEST`), and returns documents as results. Azure Cosmos DB offers a similar SQL dialect over its JSON items.

Listing 17.13: SQL++ queries on the order collection (Couchbase syntax).

```
-- Count of orders per status for one country
SELECT o.status, COUNT(*) AS n
FROM orders AS o
WHERE o.shipTo.country = "DE"
GROUP BY o.status;

-- Items as rows, joined with customers
SELECT o._id, i.sku, i.qty, c.name
FROM orders AS o
UNNEST o.items AS i
JOIN customers AS c ON c._id = o.customerId
WHERE i.qty >= 2;
```

A distinctive feature of SQL++ is that it distinguishes `NULL` (a field whose value is null) from `MISSING` (a field that does not exist), as the MongoDB filters of Section 17.3 must also do by other means. This is an honest reflection of semi-structured data, and it means that three-valued logic becomes four-valued: comparisons involving a missing value propagate "missing" through expressions. The advantage of such languages is familiarity and the ability to express joins and aggregations in one notation; the cost is a larger language whose semantics for irregular data must be learned.

17.10 Schemas and Validation

"Schemaless" is a misleading description. A document collection has a schema, implied by the code that reads and writes it; the database simply does not enforce it unless asked. Without enforcement, a misspelled field, a number stored as a string, or a missing required field enters the data and fails later in a query. *JSON Schema*, a vocabulary for describing the structure of JSON documents, is widely used for validation, and MongoDB applies it at write time.

Listing 17.14: A validator based on JSON Schema for the orders collection.

```
db.createCollection("orders", {
  validator: { $jsonSchema: {
    bsonType: "object",
    required: ["customerId", "status", "items", "total"],
    properties: {
      customerId: { bsonType: "int" },
      status: { enum: ["new", "paid", "shipped", "cancelled"] },
      items: { bsonType: "array", minItems: 1,
               items: { bsonType: "object",
                        required: ["sku", "qty", "price"] } },
      total: { bsonType: ["double", "decimal"], minimum: 0 }
    } } },
  validationLevel: "strict",
  validationAction: "error"
})
```

Validation protects the most important invariants (required fields, types, allowed values) while leaving other fields free to vary. To evolve a schema without downtime, store a `schemaVersion` in each document, let the application read all versions, migrate documents lazily or in batches, and tighten the validator when the old versions have disappeared. This is the document counterpart of the schema migrations of relational systems.

17.11 Consistency and Transactions in Queries

Document databases expose consistency choices similar to those of Chapter 16. In MongoDB, a *write concern* states how many replicas must acknowledge a write (for example `w: "majority"`), and a *read concern* states what a read may see (local data, majority-committed data, or a snapshot). Reads from secondary replicas may be stale; *causal consistency* sessions give a client read-your-writes and monotonic reads across replicas (Chapter 8). Multi-document transactions provide ACID guarantees across documents and, in recent versions, across shards, at a cost in latency and in locked resources, so they should be short and rare. The question to ask of any operation is whether it can be expressed as a single-document update; if it can, it is atomic, fast, and needs no transaction.

17.12 Operating Document Queries

17.12.1 Query Injection

Document queries are objects, and objects can be supplied by an attacker. If an application passes parts of a request body directly into a filter, a user can substitute an operator for a value. The classic example is a login query built from the request fields:

Listing 17.15: Operator injection into a filter, and a defense.

```
// Vulnerable: password comes straight from the request body
db.users.findOne({ username: req.body.username, password: req.body.password })

// An attacker sends {"username": "admin", "password": {"$ne": null}}
// and the filter becomes "password is not null", which matches any user.

// Defense: enforce types before querying (and compare password hashes, never text)
if (typeof req.body.username !== "string" ||
    typeof req.body.password !== "string") { reject(); }
```

Defenses are to validate the type and shape of every input before it reaches a query, to use libraries that reject operator objects in values, to disable server-side JavaScript evaluation if it is not needed, and to grant applications the least privileges they need. The same applies to JSONPath and SQL++ inputs.

17.12.2 Performance Diagnosis

Use the database profiler or slow query log to find queries that examine far more documents than they return, collection scans on large collections, in-memory sorts, and queries that do not use the intended index. Typical remedies are a missing or mis-ordered compound index, a regular expression that cannot use an index, deep `skip` paging, unbounded arrays, and a `$lookup` on an unindexed field. Chapter 16 lists a similar catalogue for wide-column stores; the underlying lesson is the same: the cost of a query is determined by how much data it must touch, and the model and the indexes decide that.

17.12.3 Monitoring

Monitor query latency percentiles, operations per second, cache hit ratio and working-set size, index size, replication lag, lock and transaction statistics, collection and document sizes (including the largest documents), and the number of slow queries. Review indexes periodically for unused or redundant ones.

17.13 Choosing an Approach

A sound process is: (1) list the queries and updates with their frequency; (2) choose the document boundaries so that the common operations touch one document; (3) write the filters, pipelines, and updates and note the fields they use for equality, sort, and range;

Table 17.7: Choosing a way to query JSON data.

Situation	**Suggested approach**	**Reasoning**
Application works with self-contained documents; flexible structure; scale-out expected	Document database with its native query language	Matches the access pattern; atomic single-document updates
Mostly relational data with a variable part (attributes, payloads)	JSON column in a relational database with SQL/JSON	Joins, constraints, and transactions remain available
Team fluent in SQL; semi-structured analytics	SQL++ or SQL/JSON, with `JSON_TABLE` for unnesting	Familiar notation; joins and aggregation in one language
Extract or reshape fields from API responses, logs, or files	JSONPath, jq, or JMESPath	Light tools for single documents and streams
Heavy reporting across many collections	Export to an analytical store and query with SQL	Keeps scans away from the serving database
Text relevance, facets, and fuzzy matching over documents	Search engine fed from the document store (Chapter 13)	Ranking and analysis that document queries lack

(4) create the indexes that serve them, following the equality–sort–range order, and verify with `explain` on realistic data; (5) define a validator for the invariants that matter and a versioning convention; (6) validate and type every input to prevent injection; and (7) bound arrays and document sizes, and monitor slow queries as the data grows.

Summary

Query languages for documents must navigate nested objects, arrays, and fields that are absent, null, or of varying type. Three families exist: document-native languages such as the MongoDB query language, whose queries are themselves documents; SQL extended for JSON, such as SQL/JSON and SQL++; and path and transformation languages such as JSONPath, jq, and JMESPath. In MongoDB a filter document combines conditions with AND, reaches nested fields with dot notation, and matches array elements by any-element semantics, so that conditions on the same element require `$elemMatch`; absent and null fields must be distinguished explicitly, and range-based paging scales better than `skip`. Updates are expressed as operators applied atomically to a single document, which makes the document boundary the unit of atomicity and argues for embedding data that changes together, while unbounded arrays are a modeling error. The aggregation pipeline passes documents through stages that filter, unwind, group, join, and write results, and its cost is governed by filtering early and by using indexes in the leading stages. Compound indexes follow the equality–

sort–range order, multikey indexes cover arrays, partial and TTL indexes control size and lifetime, and `explain` compares keys and documents examined with documents returned. Relational databases offer JSON types, operators, JSONPath, and `JSON_TABLE`, which suit mostly relational data with a variable part. SQL++ adds a distinction between null and missing values. A collection always has an implicit schema, and validation with JSON Schema together with a versioning convention protects it. Operationally, operator injection through unvalidated request objects is a principal risk, and slow queries usually trace back to missing indexes, unbounded arrays, and models that require joins on the hot path.

Review Questions

1. How does querying documents differ from querying flat tables? Give three constructs that a document query language must provide.
2. Name the three families of JSON query languages, and give one example and one typical use of each.
3. Explain the difference between `{"items.sku": "A", "items.qty": {$gte: 2}}` and the same conditions inside `$elemMatch`. Give an example document for which they give different results.
4. What does the filter `{discount: null}` match? How can absent and null fields be told apart?
5. Why does `skip` become slow for deep pages, and how does range-based paging avoid the problem?
6. Why are update operators preferred to reading a document, changing it in the application, and writing it back?
7. What is the scope of atomicity for an update in a document database? What does this imply for modeling?
8. Why are unbounded arrays a problem, and what are two alternatives?
9. Describe the stages of the pipeline in Listing 17.6, and explain where the number of documents grows and where it shrinks.
10. Why should `$match` come first in a pipeline, and which other stages can use an index?
11. State the equality–sort–range guideline and explain why it works. Give the index for a query with equality on `a`, range on `b`, and sort on `c`.
12. What does a healthy `explain` output look like compared with a problematic one? What is a covered query?
13. When is a JSON column in a relational database preferable to a document database, and when is it not?

14. Compare JSONPath, jq, and SQL++ in purpose and in power. What is the difference between `NULL` and `MISSING` in SQL++?

15. What is operator injection, and which three measures reduce the risk?

Exercises

1. **Filters.** Using the order of Listing 17.1, state whether each filter matches and why: (a) `{"shipTo.city": "Berlin"}`, (b) `{shipTo: {city: "Berlin"}}`, (c) `{tags: {$all: ["gift", "express"]}}`, (d) `{items: {$elemMatch: {sku: "B-02", qty: {$gte: 2}}}}`, (e) `{"items.sku": "B-02", "items.qty": {$gte: 2}}`.

2. **Queries.** Write MongoDB queries that (a) return the identifiers and totals of all orders to Germany with a total above 100, sorted by total descending, (b) count the orders per status, (c) find orders that have no `discount` field, and (d) find orders that contain the tag `express` but not the tag `gift`.

3. **Updates.** Write update statements that (a) add the tag `priority` to an order only if it is not yet present, (b) increase the price of every item with sku `A-17` by ten percent in one order, (c) ship all paid orders of customer 42 in one command, and (d) move an order from `paid` to `shipped` only if it is still `paid`, and detect whether it happened.

4. **Pipelines.** Write a pipeline that returns the five customers with the highest total revenue in 2026 from paid orders, together with their names from the `customers` collection. Say where an index would be used, and in which order you would place the stages.

5. **Indexes.** For the queries (i) equality on `customerId`, sort by `orderDate` descending; (ii) equality on `status`, range on `total`; and (iii) equality on `customerId` and `status`, sort by `orderDate`, range on `total`, propose compound indexes following the ESR guideline. Can a single index serve all three? Discuss the cost in write performance and memory.

6. **Explain.** A query returns 20 documents and `explain` reports 2 400 000 documents examined and a `COLLSCAN` stage. Describe the likely cause, three possible remedies, and the figures that you would expect after the fix.

7. **Document size.** Each event appended to the array `history` of an account takes about 120 bytes. At what number of events does a document reach 16 MiB? An account receives 500 events a day: after how many days is the limit reached, and at what size does performance begin to suffer in practice? Propose a bucketed design and write the update that appends an event to it.

8. **SQL/JSON.** Given the table of Listing 17.10, write (a) a query that returns the total revenue per country, (b) a query that finds orders with at least three items, (c) the index that makes (a) fast, and (d) the same unnesting as Listing 17.11 with

`jsonb_array_elements`. Explain when you would promote `doc->'status'` to a generated column.

9. **Schema evolution.** Orders change from `shipTo: "Berlin, DE"` (a string) to the nested object of this chapter. Write a migration plan that uses a version field, keeps the application running, handles documents of both forms in queries, and ends with a stricter validator. Include rollback.

10. **Security review.** List the injection and abuse risks of an API that accepts a JSON filter document from clients and passes it to `find`, including operators that execute code or cause expensive scans. Propose controls for each, such as allow-lists of fields and operators, type checks, query timeouts, and result limits.

References

- Bradshaw, S., Brazil, E., & Chodorow, K. (2019). *MongoDB: The Definitive Guide* (3rd ed.). O'Reilly Media.
- Bray, T. (Ed.). (2017). The JavaScript Object Notation (JSON) Data Interchange Format. RFC 8259, IETF.
- Chamberlin, D. (2018). *SQL++ for SQL Users: A Tutorial*. Couchbase.
- Gößner, S., Normington, G., & Bormann, C. (Eds.). (2024). JSONPath: Query Expressions for JSON. RFC 9535, IETF.
- ISO/IEC TR 19075-6. *Information technology – Database languages SQL – Technical Reports – Part 6: SQL support for JSON*.
- JMESPath Specification. https://jmespath.org/
- jq Manual. https://jqlang.github.io/jq/manual/
- JSON Schema Specification. https://json-schema.org/
- Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.
- Liu, Z. H., Hammerschmidt, B., & McMahon, D. (2014). JSON Data Management: Supporting Schema-less Development in RDBMS. In *Proceedings of ACM SIGMOD '14*, 1247–1258.
- MongoDB Documentation: Query Documents, Update Operators, and Aggregation Pipeline. https://www.mongodb.com/docs/manual/
- Ong, K. W., Papakonstantinou, Y., & Vernoux, R. (2014). The SQL++ Query Language: Configurable, Unifying and Semi-structured. arXiv:1405.3631.
- PostgreSQL Documentation: JSON Types and JSON Functions and Operators. https://www.postgresql.org/docs/

Chapter 18

Vector Similarity Queries and Retrieval Models

Learning Objectives

After reading this chapter, you will be able to:

- Distinguish the kinds of similarity query (top-k, range, filtered, by example, grouped, hybrid) and express them in SQL, a search engine, and a document database.
- Interpret distances and scores correctly, and choose and calibrate thresholds.
- Compare lexical, dense, sparse learned, late-interaction, and cross-encoder retrieval models in cost, quality, and failure modes.
- Combine rankings with reciprocal rank fusion or normalized score fusion, and explain when each is appropriate.
- Improve queries by rewriting, expansion, feedback, and result diversification.
- Assemble the final result list with re-ranking, grouping, paging, and a bounded context.
- Evaluate a retrieval model with recall, MRR, and nDCG on labeled queries, and avoid the usual pitfalls of benchmarks.
- Operate similarity queries safely: latency budgets, caching, monitoring, and access control.

18.1 From Matching to Ranking by Similarity

Chapter 14 described how a vector database stores embeddings and finds nearest neighbors quickly. This chapter looks from the other side, that of the person or program that *asks*: what can a similarity query say, how is it written, how are its answers interpreted, and which *retrieval model* turns a question into a good ranking. The distinction matters because the index only decides how fast the nearest vectors are found, while the retrieval model decides whether the nearest vectors are the *right* ones.

Every query language in this part of the book has a different notion of an answer. A relational query returns the rows that satisfy a predicate; a Cypher query returns the

subgraphs that match a pattern; a document query returns the documents that match a filter. These answers are *sets*: a row either matches or does not. A similarity query returns a *ranking*: every item has a degree of similarity to the question, and the answer is the top of the list. Three consequences follow, and they run through the whole chapter.

- **The answer is relative.** The "best" item is returned even when nothing is relevant, unless the query states a threshold.
- **The answer depends on a model.** A different embedding model, a different query phrasing, or a different fusion rule gives a different ranking, so evaluation takes the place of correctness proofs (Section 18.8).
- **The answer is approximate.** Both the index (Chapter 14) and the model approximate the notion of relevance.

18.1.1 Kinds of Similarity Query

Table 18.1 lists the query forms that applications use. Most real queries combine several of them.

Table 18.1: Kinds of similarity query.

Query	Question asked	Typical use
Top-k (kNN)	The k items closest to the query vector	Semantic search, retrieval for question answering
Range (radius)	All items within distance r (or above similarity s)	Duplicate detection, "nothing relevant" detection
Filtered	Top-k among items that satisfy metadata conditions	Tenant, language, date, permissions (Section 14.6 of Chapter 14)
By example	Items near one or more positive examples and away from negative ones	Recommendation, "more like this"
Batch	Top-k for many query vectors at once	Offline matching, deduplication of a collection
Grouped	Best items per group, such as one result per document	Avoiding many near-identical chunks from one source
Hybrid	A combination of keyword and vector rankings	Queries with both identifiers and paraphrases

18.2 Distances, Scores, and Thresholds

The number returned with each result is the source of frequent misunderstandings, so its meaning needs care.

18.2.1 Distance Versus Similarity

Some systems return a *distance* (smaller is closer) and others a *similarity* or *score* (larger is closer). In PostgreSQL with `pgvector`, the operator `<=>` returns the cosine *distance*, which is one minus the cosine similarity; `<->` returns Euclidean distance; and `<#>` returns the *negative* inner product, so that ascending order still means "closest first." Search engines usually convert to a positive score where larger is better; for example, Elasticsearch maps cosine similarity $c \in [-1, 1]$ to the score $(1 + c)/2$, so that scores are never negative. Sort direction, units, and range differ among systems, and a threshold written for one is wrong for another.

18.2.2 Scores Are Not Probabilities

A similarity of 0.82 does not mean that the item is 82 percent relevant. The value depends on the embedding model, on the length and style of the query and the passage, and on the corpus, and two models can assign very different typical values to the same pair. Consequently:

- Scores are comparable *within one query* and one model, and only loosely across queries; a short keyword query and a long question produce different ranges.
- A fixed threshold works only after *calibration*: take labeled queries, plot the score distributions of relevant and irrelevant results, and select a threshold that gives the desired precision. Repeat after any change of model.
- Scores from different retrieval methods (keyword, vector, re-ranker) have different scales and must not be added without normalization (Section 18.5).

18.2.3 Range Queries and Approximate Indexes

A range query ("everything with cosine similarity above 0.8") seems the natural way to say "only relevant results," but approximate indexes are built for top-k. In most systems the index is used for the pattern *order by distance, limit k*; a bare condition on distance is evaluated by scanning, or is applied *after* the index returned its candidates, which can leave too few results (the post-filtering problem of Chapter 14). Several systems add support, such as iterative index scans in `pgvector` and a minimum similarity parameter in Elasticsearch. A robust pattern is to ask for the top k with a threshold added, which returns up to k results and fewer when nothing close exists.

18.3 Writing Similarity Queries

The same concepts reappear in every system under different names, which Table 18.2 summarizes.

18.3.1 PostgreSQL

Listing 18.1: Similarity queries in PostgreSQL with pgvector ($1 is the query vector).

```
-- Top-k within a tenant; this form can use an HNSW or IVFFlat index
```

Table 18.2: Concepts of a similarity query and their names in three systems.

Concept	**PostgreSQL** (`pgvector`)	**Elasticsearch**	**MongoDB Atlas**
Query vector	Parameter of the distance operator	`query_vector`	`queryVector`
Number of results	`LIMIT`	`k`	`limit`
Search effort	`hnsw.ef_search`, `ivfflat.probes`	`num_candidates`	`numCandidates`
Metadata filter	`WHERE`	`filter`	`filter`
Metric	Operator and index operator class	`similarity` in the mapping	`similarity` in the index
Score	Computed in `SELECT`	`_score`	`$meta: "vectorSearchScore"`

```
SELECT id, content, embedding <=> $1 AS distance
FROM chunks
WHERE tenant_id = $2
ORDER BY embedding <=> $1
LIMIT 10;

-- Similarity instead of distance, and a threshold on top of top-k
SELECT id, 1 - (embedding <=> $1) AS similarity
FROM chunks
WHERE tenant_id = $2 AND embedding <=> $1 < 0.35
ORDER BY embedding <=> $1
LIMIT 10;

-- Similarity to a set of examples: the average of their vectors
SELECT id, content
FROM chunks
WHERE id NOT IN (101, 102)
ORDER BY embedding <=> (SELECT avg(embedding) FROM chunks WHERE id IN (101, 102))
LIMIT 10;
```

Because the vectors live in ordinary tables, similarity can be combined with joins, aggregates, and transactional updates in a single statement, which is the main advantage of this approach for applications that already use PostgreSQL.

18.3.2 Elasticsearch

Listing 18.2: A filtered k-nearest-neighbor query with a minimum similarity (Elasticsearch).

```
GET /docs/_search
{
  "knn": {
    "field": "embedding",
    "query_vector": [0.12, -0.03, 0.57], // abbreviated; 768 values in practice
    "k": 10,
    "num_candidates": 100,
    "similarity": 0.75, // drop results below this similarity
    "filter": { "term": { "tenant": "acme" } }
  },
  "_source": ["title", "text"]
}
```

18.3.3 MongoDB Atlas Vector Search

In a document database the similarity query is a pipeline stage (Chapter 17), so its results can be passed on to filtering, projection, joining, and grouping stages.

Listing 18.3: Vector search as the first stage of an aggregation pipeline.

```
db.chunks.aggregate([
  { $vectorSearch: {
      index: "chunks_vec",
      path: "embedding",
      queryVector: qv,
      numCandidates: 200,
      limit: 10,
      filter: { tenantId: 42, lang: "en" } } },
  { $project: { text: 1, score: { $meta: "vectorSearchScore" } } },
  { $match: { score: { $gte: 0.75 } } }
])
```

The graph database of Chapter 15 offers a similar procedure that yields nodes and scores, which can then be expanded along relationships. In all these systems the fields that a query filters on must usually be declared when the index is created, and the metric is fixed by the index and not by the query.

18.4 Retrieval Models

A *retrieval model* defines how a query and a document are compared. Several models are in common use, and they differ in what they represent, what they cost, and where they fail (Figure 18.1 places them in a typical pipeline).

18.4.1 Lexical Retrieval: TF-IDF and BM25

Lexical models rank documents by the words they share with the query. The standard formula, BM25, scores a document D for a query Q as

$$\mathrm{BM25}(D, Q) = \sum_{q \in Q} \mathrm{IDF}(q) \cdot \frac{f(q, D)\,(k_1 + 1)}{f(q, D) + k_1 \left(1 - b + b\,\frac{|D|}{\mathrm{avgdl}}\right)}, \tag{18.1}$$

where $f(q, D)$ is the number of occurrences of the term q in D, $|D|$ is the length of the document, avgdl is the average document length, k_1 (typically around 1.2) limits the effect of repeated terms, b (typically 0.75) controls length normalization, and the inverse document frequency is commonly

$$\mathrm{IDF}(q) = \ln\!\left(\frac{N - n(q) + 0.5}{n(q) + 0.5} + 1\right) \tag{18.2}$$

for a collection of N documents of which $n(q)$ contain the term (Robertson and Zaragoza, 2009). Rare terms weigh more than common ones, and the first few occurrences of a term count more than later ones. BM25 is fast, needs no training, handles identifiers, product codes, and rare names exactly, and remains a strong baseline that neural models do not always beat on out-of-domain data. Its weakness is the *vocabulary mismatch*: a query using "car" does not find a document that says "automobile" (Chapter 13).

18.4.2 Dense Retrieval: Bi-Encoders

A *bi-encoder* embeds the query and each document independently into single vectors, and the score is a similarity between them (Karpukhin et al., 2020). Because the document vectors are computed in advance, search is a nearest-neighbor lookup. Dense models capture paraphrase and meaning, and work across languages when trained for it, but they compress a whole passage into one vector, so they may blur exact terms, numbers, negations, and rare entities, and their behavior on a new domain is hard to predict. Some models are *asymmetric* and expect different prefixes or instructions for queries and passages; using the wrong one lowers quality noticeably.

18.4.3 Sparse Learned Retrieval

Learned sparse models, such as SPLADE, produce for each text a vector over the vocabulary in which most entries are zero, with weights learned so that related terms are *expanded* (a text about "car" receives weight on "automobile" as well) (Formal et al., 2021). The representation is stored in an ordinary inverted index and keeps BM25's exactness for terms while adding semantic expansion. Such models are attractive for systems built on search engines, and are interpretable, since the weights are attached to words.

18.4.4 Late Interaction: Multi-Vector Models

Late-interaction models such as ColBERT keep one vector per token and score by matching each query token to its best document token and summing:

$$\text{score}(Q, D) = \sum_{i \in Q} \max_{j \in D} \mathbf{q}_i \cdot \mathbf{d}_j. \tag{18.3}$$

The finer comparison recovers detail that a single vector loses, at the price of storing many vectors per document and of a more complex index (Khattab and Zaharia, 2020).

18.4.5 Cross-Encoders and Re-ranking

A *cross-encoder* reads the query and a candidate passage together and outputs a relevance score (Nogueira and Cho, 2019). Because every query–passage pair needs a model run, it can only be applied to a shortlist of tens or hundreds of candidates; but it is the most accurate model of the list, since the query and the passage can interact at every layer. Large language models can serve as re-rankers in the same role, with higher cost and latency and with sensitivity to the order and wording of the prompt.

Table 18.3: Retrieval models compared.

Model	**Representation**	**Strengths**	**Weaknesses**
BM25	Term counts in an inverted index	Exact terms and identifiers; no training; cheap	Vocabulary mismatch
Dense bi-encoder	One vector per passage	Meaning and paraphrase; multilingual options	Blurs exact terms; domain-dependent
Learned sparse	Weighted vocabulary vector	Exact terms plus expansion; interpretable	Larger postings lists; model required
Late interaction	One vector per token	Finer matching than one vector	Storage and index complexity
Cross-encoder	Joint scoring of query and passage	Highest accuracy on a shortlist	Cost per pair; cannot search a collection

18.5 Hybrid Retrieval and Rank Fusion

Keyword and dense retrieval fail differently: the first misses paraphrases, the second misses exact terms. Combining them gives results that are rarely worse than either and often better. There are two ways to combine rankings.

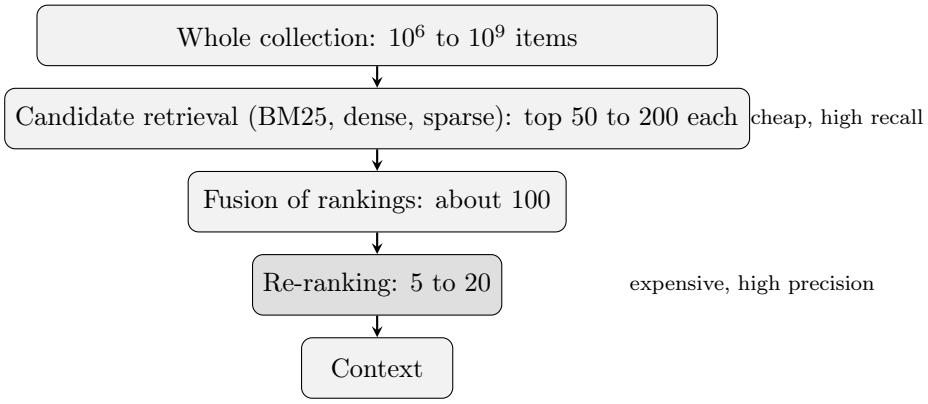


Figure 18.1: Retrieval as a funnel. Cheap models narrow the collection to a candidate set with high recall; expensive models improve precision on the few candidates that remain.

18.5.1 Reciprocal Rank Fusion

Reciprocal rank fusion (RRF) uses only the *ranks* of an item in each list, which avoids the problem that the scores of the lists are on different scales (Cormack et al., 2009):

$$\mathrm{RRF}(d) = \sum_{r \in R} \frac{1}{k + \mathrm{rank}_r(d)}, \tag{18.4}$$

where R is the set of rankings, an item missing from a list contributes nothing, and k is a constant, usually 60, that reduces the influence of the very top ranks. Suppose a vector search and a keyword search return the following ranks: document A is first in the vector list and fifth in the keyword list, document B is second and first, and document C is third and absent from the keyword list. Then

$$\begin{aligned} \mathrm{RRF}(A) &= \tfrac{1}{61} + \tfrac{1}{65} \approx 0.0318, \\ \mathrm{RRF}(B) &= \tfrac{1}{62} + \tfrac{1}{61} \approx 0.0325, \\ \mathrm{RRF}(C) &= \tfrac{1}{63} \approx 0.0159, \end{aligned}$$

so the fused order is B, A, C: the document that both lists rank high wins over the one that only the vector search ranks first. RRF needs no tuning, is robust, and is the default of many systems, but it ignores how large the score differences are.

18.5.2 Weighted Score Fusion

The alternative is to normalize each list's scores to a common scale (for example min–max scaling to $[0, 1]$ within the query's result list) and add them with weights, $s(d) = \alpha\, s_{\mathrm{vec}}(d) + (1 - \alpha)\, s_{\mathrm{kw}}(d)$. This keeps score information and allows the weight α to be tuned on labeled data, but it is sensitive to the normalization, to outliers, and to the number of candidates, and it must be re-tuned when the model changes. A reasonable practice is to start with RRF, and to move to tuned weights only if evaluation shows a clear gain.

Listing 18.4: Hybrid search with reciprocal rank fusion in one SQL statement (PostgreSQL).

```
-- $1: query vector, $2: query text, $3: tenant
WITH semantic AS (
  SELECT id, ROW_NUMBER() OVER (ORDER BY embedding <=> $1) AS rank
  FROM chunks WHERE tenant_id = $3
  ORDER BY embedding <=> $1 LIMIT 50
),
keyword AS (
  SELECT id, ROW_NUMBER() OVER (ORDER BY ts_rank_cd(tsv, q) DESC) AS rank
  FROM chunks, websearch_to_tsquery('english', $2) AS q
  WHERE tenant_id = $3 AND tsv @@ q
  ORDER BY ts_rank_cd(tsv, q) DESC LIMIT 50
)
SELECT COALESCE(s.id, k.id) AS id,
       COALESCE(1.0 / (60 + s.rank), 0) + COALESCE(1.0 / (60 + k.rank), 0) AS rrf
FROM semantic s
FULL OUTER JOIN keyword k ON s.id = k.id
ORDER BY rrf DESC
LIMIT 10;
```

Search engines offer hybrid queries natively. In Elasticsearch, a retriever combines a keyword query and a kNN query with reciprocal rank fusion in a single request (check the availability in the version and edition in use):

Listing 18.5: Hybrid retrieval with rank fusion in a single Elasticsearch request.

```
GET /docs/_search
{
  "retriever": {
    "rrf": {
      "retrievers": [
        { "standard": { "query": { "match": { "text": "reset password" } } } },
        { "knn": { "field": "embedding", "query_vector": [0.12, -0.03, 0.57],
                   "k": 50, "num_candidates": 200 } }
      ],
      "rank_window_size": 50,
      "rank_constant": 60
    }
  }
}
```

18.6 Improving the Query

Retrieval quality depends on the query as much as on the model. Users write short, ambiguous, or conversational queries, and a question and its answer rarely look alike. Several techniques transform the query before it reaches the index.

- **Query rewriting.** A language model or a rule rewrites a query into a clearer form: it expands abbreviations, resolves pronouns using the conversation ("what about its

price?" becomes "what is the price of product X?"), and removes filler. Rewriting that depends on conversation history is essential in chat applications, since a vector for "what about its price?" is meaningless alone.

- **Multi-query expansion.** Several paraphrases or sub-questions are generated, each is searched, and the rankings are fused (for example with RRF). This improves recall at the price of several searches.
- **Hypothetical documents.** The model first writes a plausible answer, and the *answer* is embedded and searched, because an answer is closer to the target passage than the question is (the HyDE technique of Gao et al., 2023). It helps for questions whose wording is far from the documents but can introduce errors when the model's guess is wrong.
- **Decomposition.** A complex question ("compare the refund policies of A and B") is split into simpler ones, each retrieved separately.
- **Relevance feedback.** A classic idea (Rocchio, 1971) moves the query vector toward results that were judged relevant and away from those that were not: $\mathbf{q}' = \alpha\,\mathbf{q} + \beta\,\overline{\mathbf{d}_{\text{rel}}} - \gamma\,\overline{\mathbf{d}_{\text{nonrel}}}$, where the bars denote averages of document vectors. The "by example" query of Listing 18.1 is this idea with positive examples only.

Each added step costs latency and, when a language model is used, money, and can add errors of its own. A rewrite should be evaluated like any other component: does it improve recall on the labeled queries, and by how much (Section 18.8)?

18.7 Shaping the Result List

18.7.1 Grouping and Diversity

A top-k over chunks often returns several near-identical passages from the same document, which wastes the limited space of a result page or of a language model's context. Two remedies are available. *Grouping* (or *collapsing*) keeps only the best result per group, such as per document, and some systems provide it as a query option. *Diversification* reorders the results to balance relevance and novelty; the classic method is *maximal marginal relevance* (MMR), which repeatedly selects the item that maximizes

$$\lambda\,\mathrm{sim}(d, q) \;-\; (1-\lambda)\,\max_{d' \in S} \mathrm{sim}(d, d'), \tag{18.5}$$

where S is the set already selected and $\lambda \in [0, 1]$ balances relevance against similarity to the chosen items (Carbonell and Goldstein, 1998). A value of λ near 1 reproduces the plain ranking; lower values spread the results over different aspects of the topic.

18.7.2 Paging

Paging through nearest-neighbor results is harder than through a sorted column. An approximate index returns the best k candidates found; asking for page ten with ten results

per page means asking for $k = 100$ and discarding the first 90, so the cost grows with the depth, and most systems cap k or the candidate count. Applications should expect users to read the first page, and design for refinement (adding a filter or rephrasing) rather than for deep paging.

18.7.3 Re-ranking and Context Assembly

When the results feed a language model (Chapter 14), the last stages decide what the model sees. A cross-encoder re-ranks the fused candidates; duplicates and near-duplicates are removed; small matched chunks may be replaced by their larger parent sections for context; and the selection is trimmed to a budget of tokens. The order in the context matters too: models tend to use information at the beginning and end of a long context better than information in the middle (Liu et al., 2024), which argues for short contexts of high-precision passages and for placing the best evidence first. Every passage should carry its source, so that answers can cite it and so that users and reviewers can check them.

18.8 Evaluating Retrieval Models

Because the answer of a similarity query is a ranking whose quality cannot be proved, it is measured. The method is the same for any retrieval model: collect realistic queries, label the relevant documents, run the system, and compute metrics that reward placing relevant documents early (Chapter 14 introduced the procedure for pipelines; this section gives the definitions).

Table 18.4: Retrieval metrics.

Metric	**Definition**	**Answers the question**
Precision@k	Fraction of the top k results that are relevant	How clean is the first page?
Recall@k	Fraction of all relevant items found in the top k	Did the retriever bring the evidence into the candidate set?
MRR	Mean over queries of 1/rank of the first relevant result	How fast does the user reach the first good answer?
MAP	Mean over queries of the average precision at each relevant result	How good is the whole ranking for many relevant items?
nDCG@k	Discounted cumulative gain, normalized by the ideal ranking	How good is the ranking when relevance has grades?

The metric nDCG accepts graded judgments (for example 0 to 3) and discounts the gain of lower positions (Järvelin and Kekäläinen, 2002):

$$\mathrm{DCG@}k = \sum_{i=1}^{k} \frac{2^{\mathrm{rel}_i} - 1}{\log_2(i+1)}, \qquad \mathrm{nDCG@}k = \frac{\mathrm{DCG@}k}{\mathrm{IDCG@}k}, \tag{18.6}$$

where rel_i is the grade of the result at position i and IDCG is the DCG of the best possible ordering. For example, if a system returns four results with grades $(3, 2, 0, 1)$, then

$$\text{DCG@4} = \frac{7}{1} + \frac{3}{1.585} + \frac{0}{2} + \frac{1}{2.322} \approx 9.323,$$

the ideal order $(3, 2, 1, 0)$ gives IDCG@4 ≈ 9.393, and nDCG@4 ≈ 0.993.

18.8.1 Pitfalls

- **Benchmarks do not transfer.** Public benchmarks such as BEIR and MTEB compare models over many datasets (Thakur et al., 2021; Muennighoff et al., 2023), and they are a good way to shortlist candidates. A model that leads a leaderboard can still lose to BM25 on a specific domain, so the final choice must be made on the application's own queries.
- **Labels are expensive and incomplete.** Unlabeled results are assumed irrelevant, which penalizes a model that finds good passages nobody labeled. Pool the results of several systems when labeling, and review disagreements. Language models can help to label but must be calibrated against human judgments.
- **Evaluate the stages separately.** Recall@100 of the candidate stage bounds what any re-ranker can achieve; precision and nDCG of the final list measure the re-ranker. A failure is first located and then fixed.
- **Look at the failures.** Aggregate numbers hide patterns such as negation, numbers, rare names, or short queries. Reading fifty failures teaches more than a decimal point of nDCG.
- **Online behavior differs from offline metrics.** Click-through, reformulation rate, and the proportion of questions answered correctly in production complement the offline numbers, and A/B tests decide between candidate systems.

18.9 Operating Similarity Queries

18.9.1 Latency Budget

A similarity query is rarely one operation. It may embed the query (a model call), run one or more searches, fuse the results, re-rank the shortlist (another model call), and possibly rewrite the query with a language model first. Each step contributes to the total and the slowest dominates the tail. Set a budget for the whole request, measure each step separately, and decide where to spend: more candidates and a re-ranker for quality, or a smaller candidate set and no rewrite for speed. Parallelize independent steps, such as the keyword and vector searches.

18.9.2 Caching and Batching

Query embeddings are deterministic for a given model, so frequent queries can be cached by their normalized text; results can be cached when the data changes slowly, with invalidation on updates or a short lifetime. Batch requests for the embedding service and use batch

similarity queries for offline jobs, which use hardware much more efficiently than one query at a time.

18.9.3 Monitoring

In addition to latency, throughput, and recall of the index (Chapter 14), monitor the *distribution of top scores* per day (a shift suggests a change in data, queries, or model), the share of queries with no result above the threshold (a measure of gaps in the content), the share of queries answered by each retrieval method, the rate of reformulated queries, and the quality metrics on a fixed set of canary queries with known answers, recomputed regularly.

18.9.4 Access Control, Privacy, and Safety

Filters for tenants and permissions belong inside the query, not applied to its results (Chapter 14); a hybrid query must apply the same filter to *every* component, since a filter that is forgotten in one branch leaks through the fusion. Logged queries may contain personal or confidential information and should be governed like any other data. Retrieved text is untrusted when it is passed to a language model, and a re-ranker or rewriter that is itself a language model can be influenced by instructions hidden in passages, so these components should see only data they need, and their outputs should be validated.

18.10 Choosing a Retrieval Approach

A sound process is: (1) collect real queries and label the relevant documents; (2) establish a baseline with BM25 and one dense model, measured with recall and nDCG; (3) add hybrid fusion with RRF and measure the gain; (4) add a re-ranker on the shortlist and measure the gain against its latency and cost; (5) try query rewriting and expansion where failures show vocabulary or conversation problems; (6) calibrate thresholds and decide what the system does when nothing relevant is found; (7) put filters and permissions in every branch of the query; and (8) monitor score distributions and canary queries in production, and re-evaluate whenever the model, the data, or the chunking changes.

Summary

A similarity query returns a ranking and not a set, so every item has a degree of similarity and the best item is returned even when nothing is relevant. Query forms include top-k, range, filtered, by example, batch, grouped, and hybrid queries, and they are written with distance operators in SQL, `knn` clauses in search engines, and pipeline stages in document databases, with the same concepts under different names. Distances and similarities differ in direction and range among systems, scores are not probabilities, and thresholds need calibration on labeled queries; approximate indexes serve the pattern of ordering by distance with a limit, so thresholds are best combined with a top-k. Retrieval models differ in representation and cost: BM25 matches exact terms and fails on vocabulary mismatch; dense bi-encoders capture meaning but blur exact terms; learned sparse models add expansion to inverted indexes; late

Table 18.5: Choosing a retrieval approach.

Situation	**Suggested approach**	**Reasoning**
Queries with identifiers, codes, exact phrases	BM25 or keyword search, with dense retrieval as a complement	Exact terms are not preserved by embeddings
Paraphrased questions over natural-language content	Dense retrieval, hybrid with BM25	Meaning plus exact terms; robust default
Search engine already in place; interpretability wanted	Learned sparse retrieval in the inverted index	Semantic expansion without a new system
Technical text with precise terminology where a single vector loses detail	Late interaction, or re-ranking with a cross-encoder	Finer matching on a bounded candidate set
Quality matters more than latency	Cross-encoder or language-model re-ranking of 20 to 100 candidates	Highest precision on the shortlist
Conversational interface	Query rewriting with history, then hybrid retrieval	Follow-up questions are meaningless without context
Results shown to users or fed to a language model	Grouping, diversification, token budget, citations	Limited space; duplicates waste it

interaction compares token vectors; and cross-encoders score query and passage together, which is accurate but possible only on a shortlist. Pipelines therefore form a funnel of cheap, high-recall stages followed by expensive, high-precision ones. Rankings from different methods are combined by reciprocal rank fusion, which uses ranks only and needs no tuning, or by weighted sums of normalized scores. Queries can be improved by rewriting, multi-query expansion, hypothetical documents, decomposition, and relevance feedback, each at a cost that must be justified by evaluation. The result list is shaped by grouping, diversification with maximal marginal relevance, limited paging, re-ranking, and assembly of a bounded context with sources. Retrieval is evaluated with precision, recall, MRR, MAP, and nDCG on labeled queries of the application's own domain, with stages evaluated separately and failures inspected. Operations center on a latency budget, caching, monitoring of score distributions and canary queries, and enforcing permissions in every branch of a hybrid query.

Review Questions

1. How does the answer of a similarity query differ from the answer of a relational or a document query? Why does this make evaluation necessary?

2. Name six kinds of similarity query and give an application of each.

3. What is the relationship between cosine similarity and the cosine distance returned by `<=>` in `pgvector`? Why does `<#>` return the negative inner product?

4. Why are similarity scores not probabilities? How would you calibrate a threshold?

5. Why is a range query on distance poorly supported by approximate indexes, and what pattern is more robust?

6. State the BM25 formula in words. What do k_1 and b control, and what is its main weakness?

7. Compare bi-encoders and cross-encoders in how they score and in what they cost.

8. What does a learned sparse model add to a lexical index? What does late interaction add to a dense one?

9. Why is retrieval organized as a funnel of stages? What does each stage optimize?

10. Explain reciprocal rank fusion and compute the fused score of a document ranked second in one list and tenth in another with $k = 60$.

11. When might weighted score fusion be preferable to RRF, and what are its risks?

12. Describe three query transformations and a situation in which each helps and one in which it can hurt.

13. What problem does maximal marginal relevance solve? What does λ control?

14. Why is deep paging expensive in nearest-neighbor search, and how should interfaces respond?

15. Define recall@k, MRR, and nDCG@k. Which would you use to evaluate the candidate stage, and which to evaluate the re-ranker?

16. Why can a model that tops a public benchmark still perform worse than BM25 in your application?

Exercises

1. **Scores.** For the vectors $\mathbf{q} = (1, 0)$, $\mathbf{a} = (3, 4)$, and $\mathbf{b} = (0.9, -0.1)$ compute the cosine similarity, the cosine distance, the Euclidean distance, and the inner product of $\mathbf{q}$ with each of $\mathbf{a}$ and $\mathbf{b}$. Which result is closest under each measure? Convert the cosine similarities to the Elasticsearch score $(1 + c)/2$.

2. **BM25.** Compute the BM25 contribution of one query term with $N = 1000$, $n = 10$, $f = 3$, $|D| = 100$, avgdl $= 120$, $k_1 = 1.2$, and $b = 0.75$. Then compute it for a document of length 300 with the same f, and explain the difference.

3. **RRF.** Three rankings are given: vector (A, B, C, D), keyword (C, A, E, B), and a title search $(B, C, \ldots)$. Compute the RRF scores with $k = 60$ and the fused order. How does the result change with $k = 1$ and with $k = 1000$?

4. **nDCG.** A system returns results with grades $(2, 0, 3, 1, 0)$ for a query. Compute DCG@5, IDCG@5, and nDCG@5 using gain $2^{\text{rel}} - 1$. Compute also precision@5 and MRR if grades of 1 and above count as relevant.

5. **Threshold calibration.** Describe an experiment to select a similarity threshold below which a question-answering system should answer "I do not know." Specify the labeled data, the metric to maximize, how to handle the trade-off between wrong answers and refusals, and how often to repeat the calibration.

6. **Hybrid query design.** A support knowledge base contains articles, error codes (for example `E-4012`), and product names. Design a retrieval query: which models, how are they combined, what is filtered, and how are error codes handled so that they are found exactly? Write the main parts in SQL or in the query language of your choice.

7. **MMR.** Five passages have query similarities $(0.90, 0.88, 0.80, 0.70, 0.60)$ and pairwise similarities such that passages 1 and 2 are near duplicates (0.95) and the others are mutually dissimilar (0.2). Select three passages with MMR for $\lambda = 1$, 0.7, and 0.5. Which passages are chosen, and in what order?

8. **Query rewriting.** A chat application receives "and what about the second one?" after a user asked for laptops with 16 GB of memory. Describe three ways to turn the message into a searchable query, discuss the risks of each, and design an evaluation that shows whether rewriting improves retrieval.

9. **Latency budget.** A request has a budget of 500 ms. The query embedding takes 40 ms, each search 60 ms (run in parallel), fusion 5 ms, a cross-encoder re-ranker 6 ms per passage on one GPU in batches of 10 (60 ms per batch), and an optional rewriting call 250 ms. How many candidates can be re-ranked within the budget with and without rewriting? Propose changes if the budget is cut to 250 ms.

10. **Failure analysis.** After deployment, 12% of queries return no relevant result in the top ten. Describe the steps to diagnose the failures, separating problems of the model, the chunking, the filters, and the queries, and propose a fix for each class of failure.

References

- Carbonell, J., & Goldstein, J. (1998). The Use of MMR, Diversity-Based Reranking for Reordering Documents and Producing Summaries. In *Proceedings of ACM SIGIR '98*, 335–336.

- Cormack, G. V., Clarke, C. L. A., & Büttcher, S. (2009). Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. In *Proceedings of ACM SIGIR '09*, 758–759.

- Elasticsearch Documentation: k-nearest neighbor (kNN) search and Retrievers. https://www.elastic.co/guide/
- Formal, T., Piwowarski, B., & Clinchant, S. (2021). SPLADE: Sparse Lexical and Expansion Model for First Stage Ranking. In *Proceedings of ACM SIGIR '21*, 2288–2292.
- Gao, L., Ma, X., Lin, J., & Callan, J. (2023). Precise Zero-Shot Dense Retrieval without Relevance Labels. In *Proceedings of ACL 2023*, 1762–1777.
- Järvelin, K., & Kekäläinen, J. (2002). Cumulated Gain-Based Evaluation of IR Techniques. *ACM Transactions on Information Systems*, 20(4), 422–446.
- Karpukhin, V., Oğuz, B., Min, S., et al. (2020). Dense Passage Retrieval for Open-Domain Question Answering. In *Proceedings of EMNLP 2020*, 6769–6781.
- Khattab, O., & Zaharia, M. (2020). ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT. In *Proceedings of ACM SIGIR '20*, 39–48.
- Liu, N. F., Lin, K., Hewitt, J., et al. (2024). Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics*, 12, 157–173.
- Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press.
- MongoDB Atlas Documentation: Atlas Vector Search. https://www.mongodb.com/docs/atlas/
- Muennighoff, N., Tazi, N., Magne, L., & Reimers, N. (2023). MTEB: Massive Text Embedding Benchmark. In *Proceedings of EACL 2023*, 2014–2037.
- Nogueira, R., & Cho, K. (2019). Passage Re-ranking with BERT. arXiv:1901.04085.
- pgvector Documentation. https://github.com/pgvector/pgvector
- Robertson, S., & Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval*, 3(4), 333–389.
- Rocchio, J. J. (1971). Relevance Feedback in Information Retrieval. In G. Salton (Ed.), *The SMART Retrieval System: Experiments in Automatic Document Processing*, 313–323. Prentice-Hall.
- Thakur, N., Reimers, N., Rücklé, A., Srivastava, A., & Gurevych, I. (2021). BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models. In *NeurIPS 2021 Datasets and Benchmarks Track*.

Part VI

Modern Data Architectures

Chapter 19

Data Lakes and Lakehouse Architectures

Learning Objectives

After reading this chapter, you will be able to:

- Explain how data warehouses, data lakes, and lakehouses differ, and which problems each was designed to solve.
- Describe the properties of cloud object storage and their consequences for analytical data systems.
- Explain why columnar file formats such as Parquet make analytical queries fast, and how row groups, statistics, and pushdown work.
- Describe how open table formats (Apache Iceberg, Delta Lake, Apache Hudi) provide transactions, snapshots, time travel, and schema evolution on top of immutable files.
- Design a layered lakehouse with ingestion, a medallion structure, a catalog, and multiple compute engines.
- Choose partitioning, clustering, and file sizes, and estimate the data read by a query.
- Operate a lakehouse: compaction, snapshot expiry, concurrency, cost, and recovery.
- Address governance, security, and privacy requirements, including deletion in systems that keep history.

19.1 From Warehouse to Lake to Lakehouse

The databases of earlier chapters mostly serve *operational* workloads: many small reads and writes on current data, with low latency. A different workload, *analytical* processing, reads very large amounts of historical data to compute aggregates, train models, and answer questions about the business as a whole. It scans billions of rows but touches few columns, runs for seconds or minutes, and tolerates writes that arrive in batches. Analytical systems have evolved through three generations, which this chapter traces.

19.1.1 The Data Warehouse

The *data warehouse* is a relational database optimized for analytics: data from operational systems is extracted, transformed into a clean, integrated, usually dimensional schema (Kimball and Ross, 2013), and loaded (the ETL process). Warehouses give fast SQL, strong consistency, and good governance, but they require the schema to be designed *before* data is loaded (*schema-on-write*), they store mostly structured data, and in their classical form they couple storage and compute so that growing either one means growing both, at high cost.

19.1.2 The Data Lake

The *data lake* appeared with Hadoop and its distributed file system, HDFS (Siadati, 2021), and then with cloud object storage as a cheap place to put *everything* as files, in the original format, without prior modeling (*schema-on-read*): logs, JSON from applications, exports from databases, images, and text. Data is loaded first and transformed later (ELT), by engines such as Spark that read the files directly (Zaharia et al., 2016). Lakes scale cheaply and serve data science and machine learning well, but a collection of plain files has no transactions, no consistent view while files are being written, no reliable schema, and no governance. Lakes without discipline become *data swamps*, and in practice organizations copied the useful parts into a warehouse, which led to two systems, two copies of the data, and the delays and inconsistencies between them.

19.1.3 The Lakehouse

The *lakehouse* combines the two: the data stays in open file formats on cheap object storage, and a metadata layer on top, the *table format*, adds the guarantees of a database, namely ACID transactions, schema enforcement and evolution, consistent snapshots, and efficient SQL (Armbrust et al., 2021). The result is one copy of the data that serves BI queries, data engineering, and machine learning, accessible to several independent compute engines. Table 19.1 compares the three architectures.

The boundaries are not sharp. Modern cloud warehouses can read open table formats, and lakehouse engines add warehouse features, so the architectures are converging. The important idea, and the subject of this chapter, is the *separation of concerns*: open files, an open table format, a catalog, and interchangeable engines.

19.2 Object Storage as the Foundation

Cloud object stores (such as Amazon S3, Google Cloud Storage, and Azure Blob Storage) hold *objects* in flat namespaces called buckets, addressed by a key such as `lake/silver/orders/part-0001.parquet`. Their properties shape the design of everything above them.

- **Cheap, durable, and practically unlimited.** Storage costs a fraction of block storage, data is replicated across failure domains, and capacity is not planned in advance.

Table 19.1: Data warehouse, data lake, and lakehouse compared.

Aspect	**Warehouse**	**Data lake**	**Lakehouse**
Storage	Proprietary, managed by the engine	Files on object storage or HDFS	Open files on object storage
Data types	Mainly structured	Any: structured, semi-structured, unstructured	Any, with tables over structured parts
Schema	On write; enforced	On read; not enforced	Enforced and evolvable, per table
Transactions	ACID	None across files	ACID through the table format
Compute	Coupled to storage (classical)	Separate, many engines	Separate, many engines
Typical users	Analysts, BI	Data engineers, data scientists	All of them
Main risk	Cost; silos; limited unstructured data	Swamp; no consistency	Operational complexity of the metadata layer

- **Immutable objects.** An object is written as a whole and replaced as a whole; there is no in-place update and no append in the database sense. All changes create new objects, which is why table formats are built on immutable files and versions.
- **No atomic multi-object operation and no cheap rename.** A table is made of many files, and the store cannot make a set of them appear atomically. Renaming is a copy followed by a delete, so protocols that rely on "write to a temporary name and rename" are neither atomic nor fast. Newer conditional-write features ("create this object only if it does not exist") provide the primitive that commit protocols need, and their availability differs among providers.
- **Consistency.** Major object stores now provide strong read-after-write consistency for single objects, but listing performance and semantics still differ, and listing a directory with millions of files is slow.
- **High latency, high throughput.** A single request takes tens of milliseconds, far slower than a local disk, but throughput scales almost without limit when many requests run in parallel. Engines therefore read large ranges of a file in parallel and avoid many small reads.
- **Cost by use.** Charges depend on stored bytes, on the number of requests, and on data leaving a region; many small files raise the request cost and slow queries.

The consequence is a design rule that returns throughout this chapter: *few, large, immutable files, with metadata that says which files make up the table.* The separation of

storage from compute (Chapter 14 noted the same trend for vector databases) lets several engines read the same files and lets each be scaled and paid for independently, at the price of network latency for every byte read.

19.3 File Formats: Row and Columnar

19.3.1 Why Columnar Formats

Row-oriented formats such as CSV, JSON, and Avro store all fields of a record together, which suits writing records one at a time and reading whole records. Analytical queries behave differently: they read a few columns of many rows. A *columnar* format stores the values of each column together, which has three advantages (Stonebraker et al., 2005; Melnik et al., 2010):

- **Projection pushdown.** A query that needs 3 of 50 columns reads only those 3 columns.
- **Better compression.** Values of one column are similar in type and often in value, so dictionary encoding, run-length encoding, and general-purpose compression reduce them strongly.
- **Vectorized processing.** Engines process batches of values of one column with efficient CPU loops.

19.3.2 Anatomy of a Parquet File

Apache Parquet is the dominant columnar format of lakes and lakehouses; ORC is a similar format. A Parquet file is divided into *row groups*, horizontal slices of typically tens to hundreds of megabytes. Inside each row group the values of each column are stored in a *column chunk*, divided into pages that are encoded and compressed. A *footer* at the end of the file holds the schema and, for every column chunk, statistics such as the minimum and maximum value and the number of nulls (Figure 19.1).

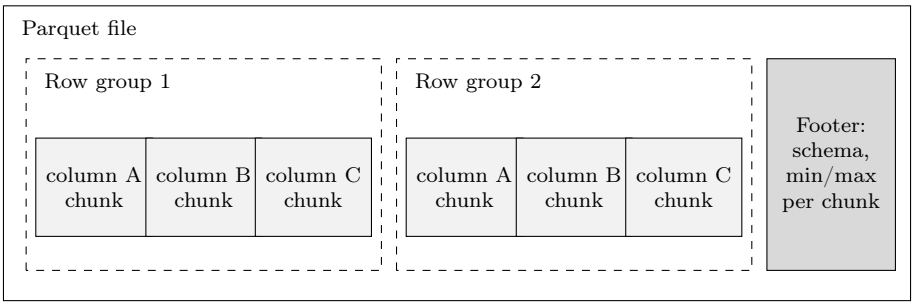


Figure 19.1: Layout of a Parquet file. The reader fetches the footer first, uses the statistics to skip row groups that cannot match, and then reads only the column chunks that the query needs.

A query engine reads the footer, applies *predicate pushdown* by comparing the filter with the min/max statistics, and skips row groups that cannot contain matches; it then fetches

only the needed column chunks by byte range. Sorting the data on a frequently filtered column makes the statistics narrower and the skipping more effective. The same idea at the level of whole files (table statistics kept by the table format) is described in Section 19.4.

19.3.3 Choosing Formats

Table 19.2: Common file formats in data lakes.

Format	Layout	Strengths	Typical use
CSV, JSON lines	Row, text	Universal; human-readable; no schema or types	Exchange, raw landing zone
Avro	Row, binary, with schema	Compact; schema evolution; fast sequential writes	Streaming and event data
Parquet	Columnar, binary	Compression; push-down; nested data; wide support	Analytical tables (default)
ORC	Columnar, binary	Similar to Parquet; strong in Hive ecosystems	Analytical tables

The usual pattern is to land data as it arrives (JSON, Avro, CSV) and convert it to Parquet once it is cleaned, as in Section 19.5. Listing 19.1 shows a conventional directory layout with *Hive-style* partitioning, in which values of the partition columns are encoded in directory names, and Listing 19.2 reads such files directly with a lightweight engine.

Listing 19.1: A partitioned layout of Parquet files on object storage.

```
s3://lake/silver/orders/
    order_date=2026-10-26/
        part-0000.parquet
        part-0001.parquet
    order_date=2026-10-27/
        part-0000.parquet
```

Listing 19.2: Querying Parquet files directly with DuckDB, and inspecting their metadata.

```
-- Only the columns and the partition needed are read
SELECT country, SUM(total) AS revenue
FROM read_parquet('s3://lake/silver/orders/*/*.parquet', hive_partitioning = true)
WHERE order_date >= DATE '2026-10-01'
GROUP BY country;

-- Row groups, column statistics, and encodings of one file
SELECT * FROM parquet_metadata('part-0000.parquet');
```

19.4 Open Table Formats

Plain Parquet files in directories are not yet a database. A table of thousands of files has no way to make a change appear atomically, so a reader may see half of a write; no way to know which files belong to the table without listing the directory; no schema history; and no safe way for two writers to work at the same time. *Open table formats* solve these problems by adding a *metadata layer* that records, in files of its own, exactly which data files belong to the table at each point in time. The three most widely used are *Apache Iceberg*, *Delta Lake*, and *Apache Hudi*.

19.4.1 The Common Idea: Snapshots and Atomic Commits

All three formats share a design built on the immutability of files. A write creates *new data files* and then records a new version of the table that lists them; old files are not changed. The commit is the atomic publication of the new version, after which readers see the whole change or none of it. Readers always work on one fixed *snapshot*, so a long query is not disturbed by concurrent writes. Because old snapshots remain until they are expired, the formats also offer *time travel*: queries and rollbacks to a previous version (Armbrust et al., 2020).

Concurrent writers use *optimistic concurrency control*: each writer reads the current snapshot, prepares its new files, and tries to commit; if another writer has committed in the meantime, the commit is rejected, and the writer checks whether its change conflicts with the intervening ones (for example, whether both modified the same files) and either retries on the new snapshot or fails. This gives snapshot isolation for readers and serializable-style protection against conflicting writes, at the cost of retries when many writers touch the same data. The trade-off is the same as for the transactions of Chapter 8: optimistic schemes are efficient when conflicts are rare.

19.4.2 Apache Iceberg

Iceberg organizes the metadata as a tree (Figure 19.2). A *catalog* stores a pointer to the current *table metadata file*, which holds the schema, the partition specification, and the list of snapshots. Each snapshot points to a *manifest list*, which points to *manifest files*, and each manifest lists data files together with per-file statistics (row counts, column minima and maxima, null counts). A commit writes new metadata files and then atomically swaps the catalog pointer to the new table metadata; this single atomic step is what makes the commit all-or-nothing.

Iceberg has several features worth knowing. *Hidden partitioning* lets a table be partitioned by a transformation of a column, such as the day of a timestamp or a hash bucket of an identifier, and queries on the original column use the partitioning automatically, without the user naming partition columns. *Partition evolution* allows the specification to change without rewriting old data, since each file records the specification under which it was written. *Schema evolution* tracks columns by identifier and not by name or position, so adding, renaming, and dropping columns is safe.

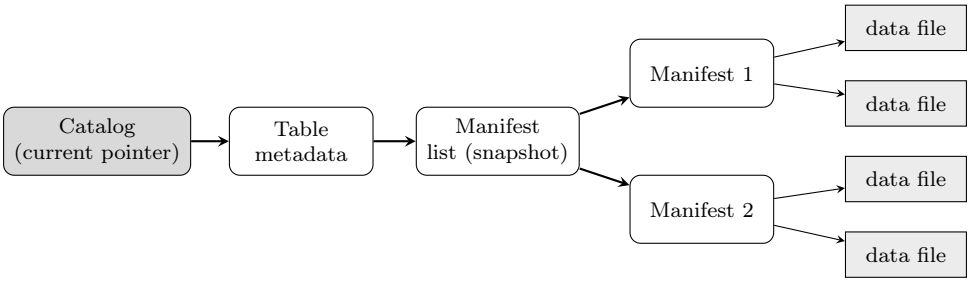


Figure 19.2: Metadata structure of Apache Iceberg. A commit adds metadata files and then atomically changes the pointer held by the catalog.

19.4.3 Delta Lake

Delta Lake keeps a *transaction log* in a directory `_delta_log` next to the data: each commit is a numbered JSON file that lists the files added and removed and any change of schema, and periodic *checkpoint* files in Parquet summarize the log so that readers do not replay it from the beginning. A commit is made atomic by creating the next numbered log file only if it does not exist yet. The table's state is obtained by replaying the log, and time travel is the replay up to an earlier version. Delta Lake originated in the Spark ecosystem and is widely used with Databricks, and it is readable by many other engines.

19.4.4 Apache Hudi

Hudi was designed for *incremental processing* and frequent upserts. It keeps a *timeline* of actions on the table and an index that maps record keys to files, so that updates and deletes of individual records are efficient. It offers two table types: *copy-on-write*, where an update rewrites the affected data files, giving fast reads and slower writes, and *merge-on-read*, where updates are written to small log files that are merged with the base files at read time or during compaction, giving fast writes and slightly slower reads. The same trade-off exists in the other formats under the names of delete files and deletion vectors.

Interoperability layers and converters allow a table written in one format to be read through the metadata of another, and the choice among formats is less irreversible than it once was. The decisive questions are which engines must read and write the tables, which catalog is used, and which features (upserts, partition evolution, streaming) the workload needs.

19.4.5 Tables in SQL

The following listings use the SQL of Spark with an Iceberg catalog named `lake`; other engines and formats offer equivalent statements with different syntax.

Listing 19.3: Creating a partitioned table and loading it (Iceberg in Spark SQL).

```
CREATE TABLE lake.shop.orders (
  order_id bigint,
  customer_id bigint,
```

Table 19.3: Open table formats compared (features evolve; check current documentation).

Aspect	**Iceberg**	**Delta Lake**	**Hudi**
Metadata	Tree of metadata, manifest lists, and manifests; catalog pointer	Ordered transaction log with checkpoints	Timeline of actions; record-key index
Commit	Atomic pointer swap in the catalog	Atomic creation of the next log file	Atomic completion of an instant on the timeline
Distinctive strength	Hidden partitioning and partition evolution; engine neutrality	Tight integration with Spark; simple log model	Efficient upserts and incremental pulls
Row-level changes	Delete files and deletion vectors	Deletion vectors; rewriting of files	Copy-on-write or merge-on-read
Typical engines	Spark, Trino, Flink, many warehouses	Spark, Trino, Flink, many warehouses	Spark, Flink, Trino

```
  order_ts timestamp,
  country string,
  total decimal(12,2)
) USING iceberg
PARTITIONED BY (days(order_ts)); -- hidden partitioning

INSERT INTO lake.shop.orders
SELECT order_id, customer_id, order_ts, country, total
FROM lake.bronze.orders_raw
WHERE order_ts >= TIMESTAMP '2026-10-01␣00:00:00';
```

Listing 19.4: An upsert with MERGE: apply a batch of changes captured from the source system.

```
MERGE INTO lake.shop.orders AS t
USING lake.staging.order_changes AS s
  ON t.order_id = s.order_id
WHEN MATCHED AND s.op = 'D' THEN DELETE
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED AND s.op <> 'D' THEN INSERT *;
```

Listing 19.5: Time travel and inspection of table history.

```
-- The table as it was at a point in time, or at a snapshot
SELECT COUNT(*) FROM lake.shop.orders TIMESTAMP AS OF '2026-10-01␣00:00:00';
SELECT COUNT(*) FROM lake.shop.orders VERSION AS OF 8271635401234567890;
```

```
-- Metadata tables describe the table itself
SELECT snapshot_id, committed_at, operation FROM lake.shop.orders.snapshots;
SELECT file_path, record_count, file_size_in_bytes FROM lake.shop.orders.files;
```

Time travel serves audits ("what did the report show last quarter?"), reproducible machine-learning datasets (train on the snapshot of a given date), debugging of faulty pipelines, and rollback after a bad write.

19.5 Architecture of a Lakehouse

A lakehouse is a set of layers, each replaceable behind an open interface (Figure 19.3).

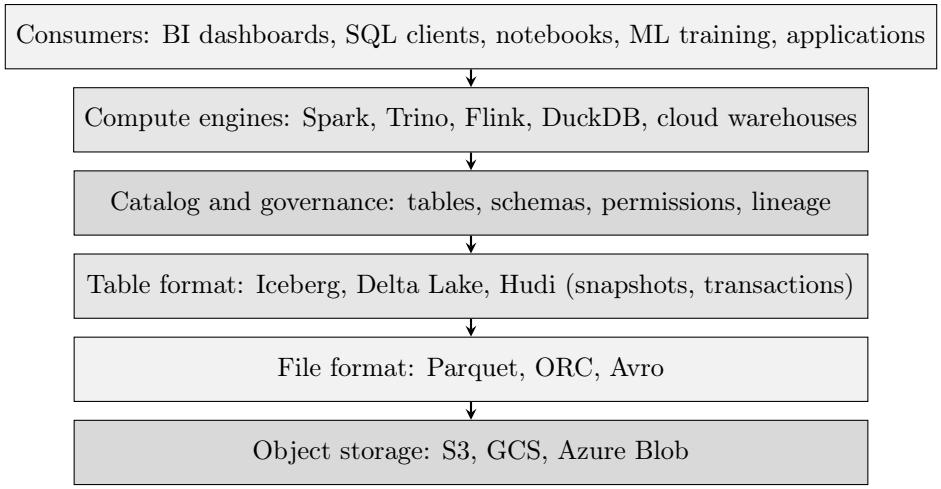


Figure 19.3: Layers of a lakehouse. Each layer has an open interface, so engines, catalogs, and formats can be combined and replaced.

19.5.1 The Medallion Pattern

A common way to organize the data is in three zones of increasing quality, often called *bronze*, *silver*, and *gold*:

- **Bronze (raw).** Data as received from sources, kept unchanged with metadata about its origin and arrival time, so that every later step can be replayed. Append-only.
- **Silver (cleaned).** Data that is deduplicated, validated, typed, and conformed to common keys and formats, modeled as tables per business entity. This is the main layer for data engineers and analysts.
- **Gold (curated).** Aggregates and dimensional models built for specific uses: reports, dashboards, features for machine learning, data products shared with other teams.

The structure separates concerns. Raw data is never lost, so a bug in a transformation is repaired by recomputing from bronze; business rules live in silver and gold, under tests; and consumers use stable gold tables whose contracts change rarely. The names are a convention and not a standard, and some organizations use different layers.

19.5.2 Ingestion

Data reaches the lakehouse in three ways (Table 19.4).

Table 19.4: Ingestion patterns for a lakehouse.

Pattern	How it works	Considerations
Batch	Scheduled jobs copy or export tables and files, for example every night	Simple; high latency; full reloads are costly for large tables
Change data capture (CDC)	The transaction log of an operational database is read (for example with Debezium) and its changes are applied with `MERGE` (Listing 19.4)	Low load on the source; keeps deletes and updates; needs ordering and idempotence
Streaming	Events from a log such as Kafka are written continuously by a stream processor	Fresh data; produces many small files unless commits are batched

Two issues recur. First, *idempotence*: a job that fails and restarts must not duplicate data, which is achieved by writing with a stable key, by merging instead of appending, and by recording progress (offsets) in the same transaction as the data. Second, *late and out-of-order data*: events arrive after the time to which they belong, so partitions of recent days must remain writable, and aggregates must be recomputable. The synchronization techniques of Chapter 9 and the outbox and CDC patterns used for search and vector indexes apply here unchanged.

19.5.3 Catalogs

The *catalog* maps table names to the location of their current metadata, and in most deployments also stores permissions. Examples include the Hive Metastore (part of Apache Hive; see Siadati, 2021), cloud catalogs such as AWS Glue, and catalogs that implement the Iceberg REST specification or provide versioned, branch-like semantics. The catalog is a critical component: it is the single place where a commit becomes visible, and if it is unavailable, no table can be read or written even though the files are intact. It must be replicated, backed up, and protected like a database, and the choice of an open catalog interface determines whether several engines can share the same tables.

19.5.4 Compute Engines

Because tables are open, several engines can work on one copy of the data, each chosen for its workload: Spark for large batch transformations, Trino or a cloud warehouse for interactive SQL, Flink for streaming, DuckDB for a single machine and for development, and Python

libraries for machine learning. Engines differ in the level of support for each table format and feature, so compatibility must be tested for every combination that is used.

19.6 Partitioning, Clustering, and File Layout

Query speed in a lakehouse is largely a question of how many bytes the engine can avoid reading. A query reads a fraction of the table given by the product of three reductions: columns not needed (projection), partitions or files excluded by the filter (pruning), and row groups skipped by statistics. For example, for a table of 10 TB with 50 columns of similar size and 30 daily partitions, a query that uses 3 columns and one day reads about

$$10\,000\ \text{GB} \times \frac{3}{50} \times \frac{1}{30} = 20\ \text{GB}, \tag{19.1}$$

before additional skipping by statistics. Good layout multiplies these savings; poor layout removes them.

19.6.1 Partitioning

A *partition* groups the files of rows with the same value of a partition expression, typically the date, so that queries on the date read only matching groups. Guidelines: partition by a column that most queries filter on and that has a modest number of distinct values (days or months, not user identifiers); keep each partition large enough to contain files of reasonable size; and avoid partitioning by columns of high cardinality, which creates huge numbers of tiny partitions. Partitioning a table by day and by 1 000 customers creates 365 000 partitions per year, each with a few small files, which is far worse than no partitioning. For high-cardinality columns, use *clustering* instead.

19.6.2 Clustering and Sorting

Sorting the rows within files on frequently filtered columns, and multi-column techniques such as Z-ordering and space-filling curves, place similar values together so that file and row-group statistics become tight and many files can be skipped. Table formats provide commands and automatic mechanisms to cluster data during compaction. Clustering is complementary to partitioning: partition by date, cluster by customer.

19.6.3 File Size and the Small-Files Problem

Each file costs a request to open, an entry in the metadata, and a task of its own in the engine. A 10 TB table in files of 1 MB would consist of 10 million files; in files of 128 MB, about 78 000. Target sizes between a hundred megabytes and about a gigabyte are common, with row groups of similar order, subject to the engine and the workload. Small files arise naturally from streaming ingestion (a commit every few seconds), from over-partitioning, and from frequent small updates, and they are cured by *compaction*: a background job that rewrites many small files into fewer large ones, which is a transaction like any other (Section 19.9).

19.7 Governance, Security, and Privacy

Putting all data in one place raises the stakes of governance. The lakehouse must say who owns data, who may read it, where it came from, and whether it is good.

- **Access control.** Permissions on catalogs, schemas, and tables, and finer control at the level of columns (hiding or masking sensitive fields) and rows (restricting a user to a region or a tenant), enforced consistently by every engine. Where engines enforce policies themselves, a client that reads the files directly can bypass them, so storage-level access should be limited to trusted services, and credentials issued to engines should be scoped and short-lived.
- **Encryption** of data in transit and at rest, with keys under the organization's control for sensitive data.
- **Lineage and cataloging.** A record of which jobs produced which tables from which inputs, and a searchable description of tables, owners, and sensitivity, support impact analysis, audits, and trust.
- **Data quality.** Tests at the boundaries between zones (non-null keys, uniqueness, value ranges, referential integrity, freshness) catch errors where they enter. A failed check should stop promotion of data to the next layer, or divert bad records to a quarantine table.
- **Data contracts.** A table that others depend on has a declared schema, semantics, and service level, and changes follow a process, because a column renamed in silver can break dashboards in gold.

19.7.1 Deletion in a System That Keeps History

Privacy laws grant individuals the right to have their personal data erased, and retention policies require old data to be removed. Immutable files and time travel make this harder: a `DELETE` statement creates a new snapshot without the rows, but the old data files, and therefore the data, remain in the earlier snapshots and on storage until they are physically removed. Complete erasure requires three steps: delete the rows (which may rewrite files); expire the snapshots that still reference the old files, and remove the unreferenced files; and make sure that copies in other layers (silver and gold tables, caches, extracts, backups, and derived data such as embeddings, as noted in Chapter 14) are deleted too, within the time that the law and the organization allow.

Listing 19.6: Erasing a customer: delete, then physically remove old versions (Iceberg in Spark SQL).

```
DELETE FROM lake.silver.customers WHERE customer_id = 42;

-- Make older snapshots (which still contain the rows) unreadable ...
CALL lake.system.expire_snapshots(
  table => 'silver.customers',
```

```
  older_than => TIMESTAMP '2026-10-27 00:00:00');

-- ... and delete data files that no snapshot references any more
CALL lake.system.remove_orphan_files(table => 'silver.customers');
```

Designing for deletion from the start eases the task: keep personal data in few tables, key them by a stable identifier, separate it from analytical facts through pseudonymous keys, and record where each identifier is used.

19.8 Beyond Tables: Unstructured Data and Machine Learning

Because a lakehouse stores files, it can hold the data that traditional warehouses cannot: documents, images, audio, and logs, with tables that describe them (paths, metadata, labels). This makes it a natural platform for machine learning. Snapshots give reproducible training sets: a model records the table version it was trained on, and the same data can be recreated later. Feature tables computed in gold can be shared by training and serving. Embedding tables, produced by the models of Chapter 14, can be stored beside the content they describe and loaded into vector indexes; the lake is the system of record, and the vector database is a derived index that is rebuilt after model changes, with the model version recorded in the table. Query-time similarity search over very large collections stored on object storage follows the disaggregated design of Chapter 14, with the same trade-off of cost and elasticity against latency of cold data.

19.9 Operating a Lakehouse

A lakehouse shifts operational work from the database engine to the data platform team. The tasks recur for every table.

- **Compaction.** Rewrite small files into large ones and, where useful, cluster the data. Schedule it per table, concentrated on partitions that are no longer written to.
- **Snapshot expiry and file cleanup.** Every commit adds metadata and keeps old files alive for time travel. Expire snapshots older than the retention period and remove files that no snapshot references, including orphan files left by failed jobs; otherwise storage cost and metadata size grow without bound. Retention sets the window of time travel and recovery, and must be consistent with deletion obligations.
- **Metadata health.** Many snapshots, manifests, or log files slow query planning; compact metadata as well as data.
- **Concurrency.** Frequent writers to the same partitions cause commit conflicts and retries. Design jobs so that they write disjoint partitions, or serialize them, and keep commits short.

- **Cost.** Monitor bytes scanned per query and per team, storage growth, request counts, and idle compute. Most savings come from partitioning, file sizes, columns selected, and avoiding repeated full scans.

- **Monitoring.** Track table sizes, file counts and size distributions, snapshot counts, commit failure rates, data freshness, query latency percentiles, and the results of data-quality checks.

Listing 19.7: Routine maintenance in Iceberg and Delta Lake.

```
-- Iceberg (Spark): compact small files, then expire old snapshots
CALL lake.system.rewrite_data_files(table => 'shop.orders');
CALL lake.system.expire_snapshots(
  table => 'shop.orders',
  older_than => TIMESTAMP '2026-10-01␣00:00:00',
  retain_last => 5);

-- Delta Lake: compact and cluster, remove old files, undo a bad write
OPTIMIZE orders ZORDER BY (customer_id);
VACUUM orders RETAIN 168 HOURS;
RESTORE TABLE orders TO VERSION AS OF 12;
```

19.9.1 Backup and Disaster Recovery

Object stores are durable but not immune to human error, and time travel is not a backup, since it disappears with expiry and does not protect against deletion of the whole table or bucket. A recovery plan replicates both data files and metadata to another region or account, and the *order* matters: data files must arrive before the metadata that references them, or a restored table points to missing files. Catalog contents need their own backup. Test the restore procedure, including the catalog and permissions, and not only the storage.

19.10 Choosing an Approach

A sound process is: (1) list the workloads (reports, ad hoc SQL, streaming, machine learning) and their latency and freshness needs; (2) choose the table format and catalog by the engines and features required, and test the combinations in use; (3) design the zones and ownership, and write the data contracts of gold tables; (4) choose partitioning, clustering, and file-size targets from the query patterns, and measure the bytes scanned; (5) build idempotent ingestion with CDC or streaming where freshness demands it; (6) automate compaction, snapshot expiry, and quality checks from the start; (7) define access control, lineage, and the deletion procedure before storing personal data; and (8) plan backup, restore, and cost monitoring, and rehearse recovery.

Table 19.5: Choosing an architecture for analytical data.

Situation	**Suggested approach**	**Reasoning**
Modest data volume; analysts, BI; one team	A single analytical database or managed warehouse	Simplest to operate; the lake adds no value
Large volume, mixed structured and unstructured data; data science and BI on the same data	Lakehouse with an open table format	One copy; open formats; separate scaling of storage and compute
Several engines or vendors must share data; avoiding lock-in matters	Open table format with an open catalog interface	Engines can be replaced without moving data
Operational changes must reach analytics within minutes	CDC or streaming into a table format with merge-on-read or frequent compaction	Fresh data without overloading the source
Sub-second point lookups and high-concurrency updates	Operational database; publish to the lakehouse via CDC	Lakehouses are not OLTP systems
Strict deletion and audit requirements	Lakehouse with planned retention, expiry, and lineage	History is useful for audit and a liability for erasure; design both

Summary

Analytical systems evolved from data warehouses, which offer strong SQL and governance but require early modeling and couple storage with compute, through data lakes, which store any data cheaply as files but lack transactions and consistency, to lakehouses, which add a metadata layer to open files on object storage. Object stores provide cheap, durable, immutable objects without atomic multi-object operations or cheap renames, which leads to a design of few large immutable files with metadata that defines the table. Columnar formats such as Parquet store column chunks within row groups, with statistics in a footer that enable projection and predicate pushdown and strong compression. Open table formats (Iceberg, Delta Lake, Hudi) record which files belong to the table in each snapshot and publish changes by an atomic commit, giving ACID transactions with optimistic concurrency, snapshot isolation for readers, time travel, schema evolution, and row-level changes through merge, delete files, or copy-on-write. A lakehouse is a stack of replaceable layers: object storage, file and table formats, a catalog with governance, interchangeable compute engines, and consumers, with data organized in bronze, silver, and gold zones and loaded by batch, CDC, or streaming ingestion that must be idempotent and tolerate late data. Query cost depends on bytes read, which partitioning, clustering, and healthy file sizes minimize, and the small-files problem is solved by compaction. Governance covers access control, lineage, quality,

and contracts, and deletion is complicated by retained history, so erasure requires deleting rows, expiring snapshots, and removing files and derived copies. Lakehouses also support machine learning through reproducible snapshots and shared feature and embedding tables. Operations consist of compaction, snapshot expiry, concurrency design, cost monitoring, and tested backup that replicates data before metadata, and the architecture is appropriate for large, varied analytical data and unsuitable for low-latency transactional workloads.

Review Questions

1. How do data warehouses, data lakes, and lakehouses differ in schema handling, transactions, and coupling of storage and compute?

2. What problems of plain data lakes motivated table formats? Why is a collection of Parquet files in a directory not a table?

3. Name four properties of object storage and explain their consequences for analytical systems.

4. Why is object-store rename a problem for commit protocols? What primitive replaces it?

5. Explain why columnar formats suit analytical queries. Describe projection pushdown, predicate pushdown, and the role of the footer.

6. Describe the structure of a Parquet file. What are row groups and column chunks?

7. How does an Iceberg commit work? Which single step makes it atomic?

8. How does Delta Lake obtain the state of a table? What are checkpoints for?

9. Compare copy-on-write and merge-on-read. When is each preferable?

10. Explain optimistic concurrency in table formats. When does it work well, and when does it lead to many retries?

11. What are time travel and snapshot isolation? Give two uses of time travel.

12. Describe the bronze, silver, and gold zones, and explain why raw data is kept unchanged.

13. What does a catalog do, and why is it a critical component?

14. Why is partitioning by a high-cardinality column harmful? What is the alternative?

15. What is the small-files problem, what causes it, and how is it solved?

16. Why is deleting personal data harder in a lakehouse than in a relational database? List the steps for complete erasure.

Exercises

1. **Bytes scanned.** A 40 TB table has 80 columns of similar size and is partitioned by day over 400 days. A query uses 4 columns and a filter on a range of 7 days. Estimate the data read, ignoring statistics, and the data read if compression of the selected columns is twice as strong as the average. By what factor is this smaller than reading the whole table?

2. **Partitioning.** A table receives 2 billion rows a day from 5 000 stores; a row takes about 100 bytes in a compressed Parquet file. Compare partitioning by day, by day and store, and by month, with respect to partition count per year, average file size if each partition is written once a day, and query performance for (a) daily sales of all stores and (b) one store over a year. Propose a layout with clustering.

3. **Small files.** A streaming job commits every 10 seconds and writes one 2 MB file per commit into the current day's partition. Compute the number of files and the data volume after one day and after 30 days. If compaction targets 256 MB files, how many files remain per day after compaction? Describe how often compaction should run and what conflicts it may cause with the writer.

4. **Parquet.** A column of country codes with 200 distinct values and 100 million rows is stored in a Parquet file as 2-byte strings. Explain how dictionary and run-length encoding reduce the size, and why sorting the file by country makes both the encoding and predicate pushdown more effective.

5. **Commit conflict.** Two jobs run concurrently on an Iceberg table: job A deletes the rows of one customer, job B appends the day's new rows. Describe the steps of each commit and decide whether the second commit succeeds, fails, or must be retried when (a) the jobs touch different files and (b) job B also rewrites a file that job A has rewritten. State the guarantee that readers have in each case.

6. **Medallion design.** Design the bronze, silver, and gold tables for an online shop with orders, customers, and web events. List the tables, keys, partitioning, ingestion mode, and quality checks of each layer, and define the contract of one gold table.

7. **Upserts.** Rewrite the MERGE statement of Listing 19.4 for a CDC feed that may contain several changes to the same order in one batch, in a nondeterministic order. State what information the feed needs to carry, and how to select the latest change per key before merging.

8. **Erasure.** A customer requests deletion. The customer's data appears in a silver table, three gold aggregates, a feature table, a vector index of support messages, and backups. Write a procedure with the steps, the verification, and the time bounds for each system. Discuss the effect of a 30-day time-travel retention.

9. **Disaster recovery.** Design the replication of a lakehouse to a second region. Explain why the order of replication of data and metadata matters, what happens if the catalog

is replicated before the data, how to measure the recovery point and recovery time, and how to test it.

10. **Choosing.** Three teams propose (a) a Delta Lake on a single cloud with Spark only, (b) an Iceberg lakehouse with Spark, Trino, and a cloud warehouse, and (c) a managed warehouse with no lake. For a company with 50 TB of structured data, 2 PB of images, and a requirement to avoid vendor lock-in, compare the three on cost, flexibility, governance, and operational effort, and state a recommendation with its assumptions.

References

- Apache Hudi Documentation. https://hudi.apache.org/docs/
- Apache Iceberg Documentation and Table Specification. https://iceberg.apache.org/
- Apache Parquet Documentation. https://parquet.apache.org/docs/
- Armbrust, M., Das, T., Sun, L., et al. (2020). Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores. *Proceedings of the VLDB Endowment*, 13(12), 3411–3424.
- Armbrust, M., Ghodsi, A., Xin, R., & Zaharia, M. (2021). Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics. In *Proceedings of CIDR 2021*.
- Behm, A., Palkar, S., Agarwal, U., et al. (2022). Photon: A Fast Query Engine for Lakehouse Systems. In *Proceedings of ACM SIGMOD '22*, 2326–2339.
- Delta Lake Documentation and Protocol Specification. https://docs.delta.io/
- Kimball, R., & Ross, M. (2013). *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling* (3rd ed.). Wiley.
- Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.
- Melnik, S., Gubarev, A., Long, J. J., et al. (2010). Dremel: Interactive Analysis of Web-Scale Datasets. *Proceedings of the VLDB Endowment*, 3(1–2), 330–339.
- Reis, J., & Housley, M. (2022). *Fundamentals of Data Engineering*. O'Reilly Media.
- Siadati, S. (2021). *Big Data Analytics and Cloud Computing*. Zenodo. https://doi.org/10.5281/zenodo.16907167
- Stonebraker, M., Abadi, D. J., Batkin, A., et al. (2005). C-Store: A Column-oriented DBMS. In *Proceedings of VLDB 2005*, 553–564.
- Zaharia, M., Xin, R. S., Wendell, P., et al. (2016). Apache Spark: A Unified Engine for Big Data Processing. *Communications of the ACM*, 59(11), 56–65.

Chapter 20

Streaming Systems and Event-Driven Architectures

Learning Objectives

After reading this chapter, you will be able to:

- Explain the difference between batch and stream processing, and between events, commands, and state.
- Describe a distributed log (topics, partitions, offsets, consumer groups, replication) and the ordering and durability guarantees that it provides.
- Distinguish at-most-once, at-least-once, and exactly-once processing, and build exactly-once *effects* with idempotence and transactions.
- Process streams with windows, event time, watermarks, and managed state, and write streaming SQL for them.
- Apply the main event-driven patterns: publish–subscribe, event-carried state transfer, event sourcing, CQRS, the outbox, change data capture, and sagas.
- Manage schemas and their evolution, and handle retries, dead-letter queues, and reprocessing.
- Size, monitor, and secure a streaming platform, and handle deletion and privacy in append-only logs.
- Decide when streaming and events are the right tool, and when a simpler design is better.

20.1 From Batches to Streams

Chapter 19 described systems that analyze data after it has been collected. Many applications cannot wait: a payment must be checked for fraud while the customer is at the till, a sensor alarm must reach an operator within seconds, and a search index should reflect a product change almost at once. These applications treat data not as a table to be queried but as a *stream*: an unbounded sequence of events that arrive continuously and must be processed as they come.

The distinction between batch and stream is one of *boundedness* and *latency*. A batch job

processes a finite, complete set of data and produces its result when it finishes; a stream job processes an infinite sequence, never finishes, and produces results continuously. Table 20.1 summarizes the contrast. The two are not opposed: a batch is a bounded stream, and good streaming systems unify them (Akidau et al., 2015).

Table 20.1: Batch and stream processing compared.

Aspect	**Batch**	**Stream**
Input	Finite, complete data set	Unbounded sequence of events
Result	Produced when the job ends	Updated continuously
Latency	Minutes to hours	Milliseconds to seconds
Completeness	All data is present at start	Data may arrive late or out of order
Failure handling	Rerun the job	Recover state and continue without gaps or duplicates
Typical uses	Reports, model training, large transformations	Monitoring, fraud detection, notifications, synchronization of derived data

20.1.1 Why Events

An *event* is an immutable record that something happened at a point in time: an order was placed, a sensor reported a temperature, a user clicked. Events differ from the *state* that databases store. State is what is true now; events are the history from which state follows. Treating events as the primary data has practical consequences that this chapter develops: several systems can consume the same events independently; a new system can be added later by replaying the past; and the sequence of events provides an audit trail that a table of current values lacks.

An event-driven architecture builds applications around the *production, transport, and reaction to events*. Services do not call each other synchronously to ask for data; they publish what happened, and interested services react in their own time. This reduces coupling between services and absorbs bursts, at the price of eventual consistency, harder debugging, and the need to design for duplicates, reordering, and failure (Chapter 8).

20.2 Events, Commands, and Messages

Three ideas are often confused:

- **Event.** A fact in the past, named in the past tense (`OrderPlaced`). It cannot be rejected; consumers may ignore it but cannot change it.
- **Command.** A request to do something in the future (`PlaceOrder`). It is addressed to one handler, which may refuse it.

- **Message.** The transport envelope that carries either of them.

The distinction guides design. Publishing events ("this happened") lets many consumers react without the publisher knowing them; sending commands ("do this") couples the sender to a handler and expects a result.

20.2.1 Designing an Event

A good event is self-describing and stable. It carries a unique identifier (so consumers can detect duplicates), a type and a schema version, a key that identifies the entity it concerns (which determines ordering), the time at which it occurred, and a payload with the data that consumers need.

Listing 20.1: An event with an envelope and a payload.

```
{
  "eventId": "7c1f8d0a-5a53-4c0e-b0a1-2f6f8f3d9a11",
  "type": "OrderPlaced",
  "version": 2,
  "key": "order-1001",
  "occurredAt": "2026-10-27T09:15:00.250Z",
  "producer": "checkout-service",
  "payload": {
    "orderId": "order-1001",
    "customerId": 42,
    "total": 129.89,
    "currency": "EUR",
    "items": [ { "sku": "A-17", "qty": 2 }, { "sku": "B-02", "qty": 1 } ]
  }
}
```

Two choices deserve thought. The *key* decides which events are kept in order (all events of one order share a key), as Section 20.3 explains. The *size of the payload* decides the coupling: a thin event ("order 1001 changed") forces consumers to call the producer for details, while a fat event carries the data that consumers need (event-carried state transfer, Section 20.7) but must be designed as a public contract.

20.2.2 Event Time and Processing Time

Every event has two times. The *event time* is when it happened, as recorded by the producer; the *processing time* is when the system handles it. They differ because of network delays, buffering, offline devices, and failures, and they differ unpredictably: a mobile app may deliver the events of a flight an hour after the landing. Computations about the real world ("orders per minute") must use event time to be correct, and that requires mechanisms to decide when all events of a period have arrived (Section 20.5).

20.3 The Distributed Log

The central component of most streaming platforms is the *distributed log*, an append-only, ordered, durable sequence of records, replicated across machines (Kreps, 2013). Apache Kafka is the best-known system; Redpanda, Apache Pulsar, and cloud services such as Amazon Kinesis and Google Pub/Sub implement variants of the idea.

20.3.1 Topics, Partitions, and Offsets

Records are organized into *topics*, and each topic is divided into *partitions*. A partition is an ordered log in which every record has a sequential number, its *offset*. Producers append to the end; consumers read from any offset and remember where they are (Figure 20.1). Records are not deleted when read: they stay until a retention rule removes them, which is what allows several consumers, replay, and the late addition of new consumers (Kreps et al., 2011).

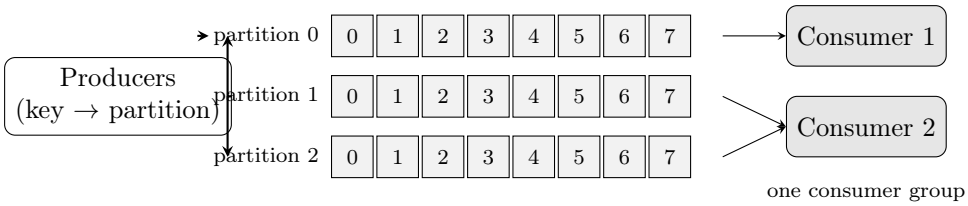


Figure 20.1: A topic with three partitions. Records are appended at the end; a key decides the partition. Within a consumer group each partition is read by exactly one consumer.

20.3.2 Ordering

A log guarantees order *only within a partition*. Records with the same key are sent to the same partition (typically by hashing the key and taking the remainder modulo the number of partitions), so all events of one order, account, or device are read in the order they were written. There is no order across partitions. This is a fundamental trade-off: more partitions mean more parallelism, but ordering exists only per key, so the key must be chosen so that the events that must be ordered share it. A popular key (a "hot key") overloads its partition, and changing the number of partitions changes the mapping of keys and therefore breaks the continuity of order for existing keys.

20.3.3 Consumer Groups and Parallelism

Consumers that cooperate form a *consumer group*; the partitions of a topic are divided among the members, so each partition is read by exactly one consumer in the group. The group scales by adding consumers, up to the number of partitions (extra consumers are idle). Different groups read the same topic independently, each with its own position, so a search indexer, a fraud detector, and a lakehouse loader can all consume the same events at their own pace. When a consumer joins or fails, the group *rebalances* and partitions move, which briefly pauses processing and requires consumers to hand over their progress correctly.

20.3.4 Replication and Durability

Each partition is replicated on several brokers: one is the *leader*, which serves reads and writes, and the others are *followers* that copy the log. Followers that keep up form the *in-sync replica set* (ISR). A producer that waits for acknowledgment from all in-sync replicas (`acks=all`), on a topic with replication factor 3 and a minimum of two in-sync replicas, gets a write that survives the loss of any one broker; if fewer than two replicas are in sync, writes are refused rather than risk loss, a deliberate choice of consistency over availability (Chapter 7). Modern Kafka manages cluster metadata with its own consensus protocol (KRaft), which replaced the external coordination service used earlier (Chapter 9).

20.3.5 Retention and Compaction

A topic keeps records according to a *retention* rule, by age or by size (for example seven days). Alternatively, a topic can be *compacted*: the log keeps at least the latest record for each key, and a record with a null value (a *tombstone*) marks the key as deleted. A compacted topic is a changelog from which the current state of every key can be rebuilt, which makes it a durable table (Section 20.6).

Listing 20.2: Creating a replicated topic with retention (Kafka command line).

```
kafka-topics.sh --bootstrap-server broker:9092 --create --topic orders \
  --partitions 12 --replication-factor 3 \
  --config min.insync.replicas=2 \
  --config retention.ms=604800000          # seven days
```

20.3.6 Logs and Queues

Traditional message queues (RabbitMQ, ActiveMQ, Amazon SQS) deliver each message to one consumer and delete it after acknowledgment; they excel at distributing tasks among workers with flexible routing. Log-based systems retain records, allow replay, and let many consumers read independently, and they provide high throughput and ordering per key. Table 20.2 lists the differences.

20.4 Delivery Guarantees and Exactly-Once Effects

When a producer sends a record and the network fails before the acknowledgment arrives, the producer does not know whether the record was stored. The same uncertainty applies to a consumer that processes a record and fails before recording its position. Three semantics result (Chapter 8 discussed the same problem for requests):

- **At-most-once.** Do not retry, and records may be lost. Acceptable only for data whose loss is harmless.
- **At-least-once.** Retry until acknowledged, and records may be *duplicated*. The usual default.

Table 20.2: Message queues and distributed logs compared.

Aspect	Message queue	Distributed log
After consumption	Message is removed	Record stays until retention expires
Consumers	Compete for messages; each message to one worker	Independent groups, each reading everything
Replay	Not possible once acknowledged	Reset the offset and read again
Ordering	Per queue, weakened by parallel consumers	Per partition (per key)
Strengths	Task distribution, routing, per-message acknowledgment, delays	Throughput, replay, many consumers, stream processing

- **Exactly-once.** Each record affects the result once. It cannot be obtained by the network alone; it is achieved by combining at-least-once delivery with mechanisms that eliminate the effect of duplicates.

20.4.1 Building Exactly-Once Effects

- **Idempotent producer.** The broker detects retried sends by a producer identifier and a sequence number and discards duplicates within a partition, so retries do not duplicate records.

- **Transactions.** A processor that reads from topics and writes to topics can commit its output records and its input offsets *atomically*, so that a failure either produces both or neither; consumers configured to read only committed records never see partial results. This covers read–process–write cycles *within* the log system.

- **Idempotent consumers.** For effects outside the log (a database write, an email, a payment), make the operation idempotent: use the event identifier as a key and ignore a second delivery, or write with an upsert keyed by the entity. An email cannot be unsent, so deduplication state must be stored with the effect.

- **Checkpointing in stream processors.** Engines such as Apache Flink periodically take consistent snapshots of all operator state together with source positions, using markers that flow through the dataflow (Carbone et al., 2015); after a failure, they restore the last snapshot and replay from the recorded positions. Transactional or idempotent sinks make the output exactly-once end to end.

The practical formula is: *exactly-once effect = at-least-once delivery + idempotent or transactional processing.* A consumer is written to be correct under redelivery, and the platform's guarantees reduce its frequency.

Listing 20.3: Producer and consumer with idempotence and manual offset commits (Python, confluent-kafka).

```
from confluent_kafka import Producer, Consumer

producer = Producer({
    "bootstrap.servers": "broker:9092",
    "enable.idempotence": True,           # no duplicates from retries
    "acks": "all",                        # wait for all in-sync replicas
})
producer.produce("orders", key="order-1001", value=event_json)
producer.flush()

consumer = Consumer({
    "bootstrap.servers": "broker:9092",
    "group.id": "billing",
    "enable.auto.commit": False,          # commit only after processing
    "auto.offset.reset": "earliest",
})
consumer.subscribe(["orders"])
while True:
    msg = consumer.poll(1.0)
    if msg is None or msg.error():
        continue
    handle(msg)                           # must be idempotent (use the event
    consumer.commit(message=msg)          # at-least-once: commit after the wo
```

Committing the offset *after* processing gives at-least-once (a crash between the two repeats the record); committing *before* gives at-most-once (a crash loses it).

20.5 Stream Processing

A *stream processor* reads events from the log, transforms them, and writes results to another log, a database, or a service. Operations are of two kinds. *Stateless* operations (filter, map, enrich from a fixed rule) treat each event alone and scale trivially. *Stateful* operations (counting, aggregating, joining, detecting patterns) must remember information across events, and state is the hard part: it must be partitioned like the stream, stored locally for speed, checkpointed for recovery, and bounded in size. The main engines are Apache Flink, Kafka Streams (a library embedded in applications), and Spark Structured Streaming, with streaming SQL available in several of them.

20.5.1 Windows

Because a stream never ends, aggregates are computed over *windows*. Table 20.3 lists the common kinds.

Table 20.3: Window types.

Window	Definition	Example
Tumbling	Fixed length, no overlap; each event belongs to one window	Orders per minute
Sliding (hopping)	Fixed length, advancing by a smaller step; windows overlap	Average over the last 10 minutes, updated every minute
Session	Closed by a gap of inactivity per key; length varies	A user's visit on a website
Global	One window for the whole stream, with a custom trigger	Running total

20.5.2 Event Time, Watermarks, and Late Data

To compute a window in event time, the processor must decide when it has seen all its events. It cannot know for certain, so it estimates progress with a *watermark*: a timestamp that asserts "no more events with an earlier event time are expected." A common strategy sets the watermark to the largest event time seen so far minus a tolerated delay. For example, with a tolerance of five seconds, after seeing events at 10:00:01, 10:00:03, and 10:00:02 (out of order), the watermark is $10{:}00{:}03 - 5\,\mathrm{s} = 09{:}59{:}58$. The window $[10{:}00{:}00, 10{:}01{:}00)$ is closed and its result emitted when the watermark passes 10:01:00, that is, when an event with a time of 10:01:05 or later has arrived.

Events that arrive after their window closed are *late*. The system may drop them, send them to a side output for separate handling, or keep windows open for an additional *allowed lateness* and emit corrected results. The choice trades latency against completeness, and it is a business decision: a real-time dashboard may accept approximate results, while billing needs complete ones, possibly computed again by a batch job.

Listing 20.4: Streaming SQL (Flink): a Kafka source with a watermark and revenue per one-minute tumbling window.

```
CREATE TABLE orders (
  order_id STRING,
  customer_id BIGINT,
  total DECIMAL(12,2),
  order_ts TIMESTAMP(3),
  WATERMARK FOR order_ts AS order_ts - INTERVAL '5' SECOND
) WITH (
  'connector' = 'kafka',
  'topic' = 'orders',
  'properties.bootstrap.servers' = 'broker:9092',
  'format' = 'json',
  'scan.startup.mode' = 'earliest-offset'
);
```

```
SELECT window_start, window_end,
       COUNT(*) AS orders,
       SUM(total) AS revenue
FROM TABLE(TUMBLE(TABLE orders, DESCRIPTOR(order_ts), INTERVAL '1' MINUTE))
GROUP BY window_start, window_end;
```

20.5.3 Joins and Enrichment

Streams can be joined, but unlike tables they have no end, so joins need bounds. A *stream–stream join* matches events within a time interval (a payment within 15 minutes of an order) and keeps both sides in state until the interval expires. A *stream–table join* enriches each event with the current row of a table, such as customer data, which is itself kept up to date from a changelog (Section 20.6); the result depends on the version of the table at the time of the event, and reprocessing may give different answers unless versions are tracked.

Listing 20.5: A stream–stream interval join (Flink SQL).

```
SELECT o.order_id, o.total, p.pay_ts
FROM orders AS o
JOIN payments AS p
  ON o.order_id = p.order_id
 AND p.pay_ts BETWEEN o.order_ts AND o.order_ts + INTERVAL '15' MINUTE;
```

20.5.4 State and Backpressure

State is held in the processor, often in an embedded key–value store on local disk, and is checkpointed to durable storage. State that grows without bound (a join without a time limit, a session window for inactive users) eventually exhausts memory or disk, so every stateful operator needs a retention rule. When a downstream stage is slower than the input, *backpressure* propagates upstream and slows the source, which is the correct behavior since the log can buffer; the symptom to watch is growing *consumer lag*, defined as the difference between the latest offset written and the offset processed.

20.6 Streams and Tables: Change Data Capture

A stream and a table are two views of the same thing. Applying a stream of changes (insert, update, delete) to a table yields the table's current state; recording every change to a table yields a stream. This *stream–table duality* explains why logs and databases fit together (Kleppmann, 2017). A compacted topic is a table, and a table's *changelog* is a stream.

Change data capture (CDC) turns a database into a source of events by reading its transaction log (the write-ahead log of PostgreSQL, the binary log of MySQL) and publishing each committed change; tools such as Debezium do this for many databases. CDC avoids the load of polling, preserves the order of commits, and captures deletes. It is the standard way to feed search indexes, caches, vector indexes, data lakes, and other services from an operational database without changing the application (Figure 20.2).

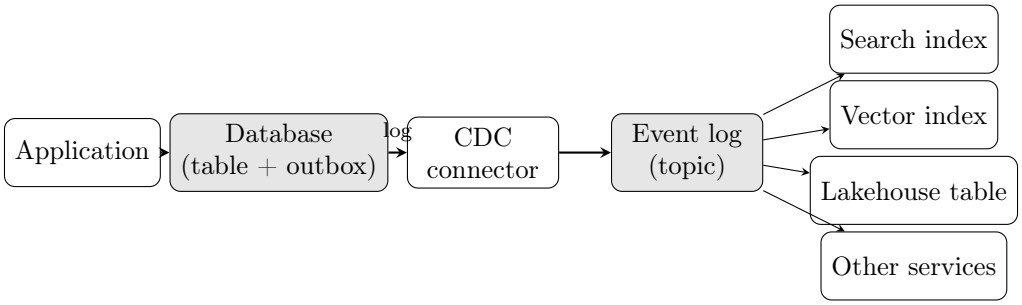


Figure 20.2: Change data capture and the outbox pattern. The application writes its data and an event in one transaction; the connector reads the database log and publishes the events, from which independent consumers build derived data.

20.6.1 The Dual-Write Problem and the Outbox

An application that updates its database and then publishes an event to a broker performs two writes that cannot be made atomic. If the process fails between them, the database and the event stream disagree: the order exists but no event announces it, or the event exists for an order that was rolled back. The *transactional outbox* pattern solves this by writing the event to an `outbox` table *in the same database transaction* as the business data; a CDC connector (or a poller) then publishes the outbox rows to the log, at least once. Consumers deduplicate by event identifier.

Listing 20.6: The transactional outbox: business data and event in one transaction.

```
BEGIN;

INSERT INTO orders (order_id, customer_id, total, status)
VALUES ('order-1001', 42, 129.89, 'placed');

INSERT INTO outbox (event_id, aggregate_id, type, payload, created_at)
VALUES (gen_random_uuid(), 'order-1001', 'OrderPlaced',
       '{"orderId": "order-1001", "customerId": 42, "total": 129.89}'::jsonb,
       now());

COMMIT; -- both rows become visible together, or neither does
```

The same pattern keeps the search indexes of Chapter 13 and the vector indexes of Chapter 14 synchronized with their sources, and loads the tables of Chapter 19 with merge statements that apply the changes.

20.7 Event-Driven Architecture Patterns

Table 20.4 lists the patterns that recur in event-driven systems, and the following subsections discuss the most important.

Table 20.4: Common event-driven patterns.

Pattern	**Idea**	**Main trade-off**
Publish–subscribe	Producers publish to a topic; any number of subscribers react	Loose coupling; hard to see who depends on what
Event notification	A thin event says that something changed; consumers fetch details	Small events; extra calls and coupling to the source
Event-carried state transfer	The event carries the data that consumers need	Autonomy of consumers; events become contracts
Event sourcing	The event log is the system of record; state is derived	Full history; complexity of replay and evolution
CQRS	Separate models for writing and for reading	Fast, tailored reads; eventual consistency
Saga	A long process as local transactions with compensations	No distributed transaction; complex failure handling

20.7.1 Event Sourcing

In *event sourcing*, the state of an entity is not stored directly. Instead, the application stores every event that changed it, and the current state is computed by replaying them. A bank account is the classic example:

Listing 20.7: An account as a sequence of events; the balance is derived by replay.

```
{ "seq": 1, "type": "AccountOpened", "accountId": "A1", "owner": "Ada" }
{ "seq": 2, "type": "MoneyDeposited", "accountId": "A1", "amount": 500.00 }
{ "seq": 3, "type": "MoneyWithdrawn", "accountId": "A1", "amount": 120.00 }
// Replay: balance = 0 + 500.00 - 120.00 = 380.00
```

The benefits are a complete audit trail, the ability to reconstruct the state at any past time, and new read models built later by replaying the history. The costs are real: replay gets slower as histories grow, so *snapshots* of the state are stored periodically; old event formats must be supported forever or migrated (Section 20.8); queries over current state need a read model; and correcting an error means appending a compensating event, never editing history. Event sourcing suits domains where history is the business (finance, logistics, compliance); it is a heavy choice for ordinary create–read–update–delete applications.

20.7.2 CQRS

Command Query Responsibility Segregation separates the model that handles commands (writes, with business rules) from the models that serve queries. Events published by the write side update one or more read models, each shaped for its queries: a table for lookups, a search index for text, a denormalized document for a screen. It combines naturally with event

sourcing and with the denormalization of Chapters 16 and 17. Its price is that read models are eventually consistent with the write side, which the user interface must accommodate (for example by showing the result of a write from the command response and not from the read model).

20.7.3 Sagas

A business process that spans several services, such as placing an order that reserves stock, takes payment, and schedules shipment, cannot use a single ACID transaction without coupling the services tightly. A *saga* breaks it into a sequence of local transactions, each followed by an event, and defines a *compensating action* for each step to undo it if a later step fails (Garcia-Molina and Salem, 1987). Table 20.5 shows the steps of an order saga.

Table 20.5: An order saga with compensating actions.

Step	Action	On success	Compensation if later step fails
1	Create order (pending)	`OrderCreated`	Cancel the order
2	Reserve stock	`StockReserved`	Release the stock
3	Charge payment	`PaymentCaptured`	Refund the payment
4	Confirm order and schedule shipment	`OrderConfirmed`	(final step)

Sagas come in two styles. In *choreography*, each service reacts to the events of the others and emits its own, with no central controller: simple for short flows, but the overall logic is spread over many services and hard to follow. In *orchestration*, a coordinator keeps the state of the process and sends commands to the participants: easier to understand and monitor, at the price of a central component. Sagas provide *atomicity by compensation* but no isolation: other transactions can see intermediate states (an order that is pending, stock that is reserved and later released), so the design must tolerate them. Helland (2007) describes the thinking behind this approach: between independent entities, work is coordinated by messages that carry the intent and are processed idempotently.

20.7.4 Retries, Dead-Letter Queues, and Poison Messages

A consumer fails on a record for transient reasons (a database is down) or permanent ones (malformed data). Transient failures are retried with *exponential backoff* and a limit; but a record that never succeeds, a *poison message*, would block its partition forever if the consumer keeps retrying it. The usual remedy is to move such records after a bounded number of attempts to a *dead-letter topic* with the error information, raise an alert, and continue. Dead-letter records are reviewed, repaired, and replayed. Retrying on a separate retry topic keeps the main partition flowing but gives up the original order for the retried records, which is acceptable only if their processing does not depend on order.

20.8 Schemas and Their Evolution

In a stream, producers and consumers are developed and deployed independently and live for years, and events written long ago stay in the log. The *schema* of events is therefore a long-lived contract. A *schema registry* stores the versions of each schema, assigns identifiers that are carried in every message, and checks each new version against a *compatibility rule* before it is accepted.

- **Backward compatible:** consumers using the new schema can read data written with the old one. Adding an optional field with a default, or removing a field, is backward compatible. Consumers are upgraded first.
- **Forward compatible:** consumers using the old schema can read data written with the new one. Producers are upgraded first.
- **Full:** both. The safest rule for logs that are replayed.

Binary formats with schemas (Avro, Protocol Buffers) are compact and support these rules; JSON is flexible but offers no guarantee unless it is paired with JSON Schema and the same checks.

Listing 20.8: Evolving an Avro schema: a new field with a default keeps old data readable.

```
{
  "type": "record", "name": "OrderPlaced",
  "fields": [
    { "name": "orderId", "type": "string" },
    { "name": "customerId", "type": "long" },
    { "name": "total", "type": "double" },
    { "name": "currency", "type": "string", "default": "EUR" } // added in v2
  ]
}
```

Breaking changes (renaming or retyping a field, removing a required one) require a new event type or topic and a period in which both exist. Because logs are replayed, the rule is stricter than for request–response APIs: *every* old version still in the log must remain readable, or must be migrated by rewriting the topic.

20.9 Architectures for Combining Batch and Stream

Two architectures describe how historical and real-time processing are combined. The *Lambda architecture* runs a batch layer that recomputes accurate results from all data and a speed layer that provides fast, approximate results for recent data, merging them at query time (Marz and Warren, 2015). It gives both accuracy and speed, but requires two codebases that must produce the same results, which is costly to maintain. The *Kappa architecture* uses a single stream pipeline: the log retains the history, and reprocessing is done by starting a new instance of the job from the beginning of the log and switching consumers to its output when it catches up (Kreps, 2014). It removes the duplicated logic but requires a log with

long retention (or access to history in a lake) and a stream engine that can process history efficiently.

Today the boundary is blurred by systems that unify the two. Streaming engines can process bounded input as a batch; lakehouse tables (Chapter 19) can be written continuously by streaming jobs and read in batch; and the lake serves as the long-term store from which a stream job can be replayed. A pragmatic design lands the raw event stream in a lakehouse table, runs streaming jobs for the use cases that need low latency, and uses batch recomputation from the lake for corrections and backfills.

20.10 Operating Streaming Systems

20.10.1 Capacity Planning

The sizing of a log has three parts. *Partitions* are chosen from the target throughput and the slowest stage: with a required throughput of 200 MB/s, a measured capacity of 10 MB/s per partition for producers and 20 MB/s for the consumer, at least $\max(200/10,\ 200/20) = 20$ partitions are needed, plus room for growth, since adding partitions later changes the key mapping. *Storage* is throughput times retention times replication: at 50 MB/s with seven days of retention and a replication factor of three,

$$50\,\text{MB/s} \times 604\,800\,\text{s} \approx 30.2\,\text{TB}, \qquad 30.2\,\text{TB} \times 3 \approx 90.7\,\text{TB}$$

of disk across the cluster, before headroom. *Consumers* are sized from processing time per record and the required throughput, with a limit of one consumer per partition in a group.

20.10.2 Monitoring

Consumer lag, both in records and in time, is the principal health signal: a growing lag means that a consumer cannot keep up. If producers write 5 000 records per second and a consumer processes 4 000, the lag grows by 1 000 per second, or 600 000 records in ten minutes, and the consumer can never catch up without more capacity. Further signals are under-replicated partitions and shrinking in-sync replica sets, broker disk and network use, produce and fetch latency, rebalance frequency, the rate of dead-letter records, end-to-end latency from event time to result, and, for stream processors, checkpoint duration and size, state size, backpressure, and the gap between watermark and wall-clock time.

20.10.3 Reprocessing and Replay

Because the log retains events, errors can be repaired by *replay*: fix the code, reset the consumer group's offsets to an earlier point or to a timestamp, and process again. Replay requires that processing be idempotent or that outputs be written to a new target and switched over, and it requires that the history still exists within retention. Tools for resetting offsets, for running a new consumer group beside the old, and for comparing results should be part of the platform from the start.

20.10.4 Security and Privacy

- **Transport and access.** Encrypt connections, authenticate clients, and restrict each client to the topics and operations it needs with access-control lists.
- **Sensitive data.** Events are copied to many consumers and retained for long periods. Keep personal data out of events where possible, or encrypt sensitive fields.
- **Erasure.** An append-only log cannot edit history. Options are short retention for personal data, compacted topics in which a tombstone removes a key's data (after compaction runs), and *crypto-shredding*: encrypting each subject's data with an individual key that is deleted on request, which makes all copies unreadable at once. Derived data in lakehouses and vector indexes must be erased too (Chapters 19 and 14).
- **Poisoning and abuse.** A producer that can write to a topic influences every consumer. Authenticate producers, validate events against schemas, and treat event content as untrusted input in consumers, especially when it is later given to language models.

20.10.5 Multi-Region Operation

Replicating topics between regions, by tools that mirror topics or by clusters that span regions, supports disaster recovery and local reads, but it is asynchronous in most designs, so a failover can lose the most recent events and consumers must handle offsets that differ between clusters. The trade-offs are those of replication in Chapter 9: synchronous replication across distance costs latency on every write, and asynchronous replication costs a recovery point.

20.11 Choosing an Approach

Events and streams solve real problems, and they add real complexity. They should be used where their advantages matter, and the simplest design that works is usually the better one.

Common mistakes include using events for interactions that need an answer, splitting a system into many small services that exchange events and become a distributed monolith, ignoring ordering by choosing a key at random, treating the log as a database without planning for retention and schema evolution, assuming exactly-once delivery without making consumers idempotent, and putting personal data into an immutable log without an erasure plan.

A sound process is: (1) identify the facts that matter to several parties and model them as events, with keys, identifiers, and versioned schemas; (2) choose the partition key from the ordering requirement and check it for hot keys; (3) decide the delivery semantics per consumer and make every effect idempotent; (4) use the outbox or CDC to publish events from databases; (5) choose windows, event-time semantics, and the allowed lateness from the business requirement for completeness versus latency; (6) define compatibility rules, a registry, retry and dead-letter handling, and replay procedures; (7) size partitions, storage, and consumers from measured throughput, and monitor lag and end-to-end latency; and (8) secure the platform and design deletion before the first personal data enters the log.

Table 20.6: Choosing between request–response, queues, and streams.

Situation	Suggested approach	Reasoning
A caller needs an immediate answer	Synchronous request–response (HTTP, gRPC)	Events add delay and complexity for no benefit
Work items distributed to workers, each processed once, with retries and delays	Message queue	Per-message acknowledgment and routing
Many consumers need the same facts; replay and new consumers later	Distributed log	Retention, independent consumer groups, ordering per key
Derived data (search, cache, vector index, lake) must follow a database	CDC or transactional outbox into a log	Avoids dual writes; preserves order and deletes
Continuous aggregates and alerts in seconds	Stream processor with event-time windows	Low latency; state and watermarks handled by the engine
Business process across services without a shared transaction	Saga with orchestration or choreography	Atomicity by compensation; accept intermediate states
Hourly or daily results are sufficient	Batch processing on a lakehouse	Simpler to build, test, and operate

Summary

Streaming systems process unbounded sequences of events continuously, with latencies of milliseconds to seconds, whereas batch systems process bounded data sets; the two can be unified, since a batch is a bounded stream. Events are immutable facts, distinct from commands and from the state held in databases, and event-driven architectures decouple services by letting them publish facts and react in their own time, at the price of eventual consistency. The distributed log, exemplified by Kafka, stores events in partitioned, replicated, append-only topics with offsets; it preserves order only within a partition, so the key decides which events stay ordered; consumer groups divide partitions among consumers, independent groups read independently, and retention or compaction determines how long data is kept. Delivery is at-most-once, at-least-once, or exactly-once, and exactly-once *effects* are built from at-least-once delivery with idempotent producers, transactions, checkpointing, and idempotent consumers. Stream processors keep partitioned, checkpointed state, aggregate over tumbling, sliding, and session windows in event time, use watermarks to decide when results are complete, handle late data by dropping, side outputs, or allowed lateness, and join streams within bounded intervals. Streams and tables are dual: change data capture publishes database changes, and the transactional outbox solves the dual-write problem. Patterns such

as event-carried state transfer, event sourcing, CQRS, and sagas organize applications around events, with trade-offs in complexity and consistency, and dead-letter topics isolate poison messages. Schemas are long-lived contracts that a registry and compatibility rules protect. Lambda and Kappa architectures combine batch and stream, and modern designs land streams in lakehouse tables and replay from them. Operations center on sizing partitions and storage, monitoring lag and latency, replay, security, erasure in append-only logs, and multi-region trade-offs, and streaming should be chosen when its benefits justify its complexity.

Review Questions

1. How do batch and stream processing differ in input, latency, and failure handling? In what sense is a batch a special case of a stream?

2. What is the difference between an event, a command, and a message? Why does the distinction matter for coupling?

3. Explain the difference between event time and processing time, and give an example in which they differ greatly.

4. Describe topics, partitions, and offsets. Why does a log guarantee order only within a partition?

5. How does a consumer group divide the work? What limits the parallelism of a group?

6. What do `acks=all`, the replication factor, and the minimum number of in-sync replicas determine together?

7. Compare a message queue with a distributed log in consumption, replay, and ordering.

8. Distinguish at-most-once, at-least-once, and exactly-once processing. Why is exactly-once delivery not obtainable from the network alone?

9. Explain how an idempotent producer, a transaction, and an idempotent consumer each contribute to exactly-once effects.

10. What is a watermark? Compute the watermark after events at 10:00:07 and 10:00:04 with a tolerance of three seconds, and say when the window ending at 10:00:10 closes.

11. What are the options for handling late events, and what does each trade off?

12. What is the stream–table duality? How does change data capture use it?

13. What is the dual-write problem, and how does the transactional outbox solve it?

14. Compare event sourcing with storing current state. Name two benefits and two costs.

15. Describe a saga with compensating actions. Why does it offer atomicity but not isolation?

16. Which schema changes are backward compatible, and why are the compatibility rules for logs stricter than for APIs?

17. How can personal data be erased from an append-only log? What are the limits of each method?

Exercises

1. **Partitions and throughput.** A topic must handle 300 MB/s. A producer can write 15 MB/s to one partition and a consumer can process 25 MB/s from one partition. Compute the minimum number of partitions, the maximum number of useful consumers in a group, and the effect of adding partitions later on the order of events with an existing key.

2. **Storage.** A cluster receives 20 MB/s on average, retains data for 14 days, and uses a replication factor of three. Estimate the disk needed, and add 30% headroom. How does it change if compression reduces the data by half, and if retention is cut to three days with the older data kept in a lakehouse table?

3. **Consumer lag.** Producers write 8 000 records per second between 08:00 and 09:00 and 3 000 per second at other times. A consumer group can process 6 000 per second. Compute the lag at 09:00, the time needed to clear it, and the number of consumers (if one consumer handles 1 500 records per second) that keep lag bounded at the peak.

4. **Watermarks.** Events arrive with event times (in seconds after 10:00:00): 12, 45, 38, 61, 52, 70, 66, 130, 25. With tumbling one-minute windows and a watermark delay of ten seconds, determine the watermark after each arrival, when each window closes, and which events are late. What changes with a delay of 40 seconds?

5. **Delivery semantics.** A consumer reads a payment event, charges a card via an external API, and then commits its offset. Describe what happens if it crashes (a) before the charge, (b) after the charge and before the commit, and (c) after the commit. Redesign the processing so that the card is charged at most once even under redelivery.

6. **Outbox.** Design the outbox table and the consumer for an order service that must publish `OrderPlaced` events. Specify the columns, the cleanup of published rows, the ordering of events per order, the deduplication in consumers, and the behavior when the connector is stopped for an hour.

7. **Saga.** Design a saga for booking a trip with a flight, a hotel, and a car. Define the events, the compensating actions, the handling of a step that times out without answering, and the intermediate states that users may see. Compare choreography and orchestration for this process.

8. **Schema evolution.** Version 1 of an event has the fields `orderId`, `total` (a number), and `status` (a string). Classify each change as backward compatible, forward compatible, both, or breaking: (a) add an optional `coupon`, (b) rename `status` to `state`, (c) change

`total` from integer cents to decimal euros, (d) add a new allowed value of `status`, (e) remove `total`. For each breaking change, describe a migration.

9. **Event sourcing.** A shopping cart is stored as events (`ItemAdded`, `ItemRemoved`, `CheckedOut`). Write the events for a session, compute the cart by replay, and propose a snapshot strategy for carts with thousands of events. Explain how a read model for "abandoned carts" would be built and kept up to date.

10. **Erasure and replay.** A customer asks for deletion. Their events are in a Kafka topic with 30 days of retention, a compacted topic of customer profiles, a stream job with state, a lakehouse table, and a vector index. Write a procedure for each store, say how crypto-shredding would simplify it, and discuss how a replay after the deletion avoids resurrecting the data.

References

- Akidau, T., Bradshaw, R., Chambers, C., et al. (2015). The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing. *Proceedings of the VLDB Endowment*, 8(12), 1792–1803.

- Akidau, T., Chernyak, S., & Lax, R. (2018). *Streaming Systems*. O'Reilly Media.

- Apache Flink Documentation. https://nightlies.apache.org/flink/

- Apache Kafka Documentation. https://kafka.apache.org/documentation/

- Carbone, P., Fra, G., Ewen, S., Haridi, S., & Tzoumas, K. (2015). Lightweight Asynchronous Snapshots for Distributed Dataflows. arXiv:1506.08603.

- Debezium Documentation. https://debezium.io/documentation/

- Garcia-Molina, H., & Salem, K. (1987). Sagas. In *Proceedings of ACM SIGMOD '87*, 249–259.

- Helland, P. (2007). Life Beyond Distributed Transactions: An Apostate's Opinion. In *Proceedings of CIDR 2007*, 132–141.

- Hohpe, G., & Woolf, B. (2003). *Enterprise Integration Patterns*. Addison-Wesley.

- Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.

- Kreps, J. (2013). The Log: What Every Software Engineer Should Know About Real-Time Data's Unifying Abstraction. LinkedIn Engineering Blog.

- Kreps, J. (2014). Questioning the Lambda Architecture. O'Reilly Radar.

- Kreps, J., Narkhede, N., & Rao, J. (2011). Kafka: A Distributed Messaging System for Log Processing. In *Proceedings of the NetDB Workshop*.

- Marz, N., & Warren, J. (2015). *Big Data: Principles and Best Practices of Scalable Realtime Data Systems*. Manning.

- Narkhede, N., Shapira, G., & Palino, T. (2017). *Kafka: The Definitive Guide*. O'Reilly Media.

- Stopford, B. (2018). *Designing Event-Driven Systems*. O'Reilly Media.

Chapter 21

Real-Time Data Pipelines and Processing Frameworks

Learning Objectives

After reading this chapter, you will be able to:

- Explain the difference between batch and stream processing, and describe the log as the backbone of real-time pipelines.
- Describe topics, partitions, offsets, and consumer groups in a distributed log such as Apache Kafka.
- Capture database changes with change data capture and manage schemas and their evolution.
- Explain delivery guarantees and how end-to-end exactly-once effects are achieved.
- Apply the concepts of stream processing: event time, windows, watermarks, state, and joins.
- Compare processing frameworks such as Kafka Streams, Apache Flink, Spark Structured Streaming, and Apache Beam.
- Design sinks, reprocessing, and architectures such as lambda, kappa, and streaming lakehouse pipelines.
- Operate pipelines: monitor lag and latency, plan capacity, handle failures, and protect data quality and privacy.

21.1 From Batch to Streams

For decades, most analytical data processing followed a simple rhythm: collect data during the day, load it into a warehouse at night, and read reports in the morning. This is *batch processing*: a computation runs over a finite, bounded input and then ends. Many modern needs cannot wait for the next batch: detecting fraud while a payment is in progress, updating a search index within seconds of a product change, alerting on a failing service, recommending an item while the user is still browsing, or keeping a dashboard current.

Stream processing treats data as an *unbounded* sequence of events that is processed

continuously as it arrives. A *real-time data pipeline* is the chain of components that moves events from their sources, through processing steps, to the systems that use them. The word "real-time" is used loosely; Table 21.1 distinguishes the latency tiers that are meant in practice.

Table 21.1: Latency tiers of data processing.

Tier	Typical latency	Typical technology and use
Batch	Hours to days	Scheduled jobs on a warehouse or data lake; reports, model training
Micro-batch	Seconds to minutes	Small frequent batches; near-real-time dashboards and loads
Stream processing	Milliseconds to seconds	Continuous processing of events; alerts, enrichment, aggregation
Request–response	Milliseconds	Synchronous service calls and database queries; user-facing transactions

Lower latency is not free: it requires always-running systems, careful handling of failures and late data, and more complex operations. A recurring theme of this chapter is therefore to decide, for each use case, how fresh the data really needs to be, and to choose the cheapest tier that meets that need.

21.1.1 Why Pipelines Rather Than Direct Integration

Without a pipeline, each producer of data connects directly to each consumer, creating a tangle of point-to-point integrations: with P producers and C consumers, up to $P \times C$ connections, each with its own format, failure handling, and timing. A pipeline built around a shared, durable event log reduces this to $P + C$ connections and decouples the teams: producers publish events without knowing who uses them, and new consumers can be added without changing producers (Kleppmann and Kreps, 2015).

21.2 The Log as Backbone

21.2.1 The Distributed Log

The central abstraction of most real-time pipelines is the *log*: an append-only, ordered sequence of records, each with a sequential position called its *offset* (Kreps, 2013). Writers append to the end; readers read from any offset onward and keep track of their own position. Because the log is durable and records are not deleted when read, many independent consumers can read the same data at their own pace, and a consumer can *replay* history by resetting its offset. This is the same structure as the write-ahead and replication logs of Chapters 9 and 10, exposed as a product.

Apache Kafka is the most widely used system of this kind (Kreps et al., 2011); Apache Pulsar, Redpanda, and the managed services of cloud providers (Amazon Kinesis, Google

Pub/Sub, Azure Event Hubs) offer related models. Its vocabulary is common to the field:

- A **topic** is a named stream of records, such as `orders`.
- A topic is divided into **partitions**, each an ordered log. Partitions allow a topic to scale across machines and to be consumed in parallel. Ordering is guaranteed *within* a partition, not across partitions.
- A **key** on each record determines its partition by hashing (Chapter 9), so all events for the same key, such as one customer, go to the same partition and stay in order.
- Each partition is **replicated** on several brokers; one replica is the leader, and followers copy it. A write is acknowledged according to the producer's setting (for example, after all in-sync replicas have it), which is the synchronous-versus-asynchronous trade-off of Chapter 9.
- A **consumer group** is a set of consumers that share the work of reading a topic: each partition is assigned to exactly one consumer in the group, so the group's parallelism is limited by the number of partitions. Different groups read independently, each with its own offsets.
- **Retention** determines how long records are kept (by time or size). **Log compaction** instead keeps only the latest record for each key, so the topic becomes a changelog that can rebuild the current state of a table.

Figure 21.1 illustrates producers, partitions, and a consumer group.

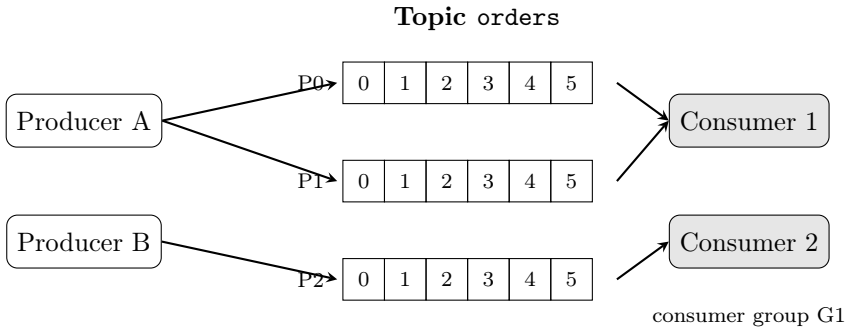


Figure 21.1: A topic with three partitions, each an ordered log with offsets. Producers append; the two consumers of a group divide the partitions between them.

21.2.2 Producers and Consumers

A producer chooses the key (and therefore the partition), the acknowledgment level, and whether to batch and compress records: larger batches raise throughput at the cost of a little latency. A consumer reads records, processes them, and *commits* its offset, recording how far it has progressed so that after a restart it resumes from there. When to commit relative to

processing determines the delivery guarantee (Section 21.4). When consumers join or leave a group, partitions are *rebalanced* among them, during which processing may pause; frequent rebalances, for example because of slow processing that exceeds the group's timeout, are a common operational problem.

21.3 Getting Data Into the Pipeline

21.3.1 Change Data Capture

Much valuable data originates in operational databases, and applications should not have to write events twice. *Change data capture* (CDC) reads the database's own replication log and publishes each committed insert, update, and delete as an event (Chapter 9 described these logical logs). Table 21.2 compares the approaches.

Table 21.2: Methods of capturing changes from a database.

Method	How it works	Trade-offs
Log-based CDC	Reads the transaction or replication log	Low overhead; captures every change including deletes, in commit order; needs log access and connector care
Query-based polling	Periodically queries for rows changed since the last run, using a timestamp or version column	Simple; misses deletes and intermediate states; adds load to the database
Triggers	Database triggers write changes to a side table	Works without log access; adds write overhead and complexity
Application events	The application publishes events itself	Rich business meaning; risk of the dual-write problem unless an outbox is used

Log-based CDC tools such as Debezium run as connectors in a framework like Kafka Connect, which handles offsets, scaling, and restarts (Debezium Documentation). Listing 21.1 shows the configuration of a connector that streams the changes of two PostgreSQL tables into topics. Such connectors typically begin with a consistent *snapshot* of the existing data and then switch to streaming the log.

Listing 21.1: Configuration of a log-based CDC connector for PostgreSQL (illustrative; option names vary by version).

```
{
  "name": "shop-postgres-cdc",
  "config": {
    "connector.class": "io.debezium.connector.postgresql.PostgresConnector",
    "database.hostname": "db.internal",
```

```
    "database.port": "5432",
    "database.user": "cdc_reader",
    "database.password": "${file:/secrets/cdc.properties:password}",
    "database.dbname": "shop",
    "plugin.name": "pgoutput",
    "topic.prefix": "shop",
    "table.include.list": "public.orders,public.customers",
    "snapshot.mode": "initial"
  }
}
```

CDC combines naturally with the transactional outbox pattern of Chapter 8: the application writes events to an outbox table in the same transaction as its data, and CDC publishes the outbox rows, so no separate relay process is needed. It is also the standard way to keep search indexes (Chapter 13) and vector indexes (Chapter 14) in sync with a system of record.

21.3.2 Schemas and Data Contracts

Producers and consumers evolve independently, so the format of events is a contract. Compact binary formats with explicit schemas, such as Apache Avro and Protocol Buffers, are preferred to untyped JSON for high-volume topics, since they are smaller, faster, and enforce structure. A *schema registry* stores each topic's schemas, gives them version numbers, and checks that a new version is compatible with earlier ones before it is accepted. Table 21.3 lists the compatibility modes.

Table 21.3: Schema compatibility modes.

Mode	**Guarantee**	**Safe changes (typical)**
Backward	Consumers using the *new* schema can read data written with the *old* one	Add a field with a default; delete a field
Forward	Consumers using the *old* schema can read data written with the *new* one	Add a field; delete a field that had a default
Full	Both of the above	Add or remove fields that have defaults

The rule of thumb is to add fields with defaults and never to change the meaning or type of an existing field; a breaking change is introduced as a new topic or a new major version of the schema. Treating schemas as code, reviewed and tested before deployment, prevents the most common cause of broken pipelines, which is a producer that silently changes its events.

21.4 Delivery Guarantees and Exactly-Once Effects

Chapter 10 explained that networks lose messages and that retries are unavoidable. For pipelines this yields the three familiar guarantees: *at-most-once* (no duplicates, but possible loss), *at-least-once* (no loss, but possible duplicates), and *exactly-once effects*. The consumer's commit order decides which one results:

- Commit the offset *before* processing: a crash after the commit loses the record (at-most-once).
- Commit *after* processing: a crash before the commit repeats the record on restart (at-least-once). This is the usual default.

21.4.1 Achieving Exactly-Once

Exactly-once results require that processing a record and recording its offset happen atomically, or that repeated processing be harmless. Three techniques are used, alone or together:

- **Idempotent producers.** The broker assigns each producer an identity and sequence numbers and discards duplicates caused by retries, so a retried send does not create a second record.

- **Transactions across read, process, and write.** Kafka supports transactions that atomically write output records to several partitions *and* commit the input offsets, so a consumer-transformer-producer step either fully happens or not at all; consumers can be configured to read only committed records. Kafka Streams uses this to provide exactly-once processing within Kafka (Wang et al., 2021).

- **Idempotent or transactional sinks.** Guarantees stop at the system's boundary. To write to an external database, the sink must either be idempotent (an upsert keyed by a unique event or entity identifier, as in Chapter 10), or take part in a two-phase commit with the processing framework, as Flink's transactional sinks do, or store the offset and the result in the same transaction.

An honest description is therefore that pipelines provide *effectively-once* results: the event may be delivered more than once, but its effect on the output is applied once. The guarantee holds end to end only if every stage, including the final sink, supports it.

21.4.2 Failures in Individual Records

Not all failures are transient. A *poison message* (for example, one that cannot be parsed or that violates a business rule) will fail on every retry and, if the consumer retries indefinitely, blocks its partition. The standard pattern is bounded retries with backoff (Chapter 10), followed by publishing the record, with the error, to a *dead-letter topic* for inspection and later reprocessing, while the pipeline continues. Dead-letter topics need monitoring and an owner; a queue of ignored failures is data loss that has merely been delayed.

21.5 Stream Processing Concepts

21.5.1 Event Time and Watermarks

Chapter 12 introduced the difference between *event time* (when something happened) and *processing time* (when the system sees it). In a pipeline, events are delayed and reordered by networks, mobile devices that were offline, retries, and backlogs, so results based on processing time would depend on accidents of delivery. Correct analytics use event time, which raises a problem: how long should a computation wait for late events before it declares a window complete? A *watermark* is the system's running estimate that no events with an event time earlier than a given moment are still expected (Akidau et al., 2015). Choosing it is a trade-off: a tight watermark gives fast, possibly incomplete results; a loose one gives complete results later. Events that arrive after the watermark has passed are *late*, and the pipeline must decide whether to drop them, send them to a side output, or update the already published result (*allowed lateness* and retractions).

21.5.2 Windows

Aggregations over unbounded streams are computed over windows, introduced in Table 12.2 of Chapter 12: tumbling windows (fixed, non-overlapping), hopping or sliding windows (fixed, overlapping), and session windows (separated by inactivity gaps). A window emits a result when its trigger fires, normally when the watermark passes the window's end, and optionally earlier to give early estimates.

21.5.3 State

Many operations are *stateless*: filtering, mapping, and parsing look at one record at a time and scale trivially. Others are *stateful*: counting, aggregating over windows, deduplicating, joining, and detecting patterns across events must remember earlier records. State is kept per key, partitioned by the same key as the data, so that each processing instance owns the state of its keys. State is the main complication of stream processing: it must survive failures, be redistributed when parallelism changes, and not grow without bound (Section 21.7).

21.5.4 Joins and Stream–Table Duality

A table can be viewed as the latest values of a changelog stream, and a stream as the history of changes to a table. This *stream–table duality* underlies joins in stream processing:

- **Stream–stream joins** match events of two streams that occur within a time window of each other, such as an ad impression and a click. Both sides' events must be buffered for the window, so state grows with the window and event rate.
- **Stream–table joins** (enrichment) look up reference data for each event, such as the customer record for an order. The table is itself a stream of changes (for example, from CDC) held as local state, which avoids a remote database call for every event. An event

is joined with the version of the table at the time it is processed, which may differ from the version at its event time.

- **Table–table joins** maintain a continuously updated joined result as either side changes.

Listing 21.2 shows these ideas in Flink SQL: a Kafka-backed table with an event-time attribute and a watermark that tolerates five seconds of disorder, and a one-minute tumbling-window aggregation. The query is written like batch SQL but runs continuously and emits a result for each window when the watermark passes its end (Apache Flink Documentation).

Listing 21.2: Event time, a watermark, and a tumbling-window aggregation in Flink SQL.

```
CREATE TABLE orders (
  order_id STRING,
  customer_id STRING,
  amount DECIMAL(10, 2),
  order_time TIMESTAMP(3),
  WATERMARK FOR order_time AS order_time - INTERVAL '5' SECOND
) WITH (
  'connector' = 'kafka',
  'topic' = 'orders',
  'properties.bootstrap.servers' = 'broker:9092',
  'scan.startup.mode' = 'earliest-offset',
  'format' = 'json'
);

SELECT window_start, window_end,
       COUNT(*) AS orders,
       SUM(amount) AS revenue
FROM TABLE(
  TUMBLE(TABLE orders, DESCRIPTOR(order_time), INTERVAL '1' MINUTE))
GROUP BY window_start, window_end;
```

21.6 Processing Frameworks

Several frameworks implement these concepts, with different architectures. Table 21.4 summarizes the main options; features and versions change quickly, so use it as a starting point and consult current documentation.

21.6.1 Architectural Differences

Two designs are worth distinguishing. A *library* such as Kafka Streams runs inside ordinary application processes: scaling means starting more instances, and the broker coordinates them as a consumer group. A *cluster framework* such as Flink or Spark accepts a job that is divided into tasks, scheduled across worker machines, and managed by a coordinator. Libraries are simple to deploy and fit the microservice style; cluster frameworks offer more powerful scheduling, resource isolation, and the ability to read from and write to many systems.

Table 21.4: Selected stream-processing frameworks and services.

Framework	Model	Strengths	Considerations
Kafka Streams	A Java library embedded in the application; reads and writes Kafka	No separate cluster; exactly-once within Kafka; state backed by changelog topics	Tied to Kafka; JVM
Apache Flink	Distributed dataflow engine; true record-at-a-time streaming	Rich event-time support, large state, SQL, exactly-once checkpoints	A cluster to operate; steeper learning curve
Spark Structured Streaming	Streaming queries on Spark, mainly as micro-batches	Unified batch and streaming API; strong ecosystem and SQL	Latency typically seconds; state handling less rich than Flink in some cases
Apache Beam	Portable programming model with unified batch and stream semantics	Runs on several engines (Flink, Spark, Google Dataflow); event-time model	Abstraction layer; features vary by runner
ksqlDB and streaming SQL databases (for example Materialize, RisingWave)	SQL over streams; incrementally maintained views	Easy to start; results served as queryable views	Less flexible than general code; maturity varies
Managed services	Cloud-provided stream processing and ingestion	Little operational work	Cost, portability, and limits

Processing may be *record-at-a-time* (Flink, Kafka Streams), giving the lowest latency, or *micro-batch* (the classic Spark model), in which small batches of data are processed periodically, giving high throughput and simple recovery at the cost of latency (Zaharia et al., 2013; Armbrust et al., 2018). The boundary has blurred, since several systems support both, but the choice still affects latency and the complexity of state handling.

21.7 State, Checkpoints, and Fault Tolerance

A stream job runs for months, so it must survive the failure of machines without losing or duplicating results. Chapter 10 introduced the principles; stream processors apply them as follows.

- **Checkpoints.** Flink periodically takes a consistent snapshot of the state of all operators together with the positions in the input streams. It does so with lightweight barriers that

flow with the data, a variant of the Chandy–Lamport distributed snapshot algorithm, without stopping the job (Chandy and Lamport, 1985; Carbone et al., 2015). After a failure, the job restarts from the last checkpoint, rewinds the inputs to the recorded positions, and replays; combined with idempotent or transactional sinks, this yields exactly-once effects.

- **Changelog-backed state.** Kafka Streams stores each state store locally and also writes every change to a compacted *changelog topic*; after a failure, a replacement instance rebuilds the state by reading that topic, or starts from a standby replica.

- **State backends.** Large state is kept in an embedded key–value store such as RocksDB on local disk (the log-structured design of Chapter 4), with periodic incremental snapshots to durable storage, so that state can exceed memory.

- **Savepoints and rescaling.** A *savepoint* is a manually triggered, portable snapshot used for upgrades, migrations, and changing the parallelism of a job; state is assigned to instances by key ranges, so it can be redistributed.

- **Backpressure.** When a downstream operator or sink is slower than the input, the system propagates the slowness upstream so that sources slow down instead of exhausting memory. Persistent backpressure shows that the pipeline lacks capacity somewhere.

- **State expiry.** State for keys that will not return, such as finished sessions or old deduplication keys, must be removed with a time-to-live, or the state grows without bound.

21.8 Sinks and Serving

A pipeline exists to feed systems that serve queries. Table 21.5 lists common sinks and how to make writes to each safe against duplicates.

Writing every event individually to a lake creates many tiny files, which makes later queries slow, so streaming writers batch events into larger files and schedule compaction. Open table formats give lake files ACID-style commits and snapshots, which makes it possible to combine a continuous stream of inserts with consistent batch reads. An important design question for each sink is whether it needs the *latest state* (an upsert target) or the *full history* (an append-only target), since the same stream can feed both.

21.9 Pipeline Architectures

Figure 21.2 shows a typical end-to-end architecture, in which sources feed a log, processors transform events, and the results are delivered to several serving systems and to long-term storage.

Table 21.5: Common sinks and idempotency strategies.

Sink	Typical use	Safe writing
Relational or key–value database	Current state, lookups, application features	Upsert by primary key; version or timestamp check to ignore stale events
Search index (Chapter 13)	Full-text and filtered search	Document identifier equals entity identifier; external version
Vector index (Chapter 14)	Semantic retrieval	Chunk identifier and content hash; replace all chunks of a changed document
Real-time OLAP store (for example ClickHouse, Druid, Pinot)	Low-latency analytics over fresh events	Deduplicate by event identifier at ingestion or query; partition by time
Data lake or lakehouse table (for example Iceberg, Delta Lake, Hudi)	Cheap, durable history and batch analytics	Transactional commits of files; periodic compaction of small files
Cache or feature store	Online model features, hot data	Overwrite with the latest value; set expiry

21.9.1 Lambda and Kappa

The *lambda architecture* combines a batch layer, which periodically recomputes accurate results over all data, with a speed layer, which provides fast approximate results for recent data; a serving layer merges the two (Marz and Warren, 2015). It gives both correctness and freshness but requires writing and maintaining the same logic twice, in two systems, and results can disagree. The *kappa architecture* drops the batch layer: because the log retains history, a single stream-processing code base serves both purposes, and to recompute results after a bug fix or logic change the job is simply restarted from an earlier offset (or a new version of the job reprocesses the log into a new output), as the dashed arrow in Figure 21.2 indicates. Kappa is simpler but demands a log with long retention (or tiered storage) and stream processing that is correct and fast enough for reprocessing at scale. In practice, many organizations use *streaming lakehouse* designs: events land continuously in table formats on object storage in successively refined layers (raw, cleaned, aggregated, often called bronze, silver, and gold), and both streaming and batch engines read the same tables.

21.9.2 Reprocessing and Backfills

Real pipelines must be able to correct the past: logic changes, defects are found, new consumers need history. Good designs make reprocessing routine:

- Keep the raw events (in the log with sufficient retention, or archived to object storage)

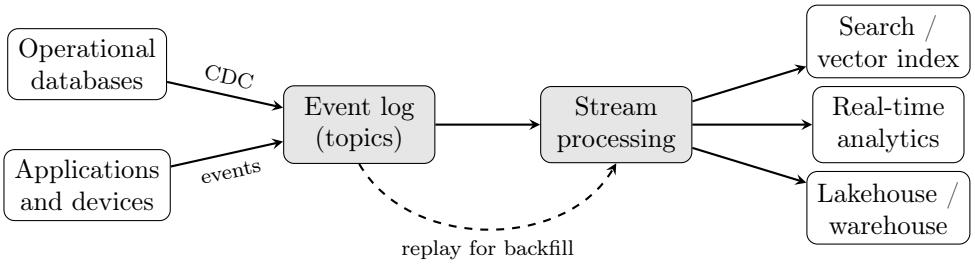


Figure 21.2: A real-time pipeline. Databases and applications publish to a durable log; stream processors transform and enrich events; sinks serve search, analytics, and long-term storage. The retained log allows reprocessing.

as the source of truth, and treat derived data as rebuildable.

- Make processing deterministic: results depend only on event contents and event time, not on the wall clock or arrival order.
- Write output to a new table or index and switch over atomically (alias or view), as for search indexes in Chapter 13.
- Throttle backfills so that they do not starve live processing or overload sinks.

21.10 Data Quality, Governance, and Privacy

Because pipelines run unattended and fan out to many consumers, one bad event can spread quickly. Controls include:

- **Validation at the edge.** Check events against their schema and business rules on entry, and route violations to a dead-letter topic.
- **Deduplication.** Include a unique event identifier and drop repeats, with state expiry for the identifiers.
- **Late-data policy.** State explicitly how late events are treated and how long corrections remain possible.
- **Lineage and ownership.** Record which topics and jobs produce and consume each dataset, and who is responsible for it, so that the effect of a change or incident can be traced.
- **Access control and encryption.** Restrict who may publish and subscribe to each topic, encrypt data in transit and at rest, and keep secrets out of event payloads and configuration files.
- **Personal data.** Logs are append-only and replicated widely, which conflicts with deletion requests under privacy law. Options include keeping personal data out of events and referencing it by an identifier, tokenizing or encrypting fields so that deleting the key makes the data unreadable (*crypto-shredding*), and using compacted topics with tombstone records for per-key deletion. Downstream copies in lakes, indexes, and caches must be deleted too, so the deletion must be propagated through the pipeline.

21.11 Operating Real-Time Pipelines

21.11.1 What to Monitor

- **Consumer lag**: the difference between the latest offset in a partition and the consumer's committed offset. Growing lag means that the consumer cannot keep up, or is stuck. It is the single most useful health signal.
- **End-to-end latency**: the time from the event's occurrence (event time) to its availability in the sink, measured at percentiles (p50, p99), with periodic synthetic marker events as in Chapter 8.
- **Watermark delay**, **checkpoint duration and failures**, **backpressure**, and **state size**.
- **Throughput and skew**: records per second per partition and per task; a single hot key or partition can cap the whole pipeline.
- **Error rates**: records sent to dead-letter topics, deserialization failures, sink errors.

21.11.2 Capacity Planning

The number of partitions bounds a consumer group's parallelism and is hard to reduce later (increasing it changes the key-to-partition mapping and the ordering of existing keys), so it should be planned. A common rule of thumb is to take the target throughput T, the measured throughput one partition can sustain for producers p and for consumers c, and provision at least

$$\text{partitions} \geq \max\left(\frac{T}{p}, \frac{T}{c}\right), \tag{21.1}$$

with headroom for growth. For $T = 200$ MB/s, $p = 10$ MB/s, and $c = 5$ MB/s this gives $\max(20, 40) = 40$ partitions. State size should be estimated similarly: 10 million keys with 200 bytes of state each is about 2 GB, plus overhead for the state backend and snapshots.

21.11.3 Common Pitfalls

- **Hot keys and skew.** A key with disproportionate traffic overloads one partition and one task. Remedies include salting the key and aggregating in two stages.
- **Unbounded state.** Windows, joins, and deduplication without expiry steadily consume memory and disk until the job fails.
- **Assuming global order.** Order exists only within a partition. Operations that depend on the order of events across keys need to be redesigned or serialized through a single partition.
- **Clock problems.** Using producer clocks for event time without handling skew, or mixing event and processing time, gives wrong windows (Chapter 12).

- **Rebalance storms.** Slow processing that exceeds timeouts causes consumers to be evicted and partitions to move repeatedly. Tune timeouts, batch sizes, and processing time.
- **Breaking schema changes.** An incompatible change deployed by one team stops every consumer. Enforce compatibility checks in the build.
- **Real time by default.** Many reports are perfectly served by hourly batches. Streaming adds cost and operational burden that should be justified by a real need for fresh data.

21.11.4 Testing

Test stream jobs as carefully as services. Unit-test the transformation logic with in-memory test harnesses provided by the frameworks; run integration tests against real brokers and databases in containers; replay recorded production events through a new version and compare its output with the old one; and test failure paths by killing workers, restarting brokers, delaying and reordering events, and sending malformed records (Chapter 10). A pipeline whose recovery has never been exercised should be assumed not to recover.

21.12 Choosing an Approach

A sound process is: (1) ask for each use case how fresh the data must be and what an error or delay costs; (2) define the events, keys, schemas, and contracts, and who owns them; (3) choose the log and the capture method (CDC or events), and plan partitions from the throughput; (4) decide the semantics: event time, windows, watermarks, and the policy for late and bad data; (5) make every sink idempotent and decide whether it needs state or history; (6) design reprocessing from the start by retaining raw events and keeping logic deterministic; (7) instrument lag, latency, and errors, and rehearse failures; and (8) start with the simplest tier and framework that meet the requirement, adding complexity only when measured needs demand it.

Summary

Real-time pipelines move events continuously from sources to the systems that use them, replacing or complementing batch processing where freshness has value. Their backbone is the distributed log, in which topics are split into replicated, ordered partitions, keys determine placement and ordering, consumer groups divide the work, and retained records can be replayed. Data enters through log-based change data capture, application events, or the outbox pattern, and schemas managed in a registry with compatibility rules keep independent teams from breaking each other. Delivery is at-least-once by default; exactly-once *effects* come from idempotent producers, transactions that tie input offsets to output, and idempotent or transactional sinks, while dead-letter topics isolate poison messages. Stream processing is built on event time, watermarks, windows, keyed state, and joins based on the duality of streams and tables, expressed in code or in streaming SQL. Frameworks differ in architecture,

Table 21.6: Choosing a real-time processing approach.

Situation	**Suggested approach**	**Reasoning**
Freshness of minutes or hours is acceptable	Scheduled batch or micro-batch jobs	Simplest to build and operate
Event-driven microservices, moderate state, Kafka already in use	Kafka Streams or similar library	No extra cluster; scales with the application
Complex event-time logic, large state, low latency	Apache Flink	Rich windowing, state, and exactly-once checkpoints
Existing Spark platform, unified batch and streaming	Spark Structured Streaming	Shared code and skills; latency of seconds is acceptable
Real-time dashboards and queryable aggregates with SQL	Streaming SQL database or real-time OLAP store	Incrementally maintained views, fast queries
Keeping search, cache, and analytics copies in sync with a database	Log-based CDC with idempotent sinks	Reliable propagation without application changes
Portability across cloud and engines	Apache Beam, or a managed service	Same model on several runners, or little operations work

from the library model of Kafka Streams to the cluster engines Flink and Spark and the portable Beam model, and they survive failures through checkpoints, changelog-backed state, state backends, savepoints, and backpressure, with state expiry to bound growth. Sinks include databases, search and vector indexes, real-time analytics stores, lakehouse tables, and caches, each needing its own way to be written safely. Lambda and kappa architectures, and streaming lakehouses, organize batch and stream work, and routine reprocessing from retained raw events is essential for correcting the past. Data quality, lineage, access control, and privacy, including deletion in append-only logs, require deliberate design. Operating pipelines means watching consumer lag, end-to-end latency, watermarks, checkpoints, and skew, planning partitions and state, and testing failure and replay. The guiding principle is to use real-time processing where the business value of freshness justifies its complexity, and to build it from the start for replay, idempotence, and observability.

Review Questions

1. Distinguish batch, micro-batch, and stream processing. What does "real-time" mean in each context?

2. Why does a shared log reduce the number of integrations from $P \times C$ to $P + C$?

3. Explain topics, partitions, offsets, keys, and consumer groups. What limits a consumer group's parallelism?

4. What ordering guarantee does a partitioned log provide, and what does it not provide?

5. Compare log-based CDC with query-based polling. What does polling miss?

6. What is a schema registry? Explain backward and forward compatibility, and give one safe and one unsafe schema change.

7. Explain how the order of committing offsets and processing produces at-most-once or at-least-once behavior.

8. Describe three techniques that together provide end-to-end exactly-once effects. Why does the guarantee depend on the sink?

9. What is a poison message, and how does a dead-letter topic help? What risk remains?

10. Distinguish event time from processing time. What is a watermark, and what trade-off does its setting involve?

11. Explain stream–table duality and use it to describe a stream–table join for enrichment.

12. How do Flink checkpoints and Kafka Streams changelog topics each restore state after a failure?

13. Compare the lambda and kappa architectures. What is the main cost of each?

14. What is consumer lag, and why is it the central health indicator of a pipeline?

15. How can personal data be deleted from an append-only log and its downstream copies?

Exercises

1. **Partitioning.** An `orders` topic must sustain 120 MB/s. A producer can write about 8 MB/s per partition, and a consumer can process about 3 MB/s per partition. How many partitions are needed? What happens to ordering if you later increase the partition count, and how would you avoid problems?

2. **Ordering.** A pipeline publishes `OrderCreated`, `OrderPaid`, and `OrderShipped` events. Choose a key and justify it. Describe what could go wrong if the events were keyed randomly, and how a consumer could detect events that arrive out of order.

3. **Delivery semantics.** A consumer reads a payment event, updates a balance in a database, and then commits its offset. Describe the outcome if it crashes (a) between the update and the commit, and (b) before the update. Redesign the processing so that the balance is updated exactly once in effect.

4. **Windows and watermarks.** Events with event times 10:00:05, 10:00:50, 10:01:10, 10:00:40, and 10:02:30 arrive in that order, and the watermark is the maximum event time seen minus 20 seconds. Using one-minute tumbling windows, state when each window closes, which event is late, and what the counts are if late events are dropped and if they are allowed within a further 30 seconds.

5. **Schema evolution.** A topic's Avro schema has fields `id`, `amount`, and `currency`. For each change, state whether it is backward compatible, forward compatible, both, or neither: (a) adding `channel` with a default; (b) removing `currency` (no default); (c) renaming `amount` to `total`; (d) changing `amount` from integer to string. Propose a safe migration for the rename.

6. **CDC pipeline.** Design a pipeline that keeps a search index (Chapter 13) in sync with a product table in PostgreSQL. Specify the capture method, topic and key, event contents, the idempotent indexing key, deletion handling, initial load, and the procedure for a full reindex.

7. **State estimate.** A job deduplicates events by identifier over a 24-hour window. The system receives 5 000 events per second, and each identifier takes 40 bytes of state with overhead. Estimate the state size, say whether it fits in memory on a machine with 16 GB, and describe the effect of a restart without checkpoints.

8. **Reprocessing.** A bug in an aggregation job has produced wrong hourly revenue for the last 12 days. Write a plan to repair the results using the retained event log, including how to run the corrected job beside the live one, how to validate it, how to switch consumers, and how to limit the load on sinks.

9. **Hot key.** A topic is keyed by merchant identifier, and one merchant generates 30% of all traffic. Describe the symptoms, and propose two remedies, including how a two-stage aggregation would restore a correct total.

10. **Architecture choice.** For (a) fraud scoring of card payments within 200 ms, (b) an hourly sales dashboard, and (c) feeding a recommendation model's online features from clickstream events, choose a latency tier, a framework from Table 21.4, and sinks, and justify each choice.

References

- Akidau, T., Bradshaw, R., Chambers, C., et al. (2015). The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing. *Proceedings of the VLDB Endowment*, 8(12), 1792–1803.

- Akidau, T., Chernyak, S., & Lax, R. (2018). *Streaming Systems*. O'Reilly Media.

- Apache Flink Documentation. https://nightlies.apache.org/flink/

- Apache Kafka Documentation. https://kafka.apache.org/documentation/
- Armbrust, M., Das, T., Torres, J., et al. (2018). Structured Streaming: A Declarative API for Real-Time Applications in Apache Spark. In *Proceedings of ACM SIGMOD '18*, 601–613.
- Carbone, P., Ewen, S., Fóra, G., Haridi, S., Richter, S., & Tzoumas, K. (2017). State Management in Apache Flink: Consistent Stateful Distributed Stream Processing. *Proceedings of the VLDB Endowment*, 10(12), 1718–1729.
- Carbone, P., Katsifodimos, A., Ewen, S., Markl, V., Haridi, S., & Tzoumas, K. (2015). Apache Flink: Stream and Batch Processing in a Single Engine. *IEEE Data Engineering Bulletin*, 38(4), 28–38.
- Chandy, K. M., & Lamport, L. (1985). Distributed Snapshots: Determining Global States of Distributed Systems. *ACM Transactions on Computer Systems*, 3(1), 63–75.
- Debezium Documentation. https://debezium.io/documentation/
- Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.
- Kleppmann, M., & Kreps, J. (2015). Kafka, Samza and the Unix Philosophy of Distributed Data. *IEEE Data Engineering Bulletin*, 38(4), 4–14.
- Kreps, J. (2013). The Log: What Every Software Engineer Should Know About Real-Time Data's Unifying Abstraction. LinkedIn Engineering blog.
- Kreps, J., Narkhede, N., & Rao, J. (2011). Kafka: A Distributed Messaging System for Log Processing. In *Proceedings of the NetDB Workshop*.
- Marz, N., & Warren, J. (2015). *Big Data: Principles and Best Practices of Scalable Real-Time Data Systems*. Manning Publications.
- Richardson, C. (2018). *Microservices Patterns*. Manning Publications.
- Wang, G., Chen, L., Dikshit, A., et al. (2021). Consistency and Completeness: Rethinking Distributed Stream Processing in Apache Kafka. In *Proceedings of ACM SIGMOD '21*, 2602–2613.
- Zaharia, M., Das, T., Li, H., Hunter, T., Shenker, S., & Stoica, I. (2013). Discretized Streams: Fault-Tolerant Streaming Computation at Scale. In *Proceedings of ACM SOSP '13*, 423–438.

Chapter 22

Polyglot Persistence and System Design

Learning Objectives

After reading this chapter, you will be able to:

- Explain polyglot persistence, and weigh its benefits against its operational cost.
- Derive store requirements from access patterns and non-functional requirements.
- Distinguish systems of record from derived data, and keep a single owner for each fact.
- Choose integration patterns (outbox, change data capture, event sourcing, CQRS) to keep several stores consistent.
- Design a multi-store architecture for a realistic system and analyze its failure modes.
- Estimate capacity and availability with back-of-envelope calculations.
- Evolve and migrate a data architecture safely, and govern data that lives in many places.
- Review a data architecture with a systematic checklist.

22.1 One Database or Many?

The preceding chapters have presented a range of data systems, each with strengths and weaknesses: relational databases for transactions and flexible queries, key–value and column-family stores for scale and simple access (Chapters 3 and 4), graph databases for connected data (Chapter 11), time-series databases for measurements (Chapter 12), search engines for relevance-ranked retrieval (Chapter 13), vector databases for similarity (Chapter 14), and streaming platforms for moving data between them (Chapter 21). Real applications rarely fit a single one of these. An online shop needs transactions for orders, fast lookups for shopping carts, ranked search for products, recommendations based on relationships, and analytics over clicks.

Polyglot persistence is the practice of using several data stores in one system, each chosen for the part of the problem it handles best (Sadalage and Fowler, 2012). It answers a long-standing observation that no single database design is best for all workloads (Stonebraker

and Çetintemel, 2005). The approach has a price: every additional technology brings new skills to learn, new failure modes, new backups, new security reviews, and new ways for data to become inconsistent. This chapter is therefore about *judgment*: when to add a store, how to connect stores safely, and how to reason about a whole architecture rather than one component.

22.1.1 Two Opposite Mistakes

- **One store for everything, stretched beyond its strengths.** A relational database used as a queue, a full-text engine, a cache, and an analytics warehouse may work for a time, but eventually one workload degrades the others, and the team fights the tool.
- **A different store for every feature.** A system with six kinds of databases, each used lightly, is expensive to operate and hard to keep consistent. The convenience of a feature-specific model is paid for with years of operations.

A sound middle path is to start with the simplest design that meets the requirements, usually one well-understood relational database, and to add a specialized store only when a measured need justifies its cost (Section 22.8). A useful principle is to limit the number of unfamiliar technologies in a system, because each one consumes a share of the team's capacity to learn and operate it.

22.2 From Workloads to Stores

Choice of storage should follow from the *workload*, not from fashion. For each part of the system, ask the following questions:

1. **Access patterns.** How is the data read and written: by key, by range, by relationship, by content, by aggregation? What are the read and write rates?
2. **Data shape.** Is it fixed records, nested documents, a graph, a time series, text, vectors?
3. **Consistency.** What anomalies can the business tolerate (Chapter 8)? Are multi-record transactions needed?
4. **Latency and scale.** What response times are required, and how large will the data and traffic grow?
5. **Durability and availability.** How much data may be lost (RPO), and how long may the service be down (RTO)?
6. **Query flexibility.** Are queries known in advance, or must users explore the data freely?
7. **Lifecycle.** How long is data kept, and who may see or delete it?
8. **Team and environment.** What can the team operate? What do the cloud or hosting platforms offer?

Table 22.1 relates typical workloads to store families, as a first approximation.

The table is a starting point, not an answer. Modern systems blur the lines: relational databases offer JSON columns, full-text search, and vector indexes; search engines offer vector

Table 22.1: Typical workloads and the store families that fit them.

Workload	**Suitable store family**	**Reason**
Transactions with invariants (orders, payments, inventory)	Relational database	ACID, constraints, joins, mature tooling
Lookup by key, sessions, carts, counters, caches	Key–value store or in-memory cache	Very low latency, simple scaling
Flexible, nested records read as a unit (catalogs, profiles)	Document or relational with JSON columns	Schema flexibility; one read per entity
Massive write throughput with known queries	Column-family or wide-column store	Partitioned, log-structured writes
Relationship traversal (recommendations, fraud, access paths)	Graph database	Efficient multi-hop traversal
Measurements indexed by time	Time-series database	Compression, retention, time-oriented queries
Text relevance, filters, facets	Search engine	Inverted index, scoring, facets
Semantic similarity	Vector index (within a search engine or database, or dedicated)	Approximate nearest-neighbor search
Large-scale analytics over history	Columnar warehouse or lakehouse	Scans and aggregations at low cost per byte
Moving and reshaping data between systems	Distributed log and stream processors	Decoupling, replay, continuous processing

search and aggregations; multi-model databases combine several models in one engine. The decision is often between *extending the database you already operate* and *adding a dedicated system*, and the former wins whenever it meets the requirements with acceptable performance.

22.3 Systems of Record and Derived Data

The most important design concept for multi-store systems is the distinction between two kinds of data (Kleppmann, 2017):

- A **system of record** (source of truth) holds the authoritative version of a fact. Each fact is written there first. If the data differs from other copies, the system of record is right.
- **Derived data** is computed from the system of record: caches, search indexes, vector

indexes, materialized views, recommendation tables, analytics copies. It can be rebuilt from the source, and it is allowed to lag.

Two rules follow. First, *every fact has exactly one owner*: one store is authoritative for it, and every other copy is derived. Second, *data flows in one direction*, from the owner to the derived stores, ideally through a durable log (Chapter 21), so that derived stores can be rebuilt by replaying it. Figure 22.1 shows the pattern.

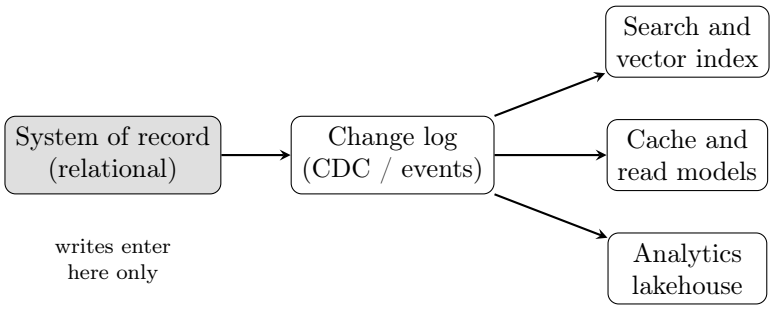


Figure 22.1: One owner, many derived copies. Writes go to the system of record; changes flow through a log to derived stores, which can be rebuilt by replaying it.

The rules prevent the worst failure of polyglot systems: two stores that both accept writes to the same fact and drift apart, with no way to tell which is right. Some data genuinely has different owners in different stages (an order is owned by the order service until it is shipped, then partly by the shipping service); the principle then applies per fact and per phase, with the handover made explicit.

22.3.1 A Data Catalog

For a multi-store system it is worth recording, for every dataset, its owner and properties. Listing 22.1 shows an entry in a data catalog. Writing this down forces decisions about ownership, recovery, retention, and personal data, and it is the basis for the governance discussed in Section 22.10.

Listing 22.1: An entry of a data catalog describing one dataset and its derived copies.

```
{
  "dataset": "orders",
  "owner": "order-service team",
  "systemOfRecord": { "store": "PostgreSQL", "consistency": "serializable",
                      "rpo": "1 minute", "rto": "15 minutes" },
  "derivedCopies": [
    { "store": "Elasticsearch", "purpose": "order search",
      "freshness": "< 10 s", "rebuild": "replay from change log" },
    { "store": "Lakehouse", "purpose": "analytics",
      "freshness": "< 15 min", "rebuild": "replay from change log" }
  ],
  "containsPersonalData": true,
  "retention": "7 years, then delete or anonymize",
```

```
  "deletionPropagation": "events to all derived copies"
}
```

22.4 Integration Patterns

Keeping several stores aligned is the central technical problem. Table 22.2 compares the patterns introduced in earlier chapters and adds two more.

Table 22.2: Patterns for integrating several data stores.

Pattern	**How it works**	**Trade-offs**
Dual write	The application writes to both stores	Simple but unsafe: a failure between the writes causes divergence; avoid
Transactional outbox	The event is stored in the same transaction as the data and relayed (Chapter 8)	Reliable; adds a relay and at-least-once delivery
Change data capture	Read the database log and publish changes (Chapter 21)	No application changes; copies the database's data model
Event sourcing	Store the sequence of events as the system of record; derive state by replaying	Full history and rebuildability; harder queries and schema evolution
CQRS (command–query responsibility segregation)	Separate the write model (commands) from one or more read models (queries) built from events	Each read model fits its queries; eventual consistency between them
API composition	A service queries several stores or services at request time and combines results	No copies to maintain; higher latency and coupling at query time
Batch ETL	Periodic jobs copy data into another store	Simple; stale between runs

22.4.1 CQRS and Read Models

In a conventional design, one model serves both writes and reads, and it is shaped by the needs of the writes: normalized tables that preserve integrity. Reads, such as a product page that combines product data, price, stock, and reviews, then require several joins. CQRS accepts that reads and writes have different needs. Writes go to the system of record, which

enforces the rules; read models are denormalized projections, built from the change stream and stored in whatever store suits the query: a search index for catalog search, a key–value store for the product page, a graph for recommendations (Richardson, 2018). The cost is that read models are *eventually consistent* with the write model, so the user interface must be designed for that, with the techniques of Chapter 8 (returning the updated object, reading from the owner right after a write, or showing a pending state).

22.4.2 Event Sourcing

Event sourcing goes a step further: instead of storing the current state and publishing changes as a side effect, it stores the *events* as the primary record, and the current state is a function of the event history. This gives a complete audit trail, the ability to rebuild any read model or answer historical questions (Chapter 12), and a natural fit with streaming (Chapter 21). The costs are significant: events must be versioned and remain interpretable for years, queries on current state need projections, and correcting mistakes means appending compensating events and not editing history. It is best applied selectively, to domains where history is itself valuable, such as accounts and ledgers, and not as a default for the entire system.

22.5 Consistency Across Stores

Chapter 10 explained that distributed transactions are costly and often unavailable across different products. In a polyglot system, *there is no transaction that spans the stores*, so consistency is constructed from the patterns above:

- **Strong consistency inside one store, eventual consistency between stores.** Put data that must change atomically into the same store (and the same partition where possible), as Chapter 10 advised. Accept that derived copies lag.
- **Sagas for multi-service processes** with compensations instead of cross-store rollbacks.
- **Idempotent consumers** everywhere, because events are delivered at least once.
- **Ordering.** Use one key (for example, the entity identifier) for all events about an entity, so that they remain ordered in a partitioned log.
- **Staleness budgets.** State for every derived store the acceptable lag (for example, search within 10 seconds, analytics within 15 minutes), monitor it, and alert when it is exceeded.
- **Reconciliation.** Run periodic jobs that compare stores, such as counts, checksums, or samples, and repair divergences, because failures and bugs will eventually cause some. Reconciliation is what makes eventual consistency trustworthy in practice.

22.5.1 Handling the Hardest Case: Inventory and Orders

A frequent difficulty is reserving a limited resource, such as stock, while an order spans several systems. The remedy is to make the reservation a local, atomic operation in the store that owns the stock, with an expiry, and to publish the result through an outbox (Listing 22.2). Payment and shipping steps then proceed as a saga; if they fail, the reservation is released by a compensating action or by its expiry.

Listing 22.2: An atomic stock reservation with expiry and an outbox event in one transaction.

```
BEGIN;

-- Guarded update: fails (0 rows) if stock is insufficient
UPDATE inventory
SET available = available - 1, reserved = reserved + 1
WHERE sku = 'B-100' AND available > 0;

-- Only continue if exactly one row was updated; otherwise ROLLBACK

INSERT INTO reservations (id, sku, order_id, expires_at)
VALUES (gen_random_uuid(), 'B-100', 5001, now() + INTERVAL '15 minutes');

INSERT INTO outbox (event_id, topic, payload)
VALUES (gen_random_uuid(), 'stock.reserved',
       '{"orderId": 5001, "sku": "B-100"}');

COMMIT;
-- A background job releases expired reservations and publishes stock.released
```

22.6 A Worked Design: An Online Shop

This section applies the preceding ideas to a concrete system. The requirements are those of a mid-sized online retailer: a catalog of two million products, ten million registered customers, five million daily active users at peak seasons, a few hundred orders per second at peak, product search with filtering, personalized recommendations, and near-real-time reporting.

22.6.1 Requirements and Constraints

- Orders, payments, and stock must never be lost or oversold (strong consistency, durable).
- Search results may lag product changes by a few seconds.
- Cart contents must be fast and survive a session, but may be lost in a total failure of the cache with acceptable inconvenience.
- Recommendations may be minutes or hours stale.
- Reports may lag by 15 minutes.
- Customer data is personal data and subject to deletion requests.
- Target availability for browsing and checkout is 99.9%.

22.6.2 Store Selection

Table 22.3 assigns each dataset to a store with its reasoning, and Figure 22.2 shows the resulting architecture.

Table 22.3: Store selection for the online shop.

Data	Store	Role	Reasoning
Orders, payments, customers, inventory	Relational database (replicated, primary–replica)	System of record	Transactions, constraints, guarded updates
Product catalog	Relational (or document) store	System of record for content	Flexible attributes; edited by staff; moderate volume
Shopping carts and sessions	In-memory key–value store	System of record for carts (short-lived)	Millisecond access; expiry; loss tolerable
Product search and autocomplete	Search engine with vector field	Derived	Relevance, facets, filters; hybrid semantic search
Product page read model	Key–value store or cache	Derived	One lookup per page view; precomputed
Recommendations	Graph database and vector index	Derived	Purchase and view relationships; similarity
Click and event stream	Distributed log	Source of events	Replayable stream feeding all derived stores
Operational metrics	Time-series database	Derived	Monitoring and alerting
Analytics and reporting	Lakehouse or columnar warehouse	Derived	Large scans, history, low storage cost

22.6.3 Key Flows

1. **Checkout.** The order service runs a local transaction in the relational database: reserve stock, create the order in a "pending" state, and write an outbox event (Listing 22.2). A saga then charges the payment; on success the order becomes "paid," and on failure the reservation is released. No transaction spans the payment provider and the database.

2. **Product updates.** Staff edit a product in the catalog database; CDC publishes the change; indexers update the search engine and the product-page read model idempotently, keyed by product identifier and version (Chapter 13).

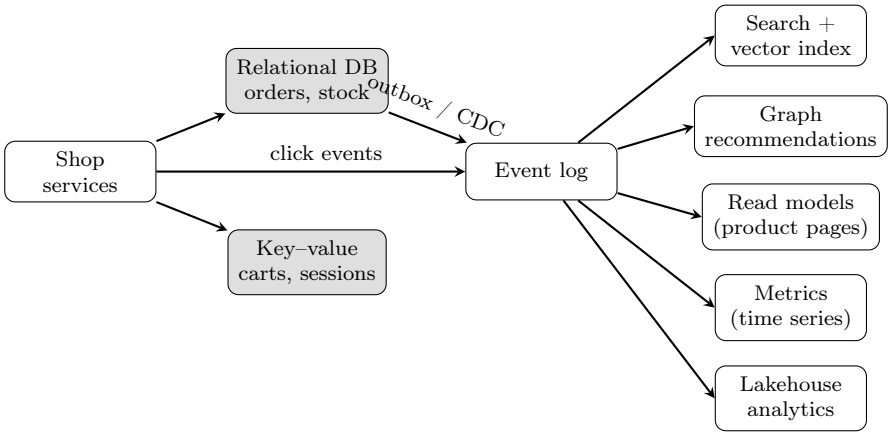


Figure 22.2: Architecture of the online shop. Orders and stock live in a relational system of record; carts live in a key–value store; all changes and events flow through a log that feeds the derived stores.

3. **Search.** Queries go to the search engine with hybrid keyword and vector retrieval (Chapter 14); results show a price and stock status that may lag, so the product page and checkout re-read authoritative values from the system of record.

4. **Recommendations.** A stream job consumes purchase and view events, updates the graph and the vector index, and publishes precomputed recommendation lists to the read-model store.

5. **Reporting.** The same events land continuously in the lakehouse, where batch and streaming queries produce reports and models.

22.6.4 Failure Analysis

A design is incomplete until its failures have been examined. Table 22.4 lists the effect of losing each store and the planned degradation.

The pattern in the table is the point of the exercise: the critical path (checkout) depends on one strongly consistent store, and everything else is derived and degrades gracefully. A design in which checkout needed the search engine, the graph database, and the cache to all be available would multiply the failure probabilities (Section 22.7).

22.6.5 Other Examples in Brief

Table 22.5 sketches two further systems to show how the same method yields different architectures.

Table 22.4: Effect of store failures in the online shop and the planned degradation.

Store lost	Effect	Planned response
Relational database primary	Checkout and order history unavailable	Fail over to replica (with fencing, Chapter 9); browsing continues from read models
Key–value store (carts)	Carts unavailable or lost	Rebuild carts from the browser or client state where possible; show a clear message; checkout of a rebuilt cart re-validates prices and stock
Search engine	Search degraded	Fall back to category browsing and a simplified database query; reindex from the log on recovery
Graph or vector index	No personalized recommendations	Show popular items from a precomputed list
Event log	Derived stores stop updating; outbox rows accumulate	Checkout continues; outbox relay catches up after recovery; staleness alerts fire
Lakehouse	Reports delayed	No customer impact; events remain in the log for replay

22.7 Estimation and Non-Functional Requirements

22.7.1 Back-of-the-Envelope Estimation

Early in a design, rough numbers guide the choice of technology and reveal impossible requirements. Typical calculations convert business volumes into rates and sizes:

- **Request rate.** 5 million users at peak, each making 50 requests per day on average, gives $5 \times 10^6 \times 50 / 86\,400 \approx 2\,900$ requests per second on average, and with a peak-to-average factor of three about 8 700 per second at peak.
- **Storage.** One million orders per day at 2 KB each is 2 GB per day, about 730 GB per year, and about 2.2 TB per year with three replicas, before indexes.
- **Bandwidth.** 8 700 requests per second at an average of 20 KB per response is about 174 MB/s of outgoing traffic.
- **Memory for caches.** Caching the hottest 20% of two million products at 5 KB each takes about 2 GB, small enough for a single cache node.

Such figures tell the designer that the order database fits comfortably on one primary with replicas, that the catalog cache is small, and that the real challenges are availability and

Table 22.5: Store selection for two further systems.

System	Main stores	Key design points
Fleet tracking and logistics	Time-series database for positions and sensor data; relational database for vehicles, drivers, and jobs; stream processor for geofence alerts; lakehouse for history	Event-time processing of late GPS data (Chapters 12 and 21); tiered retention; job assignment as a strongly consistent transaction
Enterprise knowledge assistant	Relational database for documents, metadata, and permissions; search engine and vector index for chunks; graph for entities and relationships; object storage for source files	Permissions applied inside every retrieval query; CDC from source systems; re-embedding on model change; deletion propagated to all indexes (Chapter 14)

consistency, not raw capacity. Estimates are deliberately rough; their value is to separate problems that are 10× too large for a design from those that are not.

22.7.2 Availability Arithmetic

Availability targets are often expressed in "nines." Table 22.6 shows the annual downtime allowed by each.

Table 22.6: Annual downtime allowed by availability targets.

Availability	Downtime per year
99%	about 3.65 days
99.9%	about 8.8 hours
99.99%	about 53 minutes
99.999%	about 5.3 minutes

When a request depends on several components *in series*, all of which must work, the availabilities multiply:

$$A_{\text{series}} = \prod_{i=1}^{n} A_i. \tag{22.1}$$

Five dependencies, each 99.9% available, give $0.999^5 \approx 0.995$, or 99.5%, about 44 hours of downtime per year. Redundant components *in parallel*, where one suffices, combine as

$$A_{\text{parallel}} = 1 - \prod_{i=1}^{n} (1 - A_i), \tag{22.2}$$

so two independent 99.9% replicas give 99.9999%, provided that failures are truly independent and failover works. Two lessons follow: every synchronous dependency on the critical path

lowers availability, so keep the critical path short and make other components optional; and redundancy helps only if failures are independent, so shared causes (one network, one deployment, one configuration change) limit the benefit. Latency behaves similarly: the slowest dependency determines tail latency, and fan-out to many components makes slow responses common (Dean and Barroso, 2013).

22.7.3 Service-Level Objectives

Requirements become testable as *service-level objectives* (SLOs): for example, "99.9% of checkout requests succeed within 800 ms over 30 days" or "search results reflect product changes within 10 seconds for 99% of changes." The gap between 100% and the SLO is an *error budget*, which can be spent on releases and experiments, and which, when exhausted, shifts effort toward reliability (Beyer et al., 2016). Staleness budgets for derived stores, introduced earlier, are SLOs of the same kind.

22.8 Evolving and Migrating Architectures

Architectures are not designed once; they evolve with the business. Two guidelines are useful.

22.8.1 Add Stores When Measurements Demand It

Start with a modular application and one relational database, with clean boundaries between modules so that data could later be split. Introduce a new store when evidence shows that the current one cannot meet a requirement, and not before. Table 22.7 lists typical signals.

Table 22.7: Signals that a new kind of store may be justified.

Signal	Candidate	First try
Text search is slow or poor in quality and users complain	Search engine	Database full-text search with tuned indexes
Hot reads dominate the database load	Cache or read model	Query tuning, read replicas
Queries need five or more joins over relationships, and are slow	Graph database	Recursive queries, better indexes, denormalization
Billions of measurements with retention rules	Time-series database	Time-partitioned tables and compression
Analytics queries disturb transactions	Warehouse or lakehouse	A replica for reporting
Write rate exceeds one primary despite tuning	Partitioned or distributed store	Larger machine, batching, queueing writes

22.8.2 Migrating Safely

Moving data or responsibility between stores is among the riskiest changes. A proven sequence, an application of the *strangler fig* idea of replacing a system piece by piece (Fowler, 2004), is:

1. **Capture changes** from the old store with CDC or an outbox so that nothing is missed during the migration.
2. **Backfill** historical data into the new store, replaying from the log or from a consistent snapshot, throttled to protect the source.
3. **Run both in parallel** ("dual run") and *shadow* reads: send queries to the new store as well as the old one, compare answers, and investigate differences, without affecting users.
4. **Switch gradually**, by traffic percentage or by tenant, with the ability to switch back.
5. **Reconcile** continuously until the old store is no longer needed.
6. **Retire** the old store only after a period without incidents and with a verified archive of its data.

For schema changes within one database, the *expand and contract* pattern applies: first add the new column or table (expand) while the old structure still works, migrate the data and code, and only afterwards remove the old structure (contract), so that every step is compatible with the running application versions.

22.9 The Cost of Many Stores

Each added technology has recurring costs, listed in Table 22.8. They are easy to underestimate because they appear after the system is built.

Managed services reduce the operational share of these costs but add dependency on a provider, and cost and limits that must be understood. *Multi-model databases*, and the extension of a relational database with JSON, search, and vector features, trade some depth of specialized capability for fewer systems. Neither answer is universally right; the question to ask is whether the specialized capability is actually needed at the level at which the specialized system provides it.

22.9.1 Restoring Many Stores

A disaster-recovery plan must say what happens when stores are restored to *different* moments. The consistent approach is to restore the systems of record to a chosen point, and to *rebuild all derived stores* from them or from the log, because derived data is by definition reproducible (Chapter 10 described recovery objectives). Plans that rely on restoring every store from its own backup risk a state in which stores contradict each other. The time to rebuild each derived store, for example reindexing the search engine, must be included in the recovery time objective, and tested.

Table 22.8: Recurring costs of an additional data store.

Cost	Description
Skills and on-call	Engineers must learn it, and incidents need people who can diagnose it at night
Backup and recovery	Each store needs backups and tested restores, and restores of different stores to different points in time must be reconciled
Security and compliance	Another system to patch, encrypt, audit, and include in access reviews and privacy deletion
Upgrades and drift	Version upgrades, deprecations, and licensing changes on several schedules
Observability	Separate metrics, logs, dashboards, and alerts for each technology
Data movement	Pipelines, schemas, monitoring, and reconciliation between stores
Money	Licenses or cloud charges, plus the cost of duplicated data

22.10 Governance Across Stores

When data lives in many places, governance must follow it:

- **Ownership.** Each dataset has an accountable team and a documented purpose, recorded in the catalog (Listing 22.1). Organizations that treat domain datasets as products owned by the teams that know them best follow the *data mesh* approach (Dehghani, 2022).

- **Lineage.** Track which datasets are derived from which, so that the effect of a change or an error can be traced.

- **Personal data.** Know every store that holds personal data, including caches, indexes, vector stores, logs, and analytics copies. Requests to delete data must be propagated to *all* of them, and the propagation verified (Chapters 14 and 21).

- **Retention.** Define and enforce how long each dataset is kept, including in derived copies and backups.

- **Access control.** Apply consistent permissions across stores; a restriction in the system of record is worthless if the search index returns the restricted content.

- **Quality.** Define and monitor quality rules at the points where data enters and where it is published to others.

22.11 Reviewing a Data Architecture

Design reviews are most effective with a shared checklist. The following questions cover the main concerns of this book:

1. **Workloads.** Have the access patterns, data shapes, and volumes been written down? Is each store justified by a measured or estimated need?
2. **Ownership.** Does every fact have one system of record? Do derived copies flow one way, and can they be rebuilt?
3. **Consistency.** What anomalies can users see (Chapter 8)? Where are transactions needed, and are they kept within one store?
4. **Integration.** Is there any dual write? Are outbox, CDC, or event sourcing used correctly, and are consumers idempotent?
5. **Failure.** What happens when each store, the log, and each dependency fails? Which features degrade, and which must not?
6. **Recovery.** Are RPO and RTO defined per dataset, and has recovery, including rebuilding derived stores, been tested?
7. **Scale and latency.** Do the back-of-envelope numbers fit? Where are the hot keys, the large fan-outs, and the tail latencies?
8. **Security and privacy.** Where is personal data? Can it be deleted everywhere? Are permissions enforced at every access path?
9. **Operations.** Who operates each store? Are monitoring, alerting, backups, and upgrades defined? Is the number of technologies justified?
10. **Evolution.** How will schemas change, and how would a store be replaced? Are there known signals for the next change?

Writing down the answers, with the alternatives that were considered and rejected, produces a record of the design reasoning, often kept as short *architecture decision records* (ADRs), which spares later teams from rediscovering the same trade-offs.

22.12 Choosing an Approach

A sound process is: (1) gather requirements, including non-functional targets, and estimate loads; (2) list the datasets and for each decide the owner, the consistency needed, and the access patterns; (3) choose the simplest store that meets each need, preferring one already in operation; (4) draw the data flows from systems of record to derived stores and choose the integration patterns; (5) analyze failures and define degradation, recovery objectives, and staleness budgets; (6) define governance for ownership, privacy, and retention; (7) review the design with the checklist and record decisions; and (8) plan how the architecture will be measured and how it will evolve.

Table 22.9: A decision guide for polyglot architectures.

Situation	Suggested approach	Reasoning
New product, small team, uncertain requirements	One relational database; modular code	Lowest operating cost; easy to change
One workload clearly outgrows the database	Add one specialized derived store fed by CDC or outbox	Targeted relief; system of record unchanged
Many teams and services	Domain-owned data stores, events as contracts, data catalog	Autonomy with explicit integration
History and audit are core requirements	Event sourcing for that domain, with read models	Complete record; rebuildable projections
Strict consistency across domains seems necessary	Reconsider boundaries so related data shares a store; use sagas for the remainder	Avoids cross-store transactions
Operations capacity is limited	Managed services and fewer technologies	Operational simplicity over specialized optimality

Summary

Polyglot persistence uses several kinds of data stores in one system, each for the workload it serves best, because no single database is best at everything. The price is operational complexity and the risk of inconsistency, so stores should be added when measurements justify them, starting from one well-understood relational database. Store choice follows from access patterns, data shape, consistency needs, scale, durability, and the capabilities of the team. The central design discipline is to separate systems of record from derived data, to give every fact a single owner, and to let data flow one way through a durable log, so that derived stores such as caches, search and vector indexes, graphs, and analytics copies can be rebuilt. Stores are kept aligned with the transactional outbox, change data capture, event sourcing, and CQRS rather than dual writes, and because no transaction spans stores, consistency is built from strong guarantees inside each store, sagas, idempotent consumers, staleness budgets, and reconciliation. A worked online-shop design shows a strongly consistent relational core on the critical path, with carts in a key–value store and everything else derived and gracefully degradable, and the failure analysis makes explicit what each outage costs. Back-of-envelope estimates and availability arithmetic, in which serial dependencies multiply and redundancy helps only with independent failures, check the design against its service-level objectives. Architectures evolve through measured additions, parallel runs, shadow reads, and gradual cutovers; their recurring costs in skills, backups, security, and data movement must be weighed; disaster recovery restores systems of record and rebuilds derived data; and

governance follows data into every copy, notably for privacy and deletion. A systematic checklist and recorded decisions make the reasoning reviewable. The principle that ties the chapter together is to choose the simplest architecture that meets the requirements, to make its data flows and failure behavior explicit, and to change it only as evidence demands.

Review Questions

1. What is polyglot persistence? Describe two benefits and two costs.

2. Describe two opposite mistakes in choosing data stores, and a middle path.

3. List five questions to ask about a workload before choosing a store.

4. Distinguish a system of record from derived data. Why must each fact have a single owner?

5. Why is data flow in one direction, through a log, valuable for rebuilding derived stores?

6. What fails in a dual write? Name two patterns that avoid the problem.

7. Explain CQRS and its main consistency consequence for the user interface.

8. What does event sourcing give, and what does it cost? When is it appropriate?

9. Why is there no transaction across stores in a polyglot system, and how is consistency constructed instead?

10. What is a staleness budget, and how is it enforced?

11. In the online-shop design, why is checkout kept dependent on only one strongly consistent store?

12. Compute the availability of a request that depends in series on four components with availabilities of 99.9%, 99.95%, 99.9%, and 99.99%.

13. Describe the steps of a safe migration from one store to another, including shadow reads.

14. Explain how a disaster-recovery plan should treat a system of record and its derived stores.

15. Why must a privacy deletion request reach caches, indexes, and analytics copies as well as the database?

Exercises

1. **Store selection.** A food-delivery application has restaurants and menus, customer orders and payments, live courier positions, restaurant search with filters, and personalized restaurant recommendations. Assign each dataset to a store family, state which are systems of record and which are derived, and justify each choice.

2. **Data flow.** Draw the data flow of Exercise 1 from systems of record through a log to the derived stores. Specify the keys of the events, the idempotency key for each sink, and the staleness budget for each derived copy.

3. **Estimation.** The food-delivery application has 2 million daily active users, each placing 0.1 orders per day on average and opening the application 8 times, each time making 25 requests. Compute the average and peak (factor 4) request rates and the daily and yearly order storage at 3 KB per order with 3 replicas.

4. **Availability.** A checkout request calls the API gateway (99.99%), the order database (99.95%), the payment provider (99.9%), and the search service (99.9%). Compute the availability if all four are required. Redesign the flow so that the search service is not required, and recompute. If the order database is replaced by two independent replicas with 99.95% each, with perfect failover, what is the new value?

5. **Dual write.** A service writes a new customer to PostgreSQL and then to Elasticsearch in the same request. List three failures that produce inconsistent state, and redesign the flow with an outbox. State what the relay, the indexer, and the reconciliation job each do.

6. **CQRS.** Design the read models for a banking application with write-model tables for accounts and transactions: an account-summary view, a monthly-statement view, and a fraud-detection view. For each, name the store, how it is fed, and the acceptable staleness.

7. **Failure analysis.** For the architecture you designed in Exercise 2, create a table like Table 22.4 listing the effect of losing each store and the planned degradation. Identify the single point of failure on the critical path, and propose a mitigation.

8. **Migration.** A company wants to move its product search from database full-text search to a search engine without downtime. Write a migration plan following the six steps of Section 22.8, with the specific checks you would perform during shadow reads and the criteria for switching traffic.

9. **Disaster recovery.** A regional outage destroys all stores of the online shop in Section 22.6. Write a recovery plan stating the order in which stores are restored or rebuilt, which are restored from backup and which are rebuilt from the log, and how you would estimate the achievable recovery time.

10. **Governance.** A customer requests deletion of all personal data. List every store of the online shop that may hold it, including derived copies and backups, describe how the deletion is propagated and verified, and explain how backups are handled.

11. **Architecture review.** Choose a system you know and review its data architecture with the checklist of Section 22.11. Record the answers, the problems found, and the changes you would propose, ranked by risk.

References

- Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (Eds.). (2016). *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media.
- Dean, J., & Barroso, L. A. (2013). The Tail at Scale. *Communications of the ACM*, 56(2), 74–80.
- Dehghani, Z. (2022). *Data Mesh: Delivering Data-Driven Value at Scale*. O'Reilly Media.
- Evans, E. (2003). *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley.
- Fowler, M. (2004). StranglerFigApplication. https://martinfowler.com/bliki/StranglerFigApplication.html
- Fowler, M. (2011). PolyglotPersistence. https://martinfowler.com/bliki/PolyglotPersistence.html
- Helland, P. (2007). Life Beyond Distributed Transactions: An Apostate's Opinion. In *Proceedings of CIDR '07*.
- Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.
- Newman, S. (2021). *Building Microservices* (2nd ed.). O'Reilly Media.
- Richardson, C. (2018). *Microservices Patterns*. Manning Publications.
- Sadalage, P. J., & Fowler, M. (2012). *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence*. Addison-Wesley.
- Stonebraker, M., & Çetintemel, U. (2005). "One Size Fits All": An Idea Whose Time Has Come and Gone. In *Proceedings of ICDE '05*, 2–11.
- Vernon, V. (2013). *Implementing Domain-Driven Design*. Addison-Wesley.

Chapter 23

Designing Scalable Data Systems

Learning Objectives

After reading this chapter, you will be able to:

- Define scalability, and describe load and performance with load parameters, throughput, and percentile response times.
- Apply simple quantitative models: Little's law, queueing behavior, Amdahl's law, and the Universal Scalability Law.
- Use the scale cube and the scaling ladder to choose scaling strategies in a sensible order.
- Design caches, and avoid stampedes and stale data.
- Scale reads and writes using replicas, read models, batching, queues, sharded counters, and keyset pagination.
- Protect systems under overload with rate limiting, load shedding, and backpressure.
- Model data to avoid unbounded growth and hot spots, and tier data by temperature.
- Plan capacity, test load correctly, and walk through the design of systems such as a URL shortener and a social timeline.

23.1 What Scalability Means

A system is *scalable* if it can cope with growth in load by adding resources, without a redesign and without a disproportionate rise in cost or loss of quality. Scalability is not a single property, and a statement such as "this database is scalable" is meaningless until it says *scalable in what respect*: data volume, number of concurrent users, read rate, write rate, number of tenants, or geographic spread. A system may scale well for reads and poorly for writes, or well to ten times its current load and not to a hundred times (Kleppmann, 2017).

Earlier chapters supplied the building blocks: replication and partitioning (Chapter 9), consistency trade-offs (Chapters 7 and 8), fault tolerance (Chapter 10), specialized stores, and pipelines (Chapters 22 and 21). This chapter is about using them *quantitatively and*

in the right order: measuring load, finding the bottleneck, applying the cheapest effective remedy, and verifying the result.

23.1.1 Describing Load

Before discussing growth one must describe the present load with a few numbers, called *load parameters*. Which numbers matter depends on the system: requests per second to a web server, the ratio of reads to writes in a database, the number of simultaneously active users in a chat room, the hit rate on a cache, the number of followers per user in a social network. The last example shows why the choice matters: the average number of followers is modest, but the *distribution* has a long tail of accounts with millions of followers, and that tail, not the average, determines the design (Section 23.14).

23.1.2 Describing Performance

Once load is described, one asks what happens to performance when load grows. Two measures dominate:

- **Throughput**: the number of operations per second (or records or bytes per second) that the system sustains.
- **Response time**: the time between a request and its answer, as the client sees it, including queueing, network, and processing.

Response time is a *distribution*, not a number, because identical requests take different times. The average hides the experience of the slowest requests, so performance is reported with *percentiles*: the median (p50) is the typical response, and the high percentiles (p95, p99, p99.9), called *tail latencies*, show how bad the worst cases are. Tail latency matters for two reasons. The slowest requests are often those of the most valuable users, who have the most data and make the most purchases. And in systems that fan out to many components, the chance that a request hits at least one slow component grows quickly: if each of 100 backend calls is slow in 1% of cases, then a user request that waits for all of them is slow in $1 - 0.99^{100} \approx 63\%$ of cases (Dean and Barroso, 2013). Service-level objectives (Chapter 22) should therefore be stated in percentiles, for example "99% of reads complete within 200 ms."

23.1.3 Scaling Up and Scaling Out

Vertical scaling (scaling up) means moving to a more powerful machine. It is simple, because the software need not change, but it is limited by the largest machine available, becomes disproportionately expensive near the top, and leaves a single point of failure. *Horizontal scaling* (scaling out) means spreading the load over several machines. It has no fixed ceiling and tolerates failures, but it requires software designed for distribution, with all the issues of Chapters 9–10. *Elasticity* is the ability to add and remove resources automatically as load changes, which depends on how easily the system can redistribute its state. A pragmatic view is that a single large machine with replicas goes remarkably far, and that horizontal scaling should be adopted where measurements show that it is needed.

23.2 Quantitative Models

Simple mathematical models give surprisingly useful estimates, and they explain behavior that otherwise seems paradoxical.

23.2.1 Little's Law

For any stable system, the average number L of requests in the system equals the arrival rate λ multiplied by the average time W each request spends there (Little, 1961):

$$L = \lambda W. \tag{23.1}$$

At 500 requests per second with an average response time of 0.2 s, there are on average $500 \times 0.2 = 100$ requests in progress. The law is useful for sizing connection pools, thread pools, and queues: a service that needs 100 concurrent database connections at the average cannot run on a pool of 20, and a slowdown of the database to 0.4 s doubles the number of connections required, which is how a modest slowdown turns into pool exhaustion and a cascade.

23.2.2 Queueing: Why Latency Explodes Near Saturation

A resource that serves requests one at a time, with random arrivals, behaves like a queue. For the simplest model (the M/M/1 queue), with average service time S and utilization ρ (the fraction of time the resource is busy), the average response time is

$$R = \frac{S}{1 - \rho}. \tag{23.2}$$

Table 23.1 shows the consequence for a service time of 10 ms, and Figure 23.1 plots the curve.

Table 23.1: Response time as a function of utilization (M/M/1 model, service time 10 ms).

Utilization ρ	**Response time R**	**Multiple of S**
50%	20 ms	2
80%	50 ms	5
90%	100 ms	10
95%	200 ms	20
99%	1 000 ms	100

The practical rule is that *systems must be provisioned with headroom*: running a resource at 90% utilization means that responses are ten times slower than the service time, and any burst pushes the queue toward infinity. Typical targets are 50–70% average utilization for latency-sensitive services, with the remaining capacity absorbing bursts and failures. The same behavior explains why an overloaded system becomes dramatically slower than one that is merely busy, and why overload control (Section 23.8) is essential.

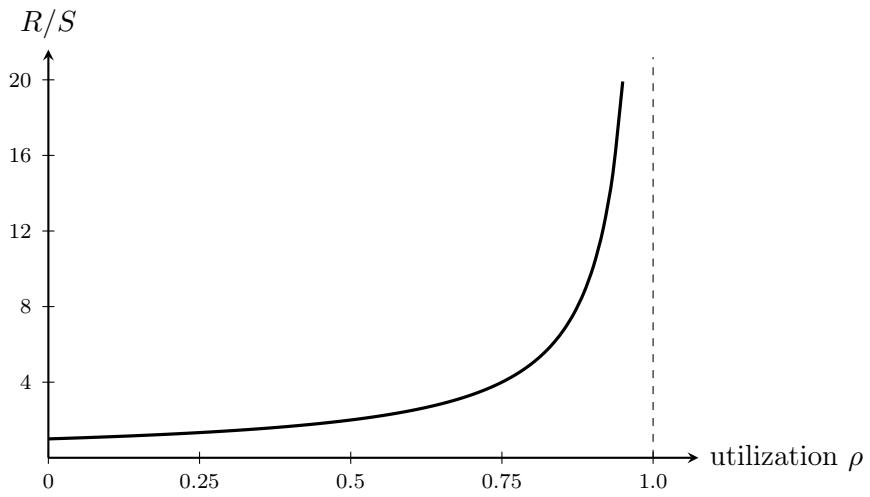


Figure 23.1: Average response time relative to service time as utilization grows (M/M/1). Response time rises slowly at first and then explodes as utilization approaches 100%.

23.2.3 Amdahl's Law and the Universal Scalability Law

Adding machines does not speed up work that cannot be parallelized. If a fraction p of the work can be done in parallel and the remainder $1 - p$ is serial, then the speedup with N machines is at most (Amdahl, 1967)

$$\text{speedup}(N) = \frac{1}{(1 - p) + p/N}. \tag{23.3}$$

With $p = 0.95$ and $N = 100$ the speedup is $1/(0.05 + 0.0095) \approx 16.8$, and no number of machines can exceed $1/0.05 = 20$. In data systems the serial part is often a single primary, a global lock, a sequence generator, or a coordinator.

Real systems do worse than Amdahl predicts, because machines also spend effort coordinating with each other. The *Universal Scalability Law* models relative capacity $C(N)$ with two penalties: contention α (waiting for shared resources, like the serial fraction) and coherency β (the cost of keeping copies consistent, which grows with the square of N) (Gunther, 2007):

$$C(N) = \frac{N}{1 + \alpha(N - 1) + \beta N(N - 1)}. \tag{23.4}$$

Because of the quadratic term, capacity reaches a maximum at $N^* = \sqrt{(1 - \alpha)/\beta}$ and *decreases* beyond it. For $\alpha = 0.05$ and $\beta = 0.0001$, $N^* \approx 97$: adding a hundredth machine would reduce throughput. The model is useful less for exact predictions than for interpreting measurements: fitting it to load-test results at several cluster sizes shows whether contention or coherency limits scaling, and therefore what to fix.

23.3 Choosing Scaling Strategies

23.3.1 The Scale Cube

Abbott and Fisher (2015) describe three independent ways to scale, which they call the scale cube:

- **X-axis: cloning.** Run several identical copies behind a load balancer. For *stateless* services this is trivial; for data stores it corresponds to replication (read replicas).
- **Y-axis: functional decomposition.** Split the system by function or domain, into services or modules with their own data, which is the microservice approach and the polyglot approach of Chapter 22.
- **Z-axis: data partitioning.** Split the data by a key (customer, region, hash), so that each instance handles only a subset, which is sharding (Chapter 9).

Stateless application servers are scaled easily on the X axis; the difficulty lies in the stateful data tier, where X scaling helps only reads, and Z scaling is required for writes and for data larger than one machine.

23.3.2 The Scaling Ladder

Techniques differ greatly in cost and complexity, and the order in which they are applied matters. Table 23.2 gives a ladder from cheapest to most expensive; the rule is to climb only as far as the measurements demand.

Premature sharding is one of the costliest mistakes in data design: it permanently complicates queries, transactions, and operations, to solve a problem that an index, a cache, or a larger machine might have solved.

23.4 Caching

A cache stores the results of expensive operations closer to the consumer, so that repeated requests are served cheaply. Caches exist at every layer: the browser, a content delivery network at the edge, an application-level cache such as Redis or memcached, and the database's own buffer pool (Nishtala et al., 2013).

23.4.1 Why Caching Is So Effective

Caches work because access is highly skewed: a small fraction of the data receives most of the requests. The load that remains on the database after a cache is the miss traffic. If the request rate is R and the hit ratio is h, the database sees

$$R_{\text{db}} = (1 - h)\, R. \tag{23.5}$$

At 100 000 requests per second, a hit ratio of 95% leaves 5 000 requests per second for the database, and raising it to 99% reduces this to 1 000. Small improvements in the hit ratio

Table 23.2: A ladder of scaling techniques for the data tier, from cheapest to most complex.

Step	Technique	What it addresses	Cost and risk
1	Measure, fix queries, add indexes	Slow queries, missing indexes, N+1 patterns	Low; often gives the largest gains
2	Cache hot data	Read load on the database	Staleness, invalidation complexity
3	Scale up the machine	General capacity	Money; single point of failure
4	Read replicas, read models	Read throughput	Replication lag; read-your-writes handling
5	Queue and batch writes	Write bursts, small writes	Added latency; eventual processing
6	Partition (shard) the data	Write throughput; data larger than a machine	Cross-shard queries and transactions; resharding
7	Change the data model or store	Fundamental mismatch of workload and store	Migration effort (Chapter 22)

therefore have large effects at the high end, and a falling hit ratio, for example after a deployment that empties the cache, can overload the database suddenly.

23.4.2 Caching Patterns

- **Cache-aside (lazy loading).** The application checks the cache; on a miss it reads the database, stores the result in the cache, and returns it. It is simple and tolerates cache failure, but a miss is slow and data can be stale.
- **Read-through.** The cache itself loads missing data from the database, so the application sees only the cache.
- **Write-through.** Writes go to the cache and the database together, so the cache stays current, at the cost of slower writes.
- **Write-behind (write-back).** Writes are acknowledged by the cache and written to the database later, which is fast but risks loss if the cache fails before the flush. It should be used only for data that can tolerate this.

23.4.3 Invalidation and Staleness

"There are only two hard things in computer science: cache invalidation and naming things" is a joke with a basis. Strategies are to use a time to live (TTL) that bounds staleness, to invalidate or update the entry when the data changes (preferably driven by the change log of Chapter 21 rather than by scattered application code), and to version keys (for example

by including a version number in the key) so that a change simply moves readers to a new key. A cache is derived data in the sense of Chapter 22: it must never be the only copy of anything that matters, and the staleness that it introduces counts against the consistency budget (Chapter 8).

23.4.4 Cache Stampedes

When a popular entry expires, many concurrent requests may miss at the same moment and all query the database to rebuild it, a *stampede* (or thundering herd) that can overload the database precisely when the entry is most popular. Remedies are: allowing only one caller to rebuild while others wait or serve the old value (request coalescing or a short lock), adding random *jitter* to TTLs so that entries do not expire together, refreshing hot entries *before* they expire, and warming the cache before directing traffic to a new instance. Listing 23.1 illustrates the lock-and-jitter approach with Redis commands.

Listing 23.1: Cache-aside with a rebuild lock and jittered expiry (Redis commands; illustrative).

```
GET product:7
-- Miss. Take a short lock so that only one caller rebuilds the entry.
SET lock:product:7 <token> NX PX 5000
-- The winner reads the database, then stores the value with a TTL of
-- 300 seconds plus a random 0-60 seconds of jitter:
SET product:7 <json> EX 347
-- Other callers either wait briefly and retry GET, or serve a stale copy.
```

23.4.5 Other Hazards

Negative results should also be cached (briefly), or requests for nonexistent keys will bypass the cache every time, which attackers can exploit. A hot key can overload a single cache node, just as in a database; it may be replicated across nodes or held in a small local cache in each application instance. And a cache that has become essential to the system's capacity is a hidden dependency: if the database cannot survive without it, its failure is an outage, and this must be designed for with sufficient database capacity, load shedding, or a gradual warm-up.

23.5 Scaling Reads

Most applications are read-heavy, with ratios of ten to a thousand reads per write, so scaling reads is the first and easiest problem.

- **Read replicas** add read capacity, with the replication-lag anomalies of Chapter 8 to be managed by routing reads that follow writes to the primary or by session tokens.
- **Read models and precomputation.** Instead of computing a complex result on each request (joins, aggregations, rankings), compute it ahead of time when data changes

and store it ready to read, as in CQRS and materialized views (Chapter 22). This converts repeated work into one-time work.

- **Denormalization.** Storing redundant data to avoid joins at read time trades write complexity and storage for read speed, and is the normal approach in the stores of Chapters 3 and 4.
- **Efficient pagination.** Offset pagination (`OFFSET 100000`) forces the database to read and discard all preceding rows, so its cost grows with the depth of the page. *Keyset* (or seek) pagination instead continues from the last row seen, using an index, at constant cost, as Listing 23.2 shows. This is the relational counterpart of `search_after` in search engines (Chapter 13).

Listing 23.2: Offset pagination versus keyset pagination in PostgreSQL.

```
-- Slow for deep pages: reads and discards 100000 rows
SELECT id, title FROM articles
ORDER BY created_at DESC, id DESC
LIMIT 20 OFFSET 100000;

-- Constant cost: continue after the last row of the previous page
CREATE INDEX articles_created_id_idx ON articles (created_at DESC, id DESC);

SELECT id, title FROM articles
WHERE (created_at, id) < ('2026-03-01␣10:00:00+00', 981233)
ORDER BY created_at DESC, id DESC
LIMIT 20;
```

23.5.1 The N+1 Query Problem

A frequent source of avoidable load is the *N+1* pattern: code fetches a list of N items with one query and then issues one more query per item to fetch related data, producing $N + 1$ round trips where one or two would suffice. Each round trip adds network latency and database work, so a page that lists 100 items triggers 101 queries. The cure is to fetch related data in bulk with a join or an `IN` list, or to use batch loading. Query logs and tracing (Section 23.12) reveal it quickly.

23.6 Scaling Writes

Writes are harder to scale than reads, because every write must reach the authoritative copy, and copies cannot simply be added.

- **Batching.** Grouping many small writes into one larger operation amortizes per-request overhead, disk synchronization, and network round trips. It is the main reason bulk loads are orders of magnitude faster than row-by-row inserts. The price is added latency, so batch sizes and time limits (for example, flush after 1 000 records or 50 ms) are tuned together.

- **Queue-based load leveling.** A durable queue or log (Chapter 21) absorbs bursts: producers append quickly, and consumers write to the database at a steady rate the database can sustain. The system accepts requests during a peak and catches up afterwards, as long as the average load is within capacity.

- **Partitioning** spreads writes over several primaries (Chapter 9), with the choice of key deciding whether the spread is even.

- **Append-only and log-structured storage** turn random writes into sequential ones, as in the stores of Chapters 4 and 12.

- **Write coalescing.** Collapsing many updates to the same item into one (for example, counting in memory and flushing the total each second) reduces write volume dramatically for hot items.

23.6.1 Hot Rows and Sharded Counters

A single row updated by every request, such as a global counter, the like count of a viral post, or the stock of a flash-sale item, becomes a bottleneck: updates to one row are serialized by locking, regardless of the number of machines. The remedy is to split the hot item into several *shards* that are updated independently and summed when read. Listing 23.3 spreads a counter over 16 rows; each increment goes to a random shard, so concurrent writers rarely collide, and a read sums the shards. The cost is a slightly more expensive read and an approximate view if one reads while writers are active, which suits counters but not invariants such as "do not oversell stock" (which need the guarded update of Chapter 22, possibly with stock allocated in blocks to each shard).

Listing 23.3: A sharded counter that spreads writes over 16 rows (PostgreSQL).

```
CREATE TABLE like_counters (
  post_id bigint NOT NULL,
  shard int NOT NULL,
  cnt bigint NOT NULL DEFAULT 0,
  PRIMARY KEY (post_id, shard)
);

-- Increment: pick a random shard to avoid contention on one row
INSERT INTO like_counters (post_id, shard, cnt)
VALUES (42, floor(random() * 16)::int, 1)
ON CONFLICT (post_id, shard)
DO UPDATE SET cnt = like_counters.cnt + 1;

-- Read: sum the shards
SELECT sum(cnt) AS likes FROM like_counters WHERE post_id = 42;
```

23.7 Partitioning Decisions

Chapter 9 described partitioning mechanisms; this section adds design guidance. The *partition key* is the most consequential and least reversible decision, because it determines which queries are cheap and which are scatter–gather over all partitions. Table 23.3 reviews typical choices.

Table 23.3: Partition key choices and their typical problems.

Key	Benefit	Typical problem
User or tenant identifier	A user's data is together; per-user queries hit one partition	Very large tenants become hot; cross-user queries scatter
Timestamp or date	Time-range scans; easy retention	All current writes hit the newest partition
Random or hash of identifier	Even spread of data and load	No locality; range queries scatter
Geographic region	Local latency; data residency	Uneven region sizes; users who move
Low-cardinality attribute (status, country)	None to speak of	Few partitions, severe skew

Guidelines that follow from experience:

- Choose the key from the *dominant access pattern*, so that the most frequent and most latency-sensitive queries touch one partition.
- The key should have *high cardinality* and an even distribution of load, not only of data.
- Keep data that is updated together in the same partition, so that transactions stay local (Chapter 10).
- Expect *hot keys* and plan for them (Section 23.6), and look at the distribution, not the average.
- Use many more logical partitions than machines, so that rebalancing moves whole partitions, and plan the resharding procedure before it is needed.
- For multi-tenant systems, consider a *cell-based* architecture, in which tenants are assigned to independent, complete copies of the system (cells). Growth is handled by adding cells, and a failure or overload affects one cell only, which limits the blast radius.

23.8 Overload Control

Because response time explodes near saturation (Section 23.2), a system that accepts all the work offered will, when overloaded, serve no one well. It is better to deliberately refuse some

work so that the rest can be served properly. Several mechanisms cooperate:

- **Rate limiting** restricts how much each client may send. The *token bucket* algorithm is common: a bucket holds up to b tokens and is refilled at r tokens per second; each request consumes a token and is rejected or delayed when the bucket is empty. It allows bursts of up to b requests while enforcing a sustained rate of r.
- **Load shedding** rejects work when the system is saturated, preferably the least valuable work first (for example, background tasks and anonymous traffic before checkout), and fast, with a clear "retry later" response, because a rejection costs far less than a timed-out request that occupied resources.
- **Admission control and concurrency limits** bound the number of requests in progress, often adaptively, to keep the system at the knee of the curve of Figure 23.1.
- **Backpressure** propagates slowness upstream so that producers slow down instead of overflowing queues (Chapter 21). Unbounded queues are a trap: they hide overload until memory is exhausted, and latency grows without limit.
- **Timeouts, circuit breakers, bulkheads, and retry budgets** (Chapter 10) keep a slow dependency from consuming all resources and prevent retry storms from amplifying an overload.
- **Graceful degradation** serves reduced functionality (cached or simplified results) when full functionality is not possible.

23.9 Data Modeling for Scale

Scalability problems often originate in the data model rather than in the infrastructure. Several principles recur:

- **Model for access patterns.** In stores that scale horizontally, design tables and keys around the queries, not around entities, and accept denormalization (Chapter 4).
- **Avoid unbounded growth in a single object.** A row, document, or partition that grows forever, such as a list of all events of a user stored in one document, or a wide-column partition per device that never rolls over, will eventually become too large to read, write, or replicate. Bound it by splitting on time or size (for example one partition per device per day).
- **Precompute aggregates** that are read often, and keep raw data for recomputation.
- **Separate hot and cold data.** Recent or frequently used data is kept on fast storage, and the rest is moved to cheaper tiers, as in Table 23.4.
- **Plan the lifecycle.** Decide how long data is kept and how it is archived or deleted; unbounded retention is unbounded cost.

- **Avoid global ordering requirements.** A single global sequence, a global unique constraint, or a total order of all events forces coordination through one node. Prefer identifiers that can be generated independently (for example time-ordered unique identifiers or the "snowflake" scheme, which combines a timestamp, a machine identifier, and a counter) and ordering per entity.

Table 23.4: Data temperature tiers.

Tier	Data	Storage	Access
Hot	Recent, frequently read or written	Memory, local SSD	Milliseconds
Warm	Older but still queried	Standard SSD or disk, replicas	Tens of milliseconds to seconds
Cold	Rarely accessed history	Object storage, compressed columnar files	Seconds to minutes
Archive	Retained for compliance	Archival storage	Hours; restore on request

23.10 Scaling Across Regions

Global systems add geography to the problem. Network latency between continents (on the order of 100 ms round trip) cannot be reduced by software, and cross-region data transfer is often the largest component of cost. Design responses, building on Chapter 9, include:

- **Edge caching and content delivery networks** serve static and cacheable responses close to users.
- **Regional deployments with home-region data**: each user's data is owned by the nearest region, so most requests are local, with cross-region replication for resilience and for global features.
- **Read replicas in each region** for locally served reads, with writes forwarded to the owning region.
- **Asynchronous replication between regions** for most data, and consensus replication only for the small subset of data that needs global strong consistency.
- **Avoiding chatty cross-region calls.** A request that makes ten sequential calls across an ocean takes a second before any work is done; batch the calls, move computation to the data, or replicate the data.

23.11 Elasticity, Capacity, and Cost

23.11.1 Capacity Planning

Capacity planning turns forecasts into resources. A basic method is to (1) measure the capacity of one unit (a node, a shard) at the target utilization by load testing, (2) forecast peak load with growth, (3) divide to find the number of units, and (4) add redundancy for failures and maintenance. If one node sustains 2 000 requests per second at 60% utilization, the forecast peak is 30 000 requests per second, and the system must survive the loss of one node ($N + 1$), then it needs $30\,000/2\,000 = 15$ nodes plus one, 16 in all. Systems that must survive the loss of a whole availability zone need correspondingly more headroom, and the calculation should be repeated as measured numbers replace estimates.

23.11.2 Autoscaling

Autoscaling adds and removes instances according to a metric. For stateless services it works well; for stateful data systems it is harder, because adding a node triggers data movement that itself consumes resources (Section 23.7), and removing one requires draining its data. Guidance includes scaling on a metric that reflects the true bottleneck (queue length or consumer lag usually reflects it better than CPU), allowing for start-up and warm-up time (a new node with a cold cache serves poorly), preventing oscillation with cool-down periods, and keeping a floor and a ceiling to limit both outages and bills. Serverless databases move this adjustment into the provider's platform, at the price of less control and less predictable cost and latency.

23.11.3 Cost as a Design Dimension

Scalability that ignores cost is not sustainable. The cost per request, per gigabyte stored, and per gigabyte transferred should be known and tracked, because they often diverge from the growth in usage: a design that scales linearly in capacity may still be uneconomical. Tiered storage, compression, reserved capacity, avoiding cross-region traffic, and removing unneeded copies are typical levers. Cost is a legitimate reason to choose a simpler design that scales less far.

23.12 Load Testing and Performance Engineering

Design claims about scalability must be verified. Good practice includes:

- **Realistic workloads.** Use production-like data volumes, key distributions (including hot keys), read/write mixes, and request sizes. A test with uniformly random keys on an empty database proves little.
- **Open versus closed workload models.** In a *closed* model, a fixed number of clients each wait for a response before sending the next request, so a slowing system automatically receives less load and hides its problems. In an *open* model, requests

arrive at a fixed rate regardless of response times, as real users do, and reveals queueing collapse. Tools and test scripts that omit the waiting time of delayed requests underreport latency, a flaw known as *coordinated omission*.

- **Kinds of test.** *Load tests* at expected peak; *stress tests* beyond capacity to find the breaking point and the failure behavior; *spike tests* with sudden surges; *soak tests* over many hours to reveal leaks and slow degradation; and *failover tests* while under load.
- **Report percentiles and throughput together**, as curves of latency against offered load, and identify the knee where latency begins to rise.
- **Find the bottleneck systematically.** The *USE method* examines each resource for **u**tilization, **s**aturation (queued work), and **e**rrors (Gregg, 2013); the *RED method* tracks the **r**ate, **e**rrors, and **d**uration of each service. Distributed tracing shows where a request spends its time, and query plans (Chapter 11 and the database chapters) show where a query does.
- **Change one thing at a time** and keep records, so that improvements and regressions can be attributed.
- **Test continuously.** Run a performance test in the delivery pipeline and compare it with a baseline, so that regressions are caught before release.

23.13 Case Study: A URL Shortener

A URL-shortening service maps short codes to long URLs and redirects visitors. It is a compact example of the method.

23.13.1 Requirements and Estimates

Assume 100 million new short URLs per month, a read-to-write ratio of 100:1, and retention for five years. Then:

- **Writes:** $10^8/(30 \times 86\,400) \approx 39$ per second on average.
- **Reads:** about $100 \times 39 \approx 3\,900$ redirects per second on average, perhaps $3\times$ that at peak, about 12 000 per second.
- **Data:** $100\text{ M} \times 12 \times 5 = 6$ billion URLs; at roughly 500 bytes per record, about 3 TB, or 9 TB with three replicas.
- **Code length:** a code of k characters from a 62-character alphabet (digits and letters) gives 62^k codes: $62^6 \approx 5.7 \times 10^{10}$, more than the 6 billion needed, so six characters suffice with room to spare.

The numbers show a modest write rate, a read rate easily served by caching, and a data volume that needs partitioning or a store that scales, but not extreme performance.

23.13.2 Design

- **Generating codes.** A central counter encoded in base 62 yields short, unique codes, but a single counter is a bottleneck and a single point of failure, and sequential codes are guessable. Alternatives are to give each application instance a range (block) of counter values to hand out locally, to generate random codes and check for collisions with a conditional insert, or to use a snowflake-style identifier encoded in base 62 (longer). Collision checks with a conditional write (Chapter 8) keep uniqueness without a global lock.

- **Storage.** The access pattern is a lookup by code, with no joins or range queries, which suits a partitioned key–value or wide-column store keyed by the code (hash partitioning spreads load evenly), or a relational database sharded by code. A relational database with a primary key on the code is entirely adequate at the write rate computed above, and the question becomes data size, not rate.

- **Caching.** Reads are heavily skewed toward popular links, so a cache of the hottest codes in front of the store can absorb the vast majority of redirects, with stampede protection for viral links (Listing 23.1). Because short URLs are almost immutable, long TTLs are safe, and invalidation is needed only for deletions.

- **Redirect semantics.** A permanent redirect (HTTP 301) can be cached by browsers, which reduces load but prevents counting visits; a temporary redirect (302) preserves counting at higher load.

- **Analytics.** Click counting is a high-volume write that must not slow the redirect. The service publishes click events to a log, and stream processing (Chapter 21) aggregates them asynchronously into an analytics store, so that the critical path remains a single cache or key lookup.

- **Abuse and overload.** Rate limiting per client, and checks against malicious destinations at creation time, protect the service.

23.14 Case Study: A Social Timeline

A timeline shows a user the recent posts of the accounts they follow. The design question is when to do the work of assembling it.

23.14.1 Fan-Out on Read and on Write

- **Fan-out on read (pull).** Store each post once. When a user opens the timeline, fetch the recent posts of everyone they follow and merge them. Writes are cheap, but each read is expensive, requiring a query for every followed account, and reads vastly outnumber writes.

- **Fan-out on write (push).** When a user posts, insert the post's identifier into a precomputed timeline of each follower, kept as a bounded list in a fast store. Reads are then a single lookup, but each post costs one write per follower.

For an account with 200 followers, fan-out on write costs 200 inexpensive writes per post, and each of their later reads is trivial, so it is clearly better. For an account with 50 million followers, a single post would need 50 million timeline writes, delaying delivery and overloading the system, and many of those followers are inactive. The load parameter that decides is the *distribution of followers per account*, not its average.

23.14.2 The Hybrid Design

The common solution is a hybrid: use fan-out on write for ordinary accounts, and fan-out on read for accounts with very many followers. At read time, the system takes the precomputed timeline of the user and merges in the recent posts of the few high-follower accounts the user follows. Table 23.5 compares the approaches.

Table 23.5: Timeline fan-out strategies.

Strategy	Write cost	Read cost	Weakness
Fan-out on read	One write per post	Many queries per view	Slow reads, heavy load on the post store
Fan-out on write	One write per follower	One lookup	Huge write bursts for popular accounts; wasted work for inactive users
Hybrid	Follower writes for ordinary accounts only	One lookup plus a small merge	More complex; threshold and merge logic to maintain

Supporting design points follow from the chapter's principles. Timelines are *derived data*, stored as bounded lists of post identifiers (for example the latest 800) in an in-memory or key–value store, and can be rebuilt from the posts and the follow graph (Chapter 11). Fan-out is done *asynchronously* from a queue, so a post is accepted immediately and distributed shortly after, with workers sized by queue length. Post content is stored once and fetched by identifier through a cache, so timelines stay small. Deletion of a post must remove it from, or be filtered when reading, the timelines that reference it. Search, ranking, and recommendation use the specialized stores of Chapters 13 and 14. The example illustrates how an understanding of the *shape* of the load, together with precomputation and asynchrony, can change the cost of the dominant operation by orders of magnitude.

Table 23.6: Frequent scalability mistakes and their remedies.

Anti-pattern	**Consequence**	**Remedy**
Premature sharding or microservices	Permanent complexity for an unproven need	Climb the scaling ladder (Table 23.2) only as measurements demand
Optimizing without measuring	Effort on the wrong bottleneck	Profile and use USE and RED methods first
N+1 queries and chatty calls	Latency and load multiplied by item count	Batch, join, or prefetch
Unbounded queries, lists, and queues	Memory exhaustion; unpredictable latency	Limits, pagination, bounded queues, backpressure
Global locks, sequences, or single coordinators	Serial bottleneck limiting scale (Amdahl)	Partition ownership; locally generated identifiers
Ignoring tail latency and fan-out	Slow user experience despite good averages	Percentile SLOs; hedging; fewer sequential dependencies
Cache as the only copy, or required for survival	Outage when the cache fails or empties	Treat the cache as derived; size the database to survive cold starts
Running at high utilization	Latency explosion on bursts	Headroom of 30–50%; load shedding
Retries without budgets	Retry storms amplify overload	Backoff with jitter, retry budgets, circuit breakers
No plan for resharding or growth	Emergency migrations	Many logical partitions; rehearsed rebalancing

23.15 Common Scalability Anti-Patterns

23.16 Choosing an Approach

A sound process is: (1) state the requirements as load parameters and percentile objectives; (2) estimate load, data volume, and growth with back-of-envelope numbers (Chapter 22); (3) design the simplest architecture that meets them, with stateless application servers and the smallest necessary stateful tier; (4) identify the likely bottlenecks with the models of Section 23.2, in particular the serial parts and the hot keys; (5) provide overload protection and graceful degradation from the start; (6) choose partition keys and data models around the dominant access patterns; (7) test with realistic open-model load, find the knee, and iterate on the measured bottleneck; and (8) plan capacity with headroom and cost in view, and revisit the design as growth and measurements change the picture.

Table 23.7: Matching the bottleneck to the remedy.

Bottleneck observed	**First remedy**	**If that is insufficient**
A few slow queries dominate database time	Indexes, query rewrite, avoid N+1	Precomputed read model
Read load on the database	Cache; read replicas	Read models; denormalization
Write bursts	Queue and batch	Partition writes
One hot row or key	Sharded counter, key splitting, local aggregation	Redesign the data model
Data larger than one machine	Partitioning by dominant access key	Cells; tiered storage
High tail latency	Remove sequential fan-out; timeouts; hedging	Precompute; reduce dependencies
Overload during peaks	Rate limiting and load shedding	More capacity and autoscaling
Cross-region latency	Regional ownership; edge caching	Regional replicas with local reads

Summary

Scalability is the ability to handle growth in a specific respect by adding resources, and it is discussed with load parameters, throughput, and percentile response times, since averages hide the tail that fan-out amplifies. Simple models guide design: Little's law sizes pools and queues, queueing theory shows that latency explodes as utilization approaches saturation and so demands headroom, and Amdahl's law and the Universal Scalability Law show that serial fractions, contention, and coordination limit what additional machines can achieve. Strategies are chosen in order of cost, along the scale cube of cloning, functional decomposition, and data partitioning, and up a ladder that begins with measurement, indexes, and caches and reaches sharding and store changes last. Caches cut database load in proportion to the hit ratio but bring staleness, invalidation, stampedes, and hidden dependencies. Reads are scaled with replicas, read models, denormalization, keyset pagination, and by eliminating N+1 queries; writes with batching, queue-based load leveling, partitioning, append-only storage, and sharded counters for hot rows. Partition keys must follow the dominant access pattern, have high cardinality and even load, and keep related data together, with cell-based designs limiting blast radius. Overload control through rate limiting, load shedding, backpressure, and degradation keeps systems useful beyond capacity. Data models must avoid unbounded growth, global ordering, and mixing of hot and cold data. Multi-region designs respect latency and cost by owning data regionally and replicating asynchronously. Capacity planning, careful autoscaling, and cost awareness make scale sustainable, and verification demands realistic, open-model load tests and systematic bottleneck analysis. The URL shortener and the social timeline show how estimates, access patterns, caching, asynchrony, and the shape of the load

determine the design. The main lesson is to measure first, scale the real bottleneck with the cheapest effective technique, and keep the design as simple as the requirements allow.

Review Questions

1. Why is "scalable" meaningless without saying in which respect? Give three different respects.

2. Why are percentiles preferred to averages for response times? Show how fan-out makes tail latency matter more.

3. State Little's law and use it to size a connection pool.

4. Explain why response time grows without bound as utilization approaches 100%, and what this implies for provisioning.

5. State Amdahl's law. What limits the speedup when 5% of the work is serial?

6. What do the contention and coherency parameters of the Universal Scalability Law represent? Why can adding nodes reduce throughput?

7. Describe the three axes of the scale cube and give a data-system example of each.

8. Why is premature sharding costly? List the steps of the scaling ladder that should normally come first.

9. Compare cache-aside, read-through, write-through, and write-behind. Which risks data loss?

10. What is a cache stampede, and what are three ways to prevent it?

11. Why does keyset pagination outperform offset pagination for deep pages?

12. What is the N+1 query problem, and how is it fixed?

13. How does a sharded counter remove a hot-row bottleneck, and what does it cost?

14. Describe the token bucket algorithm and what it permits.

15. Why is load shedding better than accepting all work during overload?

16. What is coordinated omission, and how do open and closed workload models differ?

17. Why does the social timeline use a hybrid of fan-out on read and on write?

Exercises

1. **Little's law.** A service handles 800 requests per second, with an average time in the service of 150 ms. How many requests are in progress on average? How many database connections are needed if 60% of the time is spent in the database? What happens if database time doubles?

2. **Queueing.** A database has a service time of 5 ms per query. Using the M/M/1 formula, compute the average response time at utilizations of 40%, 70%, 90%, and 98%. At which utilization would you set an autoscaling target, and why?

3. **Amdahl.** A job is 90% parallelizable. Compute the speedup with 10, 100, and 1 000 machines, and the theoretical limit. Identify what could constitute the serial part in a sharded database, and how to shrink it.

4. **USL.** With $\alpha = 0.02$ and $\beta = 0.0002$, compute $C(N)$ for $N = 10$, 50, and 100, and the value of N at which capacity peaks. Interpret the result for cluster sizing.

5. **Cache sizing.** A product catalog has 5 million items; the most popular 5% receive 80% of the requests. Each item takes 4 KB in the cache. Estimate the memory needed to cache the popular items, the expected hit ratio if only they are cached, and the database load if the request rate is 40 000 per second.

6. **Stampede.** A cache entry for a flash-sale product expires, and 5 000 requests per second request it. Describe what happens to the database, and design the cache behavior (locking, jitter, early refresh) that prevents the problem.

7. **Hot key.** A leaderboard stores a global score counter updated by every game event, at 20 000 updates per second. Explain the bottleneck and redesign it with sharding. How would you read an approximate and an exact total?

8. **Partition key.** A messaging application has 100 million users, and each user has conversations with several others. Compare partitioning messages by user, by conversation, and by time. For each, state which queries are efficient, which scatter, and where hot spots appear.

9. **Overload.** A checkout service receives twice its capacity during a promotion. Describe what happens without protection, and design the mix of rate limiting, load shedding, queueing, and degradation that keeps checkout available. State which requests are shed first.

10. **URL shortener.** Extend the case study to 1 billion new URLs per month. Recompute the write rate, read rate, and storage, determine whether six characters still suffice for five years, and state which parts of the design change.

11. **Timeline.** A social platform has 100 million users. The median user has 150 followers and the top 0.01% have more than 10 million. Estimate the daily timeline writes with pure fan-out on write assuming 20 million posts per day from ordinary users and 10 000

posts per day from the top accounts, and compare with the hybrid. Choose a follower threshold and justify it.

12. **Load test design.** Design a load test for the order database of an online shop: specify the workload model (open or closed), data volume, key distribution, request mix, test types, the metrics reported, and the acceptance criteria.

References

- Abbott, M. L., & Fisher, M. T. (2015). *The Art of Scalability: Scalable Web Architecture, Processes, and Organizations for the Modern Enterprise* (2nd ed.). Addison-Wesley.
- Amdahl, G. M. (1967). Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities. In *Proceedings of the AFIPS Spring Joint Computer Conference*, 483–485.
- Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (Eds.). (2016). *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media.
- Dean, J., & Barroso, L. A. (2013). The Tail at Scale. *Communications of the ACM*, 56(2), 74–80.
- Gregg, B. (2013). *Systems Performance: Enterprise and the Cloud*. Prentice Hall.
- Gunther, N. J. (2007). *Guerrilla Capacity Planning*. Springer.
- Harchol-Balter, M. (2013). *Performance Modeling and Design of Computer Systems: Queueing Theory in Action*. Cambridge University Press.
- Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.
- Little, J. D. C. (1961). A Proof for the Queuing Formula: $L = \lambda W$. *Operations Research*, 9(3), 383–387.
- Nishtala, R., Fugal, H., Grimm, S., et al. (2013). Scaling Memcache at Facebook. In *Proceedings of USENIX NSDI '13*, 385–398.

Glossary

Access pattern
: A specific read or write that an application performs, described by what is requested, how often, and with what latency and consistency needs.

ACID
: Atomicity, Consistency, Isolation, Durability: guarantees for reliable transactions.

Admission control
: Limiting the number of requests in progress, often adaptively, so that a system stays below the point where latency explodes.

Aggregate
: A cluster of related data treated as a single unit for storage, retrieval, and atomic update, such as an order with its line items.

Aggregation pipeline
: A sequence of stages (such as match, group, and sort) that transforms documents step by step to compute results inside the database.

Alias (index)
: A name that points to one or more indexes and can be switched atomically, used for zero-downtime reindexing.

Allowed lateness
: The extra time a stream window stays open after the watermark passes its end, so that late events can still update the result.

Amdahl's law
: A bound on parallel speedup: with a serial fraction $1 - p$ of the work, the speedup with N processors is at most $1/((1 - p) + p/N)$.

Anti-entropy
: A background process that compares replicas, often with Merkle trees, and repairs differences between them.

Append-only file (AOF)
: A log of every write operation that is replayed to rebuild the data after a restart.

Approximate nearest neighbor (ANN)
: A search that returns neighbors that are very likely, but not guaranteed, to be the true nearest ones, in exchange for much higher speed.

AP system
: A system that, during a network partition, remains available but may return stale or divergent data.

Architecture decision record (ADR)
: A short document that records an architectural decision, the alternatives considered, and the reasons for the choice.

At-least-once delivery
: A delivery guarantee in which messages are retried until acknowledged, so none are lost but duplicates are possible.

At-most-once delivery
: A delivery guarantee in which messages are never retried, so none are duplicated but some may be lost.

Atomic commit
: The problem of making all participants of a distributed transaction either commit or abort together.

Autoscaling
: Automatically adding and removing instances according to a metric such as load, queue length, or consumer lag.

Availability
: In CAP, the property that every request to a non-failing node receives a non-error response; in operations, the fraction of time a service works.

Avro
: A compact binary row format with an explicit schema, widely used for events and for schema evolution in streams.

Backfill
: Loading or recomputing historical data into a new or corrected store, usually by replaying a log or a snapshot.

Backpressure
: Propagating slowness upstream so that producers slow down instead of overflowing queues or memory.

BASE
: Basically Available, Soft state, Eventual consistency: a design philosophy favoring availability.

Batching
Grouping many small operations into one larger one to amortize per-request overhead.

Bi-encoder
A retrieval model that embeds the query and each document independently, so that scoring is a vector similarity.

Bitemporal table
A table that records both valid time and transaction time, so that corrections do not erase what was previously believed.

Bloom filter
A compact probabilistic structure that tells whether a key is definitely absent or possibly present, used to skip unnecessary disk reads.

BM25
A lexical ranking function with a saturating term-frequency factor and document-length normalization; the default scoring of Lucene-based engines.

BSON
Binary JSON: the binary encoding used by MongoDB, adding types such as dates, decimals, and binary data.

Bucketing
Adding a bounded grouping, usually a time window, to a key or document so that partitions or documents do not grow without limit.

Bulkhead
A pattern that isolates resources (threads, connections) per dependency so that one failing dependency cannot exhaust them all.

Byzantine fault
A failure in which a node behaves arbitrarily or maliciously, for example by sending conflicting information to different nodes.

Cache-aside
A caching pattern in which the application checks the cache first and, on a miss, reads the database and fills the cache.

Cache stampede
A surge of simultaneous cache misses, often when a popular entry expires, that overloads the underlying database.

Calibration (threshold)
Choosing a similarity-score threshold from labeled queries so that results above it reach a desired precision.

CAP theorem
: During a network partition, a distributed system must choose between consistency and availability.

Cardinality
: The number of distinct values or, in a time-series database, the number of distinct series; high cardinality exhausts memory and slows queries.

Cartesian product
: The combination of every row of one result with every row of another, which appears in a query plan when two patterns are not connected.

Catalog (lakehouse)
: The component that maps table names to the location of their current metadata and usually stores permissions.

Causal consistency
: A model in which operations that are causally related are seen in the same order by everyone, while concurrent ones may differ.

Cell-based architecture
: A design in which tenants are assigned to independent, complete copies of a system, which limits the blast radius of failures.

Change data capture (CDC)
: Reading a database's transaction log and publishing each committed insert, update, and delete as an event.

Chaos engineering
: Deliberately causing failures in a controlled way to confirm that a system degrades as designed.

Checkpoint
: A consistent snapshot of the state of a computation or database, from which processing can restart after a failure.

Choreography
: A saga or workflow style in which each service reacts to the events of the others, with no central controller.

Chunk
: A time-bounded partition of a time-series table, or a segment of a document split for embedding and retrieval.

Chunking
: Splitting documents into smaller passages before embedding and indexing them for retrieval.

Circuit breaker
: A pattern that stops calls to a failing dependency for a period and fails fast, closing again when trial calls succeed.

Closed workload model
: A load-test model in which a fixed number of clients wait for each response before sending the next request, which hides overload.

Clustering column
: A column that determines the sort order of rows within a partition (Cassandra).

Collection
: A group of documents in a document database, roughly analogous to a table.

Columnar format
: A file or storage layout that stores the values of each column together, which speeds analytical scans and improves compression.

Column family
: A named group of related columns in a wide-column table that is stored together.

Commit log
: An append-only file to which writes are recorded first so that they can be replayed after a crash (called the write-ahead log in HBase).

Commit wait
: In Spanner, the pause before releasing locks until the chosen commit timestamp is certainly in the past, which gives external consistency.

Compaction
: A background process that merges immutable data files, discarding overwritten and deleted data.

Compatibility mode
: A rule in a schema registry (backward, forward, or full) that decides which schema changes are accepted.

Compensating action
: An operation that undoes or corrects the effect of an earlier operation that turned out to violate a rule.

Compound index
: An index on several fields in a defined order, used for queries that filter and sort on more than one field.

Conditional write
: A write that is applied only if a condition on the current item holds; the basis of optimistic concurrency.

Consensus
The problem of getting a set of nodes to agree on a single value despite failures.

Consistency level
A per-request setting (Cassandra) that states how many replicas must acknowledge a read or write.

Consistent hashing
A partitioning scheme in which keys and nodes are placed on a hash ring, so that adding or removing a node moves only a small fraction of keys.

Consistent prefix
A guarantee that reads see writes in the order they were issued, so that an answer never appears before its question.

Consumer group
A set of consumers that share the partitions of a topic, each partition being read by exactly one member.

Consumer lag
The difference between the latest offset written to a partition and the offset a consumer has processed.

Coordinated omission
A load-testing flaw in which the waiting time of delayed requests is not recorded, so that latency is under-reported.

Coordinator
The node that handles a request and contacts other nodes on its behalf, or the node that drives a commit protocol.

Copy-on-write
A table strategy in which an update rewrites the affected data files, giving fast reads and slower writes.

Cosine similarity
A similarity measure that compares the direction of two vectors and ignores their length.

Covered query
A query answered entirely from an index, without fetching the documents.

CP system
A system that, during a network partition, preserves consistency by refusing or delaying some requests.

CQL
Cassandra Query Language: an SQL-like language that exposes partition keys and clustering columns.

CQRS
: Command Query Responsibility Segregation: separate models for writing data and for reading it, with read models built from events.

CRDT
: Conflict-free replicated data type: a data structure designed so that concurrent updates on different replicas merge automatically and deterministically.

Cross-encoder
: A model that reads a query and a passage together and outputs a relevance score; accurate, but usable only on a shortlist.

Crypto-shredding
: Erasing data by deleting the encryption key of its subject, which makes every copy unreadable at once.

Curse of dimensionality
: The effect that, in high dimensions, distances between points become nearly equal, so that classical spatial indexes stop working.

Cypher
: A declarative query language for property graphs, used by Neo4j.

Data catalog
: A record of each dataset, its owner, its derived copies, its freshness, and its retention.

Data contract
: A declared schema, meaning, and service level of a dataset on which others depend, with a process for changing it.

Data lake
: A store that keeps data in its original format as files, usually on object storage, with the schema applied on read.

Data mesh
: An approach that treats domain datasets as products owned by the teams that know them best.

Data swamp
: A data lake without governance, in which data cannot be found, trusted, or used.

Data temperature
: The classification of data as hot, warm, cold, or archive according to how often it is accessed.

Data warehouse
: A relational database optimized for analytics, with data cleaned and modeled before it is loaded.

Dead-letter queue
A queue or topic that receives records that could not be processed after bounded retries, for inspection and replay.

Debezium
An open-source platform of log-based change data capture connectors.

Deep pagination
Requesting results far down a ranking, which is expensive because every shard must produce all preceding results.

Delta Lake
An open table format that records changes in an ordered transaction log next to the data files.

Delta-of-delta encoding
A timestamp compression technique that stores the difference between successive differences, which is usually zero for regular data.

Denormalization
Storing related or duplicated data together to make common reads faster, at the cost of redundancy.

Derived data
Data computed from a system of record, such as caches, indexes, and views, which can be rebuilt and may lag.

Disaggregated storage
An architecture that keeps data in shared storage, often object storage, and uses stateless compute nodes that can be scaled independently.

Distributed log
An append-only, ordered, durable and replicated sequence of records, divided into partitions, that many consumers can read independently.

Distributed SQL
Systems that offer full SQL, joins, and distributed ACID transactions over partitioned, replicated data.

Document
A self-describing record, typically JSON or BSON, that can contain nested objects and arrays.

Doc values
A column-oriented structure in a search engine that stores each field's values by document, used for sorting and aggregations.

Dot notation
: The syntax that reaches into nested objects and arrays in a document query, such as `shipTo.city`.

Dot product
: The sum of the products of corresponding vector components; a similarity measure that is sensitive to vector length.

Downsampling
: Replacing a high-resolution series with a coarser summary to reduce volume.

Dual write
: Writing the same change separately to two systems, which is unsafe because a failure between the writes leaves them inconsistent.

Edit distance
: The number of insertions, deletions, substitutions, or transpositions needed to turn one string into another, used for fuzzy matching.

Elasticity
: The ability to add and remove resources automatically as load changes.

Embedding (document model)
: Storing related data inside its parent document.

Embedding (vector)
: A vector of numbers produced by a machine-learning model so that items with similar meaning lie close together.

Error budget
: The gap between 100% and a service-level objective, which can be spent on releases and experiments.

ETL and ELT
: Extract, transform, load, and extract, load, transform: moving data into an analytical store, transforming it before or after loading.

Euclidean distance
: The straight-line distance between two vectors.

Event
: An immutable record that something happened at a point in time.

Event-carried state transfer
: A pattern in which an event carries the data that consumers need, so that they do not have to call the producer.

Event notification
: A pattern in which a thin event announces a change and consumers fetch the details.

Event sourcing
: Storing the sequence of events as the system of record and deriving the current state by replaying them.

Event time
: The time at which an event occurred, as opposed to the time at which the system processes it.

Eventual consistency
: If no new updates occur, all replicas eventually converge to the same value.

Eviction policy
: The rule that decides which keys are removed when memory is full (for example, least recently used).

Exactly-once effect
: A result in which each event affects the output once, obtained from at-least-once delivery combined with idempotent or transactional processing.

Expand and contract
: A schema-change pattern: add the new structure, migrate data and code, and only then remove the old structure.

Explain (query plan)
: A command that shows how a query will be or was executed, including indexes used and documents examined.

Extended reference
: A modeling pattern that references a related document and also copies the few fields read most often.

External consistency
: A guarantee that the order of transaction timestamps matches real-time order; linearizability for transactions.

Facet
: A count of matching documents per category, such as brand or price range, shown beside search results.

Failover
: Promoting a replica to replace a failed leader.

Failure detector
: A mechanism, usually based on timeouts or heartbeats, that suspects which nodes have failed; it is inherently unreliable.

Fan-out
: Sending one request or event to many destinations, or the resulting number of calls.

Fan-out on read and on write
: Two timeline strategies: assemble a feed when it is read, or insert each post into every follower's feed when it is written.

Fencing token
: A monotonically increasing number attached to leadership, used by storage to reject requests from a former leader.

Follower
: A replica that copies the changes of a leader.

Fuzzy matching
: Matching terms that differ by a small edit distance, to tolerate spelling errors.

Gauge
: A metric whose value can go up and down, such as temperature or memory in use.

Global secondary index
: An index that covers all partitions and is itself partitioned by the indexed value.

Gossip protocol
: A peer-to-peer protocol by which nodes periodically exchange state to learn which nodes are alive.

GQL
: ISO/IEC 39075:2024, the international standard declarative query language for property graphs.

Graceful degradation
: Serving reduced functionality, such as cached or simplified results, when full functionality is not possible.

GraphRAG
: Retrieval-augmented generation that expands vector search results along the relationships of a graph.

Gray failure
: A degraded but not dead component, such as a slow disk, that defeats simple up-or-down health checks.

Gremlin
: A functional graph traversal language that is part of Apache TinkerPop.

Ground truth
: The exact answer, or the human-labeled relevant items, against which an index or retrieval system is measured.

Half-open interval
: A period that includes its start and excludes its end, so that adjacent periods neither overlap nor leave gaps.

Harvest and yield
Yield is the fraction of requests that receive an answer; harvest is the fraction of the complete data reflected in an answer.

Hash partitioning
Assigning each record to a partition by a hash of its key, which spreads keys evenly but loses efficient range scans.

Headroom
Spare capacity kept in reserve so that bursts and failures do not push a system toward saturation.

Hedged request
Sending a duplicate request after a delay and using the first reply, to reduce tail latency.

Hidden partitioning
An Iceberg feature in which a table is partitioned by a transformation of a column, used automatically by queries on that column.

Hinted handoff
A technique in which a healthy node temporarily holds writes for an unreachable replica and delivers them when it returns.

Hit ratio
The fraction of requests served from a cache.

HNSW
Hierarchical navigable small world: a layered proximity-graph index for approximate nearest-neighbor search.

Horizontal scaling
Adding more machines to increase capacity.

Hot key
A single key that receives a disproportionate share of traffic and can overload the node that holds it.

Hot partition
A partition that receives a disproportionate share of traffic or data and can overload the nodes that hold it.

Hybrid logical clock (HLC)
A clock that combines physical time with a logical counter, so that timestamps stay close to real time while respecting causality.

Hybrid search
Retrieval that runs keyword and vector search and merges the two rankings.

Hypertable
: A TimescaleDB table that is automatically partitioned into time-based chunks.

Iceberg (Apache Iceberg)
: An open table format that organizes metadata as a tree of metadata files, manifest lists, and manifests, committed by an atomic catalog pointer swap.

Idempotency key
: A unique identifier sent with every attempt of an operation so that the server can detect and ignore repeats.

Idempotent operation
: An operation whose repeated application has the same effect as applying it once.

Impedance mismatch
: The friction between in-memory objects and relational tables.

Index
: An auxiliary data structure, often a B-tree, that lets the database find matching documents without scanning a whole collection.

Index-free adjacency
: A graph storage property in which each node holds direct references to its relationships, so that traversal cost does not depend on total graph size.

In-doubt transaction
: A two-phase commit participant that has voted yes and cannot commit or abort until it learns the coordinator's decision.

In-memory database
: A database that keeps its working data primarily in main memory to achieve very low latency.

In-sync replica (ISR)
: A follower replica that has kept up with the leader of a partition.

Inverted index
: A mapping from each term to the list of documents that contain it.

IVF
: Inverted file index: a vector index that clusters vectors into cells and searches only the nearest cells.

Jitter
: A random component added to a delay or expiry so that clients do not retry or expire in lockstep.

JMESPath
: A small query language for selecting and reshaping JSON, used in cloud command-line tools.

jq A command-line language for selecting and transforming JSON.

JSON
JavaScript Object Notation: a text format built from objects, arrays, strings, numbers, booleans, and null.

JSONPath
A path expression language for selecting values from a JSON document, standardized as RFC 9535.

JSON Schema
A vocabulary for describing and validating the structure of JSON documents.

Kappa architecture
A design that uses a single stream pipeline and reprocesses history by replaying the log.

Keyset pagination
Paging that continues from the last row seen using an indexed key, at constant cost regardless of page depth.

Keyspace
The top-level namespace in Cassandra, at which replication is configured.

Key-value store
A database that maps unique keys to values and is accessed primarily by key.

Knowledge graph
A graph of entities and facts, often from many sources, used for integration, question answering, and retrieval.

Label (graph)
A name that classifies a node, such as Person or Product.

Lakehouse
An architecture that adds a table format with transactions and schema management to open files on object storage.

Lambda architecture
A design that combines a batch layer for accurate results with a speed layer for fast approximate results.

Lamport clock
A logical clock that orders events consistently with causality using a single counter per process.

Last-write-wins (LWW)
Conflict resolution that keeps the write with the highest timestamp, which can silently discard concurrent updates.

Late event
An event that arrives after the watermark has passed the end of the window it belongs to.

Late interaction
A retrieval model that keeps one vector per token and scores by matching each query token to its best document token (ColBERT).

Leader
The node that accepts writes for a piece of data and ships changes to followers.

Leaderless replication
A design in which clients write to several replicas directly and quorums decide success.

Learned sparse retrieval
A model that produces weighted vocabulary vectors, with expansion to related terms, stored in an inverted index.

Lightweight transaction (LWT)
A conditional write in Cassandra, using a consensus protocol among replicas of one partition.

Lineage
A record of which jobs and datasets produced which other datasets.

Linearizability
A strong consistency model in which every operation appears to take effect instantly at some point between its start and finish.

Little's law
$L = \lambda W$: the average number of requests in a system equals the arrival rate multiplied by the average time each spends there.

Load parameter
A number that describes the load on a system, such as requests per second, read/write ratio, or followers per user.

Load shedding
Rejecting work, preferably the least valuable, when a system is saturated, so that the rest can be served properly.

Local secondary index
An index in which each partition indexes only its own records; queries without the partition key must ask all partitions.

Log compaction
A retention mode that keeps at least the latest record for each key, so that a topic becomes a changelog of current state.

Logged batch
A CQL batch whose statements are guaranteed to be eventually applied together, without isolation.

Logical replication
Replication of row-level changes of selected tables, independent of the storage format.

Lookup ($lookup)
A MongoDB aggregation stage that performs a left outer join with another collection.

Lost update
An anomaly in which two concurrent read-modify-write operations overwrite each other, losing one change.

Materialized view
A query result stored as a table and maintained by the database or by application code.

Matryoshka representation
An embedding whose leading dimensions can be used alone with limited loss, allowing a choice between cost and accuracy.

Maximal marginal relevance (MMR)
A diversification method that repeatedly selects the item that balances relevance against similarity to items already chosen.

Mean reciprocal rank (MRR)
The average over queries of one divided by the rank of the first relevant result.

Medallion architecture
Organizing a lakehouse into bronze (raw), silver (cleaned), and gold (curated) zones of increasing quality.

Memtable
The in-memory sorted structure that buffers recent writes before they are flushed to disk.

Merge-on-read
A table strategy in which updates are written to small files that are merged with base files at read time or during compaction.

Merkle tree
A tree of hashes that lets replicas find differences efficiently.

Message queue
A system that delivers each message to one consumer and deletes it after acknowledgment.

Micro-batch
Processing a stream as small, frequent batches.

Monotonic reads
: A guarantee that once a session has seen a value, it never later sees an older one.

Multikey index
: An index that holds one entry per element of an array field.

Multi-leader replication
: A design in which several nodes accept writes and replicate to each other, requiring conflict resolution.

Multi-tenancy
: Serving many customers from one deployment, with isolation by separate indexes, partitions, or filters.

Multi-version concurrency control (MVCC)
: Keeping several versions of each item so that readers see a consistent snapshot without blocking writers.

N+1 query problem
: Issuing one query for a list and one more per item, producing many round trips where one or two would suffice.

nDCG
: Normalized discounted cumulative gain: a ranking metric that rewards relevant results near the top and allows graded judgments.

Nearest-neighbor search
: Finding the stored vectors closest to a query vector.

Network partition
: A failure in which groups of nodes can no longer communicate with each other.

N-gram
: A fragment of a word indexed for partial matches and autocomplete.

Object storage
: A cloud service that holds immutable objects in flat namespaces, with high durability, high latency, and no atomic multi-object operations.

Offset
: The sequential position of a record within a log partition.

openCypher
: The open specification of the Cypher language.

Open workload model
: A load-test model in which requests arrive at a fixed rate regardless of response times, as real users do.

Operator injection
: An attack in which a user supplies a query operator in place of a value, changing the meaning of a document query.

Optimistic concurrency control
: Making a write conditional on the data being unchanged since it was read, and retrying if it has changed.

Orchestration
: A saga or workflow style in which a central coordinator keeps state and tells each service what to do next.

Outbox pattern
: Writing an event to an outbox table in the same transaction as the data change, and relaying it to a broker afterwards.

PACELC
: An extension of the CAP theorem: if there is a partition, choose between availability and consistency; else, choose between latency and consistency.

PageRank
: A graph algorithm that scores each vertex by the scores of the vertices that link to it.

Paging state
: An opaque token that tells a server where to continue a paged result.

Parquet
: A columnar file format that stores column chunks within row groups, with statistics in a footer.

Partial failure
: A situation in which some components of a system fail while others continue to work.

Partial index
: An index that includes only documents or rows that satisfy a filter.

PartiQL
: A SQL-compatible query language for DynamoDB and other key-value and document stores.

Partition key
: The part of a primary key that is hashed to decide which nodes store a row; rows sharing a partition key are stored together.

Percentile
: The value below which a given share of measurements fall; p99 is the response time that 99% of requests beat.

Phi accrual failure detector
: An adaptive detector that outputs a suspicion level from the history of heartbeat arrival times.

Poison message
: A record that fails on every attempt and, if retried forever, blocks its partition.

Polyglot persistence
: Using multiple database technologies within one system, each for what it does best.

Post-filtering
: Retrieving the nearest vectors first and discarding those that fail a metadata filter, which can leave too few results.

Postings list
: The sorted list of documents, often with frequencies and positions, that contain a term.

Predicate pushdown
: Using file or row-group statistics to skip data that cannot match a filter.

Pre-filtering
: Selecting rows by metadata first and then searching only among them.

Prepared statement
: A query with placeholders that is parsed once and executed many times with bound values.

Processing time
: The time at which a system handles an event, as opposed to when it occurred.

Product quantization
: A vector compression method that splits a vector into sub-vectors and replaces each by the identifier of its nearest centroid.

Projection
: Selecting which fields or columns a query returns.

Projection pushdown
: Reading only the columns a query needs from a columnar file.

Prompt injection
: Instructions hidden in retrieved text that try to steer the language model that consumes it.

PromQL
: The query language of Prometheus, built around selecting series by label and applying functions over time ranges.

Property graph
A graph model in which nodes and directed, typed relationships can both carry properties.

Publish–subscribe
A pattern in which producers publish to a topic and any number of subscribers react.

Quantization
Storing vectors in a lossy compressed form to save memory, at some cost in recall.

Query rewriting
Transforming a user query into a clearer form, for example by resolving pronouns from the conversation, before searching.

Queue-based load leveling
Using a durable queue to absorb bursts so that consumers write at a steady rate the database can sustain.

Quorum
The minimum number of nodes, usually a majority, that must participate in an operation for it to be valid.

Raft
A consensus algorithm based on leader election and log replication, designed to be easy to understand.

Range partitioning
Assigning contiguous key ranges to partitions, which supports range scans but can create hot spots.

Range query
A similarity query that returns all items within a given distance or above a given similarity.

Rate limiting
Restricting how much each client may send, for example with a token bucket.

RDF
Resource Description Framework: a graph model built from subject–predicate–object triples with global identifiers.

Read concern
A setting that states what data a read may see, such as local, majority-committed, or snapshot data.

Read model
A denormalized projection built from events or changes and shaped for specific queries.

Read preference
A setting that chooses which replica set member serves reads.

Read repair
Writing the newest value back to a replica that returned an older one during a read.

Read-your-writes
A guarantee that a session's reads reflect its own earlier writes.

Rebalancing
Moving data between nodes when capacity is added or removed or when load becomes uneven.

Recall@k
The fraction of true relevant (or true nearest) items that appear in the top k results.

Reciprocal rank fusion (RRF)
Combining rankings by summing $1/(k + \text{rank})$ for each item in each list, which uses ranks only.

Recovery point objective (RPO)
How much recent data, measured in time, an organization can afford to lose.

Recovery time objective (RTO)
How long a service may be unavailable before it is restored.

RED method
Monitoring each service by its request rate, errors, and duration.

Referencing
Storing the identifier of a related document instead of its content.

Refresh
The operation that makes newly indexed documents searchable in a search engine.

Region
A contiguous range of row keys served by one region server in HBase.

Relationship uniqueness
The Cypher rule that a relationship is traversed at most once within a single match.

Relevance feedback
Moving a query toward results judged relevant and away from those judged irrelevant.

Replay
Resetting a consumer to an earlier offset and processing records again.

Replica set
A group of database nodes holding copies of the same data, with one primary accepting writes.

Replication factor
The number of copies of each piece of data stored in a cluster.

Replication lag
The delay between a write being applied on a leader and becoming visible on a follower.

Re-ranker
A more expensive model that reorders the top candidates from a first-stage retriever.

Response time
The time between a request and its answer as the client sees it, including queueing, network, and processing.

Retention policy
A rule that decides how long data is kept, by age or size, before it is deleted, rolled up, or moved.

Retrieval-augmented generation (RAG)
Supplying a language model with documents retrieved for each question so that answers rest on current or private information.

Retry budget
A limit on the total number of retries across layers, which prevents amplification during overload.

Retry storm
A surge of retries that overloads a struggling service and makes the outage worse.

Saga
A sequence of local transactions, each with a compensating action that undoes it, used instead of a distributed transaction.

Savepoint
A manually triggered, portable snapshot of a stream job, used for upgrades and rescaling.

Scalability
The ability of a system to cope with growth in a specific kind of load by adding resources.

Scale cube
Three independent ways to scale: cloning (X), functional decomposition (Y), and data partitioning (Z).

Scatter–gather
Sending a request to every partition and merging the partial results; latency is set by the slowest partition.

Schema evolution
Changing a schema over time, such as adding, renaming, or dropping columns, without breaking readers.

Schema-on-read
: Applying structure when data is read, as in document stores and data lakes.

Schema-on-write
: Checking every write against a declared schema, as in relational databases.

Schema registry
: A service that stores versions of schemas, gives them identifiers, and checks compatibility of new versions.

Schema validation
: Optional rules that the database checks when documents are written.

Segment
: An immutable mini-index that is the unit of storage and merging in Lucene-based engines.

Selectivity
: The fraction of items that pass a filter.

Semi-synchronous replication
: Waiting for one or a few followers to confirm a write while the others replicate asynchronously.

Serializability
: An isolation level in which the outcome of concurrent transactions equals that of some serial execution.

Service-level objective (SLO)
: A measurable target for a service, such as 99.9% of requests completing within 800 ms over 30 days.

Session guarantees
: Properties such as read-your-writes and monotonic reads that give a single client sensible behavior without global coordination.

Session token
: A logical timestamp returned with each response and sent with the next request so that a replica serves only data at least that fresh.

Shadow read
: Sending queries to a new store as well as the old one and comparing answers without affecting users.

Shard
: One partition of a dataset or index, stored and served by one or more nodes.

Sharded counter
: A counter split into several rows that are updated independently and summed when read, to avoid a hot row.

Sharding
Partitioning data across multiple independent nodes.

Shard key
The field (or fields) that determines how data is distributed across shards.

Sink
The destination of a pipeline, such as a database, an index, or a lakehouse table.

Slowly changing dimension
A warehouse technique for keeping the history of descriptive attributes, in several types.

Small-files problem
The slowdown and cost caused by tables made of very many small files, solved by compaction.

Snapshot
A point-in-time copy of a dataset or table version, used for persistence, recovery, consistent reads, and time travel.

Snapshot isolation
An isolation level in which each transaction sees a consistent snapshot of the database taken at its start.

Snowflake identifier
A unique identifier generated locally from a timestamp, a machine identifier, and a counter.

SPARQL
The standard query language for RDF data.

Split brain
A fault in which two nodes both believe they are the leader and accept conflicting writes.

SQL++
A SQL extension for nested, schema-flexible data that distinguishes NULL from MISSING.

SQL/JSON
The part of the SQL standard that adds JSON functions and operators, including `JSON_TABLE`.

SQL/PGQ
Part of SQL:2023 that adds graph pattern matching over tables viewed as a graph.

SSTable
A sorted, immutable file on disk produced when a memtable is flushed (called an HFile in HBase).

Staleness budget
: The acceptable lag of a derived store, such as search within 10 seconds, which is monitored like an objective.

Stateful operation
: A stream operation, such as aggregating or joining, that must remember information across events.

Static column
: A Cassandra column that has one value per partition, shared by all its rows.

Sticky session
: Routing all requests of a user to the same replica, which gives monotonic reads.

Storage-attached index (SAI)
: A newer Cassandra secondary index that supports ranges and performs better than classic secondary indexes.

Strangler fig
: Replacing a system piece by piece, routing traffic gradually from the old to the new.

Stream processing
: Continuous processing of an unbounded sequence of events as they arrive.

Stream–table duality
: The fact that a stream of changes yields a table when applied, and a table's changes form a stream.

Super node
: A graph node with an extremely large number of relationships, which makes traversals through it expensive.

System of record
: The store that holds the authoritative version of a fact; every other copy is derived.

System-versioned table
: A table that keeps previous row versions automatically, so that earlier states can be queried.

Tag (time series)
: An indexed key–value pair that identifies the source of a series, such as host or region.

Tail latency
: The latency of the slowest requests, usually reported as high percentiles such as p99.

Term dictionary
: The structure that holds all terms of an inverted index and supports fast lookup and prefix search.

TF-IDF
A term weight that multiplies how often a term occurs in a document by how rare it is in the collection.

Throughput
The number of operations per second that a system sustains.

Time-series database (TSDB)
A database designed for append-mostly, timestamped measurements, with compression, time partitioning, and retention.

Time to live (TTL)
A lifetime after which a key is automatically deleted.

Time travel
Querying or restoring a table as it was at an earlier snapshot or time.

Token
The hash of a partition key, which decides which nodes own the partition in Cassandra.

Token bucket
A rate-limiting algorithm in which a bucket refilled at a fixed rate allows bursts up to its capacity.

Tokenization
Splitting text into tokens before indexing or embedding.

Tombstone
A marker written to record a deletion, removed later during compaction.

Topic
A named stream of records in a distributed log, divided into partitions.

Top-k query
A similarity query that returns the k closest items to a query vector.

Transactional outbox
See outbox pattern: an event stored in the same transaction as the data it describes.

Transaction time
The time at which the database learned a fact, as opposed to when it was true in the world.

TrueTime
Spanner's clock API, which reports time as an interval whose width is the clock uncertainty.

Tunable consistency
The ability to choose, per operation, how many replicas must respond, trading consistency against latency and availability.

Two-phase commit (2PC)
: An atomic commit protocol in which a coordinator asks participants to prepare and then tells them all to commit or abort.

Two-phase locking (2PL)
: A concurrency control method in which transactions acquire locks as they go and hold them until commit.

Universal Scalability Law
: A model of capacity that adds contention and coherency penalties to Amdahl's law, so that capacity can fall as nodes are added.

Upsert
: A write that creates a record if it does not exist and updates it if it does.

USE method
: Examining each resource for utilization, saturation, and errors to find a bottleneck.

Utilization
: The fraction of time that a resource is busy.

Valid time
: The period during which a fact is true in the real world.

Vector clock
: A logical clock with one counter per process that captures causality and can detect concurrent events.

Vector database
: A system that stores embeddings and answers nearest-neighbor queries over them.

Vertical scaling
: Increasing the capacity of a single machine.

Virtual node
: One of many positions a physical node occupies on a hash ring, used to balance load.

Watermark
: A running estimate that no events with an earlier event time are still expected; it decides when a window is complete.

Wide-column store
: A database that stores rows identified by a key, whose columns are grouped into column families and can vary from row to row.

Window
: A bounded slice of a stream over which aggregates are computed: tumbling, sliding (hopping), or session.

Write-ahead log (WAL)
: A sequential log forced to stable storage before a commit is acknowledged, replayed after a crash.

Write amplification
: The amount of data physically written compared with the data logically changed, for example through repeated compaction.

Write coalescing
: Collapsing many updates to the same item into one write.

Write concern
: A setting that controls how many replicas must acknowledge a write before it is considered successful.

Write skew
: An anomaly allowed by snapshot isolation in which two transactions read overlapping data and write different data, jointly breaking a rule.

Write-through
: A caching pattern in which writes go to the cache and the database together.

Z-ordering
: A multi-column sorting technique that places similar values together so that file statistics become tight and more files can be skipped.